
Lecture Notes on Natural Language Processing

UNIVERSITY OF LJUBLJANA
Faculty of Computer and Information Science
Erasmus Mundus Joint Master's Degree in Artificial Intelligence
Spring Semester 2026

Author

Guillem Masdemont

Lecturer

Prof. Dr. Marko Robnik-Šikonja

Head of the Machine Learning and Language Technologies Lab

June 20, 2026

Abstract

These lecture notes cover the core topics of the **Natural Language Processing** course taught by Prof. Dr. Marko Robnik-Šikonja at the Faculty of Computer and Information Science, University of Ljubljana, during the spring semester of 2026, as part of the Erasmus Mundus Joint Master's Degree in Artificial Intelligence.

The notes progress from foundational tools like regular expressions, formal grammars, through classical text representations and modern deep learning architectures: recurrent networks, the Transformer, and large pre-trained models including BERT, T5, and the GPT family. In later chapters we address alignment techniques (RLHF, DPO), prompt engineering, retrieval-augmented generation (RAG), LLM-based agent architectures, and neural information retrieval.

The goal is to provide a concise yet rigorous reference that complements the course slides and supports exam preparation. I thank Pietro Sestito for his contributions, which are incorporated throughout.

Contents

1	Introduction	6
1.1	Grading	6
1.2	Understanding Natural Language	6
1.2.1	Types of Ambiguity	6
1.2.2	Disambiguation Approaches	7
1.2.3	Linguistic Analysis	7
1.2.4	Language Understanding as an AI Test	7
1.3	History of NLP: From ELIZA to 2026	8
1.3.1	Two Historical Approaches	9
1.3.2	Classical Approach to Text Understanding	9
1.3.3	NLP Success Stories	9
1.3.4	Where Is NLP Today?	9
1.3.5	CJVT UL: Center for Language Resources and Technologies	10
2	Text processing	11
2.0.1	Language-Specific Tokenization Challenges	12
2.1	Tools for NLP	12
2.2	Corpora	12
2.3	Corpus Statistics and Heaps' Law	12
2.4	Term Normalization	13
2.5	Lemmatization	13
2.6	Morphology, Stemming, and Segmentation	14
2.6.1	Morphology	14
2.6.2	Stemming vs. Lemmatization	14
2.6.3	Sentence Segmentation	15
3	Formal languages	16
3.1	Regular Expressions (RE)	16
3.1.1	Core Syntax and Operators	16
3.1.2	Examples	17
3.1.3	Errors in Pattern Matching	18
3.1.4	Application: ELIZA Using Regular Expressions	18
3.2	Formal Languages and Models	18
3.3	Formal Language Definition	19
3.4	The Chomsky Hierarchy	19
3.5	Type 3: Regular Languages	20
3.6	Type 2: Context-Free Languages	20
3.7	Type 1: Context-Sensitive Languages	20
3.8	Context-Free Languages and Parse Trees	21
3.9	Ambiguous Grammars	21
3.10	The CYK Algorithm	22
4	Sparse text representations	23
4.1	Distributional Semantics	23
4.2	Text similarity	24
4.3	Metrics for evaluation: Precision and Recall	25
5	N-gram models	27
5.1	Language Models	27
5.2	Markov assumption	28
5.3	Log probabilities	29
5.4	Intuition of Perplexity	29
5.5	Evaluating N-gram Models	30
5.5.1	Intrinsic vs. Extrinsic Evaluation	30
5.5.2	Training, Development, and Test Sets	31
5.6	Large N-gram Corpora and Tools	31

6	Revision of neural networks	32
6.1	Artificial Neural Networks	32
6.2	The Artificial Neuron and the Perceptron	32
6.3	Activation Functions	32
6.4	Feed-Forward Neural Networks	32
6.5	Gradient Descent	33
6.6	Backpropagation	33
6.7	Softmax and Cross-Entropy Loss	34
6.8	Network Topology and Practical Considerations	34
6.9	Overfitting and Regularization	34
6.10	Successes and Weaknesses of Deep Learning	35
7	Static neural embeddings	36
7.1	Word2vec	36
7.1.1	Skip-gram with Negative Sampling	37
7.1.2	CBOW (Continuous Bag of Words)	38
7.2	GloVe: Global Vectors for Word Representation	39
7.3	Subword Information and Positional Weighting	40
7.4	Evaluation of Embeddings	40
7.5	Cross-lingual Embeddings	41
7.6	Cross-lingual Applications and Limitations	42
7.7	Other Applications of Word2vec: Doc2vec	42
8	Recurrent and Convolution neural network for text	43
8.1	Recurrent Neural Networks	43
8.2	Long Short-Term Memory (LSTM)	44
8.3	Gated Recurrent Unit (GRU)	45
8.4	Bidirectional RNNs	45
8.5	The Attention Mechanism in RNNs	46
8.6	Convolutional Networks for Text Classification	47
8.6.1	Character-Level CNN Architecture (Zhang et al., 2015)	47
8.6.2	Transition from Statistical N-grams to Deep Networks in Speech Recognition	47
9	Transformers	48
9.1	Transformer Architecture	48
9.2	Very important nuances:	48
9.3	Self-Attention Mechanism	48
9.4	Multi-Head Attention	49
9.5	Positional Encoding	51
9.5.1	Sinusoidal (Fixed) Encoding – Vanilla Transformer	52
9.5.2	Learned Absolute Encoding – BERT	52
9.5.3	Relative Position Buckets – T5	52
9.5.4	Rotary Positional Encoding (RoPE) – Modern Decoder LLMs	52
9.6	Subword Encoding Tokenization and Embedding	53
9.7	Token Embedding Layer in Autoregressive Decoders (GPT)	54
9.7.1	Computational Complexity Analysis	54
10	BERT and T5 models	55
10.1	Pretrained Language Models	55
10.2	BERT	55
10.2.1	Motivation: Why Bidirectional?	55
10.2.2	BERT Architecture	55
10.2.3	Input Representation: Subword Encoding	56
10.2.4	Masked Language Modeling (MLM)	56
10.2.5	Next Sentence Prediction (NSP)	57
10.2.6	Model Sizes and Training	58
10.3	Fine-tuning BERT for Downstream Tasks	58
10.4	BERT as Contextualized Embeddings	60
10.5	Evaluating Language Understanding: GLUE and SuperGLUE	60
10.6	Macro-Evaluation via the GLUE and SuperGLUE Benchmarks	62
10.7	Pretrained Models for Different Languages	62
10.7.1	Vocabulary and Tokenization Quality	63

10.8	Cross-Lingual Transfer	63
10.8.1	Classical Cross-Lingual Transfer	63
10.8.2	Multilingual LLMs	63
10.8.3	Trilingual Models	63
10.9	Parameter-Efficient Fine-Tuning	64
10.10	Attention Efficiency and Long Sequences	64
10.11	Modern BERT	64
10.12	T5 – Text-To-Text Transfer Transformer	65
10.12.1	Motivation	65
10.12.2	Architecture: T5 vs. BERT	65
10.13	The Span Corruption Paradigm in Sequence-to-Sequence Denoising	66
10.14	Byte-Level Modeling vs. Subword Tokenization	66
10.14.1	Character-Based and Multilingual Variants: ByT5 and mT5	66
11	GPT – Generative Pre-trained Transformers	67
11.1	GPT Architecture and Training	67
11.2	GPT Evolution and Scaling	68
11.3	In-Context Learning	68
11.4	Reinforcement Learning from Human Feedback (RLHF)	69
11.5	Direct Preference Optimization (DPO)	70
11.6	LLM Evaluation: LLM Arena	70
11.7	Prompt Engineering	70
11.7.1	Prompt Elements	71
11.7.2	General Guidelines for Prompt Construction	71
11.7.3	Few-Shot Prompting	72
11.7.4	Chain-of-Thought (CoT) Prompting	72
11.7.5	Advanced Prompting Techniques	72
11.7.6	Summary of Prompt Engineering Techniques	74
11.8	Retrieval Augmented Generation (RAG)	74
12	LLM-Based Agents	76
12.1	The Agent Loop	76
12.2	From ML to LLM-Based Agents	76
12.3	LLM-Based Agents	76
12.4	How Many Agents?	77
12.5	Agent Roles	77
12.6	Self-Improvement and Feedback Loops	78
12.7	Agent Architectures	78
12.7.1	The Augmented LLM	78
12.7.2	Workflow Patterns	78
12.7.3	ReAct Loop	79
12.8	Multi-Agent Systems	80
12.8.1	Agent Frameworks: AutoGen, LangGraph, and AIOS	82
12.9	Tool Calling and the Model Context Protocol (MCP)	83
13	Neural Information Retrieval	84
13.1	Retrieval Augmented Generation (RAG) in Depth	84
13.2	Key Design Levers	84
13.3	Evaluation Metrics	85
13.4	RAG Improvements	85
13.5	RAG Variants	86
13.6	Re-Ranking	86
13.7	Toolkits for RAG Evaluation: RAGAS	87
13.8	Beyond Textual Data	87
13.9	RAG Pitfalls	88
13.10	Neural Text Ranking	88
13.10.1	BM25	88
13.10.2	Neural Ranking	89

14 POS Tagging, Dependency Parsing, and Named Entity Recognition	90
14.1 Lemmatization	90
14.2 Part-of-Speech Tagging	90
14.3 Word Classes and Tag Sets	90
14.4 Difficulty, Baselines, and Approaches	92
14.5 Slovene Morphosyntactic Tagging	92
14.6 Constituency and Dependency Parsing	92
14.7 Treebanks and Universal Dependencies	93
14.8 Modern NLP Pipelines	93
14.9 Named Entity Recognition	93
14.9.1 BIO Tagging: NER as Sequence Labelling	93
14.9.2 Algorithms and Evaluation	94
15 Word Senses and Disambiguation	95
15.1 Word Senses and Lexical Relations	95
15.2 Lexical Resources	95
15.3 WSD Task	96
15.3.1 Supervised WSD	97
15.3.2 Baselines	97
15.3.3 Semi-Supervised WSD – Bootstrapping	98
15.3.4 Word Sense Induction and WiC	98
16 Affective Computing	100
16.1 Emotion Theory	100
16.2 Sentiment and Emotion Lexicons	101
16.3 Personality Analysis	102
16.4 Sentiment Analysis	102
16.5 Related Tasks	104
17 Machine Translation	105
17.1 Motivation and Classical Paradigms	105
17.2 Statistical Machine translation (MT)	105
17.3 Neural MT	106
17.4 MT Evaluation	106
17.4.1 Human Evaluation	106
17.4.2 Automatic Evaluation Metrics	107
18 Summarization and Question Answering	108
18.1 Automatic Summarization	108
18.2 Question Answering	110
18.2.1 IR-Based and Retrieval-Augmented QA	111
18.2.2 BERT for Reading Comprehension	111
18.2.3 Closed-Book QA and LLM-Based QA	112
18.2.4 QA Datasets	113
19 Parameter-Efficient Fine-Tuning and Language Agents	114
19.1 Parameter-Efficient Fine-Tuning (PEFT)	114
19.1.1 Adapter Fine-Tuning	114
19.1.2 Prefix Tuning and Prompt Tuning	114
19.1.3 LoRA (Low-Rank Adaptation)	115
19.1.4 QLoRA: Quantised LoRA	115
19.1.5 Types of Fine-Tuning: A Summary	116
19.2 Knowledge Distillation (Teacher-Student Framework)	117
19.3 Language Agents	117
Summary of Key Concepts by Chapter	119

1 Introduction

The course structure is divided into two primary components: lectures and tutorials.

The **Lectures** serve as an introduction to the core topics and facilitate high-level discussion. They provide practical examples and offer a broader view of the theoretical landscape within NLP.

The **Tutorials** focus on the application of these concepts through exercises and assignments motivated by practical use. During these sessions, the assistant presents the assignments, provides implementation tips, and moderates discussions. To get the most out of these sessions, students are expected to come prepared and pose questions. These sessions also introduce specific problem-solving tools and software, focusing primarily on the English and Slovene languages.

1.1 Grading

The final grade is determined by the following obligations. Please note the minimum requirements for each component:

Obligation	% of Total	Minimum Requirement
Five Quizzes	0%	$\geq 50\%$ altogether
Projects	50%	$\geq 50\%$ of points
Written Exam	50%	$\geq 50\%$ of points

Definition 1.1 (NLP). *Natural Language Processing involves:*

- **Techniques for language processing:** Including syntax, grammars, language resources, text representation, and text classification.
- **Attempts to understand language:** Exploring the semantics and pragmatics of language, closely related to the ultimate goals of Artificial General Intelligence (AGI).

1.2 Understanding Natural Language

Understanding natural language by a computer requires far more than simple word matching. A system must grapple with words, syntax, semantics, context, as well as the writer's intentions, knowledge, background, assumptions, and biases. The central difficulty is **ambiguity**: natural language is fundamentally ambiguous at multiple levels.

1.2.1 Types of Ambiguity

Lexical Ambiguity The same word can have multiple meanings. In "*The bat flew through the air*", *bat* may refer to a flying mammal or a sports implement.

Syntactic Ambiguity The grammatical structure of a sentence can be interpreted in more than one way. The classic example is "*Flying planes can be dangerous*", where *flying* can be read as:

- An adjective modifying *planes*: *Planes that are flying can be dangerous.*
- A verb in gerundive form: *It can be dangerous to fly planes.*

Newspaper headlines exploit intentional ambiguity for effect (clickbait): "*Juvenile court to try shooting defendant*", "*Kids make nutritious snacks*".

Semantic Ambiguity Even grammatically well-formed sentences can be semantically ambiguous. The sentence "*I made her duck*" has at least five interpretations: *I cooked waterfowl for her*, *I cooked her waterfowl*, *I created the duck she owns*, *I caused her to lower her head*, or *I waved a magic wand and turned her into a duck*. Further examples: "*The girl told the story cried*", "*Put the box on the table in the kitchen*".

Referential Ambiguity Pronouns and referring expressions often have unclear antecedents. In "*The boy and the dog were playing in the park. He ran into a tree.*", it is unclear whether *He* refers to the boy or the dog.

1.2.2 Disambiguation Approaches

Resolving ambiguity is a central problem in NLP. Common approaches include:

- **Part-of-speech (POS) tagging:** Assigning grammatical categories to resolve syntactic ambiguity.
- **Word sense disambiguation:** Selecting the correct meaning of a word based on context (e.g., *jaguar* as a car or an animal).
- **Probabilistic parsing:** Assigning probabilities to alternative parse trees.
- **Speech act interpretation:** Determining whether an utterance is a statement or a question—“*We made it.*” vs. “*We made it?*”.
- **User profiles:** Personalizing disambiguation based on known user context (e.g., a car enthusiast searching *Paris* may mean Paris Hilton or Paris, France).

Linking Linking refers to the ability of a reader to connect units of information at the word, sentence, or discourse level. An example from syntactic theory is the *self-embedded structure*: “*The boy the girl the men left watched then left.*”—it is difficult to identify which noun phrase (*the boy, the girl, the men*) links to each verb (*left, watched, left*). Readability is determined by several linguistic factors: syntactic, semantic, morphological, and discourse-level properties.

1.2.3 Linguistic Analysis

Linguists decompose the knowledge required to understand language into several interacting levels:

1. **Prosody:** The patterns of stress and intonation (rhythm) in speech.
2. **Phonology:** The system of sounds and their relationships that constitute the fundamental components of a language.
3. **Morphology:** The admissible arrangement of sounds in words; how to form words, prefixes, and suffixes.
4. **Syntax:** The arrangement of words and phrases to create well-formed sentences. Syntax represents the kind of implicit knowledge of a native language mastered by age 3–4 without explicit instruction, not merely the rules later taught in school.
5. **Semantics:** The meaning of a word, phrase, sentence, or text.
6. **Pragmatics:** Language in use and the contexts in which it is used, including deixis (words whose meaning changes with context: *I, he, here, there, soon*), turn-taking in conversation, text organization, presupposition, and implicature. For example, “*Can you pass me the salt?*” is pragmatically a request, not a question about capability.
7. **Knowledge of the world:** Knowledge of the physical world, humans, society, and communicative intentions.

These levels are not independent: for example, intonation affects semantics (sarcasm), and discourse context affects word-sense resolution.

Linguistic Priming Language shapes perception. In a classic experiment (Enfield 2022), subjects who were given a word label before viewing ambiguous geometric figures reproduced the figure in a way influenced by the label—demonstrating that the words we know prime how we perceive and recall the world.

1.2.4 Language Understanding as an AI Test

The ability to understand natural language on a human level has long been considered a proxy for general intelligence. A system that truly understands language would contain a large portion of human knowledge and its responses would be indistinguishable from a human’s.

The Turing Test Proposed by Alan Turing, the test asks: can a person (Person A), communicating only via text, determine whether they are talking to a machine (Machine Z) or another person (Person X)? Passing this test is considered a hallmark of human-level language ability.

Chatbots Attempting the Turing Test Systems designed to pass include: CYC, ELIZA, ALICE, chatterbot, VIDA, TIA, Siri, and modern LLMs such as ChatGPT, Gemini, and DeepSeek.

Remark 1.1 (The ARC-AGI Challenge). *A more rigorous test of general reasoning is the ARC-AGI benchmark, which presents pattern-completion puzzles trivial for humans but challenging for LLMs. Current models still fall well short of human performance, suggesting that fluent language generation does not imply general intelligence.*

1.3 History of NLP: From ELIZA to 2026

The history of Natural Language Processing is generally categorized into four distinct paradigms, shifting from hand-coded logic to massive-scale data learning.

1. Symbolic Era (1950s–1980s): Focused on formal languages and rule-based systems.

- **ELIZA (1966):** Created by Joseph Weizenbaum, ELIZA simulated a Rogerian psychotherapist using simple pattern matching and substitution rules. For example, the pattern `s/. * I'M (depressed|sad) .*/I AM SORRY TO HEAR YOU ARE \1/` would transform the user's statement into a reflective question. Despite having no actual understanding, users repeatedly became emotionally attached to the system—the *ELIZA effect*. Weizenbaum later became a vocal critic of AI upon realizing that “even once the processes were explained, many people still bought into the illusion.” The same dynamic recurs today with systems like ChatGPT.
- **SHRDLU (1972, Winograd):** Demonstrated language understanding within a constrained *micro world* of simple geometric objects (blocks, pyramids, cylinders on a table). The system could answer questions such as “What is sitting on the red block?” and execute commands such as “Place the green pyramid on the red brick.” Its success depended entirely on the closed-world assumption—once that was relaxed, the approach did not scale.

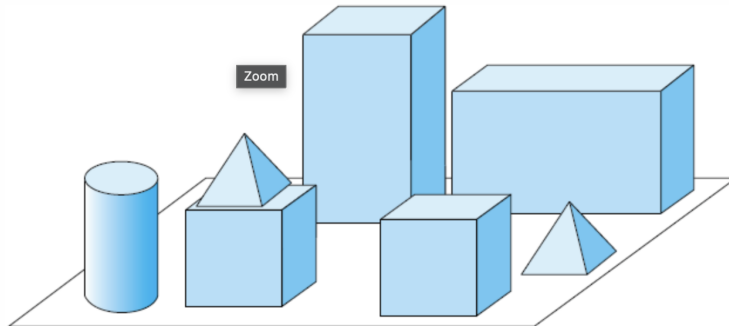


Figure 1: Input of the BERT model

2. Statistical Era (1990s–2010s): A shift toward probabilistic modeling fueled by the availability of large text corpora.

- Introduction of **Hidden Markov Models (HMM)** for tagging and **N-grams** for predicting word sequences.
- Development of Statistical Machine Translation (SMT), which dominated until the arrival of deep learning.

3. Neural Era (2013–2017): The rise of “Word Embeddings” and Recurrent Neural Networks (RNNs).

- **Word2Vec (2013):** Mapped words into vector spaces where spatial distance represents semantic similarity.
- **LSTMs:** Allowed models to “remember” longer sequences, though they remained computationally slow due to sequential processing.

4. Transformer & LLM Era (2017–Present): The current state of the art based on the Attention mechanism.

- **Attention is All You Need (2017):** Introduced the Transformer, enabling parallel processing of text.

- **LLMs (GPT-4, Claude, Llama):** Massive scaling of parameters leading to emergent reasoning, multimodality (text/image/video), and agentic behavior.

1.3.1 Two Historical Approaches

Historically, NLP research has proceeded along two parallel tracks:

- **Symbolic (rule-based):** Knowledge is injected manually in the form of grammars, frames, parse trees, and semantic rules. This top-down, "Good Old-Fashioned AI" (GOFAI) approach is interpretable but brittle and expensive to scale.
- **Statistical (data-driven):** Knowledge is extracted from large corpora via bottom-up learning—patterns, links, and probabilistic reasoning. Even outputs that are syntactically or semantically wrong can emerge. Large pretrained language models (BERT, GPT, T5, LLaMA, Gemini, Claude, DeepSeek, GAMS) are the current apex.

The emerging paradigm merges both: injecting symbolic knowledge into LLMs (via retrieval, fine-tuning, or tool calling) and letting LLMs call external services.

1.3.2 Classical Approach to Text Understanding

The classical pipeline for text understanding comprises three sequential phases:

1. **Syntactic Analysis:** Find the syntactical structure via POS tagging, identify the role of each word in the sentence (subject, object, predicate). The result is a *parse tree*. Required knowledge: syntax, morphology, and some semantics.
2. **Semantic Interpretation:** Apply knowledge of word meanings and their language use. The result is an internal representation such as conceptual graphs, frames, or a logical program.
3. **Use of World Knowledge:** Extend the interpretation with background knowledge. Consider the purpose of the system (summarization, database interface, etc.). Knowledge bases such as Cyc and OpenCyc provide ontologies of common-sense knowledge (e.g., "*Every tree is a plant*", "*Plants die eventually*"). Processing proceeds incrementally, adding the meaning of previous sentences.

1.3.3 NLP Success Stories

NLP has produced numerous practical successes:

- **2011:** IBM Watson won against two human champions in *Jeopardy!*
- Widely used tools: Google Translate, Siri, Cortana, Alexa, search engines.
- Information extraction and retrieval, speech recognition, text classification.
- Summarization, question answering, ChatGPT.
- LLMs as a general approach to AI.

1.3.4 Where Is NLP Today?

NLP is an active research area with many commercial applications:

- Speech recognition and synthesis, automatic reply engines.
- Machine translation, text summarization, question answering.
- Language generation, database interfaces.
- Intelligent search, information extraction, sentiment detection.
- Named entity recognition and linking.
- Categorization and classification of documents, messages, tweets.
- Prevalence of deep neural network approaches, cross-lingual methods.
- LLMs are ubiquitous—*LLMs for everything*.

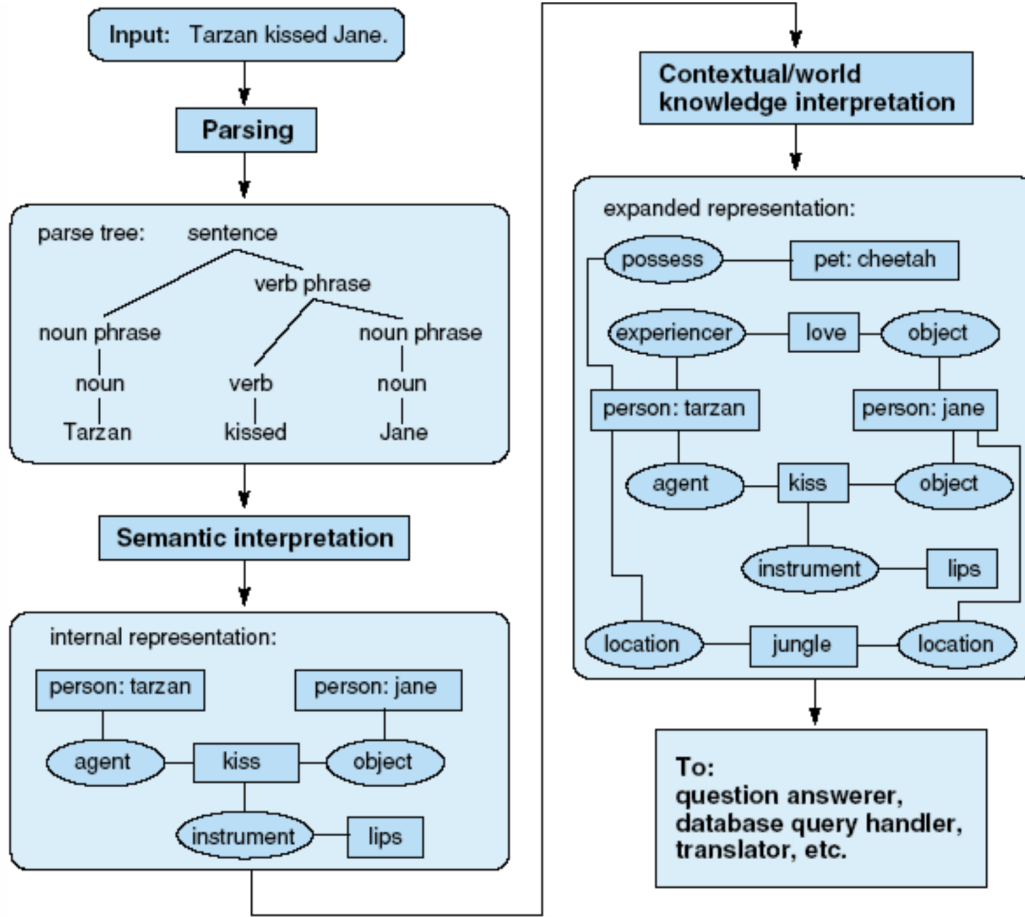


Figure 2: Tarzan kissing Jane, a syntactic representation and use of world language.

Open questions remain about language understanding itself: Can current systems pass the Turing test? Do large language models truly understand language, or are they sophisticated autocomplete? We do not yet have good enough models for knowledge representation. A key research direction is injecting different kinds of knowledge into deep neural networks: factual, linguistic, common-sense, and domain-specific.

Remark 1.2 (ChatGPT’s Current Limitations). *Despite impressive fluency, ChatGPT and similar LLMs exhibit systematic failures: bias, errors in logical reasoning and arithmetic, ethical issues, and hallucinations (stating false information confidently). They are convincing but sometimes wrong—a concern that echoes the ELIZA effect at a much larger scale.*

1.3.5 CJVT UL: Center for Language Resources and Technologies

The Centre for Language Resources and Technologies of the University of Ljubljana (CJVT UL) develops many practical open-source NLP tools and resources for Slovene, including: corpora and datasets, thesauri, dictionaries of collocations, lexicons of wordforms, sentiment lexicons, machine translation, speech recognition, neural POS taggers, comma-placement and readability models, summarizers, cross-lingual models, embeddings, and LLMs such as GaMS.

2 Text processing

The classical NLP pipeline follows a hierarchical structure, breaking down complex documents into manageable units through a series of deterministic and probabilistic steps.

The process typically flows from coarse to fine-grained analysis:

1. **Segmentation:** A document is first decomposed into its constituent **paragraphs**, which are further split into **sentences**.
2. **Tokenization:** Sentences are broken down into individual **words** (tokens). While classical methods focus on whitespace and punctuation, modern neural pipelines often utilize **subword tokens** (e.g., BPE or WordPiece) to better handle rare words and morphologically rich languages like Slovene.
3. **Morphosyntactic Tagging (POS):** Each token is assigned a **Part-of-Speech tag** (noun, verb, adjective, etc.), providing the grammatical category necessary for further analysis.
4. **Syntactic and Grammatical Analysis:** Sentences undergo dependency or constituency parsing to reveal the underlying **grammatical structure** and relationships between words (e.g., identifying the subject, object, and predicate).

Definition 2.1 (Token). A **token** is an instance of a sequence of characters in some text processing task that are grouped together as a useful semantic unit for processing.

Definition 2.2 (Type). A **type** is the class of all tokens containing the same character sequence.

Definition 2.3 (Term). A **term** is a (perhaps normalized) type that is included in the system’s dictionary.

Example 2.1. Consider the sentence: "The cat chased the big cat."

- **Tokens (7):** Every individual word instance, counting duplicates.
[The] [cat] [chased] [the] [big] [cat] [.]
- **Types (6):** The unique character sequences (case-sensitive).
{The, cat, chased, the, big, .}
- **Terms (4):** The normalized dictionary entries (lowercased, punctuation stripped, and stop-words removed like "the").
{cat, chase, big}

Tokens	Types	Terms
The, the	The, the	– (Removed as stop-word)
cat, cat	cat	cat
chased	chased	chase (Lemmatized)
big	big	big
.	.	– (Punctuation stripped)

Tokenization is frequently oversimplified as the process of splitting text by whitespace. However, natural language contains structural ambiguities where a single space does not necessarily denote a semantic boundary, and punctuation does not always signify a terminal break.

Example 2.2 (The Ambiguity of Punctuation and Compounds). Consider the sentence: "Dr. Jones’s assistant didn’t leave N.Y.C. at 5 p.m."

A naive whitespace tokenizer would fail to distinguish **Abbreviations** like "Dr.", "N.Y.C.", and "p.m.", where periods are internal to the token rather than sentence boundaries. It also struggles with **Clitics and Contractions** such as "didn’t," which should be decomposed into **did** and **n’t** for accurate downstream grammatical analysis. Furthermore, **Possessives** like "Jones’s" require the separation of the entity from the possessive marker, while **Multi-word Expressions** such as "New York City" should ideally be treated as a single semantic unit despite the presence of internal spaces.

UNIX-style Tokenization A simple UNIX tokenization approach uses regular expressions to identify token boundaries. For example, splitting on whitespace and punctuation, then applying substitution rules:

```
s/([.,;:?!()"'])/ \1 /g # space around punctuation
```

This handles many common cases but fails on abbreviations and clitics. More robust tokenizers (e.g., NLTK’s `word_tokenize`) use a combination of RE rules, abbreviation lists, and trained models.

2.0.1 Language-Specific Tokenization Challenges

French: Elision and Clitics French contracts articles before vowels: *l'ensemble* must be split into *l'* and *ensemble*. Similarly, clitic pronouns attach to verbs: *j'ai*, *n'est-ce pas*.

German: Compound Words German routinely forms very long compound nouns: *Lebensversicherungsge-sellschaftsangestellter* (life-insurance-company employee). These must be split into constituent morphemes for meaningful analysis.

Chinese and Japanese: No Whitespace Chinese and Japanese do not use spaces between words. Tokenization requires **word segmentation**: finding the most probable sequence of words for a given character string. For example, a sequence of Chinese characters meaning “Sharapova now lives in the southeast of the United States” must be segmented by a statistical or neural model to identify named entities, place names, and function words—there is no whitespace signal to exploit. This is fundamentally a different problem from European-language tokenization and requires dedicated, language-specific tools.

2.1 Tools for NLP

Several open-source libraries provide robust implementations of the preprocessing pipeline described above:

- **NLTK (Natural Language Toolkit)**: A comprehensive Python library covering tokenization, POS tagging, parsing, stemming, and much more. Widely used in teaching and research.
- **spaCy**: A fast, production-oriented Python library with pretrained models for tokenization, NER, POS tagging, and dependency parsing across many languages.
- **Gensim**: Specialized in topic modeling and word embeddings (Word2Vec, FastText, LSI).
- **CLASSLA-Stanza**: A pipeline built on top of Stanford’s Stanza, specifically fine-tuned for South Slavic languages including **Slovene**. Provides tokenization, lemmatization, POS tagging, and dependency parsing for Slovene with state-of-the-art accuracy.

2.2 Corpora

A **corpus** (plural: corpora) is a large, principled collection of text used to train, evaluate, and study NLP systems. Corpora vary along several dimensions:

- **Language and variety**: A corpus may be monolingual (e.g., English only) or multilingual. Within a language, it may cover a single variety (standard British English) or include regional dialects and non-standard forms (e.g., African American Language).
- **Code-switching**: Some corpora capture speakers fluidly alternating between languages within a single utterance, common in multilingual social media.
- **Genre**: Linguistic characteristics differ markedly across news, fiction, scientific text, social media, or legal documents.
- **Author demographics**: Age, gender, race, and socioeconomic status all influence the linguistic signal in a corpus.

Corpus Datasheets To promote transparency and reproducibility, modern corpus creation follows the practice of **corpus datasheets**—structured documentation that records the purpose, collection methodology, potential biases, and licensing of a dataset, analogous to datasheets for electronic components. This practice is important because many historic NLP systems were trained on corpora whose demographic and genre biases were never disclosed.

2.3 Corpus Statistics and Heaps’ Law

Quantifying a corpus requires a clear distinction between the total number of instances (**tokens**) and the unique set of words (**types**).

Definition 2.4 (Corpus Metrics). *Let N represent the total number of tokens in a document or corpus. The **Vocabulary** (V) is the set of all unique types, where $|V|$ denotes the vocabulary size.*

The growth of the vocabulary relative to the corpus size is governed by **Heaps' Law** (also known as Herdan's Law). It states that the vocabulary size V grows as a function of the number of tokens N according to the power law:

$$V = kN^\beta$$

In most natural language contexts, the exponent β typically falls between 0.67 and 0.75. This implies that the vocabulary size grows at a rate slightly greater than the square root of the number of word tokens, meaning that as a corpus grows, we continue to encounter new types, albeit at a diminishing rate.

Corpus	Tokens (N)	Types ($ V $)
Shakespeare	884,000	31,000
Google n -grams (2006)	1,024,908,267,229	13,588,391

Table 1: Corpus statistics illustrating Heaps' Law: vocabulary grows sublinearly with corpus size.

This essentially says that Shakespeare published about 884,000 words in total, with around 31,000 unique words (Shakespeare vocabulary).

Remark 2.1 (Linguistic Diversity and Variance). *NLP must account for the vast diversity across the 7,097 global languages and their internal variations. Key challenges include:*

- **Dialects and Varieties:** *Non-standard forms, such as African American Language (AAL) (e.g., "iont" for "I don't"), require inclusive modeling.*
- **Code-switching:** *The fluid alternation between languages (e.g., Spanish/English or Hindi/English) within a single utterance is common in multilingual social media contexts.*
- **Genre and Demographics:** *Textual characteristics shift significantly across **genres** (newswire vs. fiction) and **author demographics** (age, gender, race, and socioeconomic status), all of which influence the linguistic signal.*

2.4 Term Normalization

Term normalization is the process of mapping varied surface forms of a token to a single, standardized "term" to ensure consistency in Information Retrieval (IR) and downstream analysis.

Remark 2.2 (The Need for Normalization). *For a system to successfully match a user's query with indexed text, both must share the same form. Without normalization, a search for **USA** would fail to return documents containing **U.S.A.**, and a search for **fed** might miss the capitalized **Fed**. This process implicitly defines **equivalence classes**, such as treating *uhhuh* and *uh-huh* as identical units or mapping the inflected forms *am*, *is*, *are* to the lemma *be*.*

Asymmetric Expansion An alternative to rigid normalization is asymmetric expansion, which provides more granular control at the cost of efficiency. Unlike standard normalization (where all forms are reduced to one), expansion broadens the search based on the specific input:

- A search for *window* expands to include *window* and *windows*.
- A search for *Windows* (capitalized) may be restricted to the operating system only.

While this method is potentially more powerful for capturing user intent, it is significantly less efficient as it increases the complexity of the query at runtime.

Case Folding The simplest normalization is **case folding**: converting all characters to lowercase. This allows *Woodchuck* and *woodchuck* to be treated identically. For **information retrieval**, case folding is almost always beneficial. However, for named entity recognition and other tasks, capitalization carries meaningful signal (e.g., *US* the country vs. *us* the pronoun), so case folding must be applied selectively.

2.5 Lemmatization

Lemmatization is the process of reducing inflections or variant surface forms of a word to their base dictionary form, known as the **lemma**. Unlike simple stemming, which chops off affixes, lemmatization involves a full morphological analysis to identify the correct "headword."

Example 2.3 (English and Slovene Lemmatization). *The goal is to map all related forms to a single semantic anchor:*

- **English:** *am, are, is* → **be**; *car, cars, car's, cars'* → **car**.
- **Slovene:** The forms *hočem* (I want) and *hočeš* (you want) are mapped to the lemma **hoteti** (to want).

A fully lemmatized sentence like "the boy's cars are different colors" would be reduced to: **the boy car be different color**.

This process is critical for tasks like **Machine Translation** and **Information Retrieval**, as it allows the system to recognize that different grammatical versions of a word convey the same underlying concept. However, it requires a comprehensive dictionary and a morphological parser to handle ambiguity (e.g., determining if "saw" is the lemma for a tool or the past tense of "see").

The difficulty of lemmatization varies dramatically by language. English has relatively few inflectional forms, making it tractable with small dictionaries. **Slovene**, by contrast, has a rich case system (six cases), three genders, three numbers (singular, dual, plural), and extensive verb conjugation—meaning that a single lemma may have dozens of surface forms. High-quality Slovene lemmatization requires large, manually compiled lexicons and probabilistic disambiguation models (e.g., those provided by CLASSLA-Stanza).

2.6 Morphology, Stemming, and Segmentation

To process natural language, we must understand the internal structure of words and the boundaries that define separate thoughts.

2.6.1 Morphology

Morphology is the study of how words are formed and their relationship to other words in the same language. Every word is built from **morphemes**—the smallest meaning-bearing units. These fall into two broad classes:

- **Stems:** The core, meaning-bearing part of the word (e.g., *teach* in *teacher*).
- **Affixes:** Prefixes, suffixes, infixes, or circumfixes that modify meaning or grammatical function.

Morphology is further divided by what the affixes do:

- **Inflectional Morphology:** Modifying a word to express grammatical categories like tense, number, or gender without changing its core meaning (e.g., *run* → *runs*).
- **Derivational Morphology:** Creating new words from existing ones, often changing the part of speech (e.g., *teach* → *teacher*).

Morphological Parsers A **morphological parser** takes a word and returns its morphemes together with their grammatical labels. This is straightforward in English (with its relatively impoverished inflection system) but becomes highly complex in morphologically rich languages.

Example 2.4 (Complex Morphology: Turkish Agglutination). *Turkish is an **agglutinative** language: many morphemes attach in sequence to a single root, each encoding a distinct grammatical feature. For example:*

uygarlaştıramadıklarımızdanmızsınızcasına

*meaning approximately “(behaving) as if you are among those whom we could not civilize.” This single word encodes the root *uygar* (civilized) plus a cascade of derivational and inflectional morphemes. Such languages require dedicated finite-state morphological analyzers.*

2.6.2 Stemming vs. Lemmatization

Both techniques aim to reduce inflectional forms to a common base, but they differ significantly in their approach:

Stemming A crude, heuristic process that chops off the ends of words in the hope of reaching a "stem." The most famous implementation is **Porter's Algorithm**, which applies a cascade of suffix-stripping rules (e.g., *caresses* → *caress*, *relational* → *relate*). Key characteristics:

- Operates **without context**—the same surface form is always reduced the same way, regardless of surrounding words.
- Fast but often produces non-words (e.g., *university* → *univers*).
- Rare in modern NLP pipelines, which prefer subword tokenization (BPE, WordPiece) or lemmatization.

Lemmatization A sophisticated morphological analysis that uses a vocabulary and part-of-speech information to return the **lemma** (the dictionary headword). While more computationally expensive, it is far more accurate (e.g., "saw" → "see" or "saw" depending on context).

2.6.3 Sentence Segmentation

Sentence segmentation (or Boundary Detection) is the task of deciding where one sentence ends and the next begins. While the period (.) is the primary indicator, it is highly ambiguous.

Example 2.5 (Segmentation Ambiguity). *In the string "The NASA rep. said the launch is at 5 a.m. on Tuesday.", the periods in "rep." and "a.m." are not boundaries. A segmenter must use a list of known abbreviations or a machine learning classifier (such as a Decision Tree or an HMM) to distinguish between a terminal period and a decimal or abbreviation point.*

Machine Learning Approach to Sentence Segmentation A common approach is to train a **binary classifier** (e.g., a Decision Tree) on features extracted around each period character. The classifier decides whether a given period marks the *end-of-sentence* (EOS) or not.

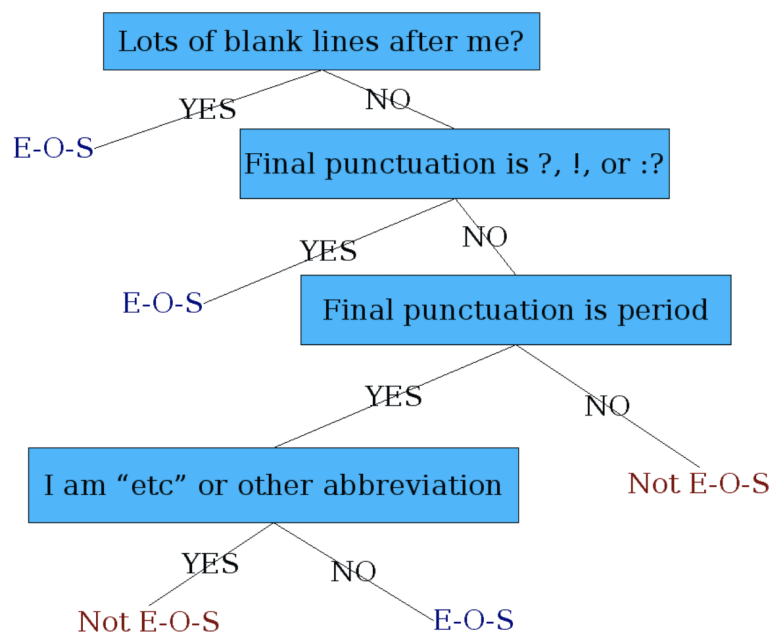


Figure 3: Decision tree

Features used for end-of-sentence detection include:

- **Case of the word containing the period:** Proper nouns and sentence-initial words are capitalized.
- **Case of the word following the period:** A capitalized word after the period is evidence of a new sentence.
- **Length of the word preceding the period:** Short words are often abbreviations.
- **Membership in an abbreviation list:** Known abbreviations (Mr., Dr., a.m.) are marked.
- **Numeric word:** Periods after numbers are often decimals or list markers.
- **Prefix or suffix patterns** of the word before and after.

3 Formal languages

3.1 Regular Expressions (RE)

Definition 3.1 (Regular Expression). *A Regular Expression is a standard notation for characterizing text sequences. If A and B are REs, then their concatenation AB is also an RE. The primitive elements include alphanumeric characters $(a, b, \dots, z, 0, 1, \dots, 9)$, which match themselves.*

3.1.1 Core Syntax and Operators

While multiple syntaxes exist, we focus on the Python-standard notation. The following operators define the logic of REs:

1. Anchors and Wildcards

- Dot (`.`): Matches any single character except a newline. E.g., `va.a` matches *vaba*, *vaza*, or *vaya*.
- Caret (`^`): Matches the start of a string. E.g., `^noga` matches *noga* at the start, but not inside *nadloga*.
- Dollar (`$`): Matches the end of a string. E.g., `end$` matches a line that ends with the word *end*.

2. Quantifiers and Greediness

Quantifiers determine how many times a preceding element can repeat. By default they are **greedy**, meaning they match the longest possible string.

- `*`, `+`, `?`: Match 0 or more, 1 or more, and 0 or 1 repetitions, respectively. E.g., `go*d` matches *gd*, *god*, *good*; `go+d` requires at least one *o*, so it matches *god* and *good* but not *gd*.
- Non-greedy variants: Appending `?` to a quantifier (e.g., `*?`, `+`?) causes **minimal** matching—it stops as soon as the rest of the pattern can match.
- Explicit bounds: `{m,n}` matches between *m* and *n* repetitions; `{m,}` means at least *m*. E.g., `\d{2,4}` matches 2, 3, or 4 consecutive digits.

Example 3.1 (Greedy vs. Non-Greedy Matching). *Consider the string `<a> b <c>`, which contains two angle-bracket tags.*

- Greedy `<.*>`: `.*` expands as far right as possible, so the whole string `<a> b <c>` is consumed as a single match.
- Non-greedy `<.*?>`: `.*?` stops at the first closing `>`, returning only `<a>`. Running the pattern again on the remainder then yields `<c>`.

The non-greedy variant is essential whenever the target pattern can appear multiple times in the same string.

Example 3.2 (Explicit Bounds — `{m,n}`). *Suppose we want to match any word that is between 4 and 6 letters long:*

`\b[a-zA-Z]{4,6}\b`

- `\b` is a word-boundary assertion: it matches the transition between a word character and a non-word character (zero width).
- `[a-zA-Z]{4,6}` requires between 4 and 6 letters.

This matches word (4 chars), hello (5), and Python (6), but not hi (2) or elephant (8). Replacing `{4,6}` with `{4,}` removes the upper bound, matching any word of at least 4 letters.

3. Sets and Groups

- Character sets `[...]`: Matches any *one* character listed inside the brackets. Ranges are allowed. E.g., `[aeiou]` matches any vowel; `[a-z]` matches any lowercase letter; note that `*` and `.` lose their special meaning inside `[]`.
- Complements `[^...]`: Matches any character *not* in the set. E.g., `[^aeiou]` matches any non-vowel character.
- Alternation `|`: Logical OR, tested left to right. E.g., `cat|dog` matches either *cat* or *dog*.
- Grouping `(...)`: Groups part of the pattern and *captures* the matched text. E.g., `(un)?happy` matches both *happy* and *unhappy*; the captured group `\1` then holds *un* (or is empty).

Example 3.3 (Sets, Complement, Alternation, and Grouping). *Consider the task of matching a simple CSV field that is either a quoted string or a plain number:*

`"[^"]*" / \d+`

Breaking it down:

- `"[^"]*"`: a double-quote, then `[^"]*` (zero or more characters that are not a double-quote), then a closing double-quote. This matches `"hello"`, `"foo bar"`, and even `"` (empty field).
- `|`: alternation — try the left branch first; if it fails, try the right.
- `\d+`: one or more digit characters, matching plain numbers such as `42` or `2024`.

Wrapping the whole pattern in a group, `("[^"]*" / \d+)`, allows the matched field to be captured as group 1 for further processing. Applied to the string `42,"hello",7` the pattern finds three non-overlapping matches: `42`, `"hello"`, and `7`.

4. **Special Flags** Flags are written as `(?flag)` at the start of the pattern and change how matching is performed:

- `i` (ignore case): `(?i)hello` matches `hello`, `Hello`, and `HELLO`.
- `s` (dot-all): makes `.` match newline characters as well, useful for multi-line text blocks.
- `m` (multi-line): `^` and `$` match the start and end of *each line* rather than the whole string.
- `a` (ASCII): restricts `\d`, `\w`, etc. to ASCII characters only.

Example 3.4 (Combining Flags — Case-Insensitive Multi-Line Search). Suppose a log file contains lines such as:

```
ERROR: disk full
warning: low memory
Error: timeout
```

To find every line that starts with either `error` or `warning` regardless of capitalisation, combine the `(?i)` (ignore case) and `(?m)` (multi-line) flags:

`(?im)^(error/warning)`

- `(?im)`: enables both flags simultaneously.
- `^`: with `(?m)` active, this anchors to the start of each line, not just the start of the whole string.
- `(error/warning)`: alternation captures either keyword.

All three lines above match, whereas a plain `error/warning` (no flags, no anchor) would also match mid-line occurrences and would be case-sensitive.

3.1.2 Examples

The following examples show regular expressions of increasing complexity, each building on the operators introduced above.

Example 3.5 (Spelling Variants — using `?` and `|`). English has many words with British and American spellings that differ by a single character. The `?` quantifier makes the preceding character optional, so:

`colour?r`

matches both `color` (the `u` is absent) and `colour` (the `u` is present). Alternation handles larger differences: `cent(er|re)` matches both `center` and `centre`. These tiny patterns already cover a real normalization problem in corpus preprocessing.

Example 3.6 (Finding Phone Numbers — using `\d`, `{m,n}`, and `[]`). A simple pattern for a European-style phone number (digits grouped with optional spaces or hyphens) is:

`\d{2,3}[- ]?\d{3}[- ]?\d{3,4}`

- `\d{2,3}`: the area code (2 or 3 digits).
- `[- ]?`: an optional hyphen or space separator.
- `\d{3}` and `\d{3,4}`: the subscriber number split into two groups.

This matches `01 234 5678`, `01-234-5678`, and `012345678`, but not `1234` (too short).

Example 3.7 (Email Address Matching — combining multiple operators). *Matching an email address requires combining character sets, quantifiers, and literal characters. A standard pattern is:*

`[a-zA-Z0-9.%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}`

Read it in three parts, separated by the literal @:

- **Local part** `[a-zA-Z0-9.%+-]+`: one or more characters that may be letters, digits, dots, percent signs, plus signs, or hyphens (e.g. `john.doe+work`).
- **Domain** `[a-zA-Z0-9.-]+`: the hostname, allowing letters, digits, dots, and hyphens (e.g. `uni.ac`).
- **Top-level domain** `\.[a-zA-Z]{2,}`: a dot followed by at least two letters (e.g. `.com`, `.si`).

The pattern matches `user@example.com` and `john.doe+work@uni.ac.uk`, but rejects `user@` (no domain) or `user@com` (no dot before TLD).

Remark 3.1 (The Role of Regular Expressions in Modern NLP). *Despite the shift toward deep learning, regular expressions remain a foundational tool in the NLP pipeline for several reasons:*

- **First-Pass Modeling:** *They are often the first model implemented for any text processing task due to their deterministic nature and ease of implementation.*
- **Hybrid Systems:** *While machine learning classifiers are preferred for “hard” tasks involving nuance and context, REs are still used extensively for pre-processing data or as deterministic features within those very classifiers.*
- **Capturing Generalizations:** *They are exceptionally powerful at capturing rigid linguistic generalizations (e.g., phone numbers, email addresses, or specific morphological patterns) that might require thousands of examples for a neural model to learn less reliably.*

3.1.3 Errors in Pattern Matching

Any pattern-matching system in NLP must contend with two antagonistic error types:

1. **False Positives (Type I errors):** Matching strings that should *not* have been matched. For example, the pattern `[tT]he` intended to find the word *the* will also match *other*, *theology*, and *there*.
2. **False Negatives (Type II errors):** Not matching strings that *should* have been matched. For example, the pattern `the` (lowercase only) misses the capitalized *The* at the start of a sentence.

Reducing the error rate involves two antagonistic efforts: **increasing precision** (minimizing false positives) and **increasing recall** (minimizing false negatives). In NLP, we are always dealing with this trade-off.

3.1.4 Application: ELIZA Using Regular Expressions

ELIZA (Section 1) is a direct application of regular expression substitution. Its pattern-response rules are RE substitutions of the form `s/pattern/response/`:

```
s/. * I'M (depressed|sad) .*/I AM SORRY TO HEAR YOU ARE \1/
s/. * I AM (depressed|sad) .*/WHY DO YOU THINK YOU ARE \1/
s/. * all .*/IN WHAT WAY?/
s/. * always .*/CAN YOU THINK OF A SPECIFIC EXAMPLE?/
```

The captured group `\1` allows the response to echo back the user's own words, creating the illusion of comprehension.

3.2 Formal Languages and Models

In the context of computational linguistics, a language is treated as a mathematical object that can be formally defined, recognized, and generated.

Definition 3.2 (Formal Language). A **Language** is a (possibly infinite) set of strings composed of symbols from a finite alphabet Σ .

Definition 3.3 (Model of a Language). A **Model** is a computational mechanism (such as an automaton or a grammar) that can recognize and generate all—and only—the strings belonging to a specific formal language. In this sense, the model serves as the literal definition of the language.

Notation and Preliminaries To define these sets precisely, we use the following standard notation:

- **Alphabet Σ :** A finite set of symbols. E.g., $\Sigma = \{0, 1\}$ (binary) or $\Sigma = \{a, b, c, d\}$ (four-letter alphabet).
- **String:** A finite sequence of symbols chosen from Σ . E.g., over $\Sigma = \{a, b\}$, the sequence aba is a valid string of length 3.
- **Empty String ϵ :** A string of length zero containing no symbols. E.g., $|\epsilon| = 0$; concatenating it with any string w gives $\epsilon w = w$.
- **Kleene Star Σ^* :** The set of all possible strings over Σ , including ϵ .
- **Concatenation:** Symbols or strings can be combined sequentially. E.g., $\Sigma \cup \Sigma\Sigma$ represents all strings of length exactly 1 or 2; over $\Sigma = \{a, b\}$ this yields $\{a, b, aa, ab, ba, bb\}$.

Remark 3.2 (Contextual Ambiguity). *Formal notation is often slightly imprecise: for instance, the symbol 0 may refer to a single character in an alphabet or a string of length one, depending on the context of the operation.*

3.3 Formal Language Definition

A formal language is not merely a collection of words, but a mathematically defined subset of all possible combinations of a given alphabet.

Definition 3.4 (Language over an Alphabet). *For a given finite alphabet Σ , a **Language** L is any subset of Σ^* , where Σ^* denotes the **Kleene star** of Σ —the infinite union of all finite-length string sets:*

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

Here Σ^k is the set of all strings of exactly length k (so $\Sigma^0 = \{\epsilon\}$). Thus Σ^* contains every string of any finite length that can be formed from Σ , including ϵ . The $*$ is not an arithmetic exponent—it is a dedicated operator meaning “zero or more concatenations.”

To define a specific language, we often apply constraints or rules to the symbols allowed in the sequence.

Example 3.8 (Constrained Binary Language). *Consider the alphabet $\Sigma = \{0, 1\}$. We define the language L as the set of all strings where there are **no two consecutive 1s**. The resulting set includes:*

$$L = \{\epsilon, 0, 1, 00, 01, 10, 000, 001, 010, 100, 101, 0000, 0001, 0010, 0100, 0101, 1000, 1001, 1010, \dots\}$$

3.4 The Chomsky Hierarchy

We want to formally define what counts as a “valid sentence” in some language — could be English, could be a programming language, could be an artificial toy language like “strings of a’s and b’s with no two consecutive 1s.” A formal language is just a set of strings (possibly infinite) built from some alphabet.

The question is: how complicated does our machinery need to be to describe that set? It turns out not all languages are equally hard to describe. Some can be checked with a very dumb, memoryless mechanism. Others need a mechanism with a memory stack. Others need even more power. The Chomsky Hierarchy ranks these by power, from weakest to strongest:

Definition 3.5 (The Chomsky Hierarchy). *A containment hierarchy of formal grammars consisting of:*

1. **Regular (Type 3):** Modeled by Finite State Automata and Regular Expressions.
2. **Context-Free (Type 2):** Modeled by Pushdown Automata.
3. **Context-Sensitive (Type 1):** Modeled by Linear Bounded Automata.
4. **Unrestricted (Type 0):** Modeled by Turing Machines.

Remark 3.3 (Notation in the Production Rules Column). *In the Production Rules column of Table 1: capital letters (A, B, C) denote individual **non-terminal (variable)** symbols; Greek letters (α, β, γ) denote arbitrary **strings** of zero or more variables and/or terminals; a lowercase letter a denotes a single **terminal** symbol.*

Type	Grammar Class	Language Class	Production Rules	Automaton
Type-3	Regular / Regex	Regular	$A \rightarrow a$ or $A \rightarrow aB$	Finite State Automaton
Type-2	Context-Free	Context-Free	$A \rightarrow \gamma$	Pushdown Automaton
Type-1	Context-Sensitive	Context-Sensitive	$\alpha A \beta \rightarrow \alpha \gamma \beta$	Linear Bounded
Type-0	Turing / Unrestricted	Recursively Enumerable	$\alpha \rightarrow \beta$	Turing Machine

Table 2: The Chomsky Hierarchy of Grammars

3.5 Type 3: Regular Languages

Regular languages are exactly what regular expressions (and finite state automata) can describe. The production rule is restricted to $A \rightarrow a$ or $A \rightarrow aB$: a non-terminal expands into a single terminal, or a terminal followed by at most one more non-terminal. There is no mechanism for recursive wrapping or counting.

Example 3.9 (Type-3, Regular). $L = \{a^n \mid n \geq 0\}$ - all strings of zero or more a's. Captured by the regex a^* and by the single rule $A \rightarrow aA \mid \epsilon$. No counting is required here: the language only asks for "any number of a's," not a comparison between two counts, so a finite automaton with no memory is sufficient.

3.6 Type 2: Context-Free Languages

Context-free grammars relax the rule to $A \rightarrow \gamma$, where A is a single non-terminal but γ can be *any* string of terminals and non-terminals, including ones that recursively wrap around A . Crucially, the rule fires regardless of context - there is no requirement to check neighboring symbols.

Definition 3.6 (Context-Free Grammar). A Context-Free Grammar is a formal notation for describing languages using a set of recursive production rules. It is defined by a 4-tuple (V, Σ, R, S) , where V is a set of non-terminal symbols, Σ is the alphabet of terminal symbols, R is the set of rules mapping non-terminals to sequences of symbols, and S is the start symbol.

Example 3.10 (Type-2, Context-Free). $L = \{a^n b^n \mid n \geq 1\}$ - balanced pairs like $ab, aabb, aaabbb$. Requires the CFG $S \rightarrow aSb \mid ab$; no finite automaton can enforce the equal-count constraint.

Example 3.11 (Recursive Derivation). Consider the analogous grammar over $\{0, 1\}$ with two productions:

- $S \rightarrow 01$ (base case: directly produces the shortest valid string)
- $S \rightarrow 0S1$ (recursive case: wraps the current S with one 0 on the left and one 1 on the right)

To derive the string 000111 we apply the recursive production twice and then terminate with the base case:

$$\begin{array}{ll}
S \Rightarrow 0S1 & \text{apply } S \rightarrow 0S1 \\
\Rightarrow 00S11 & \text{apply } S \rightarrow 0S1 \text{ to the inner } S \\
\Rightarrow 000111 & \text{apply } S \rightarrow 01 \text{ to the inner } S \text{ (base case)}
\end{array}$$

Each application of the recursive rule adds exactly one 0 and one 1, keeping both sides in balance. The language generated by this grammar is $L = \{0^n 1^n \mid n \geq 1\}$ - a classic example of a context-free language that is not regular (no finite automaton can verify that the counts of 0s and 1s are equal).

The underlying automaton, a pushdown automaton, can verify this via a single stack: push a symbol per 0, then pop one per 1, accepting if the stack empties exactly at the end. This single-stack mechanism can verify one count-equality relationship, but no more - which is exactly the ceiling that motivates Type-1.

3.7 Type 1: Context-Sensitive Languages

The rule shape becomes $\alpha A \beta \rightarrow \alpha \gamma \beta$: A may only be rewritten as γ when it appears in the specific context $\alpha \cdots \beta$, and that context is preserved unchanged on both sides. This is the opposite of context-free: the legality of the rewrite now depends on neighboring symbols.

Example 3.12 (Type-1, Context-Sensitive). $L = \{a^n b^n c^n \mid n \geq 1\}$ - three equal-count blocks like $abc, aabccc$. Cannot be generated by any CFG (two simultaneous counting dependencies exceed what a pushdown automaton can handle, since its single stack can only verify one count-equality relationship).

Context-sensitive grammars manage this by using context-conditioned rules to reorganize symbols generated in one order into the correctly grouped final string. For example, a rule such as $CA \rightarrow AC$ is legal only because C happens to be immediately followed by A ; repeated application of such context-dependent swaps lets the grammar coordinate three counts at once, something no context-free rule (which cannot inspect neighbors) is able to do.

3.8 Context-Free Languages and Parse Trees

Definition 3.7 (Context-Free Language (CFL)). *A language is considered Context-Free if it can be defined by at least one Context-Free Grammar.*

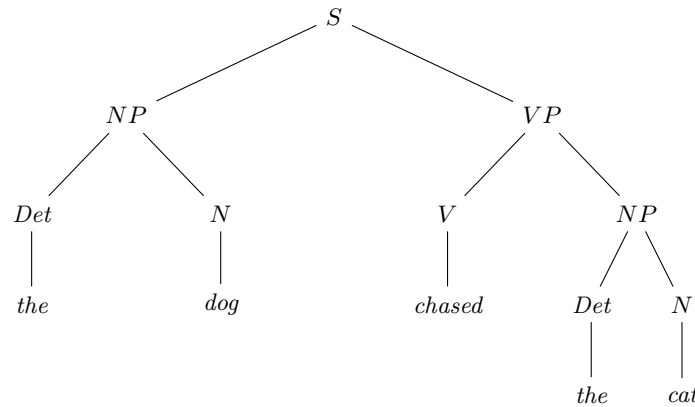
While all regular languages are context-free, the inverse is not true. CFLs possess greater expressive power, notably the ability to handle recursive nesting and "counting" dependencies between two elements (e.g., $0^n 1^n$). However, CFLs cannot model all languages; intuitively, they can track relationships between two items but fail when a third dependent item is introduced (e.g., $0^n 1^n 2^n$ is not context-free).

Definition 3.8 (Parse Tree). *A parse tree is a graphical representation of the derivation of a string in a CFG, organized by the following structural rules:*

1. **Root:** Must be labeled by the start symbol of the grammar.
2. **Interior Nodes:** Labeled by variables (non-terminals).
3. **Children:** Labeled by the symbols on the right side of a production rule applied to the parent node.
4. **Leaves:** Labeled by a terminal symbol or the empty string ϵ .

The "yield" of the parse tree—the sequence of leaves read from left to right—represents the final string in the language.

Example 3.13 (Parse Tree for "The dog chased the cat"). *Using the CFG rules $S \rightarrow NP VP$, $NP \rightarrow Det N$, $VP \rightarrow V NP$ and lexical rules $Det \rightarrow the$, $N \rightarrow dog \mid cat$, $V \rightarrow chased$, the parse tree for the sentence "The dog chased the cat" is:*



The root is labelled by the start symbol S ; interior nodes carry non-terminal labels (NP , VP , V , N , Det); leaves carry the actual words (terminals). Reading the leaves left-to-right recovers the original sentence.

3.9 Ambiguous Grammars

Definition 3.9 (Ambiguous Grammar). *A CFG is **ambiguous** if there exists a string in its language that is the yield of two or more distinct parse trees. Ambiguity is a property of the grammar, not the language—many languages have both ambiguous and unambiguous grammars.*

Example 3.14 (Balanced Parentheses with Ambiguity). *Consider: $S \rightarrow SS \mid (S) \mid ()$. The string $()()()$ can be derived by two different parse trees—corresponding to associating the three pairs left-to-right or right-to-left. This ambiguity can be resolved by rewriting the grammar as:*

$$\begin{aligned}
 B &\rightarrow (RB \mid \epsilon \\
 R &\rightarrow) \mid (R
 \end{aligned}$$

where B derives balanced strings and R derives strings with one extra right bracket. This unambiguous grammar generates the same language.

Definition 3.10 (Inherently Ambiguous Language). *A context-free language is **inherently ambiguous** if every CFG for it is ambiguous. An example is the language $\{0^i 1^j 2^k \mid i = j \text{ or } j = k\}$: strings of the form $0^n 1^n 2^n$ must be generated by two different parse trees (one counting 0s and 1s, the other counting 1s and 2s), so no unambiguous grammar exists.*

3.10 The CYK Algorithm

Definition 3.11 (CYK Algorithm). *The CYK algorithm is a bottom-up parsing algorithm used to determine whether a string w can be generated by a given Context-Free Grammar. It utilizes **dynamic programming** to build a table of all possible derivations for every substring of the input.*

For the algorithm to function, the grammar must be in **Chomsky Normal Form (CNF)**, where all production rules are restricted to the forms:

- $A \rightarrow BC$ (a variable goes to exactly two variables)
- $A \rightarrow a$ (a variable goes to exactly one terminal)

Algorithmic Logic Given a string of length n , the algorithm populates a triangular table T :

1. **Base Case:** Fill the bottom row by identifying which variables produce each individual terminal w_i .
2. **Recursive Step:** For substrings of increasing length, check all possible split points. If a substring can be split into two parts derived by B and C , and a rule $A \rightarrow BC$ exists, add A to the table for that substring.
3. **Termination:** The string w is in the language if the start symbol S appears in the top cell of the table, representing the span of the entire string.

Remark 3.4 (Computational Complexity). *The CYK algorithm has a time complexity of $O(n^3 \cdot |G|)$, where n is the length of the string and $|G|$ is the number of rules in the grammar. This makes it much more efficient than exhaustive search for deep or complex derivations.*

Example 3.15 (CYK Parse of “the dog chased the cat”). *Consider the following grammar already in CNF (all rules are either $A \rightarrow BC$ or $A \rightarrow a$):*

$$S \rightarrow NP VP, \quad VP \rightarrow V NP, \quad NP \rightarrow Det N$$

$$Det \rightarrow the, \quad N \rightarrow dog \mid cat, \quad V \rightarrow chased$$

Input string $w =$ the dog chased the cat (positions 1–5).

Step 1 — single words (length-1 substrings):

Position	1 (the)	2 (dog)	3 (chased)	4 (the)	5 (cat)
Variables	Det	N	V	Det	N

Step 2 — length-2 substrings (check all binary splits):

- $T[1, 2] =$ “the dog”: split $1|2 \Rightarrow Det \cdot N \rightarrow NP$. $T[1, 2] = \{NP\}$
- $T[2, 3] =$ “dog chased”: $N \cdot V$ — no rule. $T[2, 3] = \emptyset$
- $T[3, 4] =$ “chased the”: $V \cdot Det$ — no rule. $T[3, 4] = \emptyset$
- $T[4, 5] =$ “the cat”: $Det \cdot N \rightarrow NP$. $T[4, 5] = \{NP\}$

Step 3 — length-3 substrings:

- $T[1, 3] =$ “the dog chased”: splits give $NP \cdot V$ and $Det \cdot \emptyset$ — no rule. $T[1, 3] = \emptyset$
- $T[2, 4] =$ “dog chased the”: all splits yield $\emptyset$. $T[2, 4] = \emptyset$
- $T[3, 5] =$ “chased the cat”: split $3|4-5 \Rightarrow V \cdot NP \rightarrow VP$. $T[3, 5] = \{VP\}$

Step 4 — length-4 substrings:

- $T[1, 4], T[2, 5]$: all splits involve an $\emptyset$ cell. both $= \emptyset$

Step 5 — full string $T[1, 5] =$ “the dog chased the cat”:

- split $1-2|3-5$: $NP \cdot VP \rightarrow S$. ✓

Since $S \in T[1, 5]$, the string is accepted. The completed upper-triangular table is:

$i \backslash j$	1	2	3	4	5
1	Det	NP	$\emptyset$	$\emptyset$	S
2		N	$\emptyset$	$\emptyset$	$\emptyset$
3			V	$\emptyset$	VP
4				Det	NP
5					N

4 Sparse text representations

4.1 Distributional Semantics

Instead of defining a word using a traditional dictionary, the idea of **distributional semantics** is to define a word by looking at its "neighbors" (the company it keeps) across millions of sentences. Now we formally call this **embedding**.

- "You shall know a word by the company it keeps" — J. R. Firth (1957), *A synopsis of linguistic theory 1930–1955*.
- "The meaning of a word is its use in the language" — Ludwig Wittgenstein, *Philosophical Investigations* §43.

Distributional semantics provides a baseline for word similarity via two types of co-occurrence:

- **First-order co-occurrence** (syntagmatic association): words that typically appear near each other, e.g. *wrote, book, poem*.
- **Second-order co-occurrence** (paradigmatic association): words that share similar neighbors and are therefore interchangeable in many contexts, e.g. *wrote, said, remarked*.

The standard way to operationalize this in NLP is the **embedding** approach: represent each word as a vector so that syntactic and semantic similarity becomes distance and direction in vector space. There are two main kinds:

1. **Sparse** (e.g. tf-idf): a common baseline; sparse vectors where each dimension corresponds to a vocabulary word and the value is a simple function of co-occurrence counts.
2. **Dense** (e.g. word2vec): dense, low-dimensional vectors obtained by training a classifier to distinguish nearby from far-away words.

Definition 4.1 (Sparse embeddings). Let $|V|$ be the vocabulary size. The sparse representation of a document is a vector of dimension $|V|$, where entry i counts how many times the i -th vocabulary word appears in the document.

Example 4.1. Consider the four sentences: *An elephant is a mammal. Mammals are animals. Humans are mammals, too. Elephants and humans live in Africa.* After lemmatization and stop-word removal the bag-of-words count vector (over the 9 content words) is:

<i>Africa</i>	<i>animal</i>	<i>be</i>	<i>elephant</i>	<i>human</i>	<i>in</i>	<i>live</i>	<i>mammal</i>	<i>too</i>
1	1	3	2	2	1	1	3	1

This gives the 9-dimensional vector $(1, 1, 3, 2, 2, 1, 1, 3, 1)$. In practice the vector has dimension $|V|$ (on the order of 10,000+), making it a sparse vector — mostly zeros.

Both words and documents can be embedded in the **term-document matrix**:

Definition 4.2 (Term-document matrix). A term-document matrix M of dimension $|V| \times |D|$ has vocabulary words as rows and documents as columns; entry $M_{t,d}$ is the count of term t in document d .

Example 4.2 (Shakespeare corpus). A small slice of the Shakespeare term-document matrix:

	<i>As You Like It</i>	<i>Twelfth Night</i>	<i>Julius Caesar</i>	<i>Henry V</i>
<i>battle</i>	1	0	7	13
<i>good</i>	114	80	62	89
<i>fool</i>	36	58	1	4
<i>wit</i>	20	15	2	3

The two comedies (*As You Like It*, *Twelfth Night*) have similar vectors — many fools and wit, few battles — whereas the history plays (*Julius Caesar*, *Henry V*) cluster separately. This illustrates how the term-document matrix captures genre differences in vector space.

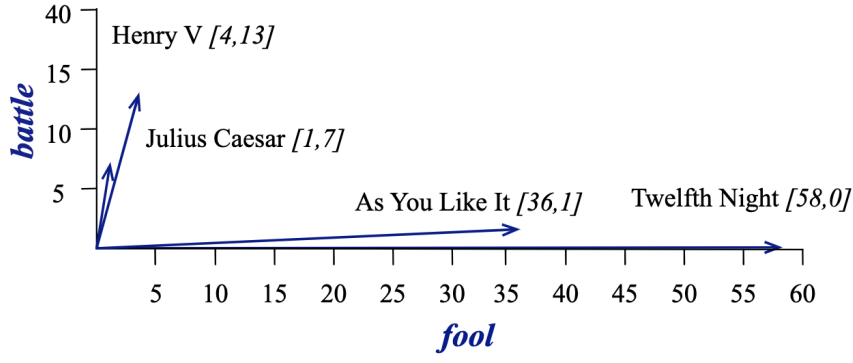


Figure 4: Vectorial plot of the document-query similarity space.

4.2 Text similarity

Text similarity can be defined at multiple levels:

- **Morphological**: surface variants of the same root (e.g. *respect*–*respectful*, Slovene *podoba*–*podobnost*).
- **Spelling**: variant spellings (e.g. *theater*–*theatre*).
- **Synonymy**: different words with the same meaning (e.g. *talkative*–*chatty*).
- **Homophony**: words that sound alike but differ in meaning or form.
- **Semantic**: words in the same semantic field (e.g. *cat*–*tabby*).
- **Sentence-level**: paraphrases.
- **Document-level**: two news articles covering the same event.
- **Cross-lingual**: translated documents or cognates across languages.

We want to identify how similar two strings are. For example, if a word is misspelled, which dictionary word is closest? If we want to retrieve information, which document best matches a query? Two complementary notions address this: **edit distance** for character-level comparison and **cosine similarity** for vector-space comparison.

Definition 4.3 (Edit distance). *The minimum edit distance between two strings is the minimum number of editing operations (insertion, deletion, substitution) needed to transform one into another.*

Example 4.3. *The edit distance between intention and execution is 5.*

Edit distance also appears in computational biology (sequence alignment of nucleotides), machine translation evaluation, and speech recognition.

Document similarity via cosine To compare document vectors we use **cosine similarity**, which measures the angle between two vectors rather than their absolute magnitudes (thus normalizing for document length):

$$\cos(\Theta) = \frac{A \cdot B}{|A| |B|} = \frac{\sum_{i=1}^N a_i b_i}{\sqrt{\sum_{i=1}^N a_i^2} \sqrt{\sum_{i=1}^N b_i^2}} \quad (1)$$

A value of +1 means the vectors point in the same direction (maximally similar); 0 means orthogonal (no shared terms); −1 means opposite. Since term frequencies are non-negative, cosine values for text vectors lie in $[0, 1]$.

Now we want to study how we can define **document similarity** more carefully. A naive approach would be to count raw word frequencies and compare them across documents. However, overly frequent words like *the* or *it* are not informative about the specific topic of a document. We need a weighting scheme that resolves this frequency paradox.

Weighting schemes Three weighting schemes of increasing sophistication are commonly used:

- **One-hot:** Binary presence/absence: 1 if the term appears in the document, 0 otherwise. Ignores term frequency entirely.
- **Term Frequency (TF):** Log-compressed count: $tf_{t,d} = 1 + \log_{10}(\text{count}(t, d))$ if the term is present, else 0. Accounts for how often a term appears but dampens the effect of very high counts.
- **TF-IDF:** Combines TF with the Inverse Document Frequency to downweight terms that appear in many documents (e.g. *the*, *is*), making the weight specific to the document-term pair.

Definition 4.4 (TF-IDF retrieval). *We define the weight of term t in document d as:*

$$w_{t,d} = tf_{t,d} \times idf_t \quad (2)$$

where:

$$tf_{t,d} = \begin{cases} 1 + \log_{10}(\text{count}(t, d)) & \text{if } \text{count}(t, d) > 0 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

and

$$idf_t = \log_{10} \left(\frac{|D|}{df_t} \right) \quad (4)$$

Where $|D|$ is the total number of documents in the collection and df_t is the number of documents that contain term t .

Note the goal of this definition is pricing words that appear a lot in a document and that rarely appear among all documents. In such case, we power spike the words that are important or mainly define a document. From this, we can define the relation between a query and a document as:

$$\text{sim}(q, d) = \frac{\sum_t w_{t,d} \cdot w_{t,q}}{\sqrt{\sum_t w_{t,d}^2} \sqrt{\sum_t w_{t,q}^2}} \quad (5)$$

The previous definition ranks the similarity between a query and a document.

TF-IDF in Practice TF-IDF remains a strong and widely used baseline. It is fast to compute, requires no training data, and is still competitive for tasks such as keyword search, document ranking, and feature extraction for classical classifiers. Its main weaknesses are that it is entirely **bag-of-words**: word order is ignored, and two words with the same meaning (e.g. *car* and *automobile*) get no credit for being synonymous because they occupy different dimensions in the vector. It also produces very high-dimensional sparse vectors, which scale poorly to large vocabularies. In modern NLP, TF-IDF has largely been superseded by dense neural embeddings (Word2Vec, BERT, sentence transformers) for tasks requiring semantic understanding, but it is still routinely used as a fast baseline, in hybrid retrieval systems, and wherever interpretability and low computational cost matter.

4.3 Metrics for evaluation: Precision and Recall

We present some performance measures for information retrieval. We first present the standard definition for precision, recall and accuracy and then generalize for natural language processing.

Definition 4.5 (Precision, Recall and Accuracy). *The precision is defined as:*

$$P = \frac{TP}{TP + FP}$$

Recall is defined as:

$$R = \frac{TP}{TP + FN}$$

And accuracy is defined as:

$$A = \frac{TP + TN}{TP + FP + FN + TN}$$

We note that the accuracy is similar to the F_1 score.

For document retrieval a parallel definition is used:

Definition 4.6 (Precision and Recall for Document Retrieval). *Let N be the total number of documents in the collection, n the number of documents relevant to a given query, and m the number of documents returned by the system. Suppose the system returns m documents of which a are relevant. Then:*

$$P = \frac{a}{m}$$

*is the **precision**: the proportion of retrieved documents that are relevant. And:*

$$R = \frac{a}{n}$$

*is the **recall**: the proportion of all relevant documents that were successfully retrieved.*

We then define the score-measurement in terms of the precision and recall as:

Definition 4.7. *We define the F -measure in terms of precision P and recall R as*

$$F_\beta = (1 + \beta^2) \frac{P \cdot R}{\beta^2 \cdot P + R} \quad 0 < \beta < \infty$$

If $\beta = 1$ we obtain the F_1 score, which means equal weight for precision and recall. If $\beta < 1$ then we emphasize precision, whereas if $\beta > 1$ we emphasize recall.

Note that for large numbers, the precision may no longer be a relevant measure, because the quantity of relevant documents is somehow bounded. Then it is also common to define the precision, recall and F_1 -score for the first k returned documents.

Definition 4.8 (Precision, Recall and F_1 at rank k). *Precision and recall at rank k evaluate quality considering only the top k results, rather than all documents. Let a_k be the number of relevant documents among the first k retrieved. Then:*

$$P@k = \frac{a_k}{k}, \quad R@k = \frac{a_k}{n}, \quad F_1@k = 2 \cdot \frac{P@k \cdot R@k}{P@k + R@k}$$

Here, n continues to be all the relevant document for that query. This is a well known limitation because if there is $n = 1000$ relevant documents in the collection and you impose $k = 10$ the maximum achievable recall is $10/1000 = 1\%$. These rank-aware metrics are especially useful for web search, where users typically inspect only the first page of results.

The β parameter of the F_β score allows tuning the trade-off between precision and recall depending on the application:

- $\beta < 1$ (e.g. $F_{0.5}$): Precision receives more weight than recall. Useful when false positives are costly, e.g. spam detection.
- $\beta = 1$ (F_1): Equal weight; the most common choice.
- $\beta > 1$ (e.g. F_2): Recall receives more weight. Useful when missing a relevant item is costly, e.g. disease screening.

Remark 4.1 (Precision–Recall trade-off). *Increasing recall generally decreases precision and vice versa: retrieving more documents raises the chance of capturing all relevant ones but also brings in more irrelevant ones. This inverse relationship is visualized with a **precision–recall curve**; the area under this curve (AUC-PR) summarizes overall retrieval quality.*

As we can see, when we decrease the precision, the recall then to increase and viceversa. For example, we can achieve a recall of 1 simply classifying all document as rellevant or positive, but then we achieve a disastrous precision.

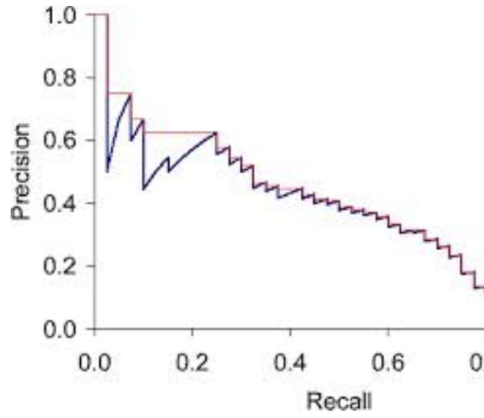


Figure 5: Precision-recall curve showing the inverse relationship between precision and recall.

5 N-gram models

5.1 Language Models

A **language model** (LM) is a system that can predict upcoming words. Concretely, it can:

- assign a probability to each potential next word given the preceding context;
- assign a probability to a whole sentence.

Why word prediction? Word prediction is a useful component of many language tasks:

- **Grammar and spell checking:** a language model can distinguish *Their are two midterms* (wrong) from *There are two midterms* (correct) and *Everything has improve* from *Everything has improved*.
- **Speech recognition:** *I will be back soonish* vs. *I will be bassoon dish* — the language model prefers the first because it assigns it higher probability.

More formally, the goal is to assign a probability to a sentence or a sequence of words. This matters for:

- **Machine Translation:** $P(\text{high winds tonight}) > P(\text{large winds tonight})$.
- **Spell Correction:** $P(\text{about fifteen minutes from}) > P(\text{about fifteen minuets from})$.
- **Speech Recognition:** $P(\text{I saw a van}) \gg P(\text{eyes awe of an})$.
- Summarization, question-answering, etc.

Hence, the goal is to compute the probability of a sentence or sequence of words:

$$P(W) = P(w_1, w_2, \dots, w_n) \quad (6)$$

A related task is computing the probability of an upcoming word:

$$P(w_5 \mid w_1, w_2, w_3, w_4) \quad (7)$$

Definition 5.1 (Language Model). *A model that computes either $P(W)$ or $P(w_n \mid w_1, w_2, \dots, w_{n-1})$ is called a **language model** (LM). An alternative name sometimes used is **grammar model**, though language model is standard.*

The Chain Rule of Probability To compute joint probabilities, we rely on the chain rule:

$$P(x_1, x_2, \dots, x_n) = P(x_1) P(x_2 \mid x_1) P(x_3 \mid x_1, x_2) \cdots P(x_n \mid x_1, \dots, x_{n-1}) = \prod_{i=1}^n P(x_i \mid x_1 \dots x_{i-1}) \quad (8)$$

Example 5.1.

$$\begin{aligned} P(\text{its water is so transparent}) &= P(\text{its}) \times P(\text{water} \mid \text{its}) \\ &\quad \times P(\text{is} \mid \text{its water}) \times P(\text{so} \mid \text{its water is}) \\ &\quad \times P(\text{transparent} \mid \text{its water is so}) \end{aligned}$$

Could we simply estimate these probabilities by counting? For instance:

$$P(\text{the} \mid \text{its water is so transparent that}) = \frac{\text{Count}(\text{its water is so transparent that the})}{\text{Count}(\text{its water is so transparent that})}$$

The answer is no: there are too many possible sentences, and we will never observe enough data to estimate all these conditional probabilities. We need an approximation.

5.2 Markov assumption

If we don't use a neural network, we might need to store the values of $P(x_1, \dots, x_n)$. In such case, if our memory storage is short, we use the Markov assumption:

Definition 5.2. We define the Markov assumption of a probability distribution P over a set of words or tokens $x_1, \dots, x_n$ as:

$$P(x_n \mid x_1, \dots, x_{n-1}) \approx P(x_n \mid x_{n-1}) \quad (9)$$

Similarly, we use the second order assumption:

$$P(w_n \mid w_{n-2}, w_{n-1}, \dots, w_1) \approx P(w_n \mid w_{n-2}, w_{n-1}) \quad (10)$$

From this it falls that, using a Markov assumption of order k we obtain that:

$$P(w_1, w_2, \dots, w_n) \approx \prod_{i=1}^n P(w_i \mid w_{i-k}, \dots, w_{i-1}) \quad (11)$$

Namely, approximating $P(w_i \mid w_1, w_2, \dots, w_{i-1}) \approx P(w_i \mid w_{i-k}, \dots, w_{i-1})$.

The models obtained from the Markov assumption are called:

Definition 5.3. From the zero-th Markov assumption we obtain the unigram model:

$$P(w_1, w_2, \dots, w_n) \approx \prod_{i=1}^n P(w_i) \quad (12)$$

From the first Markov assumption we obtain the bigram model:

$$P(w_1, w_2, \dots, w_n) \approx \prod_{i=1}^n P(w_i \mid w_{i-1}) \quad (13)$$

The way to estimate the probabilities in bigram models is by Maximum Likelihood Estimation (MLE) from a training corpus—simply count and divide:

$$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})} \quad (14)$$

Example 5.2 (Bigram probabilities on a tiny corpus). Consider the training corpus (with sentence markers $\langle s \rangle$ and $\langle /s \rangle$):

$\langle s \rangle$ I am Sam $\langle /s \rangle$

$\langle s \rangle$ Sam I am $\langle /s \rangle$

$\langle s \rangle$ I do not like green eggs and ham $\langle /s \rangle$

Some bigram probabilities computed by MLE:

$$\begin{aligned} P(I \mid \langle s \rangle) &= \frac{2}{3} = 0.67 & P(\text{Sam} \mid \langle s \rangle) &= \frac{1}{3} = 0.33 & P(\text{am} \mid I) &= \frac{2}{3} = 0.67 \\ P(\langle /s \rangle \mid \text{Sam}) &= \frac{1}{2} = 0.5 & P(\text{Sam} \mid \text{am}) &= \frac{1}{2} = 0.5 & P(\text{do} \mid I) &= \frac{1}{3} = 0.33 \end{aligned}$$

On larger corpora, bigram counts are stored as a sparse matrix. For example, from 9,222 restaurant-query sentences:

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

The many zeros highlight the **sparsity problem**: most bigrams are never observed in training, yet may appear at test time.

Normalized bigram probabilities Dividing each row by the corresponding unigram count $c(w_{i-1})$ yields the **MLE** (Maximum Likelihood Estimation) probability estimates. The name comes from the fact that these are the probability values that maximise the likelihood of observing the training corpus under the bigram model—intuitively, the most “faithful” assignment of probabilities given the data we have seen. For a bigram, this estimate is simply the relative frequency: $\hat{P}(w_i | w_{i-1}) = c(w_{i-1}, w_i) / c(w_{i-1})$, i.e. out of all times w_{i-1} appeared, how often was it followed by w_i . For this corpus the unigram counts are approximately $c(i) = 2,533$, $c(want) = 927$, $c(to) = 2,417$, $c(eat) = 746$, $c(chinese) = 158$, $c(food) = 1,093$, $c(lunch) = 341$, $c(spend) = 278$.

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.326	0	0.004	0	0	0	0.001
want	0.002	0	0.656	0.001	0.006	0.006	0.005	0.001
to	0.001	0	0.002	0.284	0.001	0	0.002	0.087
eat	0	0	0.003	0	0.021	0.003	0.056	0
chinese	0.006	0	0	0	0	0.519	0.006	0
food	0.014	0	0.014	0	0.001	0.004	0	0
lunch	0.006	0	0	0	0	0.003	0	0
spend	0.004	0	0.004	0	0	0	0	0

Example 5.3 (Sentence probability using bigrams). *Using these estimates, the probability of the sentence “ $\langle s \rangle$ I want English food $\langle /s \rangle$ ” is:*

$$\begin{aligned}
 P(\langle s \rangle \text{ I want English food } \langle /s \rangle) &= P(I | \langle s \rangle) \times P(\text{want} | I) \times P(\text{English} | \text{want}) \\
 &\quad \times P(\text{food} | \text{English}) \times P(\langle /s \rangle | \text{food}) \\
 &\approx 0.25 \times 0.33 \times 0.0011 \times 0.50 \times 0.68 \\
 &\approx 3.1 \times 10^{-5}
 \end{aligned}$$

The product of many small probabilities quickly approaches zero, motivating storage and computation in *log space* (see below).

5.3 Log probabilities

Language model probabilities involve products of many small numbers, which easily underflow to zero in floating-point arithmetic. In practice, all computations are performed in **log space**:

$$\log P(w_1, w_2, \dots, w_n) = \sum_{i=1}^n \log P(w_i | w_{i-1}) \quad (15)$$

Multiplications become additions, eliminating underflow. The original probability can be recovered by exponentiation:

$$P(w_1, \dots, w_n) = \exp\left(\sum_{i=1}^n \log P(w_i | w_{i-1})\right) \quad (16)$$

This procedure extends directly to trigrams, 4-grams, and 5-grams. However, N-gram models have a fundamental limitation: language has **long-distance dependencies**. For example, in “*The computer(s) which I had just put into the machine room on the fifth floor is (are) crashing*”, the verb must agree with the subject despite many intervening words. N-gram models also work better in English than in Slovene and other morphologically rich languages, because English has far larger available corpora and simpler word-form structure.

5.4 Intuition of Perplexity

The motivation of perplexity comes from Shannon game. For example ‘How well can we predict the next word?’

Definition 5.4. *The perplexity of a **sentence** is the inverse probability of the test set, normalized by the number of words:*

$$PP(W) = P(w_1, w_2, \dots, w_n)^{-1/n} = \sqrt[n]{\frac{1}{P(w_1, w_2, \dots, w_n)}} \quad (17)$$

If we use the chain rule we would obtain:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i|w_1, \dots, w_{i-1})}} \quad (18)$$

For the case of bigram models then we have:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i|w_{i-1})}} \quad (19)$$

Note that the perplexity is motivated by the geometric average. Minimizing perplexity is the same as maximizing the probability.

Example 5.4. *Let us suppose we have a sentence consisting of random digits. What is the perplexity of this sentence according to a model that assigns a probability of 1/10 to each digit?*

$$PP(W) = (1/10)^{-1} = 10 \quad (20)$$

In general the lower the perplexity, the better the model is, because it means the model is more robust, it doesn't select guesses randomly. Hence, generally unigram models, contain sentences with more perplexity than a bigram model.

The **Shannon Visualization method** consists in making sentences by choosing a random bigram ($\langle s \rangle, w$) according to the probability, and iterating recursively on the next word, i.e. choosing (w, x) until we reach $\langle /s \rangle$.

Generated text by N-gram order The coherence of generated text improves markedly with n :

- **Unigram:** “fifth, an, of, futures, the, an, incorporated, a, a, the, inflation, most, dollars, quarter, in, is, mass” — no coherence.
- **Bigram:** “the Nasdaq composite index rose 4.31 percent in this trading” — locally fluent phrases, no global structure.
- **Trigram:** “Microsoft was at a loss in trading, said Mike Lazaridis, president of the Federal” — longer coherent phrases, occasional incoherence.
- **4-gram:** “They also point to ninety nine point six billion dollars from two hundred” — approaches natural-sounding language.

The problem with N-gram models is that they strongly reduce the perplexity, in trade-off with overfitting to the training text. For example, a 4-gram model trained on Shakespeare text is basically reproducing sentences from Shakespeare. Our goal is to **generalize**, and for doing this, we use neural language models.

5.5 Evaluating N-gram Models

5.5.1 Intrinsic vs. Extrinsic Evaluation

There are two complementary paradigms for evaluating language models:

- **Intrinsic evaluation:** Measures the model in isolation, independently of any application, using a metric that is internal to the model itself—typically **perplexity** on a held-out text. It is fast, cheap, and convenient during development. The drawback is that it is not guaranteed to correlate with real-world utility: a model with lower perplexity does not always produce better translations or fewer spelling errors.
- **Extrinsic evaluation** (also called *in-vivo* or *task-based* evaluation): Plugs the language model into a complete downstream application and measures performance on that application’s own metric (e.g. word error rate for speech recognition, BLEU for machine translation (see Definition 17.7), accuracy for spell-checking). This is the gold standard—it tells you whether the model actually helps—but it is expensive because changing the LM requires re-running the whole pipeline.

Example 5.5 (Intrinsic vs. Extrinsic for a Spell Checker). *Suppose we train two trigram language models, A and B, on different corpora. Intrinsically, model A achieves a perplexity of 120 on held-out news text while model B achieves 105—B looks better. However, when both models are embedded in a spell-checking application and tested on real user corrections, model A corrects 87% of errors correctly while model B corrects only 83%. The extrinsic result reverses the ranking, illustrating that intrinsic metrics are a useful but imperfect proxy.*

5.5.2 Training, Development, and Test Sets

N-gram models must be evaluated on **held-out data** — text never seen during training. The standard partition is:

- **Training set:** Used to estimate all parameters (n -gram counts and probabilities).
- **Development set (devset):** Used repeatedly during model selection and hyperparameter tuning. Allows multiple evaluation cycles without contaminating the final report.
- **Test set:** Used *once* to report final results. If the model is tested repeatedly on the same test set, reported perplexity will be artificially low—a form of data leakage. This constitutes bad scientific practice.

Perplexity comparison across model orders On a held-out sample from the Wall Street Journal (38M training words, 1.5M test words), standard n -gram models achieve the following perplexities:

Model	Perplexity
Unigram	962
Bigram	170
Trigram	109

Each additional order significantly reduces perplexity, but returns diminish quickly and higher-order models face severe data sparsity.

5.6 Large N-gram Corpora and Tools

Corpora

- **COCA** (Corpus of Contemporary American English): approximately 1 billion words.
- **Google Web 5-grams** (Franz & Brants, 2006): 1 trillion words; pre-computed 5-gram counts released publicly. Probabilities can be quantized to 4–8 bits instead of 8-byte floats to drastically reduce memory footprint.
- **Infini-gram** (Liu et al., 2024): uses suffix arrays to compute n -gram probabilities for *any* n on-the-fly, without pre-computing all orders, enabling flexible large-scale lookup.

Toolkits

- **SRILM:** A comprehensive toolkit for N-gram language model training and evaluation.
- **KenLM:** A highly efficient toolkit for large-scale N-gram estimation and querying, with memory-mapped storage and fast lookup.

6 Revision of neural networks

This section provides a brief revision of the fundamental concepts underlying modern neural networks, serving as the technical foundation for the neural embedding methods that follow. The recommended reading is Chapter 6 of Jurafsky & Martin (3rd edition), as well as Simon J.D. Prince’s *Understanding Deep Learning* (MIT Press, 2023) and Ian Goodfellow, Yoshua Bengio, and Aaron Courville’s *Deep Learning* (MIT Press, 2016).

6.1 Artificial Neural Networks

Definition 6.1 (Artificial Neural Network (ANN)). *An Artificial Neural Network (ANN) is a computational model inspired by the structure of the brain. It acts as a **universal function approximator**: given enough hidden units and training data, it can closely approximate any continuous function. The intuition is that neurons in successive layers encode increasingly abstract and useful features of the input.*

The biological analogy motivates the architecture: in the brain, a neuron receives electrical signals via **dendrites**, integrates them in the **soma** (cell body), and transmits a signal along the **axon** to **axon terminals** that synapse onto other neurons. There are more than a hundred distinct types of neurons in the human brain. The artificial model simplifies this drastically—a neuron cell is far more complex than a weighted sum, and so are its interconnections.

6.2 The Artificial Neuron and the Perceptron

Definition 6.2 (Artificial Neuron). *An artificial neuron takes inputs $x_1, x_2, \dots, x_n$ (plus a bias $x_0 = 1$) with associated weights $w_0, w_1, \dots, w_n$. It computes the weighted sum:*

$$net = \sum_{i=0}^n w_i x_i = \mathbf{w} \cdot \mathbf{x}$$

and passes the result through an **activation function** g to produce an output $y = g(net)$.

The simplest network unit is the **Perceptron**, which uses a step (threshold) activation function:

$$\text{output} = \begin{cases} 1 & \text{if } \sum_{i=0}^n w_i x_i > 0 \\ -1 & \text{otherwise} \end{cases}$$

6.3 Activation Functions

A key question is why nonlinear activation functions are necessary. The reason is that the product of two linear transformations is itself a linear transformation—stacking multiple linear layers collapses into a single linear mapping regardless of depth. Nonlinearity is what gives deep networks their expressive power.

Common activation functions include:

- **Step function:** $g(x) = \mathbf{1}[x > 0]$. Discontinuous; used in the original perceptron.
- **Sigmoid (logistic):** $\sigma(x) = \frac{1}{1 + e^{-x}}$. Smooth, bounded in $(0, 1)$; classically used but susceptible to vanishing gradients in deep networks.
- **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$. The default activation in modern deep networks; computationally cheap and avoids vanishing gradients for positive inputs.
- **Softplus:** $f(x) = \ln(1 + e^x)$. A smooth approximation of ReLU with a continuous derivative everywhere.

6.4 Feed-Forward Neural Networks

Definition 6.3 (Feed-Forward Neural Network (FFNN)). *A multi-layer feed-forward neural network organizes neurons into an **input layer**, one or more **hidden layers**, and an **output layer**. Values are propagated from the input through the network until the output layer returns the prediction. The network is **feed-forward** if no weights cycle back to a previous layer; networks with backward connections are called **recurrent neural networks (RNNs)**.*

More precisely:

- The inputs correspond to the attributes measured for each training tuple and are fed simultaneously into the input layer.

- They are weighted and fed to a hidden layer.
- If more than one hidden layer is used, the network is called a **deep neural network**.
- The weighted outputs of the last hidden layer feed into the output layer, which emits the prediction.
- From a statistical perspective, networks perform **nonlinear regression**: given enough hidden units and training samples, they can closely approximate any continuous function.

Definition 6.4 (Deep Learning). *Deep learning refers to the use of deep neural networks trained on large datasets with GPU acceleration, combined with many algorithmic innovations. It enables the learning of **hierarchical representations**: successive layers extract features from low-level (edges, character n-grams) to mid-level to high-level abstractions (objects, semantic concepts).*

6.5 Gradient Descent

Training a neural network requires minimizing a loss function over its parameters (weights). This is achieved by **gradient descent**.

Definition 6.5 (Gradient Descent (GD)). *Gradient descent is an iterative local optimization algorithm in $\mathbb{R}^n$. Given a differentiable function $f: \mathbb{R}^n \rightarrow \mathbb{R}$, the gradient is:*

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$$

The algorithm moves iteratively in the direction of $-\nabla f(\mathbf{x})$ (steepest descent) with step size η (learning rate):

$$\mathbf{x}^{t+1} = \mathbf{x}^t - \eta \cdot \nabla f(\mathbf{x}^t)$$

A local minimum is a point $\mathbf{x}$ for which $f(\mathbf{x}) \leq f(\mathbf{x}')$ for all nearby $\mathbf{x}'$.

6.6 Backpropagation

Definition 6.6 (Error Backpropagation). *Backpropagation is the standard neural network learning algorithm—originally developed by psychologists and neurobiologists to model computational analogues of neurons. During training, the network learns by adjusting weights to minimize prediction error, working in the **backwards** direction from the output layer through each hidden layer down to the first hidden layer. It is also referred to as **connectionist learning** due to the learned connections between units.*

In a network with weight matrices $\vec{\theta} = \{w_{ij}, w_{jk}, w_{kl}\}$, each node j has a pre-activation $a_j = \sum_i w_{ij} z_i$ and a post-activation $z_j = g(a_j)$. The **empirical risk** over N training examples is:

$$R(\theta) = \frac{1}{N} \sum_{n=0}^N \frac{1}{2} (y_n - f(x_n))^2$$

Computing gradients of this nested expression relies on the **chain rule of derivation**:

$$f(g(x))' = f'(g(x)) g'(x)$$

In a network, the output of each neuron is a function of its activation and all its inputs, which are themselves functions of neurons in previous layers. Backpropagation unravels the multivariate chain rule and solves the gradient for each partial component separately; target values for each layer come from the next layer, feeding errors back along the network.

The complete gradient for each weight layer is:

$$\frac{\partial R}{\partial w_{kl}} = \frac{1}{N} \sum_n \delta_{l,n} z_{k,n}$$

$$\frac{\partial R}{\partial w_{jk}} = \frac{1}{N} \sum_n \delta_{k,n} z_{j,n}$$

$$\frac{\partial R}{\partial w_{ij}} = \frac{1}{N} \sum_n \delta_{j,n} z_{i,n}$$

where $\delta_{l,n} = -(y_n - z_{l,n}) g'(a_{l,n})$ is the error signal from the output layer, propagated inward by the multivariate chain rule.

Once gradients are computed, all weights are updated simultaneously:

$$w^{t+1} = w^t - \eta \frac{\partial R}{\partial w}$$

Backpropagation algorithm steps:

1. Initialize weights to small random numbers (associated with biases).
2. Propagate inputs forward (applying activation functions layer by layer).
3. Backpropagate the error (update weights and biases from output toward input).
4. Repeat until a termination condition is met (e.g., error falls below a threshold).

Weight updates can be performed in different modes: **batch** (full dataset per update), **stochastic** (one example at a time), or **mini-batch** (small random subsets).

6.7 Softmax and Cross-Entropy Loss

For multi-class classification, the output layer typically applies the **softmax** function, which normalizes raw output scores into a valid probability distribution:

$$y_i = \frac{e^{z_i}}{\sum_{j \in \text{group}} e^{z_j}}, \quad \frac{\partial y_i}{\partial z_i} = y_i(1 - y_i)$$

Together with softmax, the standard loss function is **cross-entropy**:

$$C = - \sum_j t_j \log y_j, \quad \frac{\partial C}{\partial z_i} = y_i - t_i$$

where t_j is the target (one-hot encoded) value. The elegant form of the gradient $y_i - t_i$ makes this combination computationally attractive for classification tasks.

6.8 Network Topology and Practical Considerations

Designing a network requires specifying the number of units in the input layer, the number of hidden layers, the number of units in each hidden layer, and the number of output units. Inputs are normalized to $[0.0, 1.0]$; for classification with more than two classes, one output unit per class is used. If a trained network remains inaccurate, one may retry with a different topology or different random initial weights.

Strengths and weaknesses of neural networks as classifiers:

- **Weaknesses:** Long training time; many hyperparameters that must be determined empirically (e.g., network topology); poor interpretability—it is difficult to extract the symbolic meaning of learned weights or hidden units, causing networks to be treated as **black boxes**.
- **Strengths:** High tolerance to noisy data; ability to classify untrained patterns; well-suited for continuous-valued inputs and outputs; successful on diverse real-world data (e.g., handwritten letters); inherently parallelizable.

The efficiency of backpropagation is $O(|D| \cdot w)$ per epoch (one full pass through the training set), where $|D|$ is the number of training tuples and w is the number of weights. The number of epochs needed can be large, in the worst case exponential in the number of inputs.

6.9 Overfitting and Regularization

Neural networks are especially prone to **overfitting** due to their large number of parameters: the model fits the training data too closely and fails to generalize to unseen examples. Common approaches to prevent overfitting include:

- **Weight decay:** Penalize large weights via L2 regularization.
- **Weight sharing:** Reuse weights across the network (as in CNNs).
- **Early stopping:** Halt training when validation error begins to increase.
- **Dropout:** Randomly set a fraction of activations to zero during each training step.
- **Model averaging / Bayesian fitting:** Combine predictions from multiple models.
- **Generative pre-training:** Pre-train layers in an unsupervised manner before fine-tuning.

6.10 Successes and Weaknesses of Deep Learning

Deep learning has achieved remarkable results, including super-human performance on image recognition (ImageNet ILSVRC), medical image analysis, autonomous driving, robotics, game playing (AlphaGo defeating world champion Lee Sedol), and speech recognition (Microsoft reporting a 5.1% error rate below human-level).

However, deep networks exhibit important weaknesses:

- **Adversarial examples:** Imperceptible perturbations to inputs cause dramatic misclassification. Images that appear as random noise or abstract textures to humans are confidently classified as specific objects by a neural network. This is a fundamental vulnerability, not a corner case.
- **Failures on out-of-distribution inputs:** Unusual viewpoints or poses of familiar objects (e.g., a school bus photographed from an unusual angle) cause high-confidence misclassifications (Alcorn et al., 2018, arXiv:1811.11553).
- **Physical adversarial attacks:** Adversarial patches placed in a scene can fool deployed classifiers into misidentifying objects entirely, posing safety risks in real-world systems.

7 Static neural embeddings

The goal of neural embeddings is to represent words as dense vectors so that semantically similar words occupy nearby regions of a continuous vector space. This allows a system to relate sentences that are written in completely different ways but mean the same thing, such as *She went for a walk* and *She was strolling along a path*.

Why *static*? The embeddings studied in this chapter are called **static** because each word receives a **single fixed vector** regardless of the context in which it appears. This has two key consequences:

- **Polysemy is ignored:** A word like *bank* (financial institution vs. river bank) gets one vector that mixes all its meanings.
- **Context-sensitivity is absent:** The meaning of a word can shift slightly depending on its neighbors, but static embeddings cannot capture this.

These limitations are addressed by **contextual embeddings** (ELMo, BERT), which will be covered in later chapters.

7.1 Word2vec

Definition 7.1 (Word2vec (Mikolov et al., 2013)). *Word2vec is a neural embedding method that trains a shallow network with three layers:*

- **Input layer** ($1 \times |V|$): a one-hot vector for the target word w_t .
- **Projection layer** ($1 \times d$): a dense embedding obtained by multiplying the input by the target embedding matrix W of size $|V| \times d$. Since the input is one-hot, this is simply a row lookup in W .
- **Output layer** ($1 \times |V|$): obtained by multiplying the projection by the context embedding matrix C of size $d \times |V|$; after a softmax this gives the probabilities of each vocabulary word being a context word of w_t .

After training, the rows of W are used as the final word embeddings.

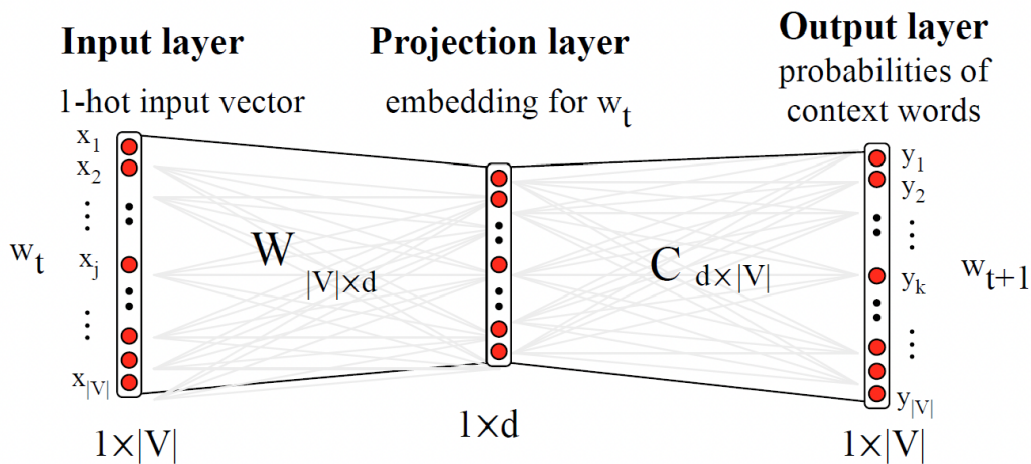


Figure 6: Architecture of a Word2vec model.

Key idea. Instead of *counting* how often word w occurs near a target word (as TF-IDF does), Word2vec trains a binary classifier on the prediction task: *is w likely to appear near this target?* The task itself is not the end goal—what matters is that the **learned classifier weights become the word embeddings**. Words near the target act as positive (correct) answers; randomly sampled noise words act as negative answers. Crucially, no hand-labelled supervision is needed: the corpus itself generates the training signal. This idea generalises widely and is the origin of the “something-to-vec” naming convention.

Word2vec comes in two variants:

- **Skip-gram:** given a target word, predict its context words (position-independent).
- **CBOW** (Continuous Bag of Words): given the context words, predict the target word.

7.1.1 Skip-gram with Negative Sampling

Training data. For each word in the corpus, we slide a window of $\pm L$ words around it (in practice $L \approx 5-10$). The target word t and each word within the window form a **positive pair**. For each positive pair we also sample k random **noise words** from the vocabulary to form negative pairs. For example, given the sentence fragment:

... lemon, a tablespoon of apricot jam a pinch ...
 $c_1 \qquad c_2 \qquad t \qquad c_3 \qquad c_4$

with target $t = \text{apricot}$ and window size $L = 2$, the positive pairs are $(\text{apricot}, \text{tablespoon})$, $(\text{apricot}, \text{of})$, $(\text{apricot}, \text{jam})$, $(\text{apricot}, \text{a})$. With $k = 2$ negative samples per positive pair, we might add noise pairs such as $(\text{apricot}, \text{aardvark})$, $(\text{apricot}, \text{twelve})$, etc.

Algorithm. The Skip-gram algorithm proceeds in four steps:

1. Treat (t, c) pairs from the corpus window as **positive** examples.
2. Randomly sample k words from the vocabulary for each positive pair to form **negative** examples.
3. Train a logistic regression classifier to distinguish positive from negative pairs.
4. Discard the classifier; keep the learned weights as the **word embeddings**.

What are t and c as vectors? t correspond to a row of matrix W and c to a column of matrix C . We construct them by:

- **Target vector t :** The target word is encoded as a one-hot vector $\mathbf{x} \in \mathbb{R}^{1 \times |V|}$ with a single 1 at position j (the word's index). Multiplying by W (of size $|V| \times d$) gives $\mathbf{x}W \in \mathbb{R}^{1 \times d}$, which simply **selects row j of W** . This row is the target embedding t . Every word has one row in W , so W stores all target embeddings.
- **Context vector c :** The context matrix C has size $d \times |V|$, so each **column k** of C is a d -dimensional vector representing word k when it plays the role of a context word. We use a column (not a row) because we want the dot product $t \cdot c$ to produce a scalar: t is a row vector ($1 \times d$), so c must be a column vector ($d \times 1$), i.e. column k of C .

Example 7.1 (Dimension check). Suppose $|V| = 10,000$ and $d = 300$. Then W is $10,000 \times 300$ and C is $300 \times 10,000$. The target embedding t is one row of W , a vector of length 300. The context embedding c is one column of C , also a vector of length 300. Their dot product $t \cdot c$ is a single number, which the sigmoid then maps to a probability.

Computing $P(+ | t, c)$. The classifier must output the probability that c is a genuine context word of t . The natural measure of word similarity is the **dot product** $t \cdot c$: if the two vectors point in similar directions, the dot product is large. However, a dot product is not a probability. We fix this by squashing it through the **sigmoid**:

$$P(+ | t, c) = \sigma(t \cdot c) = \frac{1}{1 + e^{-t \cdot c}}, \quad P(- | t, c) = 1 - P(+ | t, c) = \frac{e^{-t \cdot c}}{1 + e^{-t \cdot c}} \quad (21)$$

Assuming context words are independent, the joint probability over all k context words $c_{1:k}$ is:

$$P(+ | t, c_{1:k}) = \prod_{i=1}^k \sigma(t \cdot c_i), \quad \log P(+ | t, c_{1:k}) = \sum_{i=1}^k \log \sigma(t \cdot c_i) \quad (22)$$

Objective function. For a single target word t with one true context word c and k noise words $n_1, \dots, n_k$, the per-example log-likelihood to maximise is:

$$L(\theta) = \log \sigma(c \cdot t) + \sum_{i=1}^k \log \sigma(-n_i \cdot t) \quad (23)$$

The first term rewards making t and its real context similar; the k negative terms penalise assigning high similarity to noise words. Summing over all (t, c) pairs in the corpus gives the global objective:

$$J(\theta) = \sum_{(t,c) \in +} \log \sigma(t \cdot c) + \sum_{(t,c) \in -} \log \sigma(-t \cdot c) \quad (24)$$

This is maximised using **stochastic gradient descent**: starting from $|V|$ random d -dimensional vectors (typically $d = 300$), the algorithm iterates over the corpus and updates embeddings to push positive pairs closer and negative pairs apart.

Noise word distribution. Noise words are not sampled uniformly. A frequency-weighted distribution with exponent α is used:

$$P_\alpha(w) = \frac{\text{count}(w)^\alpha}{\sum_{w'} \text{count}(w')^\alpha} \quad (25)$$

The value $\alpha = 0.75$ is standard: compared to plain unigram sampling ($\alpha = 1$), it boosts the sampling probability of rare words, ensuring the model sees diverse negative examples.

Example 7.2 (Effect of $\alpha = 0.75$). Suppose two words have unigram probabilities $p(a) = 0.99$ and $p(b) = 0.01$. Under $\alpha = 0.75$:

$$P_{0.75}(a) = \frac{0.99^{0.75}}{0.99^{0.75} + 0.01^{0.75}} \approx 0.97, \quad P_{0.75}(b) \approx 0.03$$

The rare word b triples its sampling probability (from 1% to 3%), giving it a fairer chance of appearing as a noise word.

Output. Training learns two matrices W (target embeddings) and C (context embeddings). In practice we use the rows of W as the final word vectors, and discard C ; alternatively, W and C can be summed or averaged.

7.1.2 CBOW (Continuous Bag of Words)

CBOW reverses the Skip-gram direction: given the context words $w_{t-L}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+L}$, predict the target word w_t . The context word vectors are **summed** (or averaged) in the projection layer to produce a single d -dimensional vector, which is then passed through a matrix W' to predict the target. The objective is to maximise the log conditional probability of the target given its context.

CBOW is **faster to train** than Skip-gram because it makes one prediction per window rather than $2L$ predictions, and it tends to perform better on frequent words. However, it ignores global vocabulary information and is less common in research papers today.

Observation 7.1. Pre-trained Word2vec embeddings are publicly available (e.g. Google’s 300-dimensional vectors trained on 100 billion words of Google News). It is also straightforward to train domain-specific embeddings using the **gensim** library in Python. Note that Word2vec embeddings are **position-independent**: all words in the window contribute equally regardless of their distance from the target. This is addressed by position-dependent weighting (see Section 7.3).

Phrase representations (Word2phrase). Word2vec operates at the unigram level, so multi-word expressions like *New York* or *ice cream* are split into separate tokens. The **Word2phrase** method identifies frequent collocations and merges them into a single token before training. A bigram (w_i, w_j) is merged if its score exceeds a threshold δ :

$$\text{score}(w_i, w_j) = \frac{\text{count}(w_i, w_j) - \delta}{\text{count}(w_i) \cdot \text{count}(w_j)} \quad (26)$$

The numerator discounts rare bigrams; the denominator normalises by the individual word frequencies, so the score is high only when the pair co-occurs far more than chance. The process is usually repeated 2–6 times with a decreasing threshold to capture longer phrases. The result is a word2vec with amplified vocabulary $|V'| \geq |V|$ of the most important bigrams.

FastText (Bojanowski et al., 2016). FastText extends the Skip-gram model by representing each word as a **sum of character n -gram vectors**. For the word *organization* with n -grams of length 3–6, the representation is:

$$v(\text{organization}) = v_{\text{organization}} + \frac{1}{|N|} \sum_{n \in N} x_n \quad (27)$$

where N is the set of n -character grams and x_n is the embedding of n -gram n through the embedding matrix. In matrix terms, fastText maintains **two embedding matrices**: the usual word matrix W ($|V| \times d$) and a separate character n -gram matrix G ($|G| \times d$), where $|G|$ is the number of unique n -grams seen in the corpus. Here:

- $v_{\text{organization}}$ is simply **row j of W** , standard Word2vec lookup.
- Each x_n is **row n of G** , a d -dimensional vector for one character piece.
- The sum averages those n -gram rows and adds the word row to produce the final vector $v(\text{organization})$.

This means fastText’s total parameter count is $|V| \cdot d + |G| \cdot d$ instead of just $|V| \cdot d$. However, $|G|$ is **bounded**: with a fixed alphabet and $n \in \{3, \dots, 6\}$, there are only so many possible character sequences regardless of corpus size, so the overhead is manageable. Crucially, n -gram vectors are **shared across all words** — the piece **org** is the same vector whether it appears in *organize*, *organization*, or *organ*.

Example 7.3 (Character n -grams of *organization*). *FastText* wraps the word in boundary markers $<$ and $>$ before extracting n -grams. For $n = 3$, *organization* yields the pieces:

$<or, org, rga, gan, ani, niz, iza, zat, ati, tio, ion, on>$

(plus longer n -grams for $n = 4, 5, 6$, and the whole-word token $<organization>$). Each piece n has its own learned vector x_n . The final embedding is:

$$v(\text{organization}) = \underbrace{v_{\text{organization}}}_{\text{full-word vector}} + \frac{1}{|N|} \underbrace{\sum_{n \in N} x_n}_{\text{avg. of subword pieces}}$$

The subword average captures morphological regularities: *org, rga, ...* appear in *organize, organizational, etc.*, so their shared pieces pull those words' vectors closer together.

Handling out-of-vocabulary words. Because *fastText* operates at the character n -gram level, it can construct a representation for any word — even one never seen during training. An unseen word like *LLMs* is decomposed into its character pieces (e.g. *LL, LM, Ms, ...*), each of which has a trained vector; their sum yields the OOV embedding. This is the key advantage over *Word2vec* and *GloVe*, which assign no vector at all to words outside their fixed vocabulary.

Comparing the two models: *Skip-gram* performs better on **semantic tasks** for English (2–5% more accurate than *fastText*), while *fastText* outperforms *Skip-gram* on **syntactic tasks** and morphologically rich languages. *FastText* also takes roughly $1.5\times$ longer to train.

Implementation. *Word2vec* can be trained using the **gensim** Python library. Pre-trained *fastText* models in 157 languages with aligned vectors are available at fasttext.cc. For Slovene, several variants are available via Clarín.si.

7.2 GloVe: Global Vectors for Word Representation

GloVe (Pennington, Socher & Manning, 2014) is an alternative static embedding method that directly exploits global co-occurrence statistics, in contrast to the local context windows of *Word2vec*.

Definition 7.2 (Word–word co-occurrence matrix). Let X be the word–word co-occurrence matrix, where X_{ij} counts how often word j appears in the context of word i across the entire corpus. Context here means within a fixed window of L words on either side: for each occurrence of word i in the corpus, every word within L positions of it is counted as a context word. Define $X_i = \sum_k X_{ik}$ (the total number of times any word appeared in the context of i) and the conditional co-occurrence probability $P_{ij} = X_{ij}/X_i$.

Example 7.4 (Building the co-occurrence matrix). Consider the sentence “the cat sat on the mat” with window $L = 1$ (one word on each side). For the target word *sat* (position 3), its context words are *cat* (left) and *on* (right). So this sentence contributes +1 to $X_{\text{sat}, \text{cat}}$ and +1 to $X_{\text{sat}, \text{on}}$. After processing the full corpus, $X_{\text{sat}, \text{cat}}$ equals the total number of times *cat* appeared within one position of *sat*. The probability $P_{\text{sat}, \text{cat}} = X_{\text{sat}, \text{cat}}/X_{\text{sat}}$ is then the fraction of *sat*’s context slots that were occupied by *cat*.

The central insight of *GloVe* is that *ratios* of co-occurrence probabilities carry meaning more reliably than raw counts. For a probe word k :

- If k is related to word i but not to word j , then $P_{ik}/P_{jk} \gg 1$.
- If k is related to j but not to i , then $P_{ik}/P_{jk} \ll 1$.
- If k is equally related (or unrelated) to both, then $P_{ik}/P_{jk} \approx 1$.

For example, with $i = \text{ice}$ and $j = \text{steam}$: the probe *solid* gives a large ratio ($\text{solid} \leftrightarrow \text{ice}$), *gas* gives a small ratio ($\text{gas} \leftrightarrow \text{steam}$), and *water* gives a ratio near 1.

Definition 7.3 (*GloVe* objective). *GloVe* trains word vectors w_i and context vectors $\tilde{w}_j$ (plus scalar biases $b_i, \tilde{b}_j$) by minimizing the weighted least-squares loss:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 \quad (28)$$

where the weighting function

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & x < x_{\max} \\ 1 & \text{otherwise} \end{cases} \quad (29)$$

caps the contribution of very frequent co-occurrences (typical values: $x_{\max} = 100$, $\alpha = 3/4$).

Remark 7.1. *GloVe is essentially a Word2vec-style approach — the embeddings w_i and $\tilde{w}_j$ play the same role as the target and context vectors in Skip-gram but with a different objective: instead of predicting context words one at a time, GloVe directly fits the global log co-occurrence counts. The scalar biases b_i and $\tilde{b}_j$ absorb each word’s overall frequency in the corpus (frequent words like the co-occur with almost everything at high counts purely due to frequency, not semantics). Essentially $\log(P_{ij}) = \log(X_{ij}) - \log(X_i)$, so we need the bias term.*

Because GloVe trains on the global co-occurrence matrix (computed once over the entire corpus), it is efficient once X is precomputed. In practice, GloVe and Word2vec produce embeddings of comparable quality on most benchmarks; fastText tends to outperform both on morphologically rich languages.

Model	Advantages	Disadvantages
CBOW	Faster to train; predicts frequent words well	Ignores global vocabulary information; less common in practice
Skip-gram	Good semantic accuracy; pre-trained models available	Does not handle out-of-vocabulary words
GloVe	Uses global co-occurrence statistics; good syntactic and semantic accuracy; pre-trained models available	Does not handle out-of-vocabulary words
FastText	Handles OOV words; strong on syntactic tasks; 157-language pre-trained models with aligned vectors	Takes $\approx 1.5\times$ longer to train than Skip-gram; 2–5% less accurate on English semantic tasks

Table 3: Comparison of static word embedding models.

7.3 Subword Information and Positional Weighting

Standard **CBOW** aggregates all context words into a single vector v_C by averaging their embeddings, treating every position in the window equally. A word immediately next to the target, however, is typically more semantically relevant than one L steps away. To capture this, we replace the plain average with a **position-weighted sum**:

$$v_C = \sum_{p \in P} d_p \circ u_{t+p} \quad (30)$$

where P is the set of relative positions in the window, u_{t+p} is the embedding of the word at relative position p , and d_p is a learned position-dependent weight vector. The operator $\circ$ denotes the **Hadamard product** (element-wise multiplication): rather than scaling the whole embedding by a scalar, each dimension can be weighted differently depending on distance. The key idea is that this adds a form of **positional encoding** to each context word’s embedding before aggregation, without changing the word embedding itself. Note this extension applies to **CBOW**, not Skip-gram: Skip-gram scores each context word independently and has no aggregation step to weight.

7.4 Evaluation of Embeddings

There are two main approaches to evaluation: intrinsic and extrinsic measures. While intrinsic tasks are useful for development, we are usually interested in extrinsic measures tailored to the final downstream application.

1. Intrinsic tasks:

- Evaluation on a specific/intermediate subtask, such as the **Word Analogy** test (a is to b as c is to d) or word similarity correlations (e.g., WordSim-353).
- Fast to compute and requires no heavy downstream training.
- Helps to understand the geometric properties of the latent space.
- A major drawback is that improvements in intrinsic scores do not always correlate with better performance on real-world applications.

2. Extrinsic tasks:

- Evaluation on a real-world task like **Named Entity Recognition (NER)**, **Sentiment Analysis**, or **Machine Translation**.
- Can take a long time to compute because it requires training a full supervised model using the embeddings as features.

- Provides a true measure of the embedding’s utility in a practical system.
- A practical difficulty: when accuracy changes, it is often **unclear which subsystem is responsible**—the embeddings, the model architecture, or their interaction. If replacing exactly one subsystem with another improves accuracy, then we have clear evidence of improvement.

Intrinsic Human-Based Evaluation

We compare model performance against human scores on word similarity-type tasks. Standard benchmarks include:

- **WordSim-353** (Finkelstein et al., 2002): 353 word pairs with human similarity scores.
- **SimLex-999** (Hill et al., 2015): 999 word pairs; distinguishes similarity from relatedness.
- **CoSimLex** (SemEval 2020): extends SimLex by providing word pairs *in context*, accounting for polysemy.
- **SCWS** (Stanford Contextual Word Similarity, Huang et al., 2012): contextual word similarity.
- **TOEFL**: synonym selection multiple-choice questions, e.g. “*Levied is closest in meaning to: imposed / believed / requested / correlated.*”

For a comprehensive comparison across many tasks, one should check the **MTEB** (Massive Text Embedding Benchmark).

Note: The type of similarity captured depends heavily on the window size L , a detail that is often difficult to notice.

- **Small windows** tend to capture *functional* or *syntactic* similarity (e.g., “red” and “blue”).
- **Large windows** tend to capture *topical* or *associative* similarity (e.g., “doctor” and “hospital”).

If an embedding is trained properly, semantically related words are embedded in nearby regions of the latent space.

For example, there is **SimLex-999**, which consists of human judgments on how similar words are. **CoSimLex** improves upon this by taking into account the specific context of the word within the sentence during human evaluation.

The strength of **Word2vec** is that it preserves linear relationships in the latent space. This allows for morphological and syntactic analogies:

$$v_{\text{apple}} - v_{\text{apples}} \approx v_{\text{car}} - v_{\text{cars}} \approx v_{\text{family}} - v_{\text{families}} \quad (31)$$

As a result, these encodings are highly effective at capturing both general similarity and specific dimensions of meaning. In this framework, **directions in space correspond to meaning**.

Observation 7.2. *There are many more relational similarities that have been shown to be true in the latent space, such as country-capital pairs, noble titles, and family relations (e.g., aunt and uncle, mother and son, father and mother). There are also relationships based on adjectives: slow, slower, slowest; or short, shorter, shortest.*

Note: It is possible to see embedding visualizations in different applications (such as the Embedding Projector). Embeddings can also help study **diachronic linguistics**—how word meanings change over time. For this, researchers have used corpora like Google Books from 1850 until 2000, training separate embeddings every 10 years to track the “trajectory” of a word’s meaning.

Embeddings also reflect **cultural bias** present in the training data. For example, studies have shown that African-American names can be implicitly associated with unpleasant words, or that male and female words are often associated with different occupations or stereotypical meanings.

7.5 Cross-lingual Embeddings

Typically, embeddings are trained on monolingual resources. Words of one language form a “cloud” in high-dimensional space. For example, we have an embedding matrix for English and another for Slovenian. We can align these clouds for different languages via a transformation matrix T (often solved using the Procrustes problem) such that:

$$T \cdot v_{\text{slovenian}} \approx v_{\text{english}} \quad (32)$$

In a simplified view, if S is the Slovenian matrix and E is the English matrix, we seek a mapping such that $SW \approx E$.

Definition 7.4 (Alignment). *We perform a series of linear transformations such that the two language clouds align to a reasonable degree. There are several techniques for this, such as using a small seed dictionary of parallel words. While this is still useful, it is increasingly being replaced by **Multilingual Language Models** (like mBERT or XLM-R) which learn a shared space for all languages simultaneously.*

7.6 Cross-lingual Applications and Limitations

The benefit of cross-lingual alignment is that it allows us to transfer AI tools developed for a high-resource language, like English, to other languages such as Catalan. By mapping one language's structure into the space of another, we can perform downstream tasks without retraining models from scratch.

However, a major limitation of these models is that they are **static**. A single word vector must represent all possible meanings of a word. For example, the word "bank" has the same representation whether it refers to a *river bank* or a *financial institution*. In these cases, the vector represents a statistical average of its usages.

The next evolutionary step is **contextual embeddings** (e.g., Transformers). In this framework, the embedding is not produced for a single word in isolation, but for the word within its specific context. Depending on the complexity of the task, static embeddings might still be useful for their efficiency, while contextual embeddings are necessary for nuance.

7.7 Other Applications of Word2vec: Doc2vec

Definition 7.5 (Doc2vec). *Doc2vec (also known as Paragraph Vector) is an extension of Word2vec designed to encode entire documents, sentences, or paragraphs into a single continuous vector.*

The core idea is to treat a **Document ID** as a unique vector that acts as a "memory" of the topic, which is used alongside word vectors to predict words within that document.

Doc2vec Variants

There are two primary flavors of Doc2vec, mirroring the original Word2vec architectures:

1. **PV-DM (Distributed Memory)**: This is the analog to CBOW. The model takes a context of words *and* the Document ID as inputs to predict a target word. The Document ID acts as a global context, helping the model remember what is missing from the local window.
2. **PV-DBOW (Distributed Bag of Words)**: This is the analog to Skip-gram. Instead of using context words, the model uses only the Document ID as input to predict a set of words randomly sampled from the document. This method is faster and requires less storage.

Observation 7.3. *If we have a new book or document to encode, we fix the trained word weights and perform an iterative process to find the optimal Document ID vector for this new content.*

Doc2vec learns which words are specific to a document's theme. Its advantages include inheriting semantic word knowledge from Word2vec while excelling at finding similar documents or performing document-level classification.

We will extend these concepts to Transformers in the future.

8 Recurrent and Convolution neural network for text

Eventhough the transformer architecture is dominating the panorama right now, in some aspects, recurrent neural networks are still used nowadays. Advise: The SOTA it is not necessarily the most convenient one.

8.1 Recurrent Neural Networks

The fundamental question is: Can we design a neural network with "back" connections? The goal is to introduce **memory** into the architecture. Memory is essential for processing text because language relies on long-range dependencies; to understand a word, we often need the context of previous words or even previous sentences.

A significant limitation of standard Feed-Forward Neural Networks (FFNN) is their fixed input size. In Recurrent Neural Networks (RNNs), we solve this problem by partitioning the input into sequential chunks. This allows us to input data gradually, one step at a time, making the model flexible enough to handle sequences of any length.

Definition 8.1 (Recurrent Neural Networks). *Informally, Recurrent Neural Networks (RNNs) are networks with loops in them, allowing information to persist over time. This makes them ideal for sequential data where the current output depends on previous computations. Formally, at each time step t an RNN takes the current input x_t and the previous hidden state h_{t-1} and computes:*

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h) \quad (33)$$

$$\hat{y}_t = \text{softmax}(W_{hy} h_t + b_y) \quad (34)$$

where W_{hh} , W_{xh} , W_{hy} are learnable weight matrices and b_h , b_y are bias vectors. Crucially, **the same weights are shared across all time steps**, allowing the network to generalize to sequences of arbitrary length.

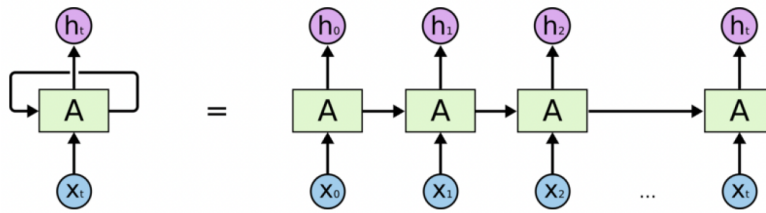


Figure 7: Architecture of a recurrent neural network showing the hidden state loop.

Observation 8.1. *The input token x_t is projected into a dense space via the embedding matrix $E \in \mathbb{R}^{|V| \times d}$, where $|V|$ is the vocabulary size (e.g., 10,000) and d is the embedding dimension. Because $|V| \gg d$, E dominates the network's parameter count. The core recurrent weights—the input-to-hidden matrix $U \in \mathbb{R}^{d \times h}$, hidden-to-hidden matrix $W \in \mathbb{R}^{h \times h}$, and hidden-to-output matrix $V \in \mathbb{R}^{|V| \times h}$ (if not tied to E)—collectively constitute only a tiny fraction (often $\approx 1\%$) of the total network parameters.*

There are many architectures for Recurrent Neural Networks depending on the input and output requirements:

- **One-to-one:** Standard neural network (e.g., image classification).
- **One-to-many:** A single input produces a sequence (e.g., image captioning).
- **Many-to-one:** A sequence produces a single output (e.g., sentiment analysis).
- **Many-to-many:** A sequence produces another sequence (e.g., machine translation).

Example 8.1 (RNN for text). *To process a sentence, we first tokenize the text, convert tokens to indices, and project them into an embedding space before passing the vectors into the RNN.*

Example 8.2 (Ilya Sutskever, 2011). *Produced some of the first effective neural language models. These were based on RNNs at the character level, trained on half a billion characters from Wikipedia to predict the next character in a sequence.*

Example 8.3. *Next-word prediction is crucial in Automatic Speech Recognition (ASR) because real-time processing requires the context of previous words to disambiguate phonemes. Traditional approaches involved trigram models, but RNNs capture longer dependencies.*

In vanilla RNNs, we face significant challenges during training, specifically **vanishing gradients** and **exploding gradients**. Due to the structure of **Backpropagation Through Time (BPTT)**, gradients are multiplied repeatedly by the same weights, causing them to shrink or grow exponentially, which effectively prevents the network from learning long-term dependencies. Moreover, RNN struggle to be parallelizable.

To avoid the vanishing gradient problem, we use **LSTM** (Long Short-Term Memory) solutions. LSTMs utilize a specialized **memory cell** to protect the backpropagated information, ensuring we do not lose long-term dependencies or "forget" essential context. LSTMs were the dominant architecture for sequence modeling approximately from 2011 to 2017.

8.2 Long Short-Term Memory (LSTM)

Definition 8.2 (LSTM). A Long Short-Term Memory unit (Hochreiter & Schmidhuber, 1997) extends the vanilla RNN by introducing a **cell state** c_t and three multiplicative **gates** that regulate the flow of information:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{forget gate}) \quad (35)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{input gate}) \quad (36)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (\text{candidate cell state}) \quad (37)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{cell state update}) \quad (38)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{output gate}) \quad (39)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{hidden state}) \quad (40)$$

where σ is the sigmoid function, $\odot$ denotes element-wise (Hadamard) multiplication, and $[h_{t-1}, x_t]$ denotes the concatenation of the previous hidden state and the current input.

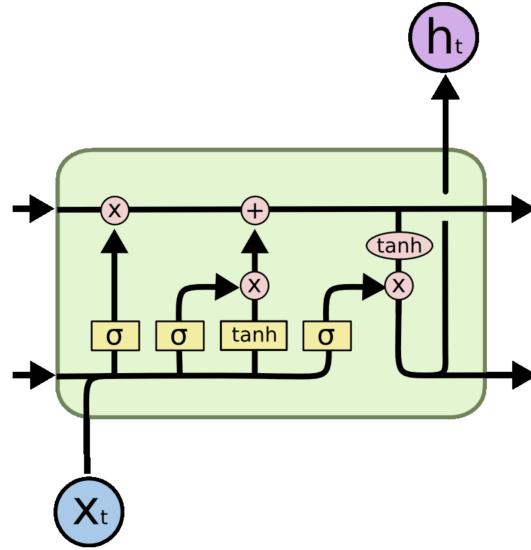


Figure 8: LSTM architecture.

The three gates serve distinct roles:

- **Forget gate** $f_t \in (0, 1)^d$: decides what fraction of the previous cell state to retain. A value near 0 discards past information; near 1 it is preserved.
- **Input gate** i_t : controls how much of the new candidate $\tilde{c}_t$ is written into the cell state.
- **Output gate** o_t : determines what portion of the cell state is exposed as the hidden state h_t .

The cell state c_t acts as a "conveyor belt" carrying information across many time steps through an *additive* update. Because gradients flow through the additive path $c_t = f_t \odot c_{t-1} + \dots$ without being repeatedly multiplied by small weights, LSTMs mitigate the vanishing gradient problem of vanilla RNNs.

8.3 Gated Recurrent Unit (GRU)

Definition 8.3 (GRU). *The Gated Recurrent Unit (Cho et al., 2014) is a streamlined variant of the LSTM that merges the cell state and hidden state and reduces the number of gates to two:*

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad (\text{update gate}) \quad (41)$$

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad (\text{reset gate}) \quad (42)$$

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h) \quad (\text{candidate hidden state}) \quad (43)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (44)$$

- **Update gate** z_t : interpolates between the previous hidden state and the new candidate. When $z_t \approx 1$, the hidden state is fully replaced; when $z_t \approx 0$, past information is preserved. This single gate combines the roles of the LSTM’s forget and input gates.
- **Reset gate** r_t : controls how much of the previous hidden state contributes to the candidate. When $r_t \approx 0$, the model effectively ignores the past and reads the current input fresh.

GRUs have fewer parameters than LSTMs and are faster to train, while achieving comparable performance on most tasks. LSTMs may retain a slight advantage for sequences requiring very long-range dependencies.

8.4 Bidirectional RNNs

Definition 8.4 (Bidirectional RNN). *A Bidirectional RNN (BiRNN) runs two independent RNNs over the same input sequence: one processes tokens from left to right, producing forward hidden states $\vec{h}_t$, and one from right to left, producing backward hidden states $\overleftarrow{h}_t$. At each position t , the two states are concatenated:*

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (45)$$

Example 8.4 (Disambiguation in POS Tagging). *Consider the word “estimate” in the following two contexts:*

1. “The precise **estimate** was given yesterday.”
2. “We **estimate** the project will take three weeks.”

A standard left-to-right RNN reading “estimate” lacks access to the subsequent tokens. In contrast, a BiRNN processes the future context simultaneously via its backward pass. For the first sentence, the backward pass immediately registers the verb “was”, signaling that “estimate” must act as a **Noun**. For the second sentence, it captures the noun phrase “the project”, allowing the model to correctly classify it as a **Verb**.

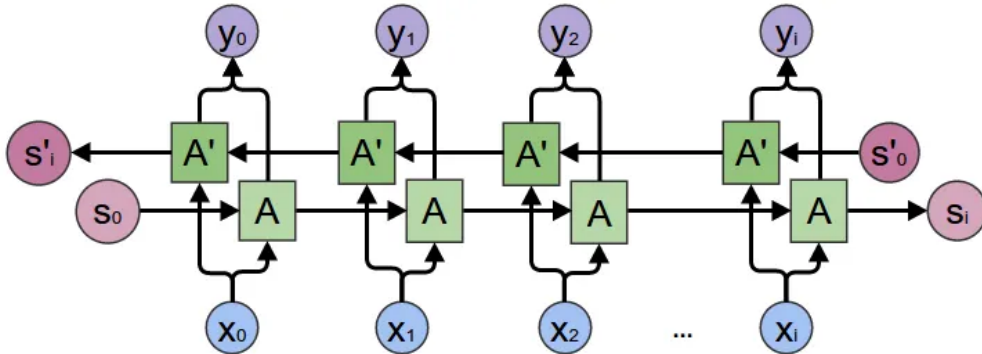


Figure 9: Architecture of a bidirectional recurrent neural network showing the hidden states loop.

By combining both directions, each position’s representation captures context from the *entire* sequence. BiLSTMs (bidirectional LSTMs) were the dominant architecture for NLP sequence-labeling tasks — **Named Entity Recognition (NER)**, **POS tagging**, **dependency parsing** — before Transformers superseded them.

Remark 8.1. *BiRNNs cannot be applied directly to autoregressive generation tasks (e.g., language modeling or translation decoding), since the right context is unavailable at generation time. They are therefore most commonly used as encoder components or for classification and labeling tasks where the full sequence is available at inference.*

Seq2Seq (Sequence-to-Sequence) architectures are specifically designed to map an input sequence to an output sequence where the two lengths may differ. This structure serves as the foundation for tasks like **Neural Machine Translation**, where input and output languages have entirely different syntactic structures and word counts. It consists of two major components:

- **Encoder:** A recurrent network that processes the input sequence $\mathbf{x} = (x_1, x_2, \dots, x_T)$ step-by-step. Its primary objective is to compress the entire source context into a single dense representation called the **context vector $\mathbf{c}$** . Typically, $\mathbf{c}$ is chosen to be the final hidden state of the encoder RNN ($\mathbf{c} = h_T$), capturing a summary of the whole input.
- **Decoder:** An autoregressive recurrent network initialized by the context vector ($\mathbf{s}_0 = \mathbf{c}$). The decoder acts as a language model conditioned on the encoder’s summary, responsible for unfolding $\mathbf{c}$ into the target sequence $\mathbf{y} = (y_1, y_2, \dots, y_{T'})$ one token at a time.

Remark 8.2 (The Autoregressive Loop in Action). *During text generation (inference), the decoder generates tokens sequentially by consuming its own previous choices. At step t , the decoder takes the previous emitted word token $\hat{y}_{t-1}$ and its current hidden state $\mathbf{s}_{t-1}$ to compute the next hidden state $\mathbf{s}_t$. This hidden state is projected to a softmax distribution over the target vocabulary to sample the new token $\hat{y}_t$. This newly sampled output is then fed directly back into the network as the input for step $t + 1$:*

$$\mathbf{s}_t = f(\mathbf{s}_{t-1}, \mathbf{E}[\hat{y}_{t-1}]) \quad (46)$$

This feedback loop continues recursively until the model predicts a special end-of-sequence token ($\langle \text{EOS} \rangle$).

This structure is the foundation for tasks like **Neural Machine Translation**, where the input and output languages have different syntactic structures and lengths. The critical bottleneck is, of course, the compression of the sentence in a single latent vector. That motivated the development of the Attention mechanism.

8.5 The Attention Mechanism in RNNs

The fundamental bottleneck in the vanilla Seq2Seq architecture is not a lack of representation capacity, but rather an information retrieval problem. Forcing an entire source sentence into a single, fixed-length context vector causes severe degradation on long sequences. The Attention mechanism resolves this by installing a dynamic alignment layer on top of the encoder-decoder framework, allowing the decoder to selectively “look back” at the entire history of input vectors.

1. **Bidirectional Encoding:** The encoder remains a core component, but instead of discarding intermediate states, we stack every hidden representation. Using a bidirectional RNN to capture context from both temporal directions, we concatenate the forward and backward hidden states for each source token i :

$$h_i = [\vec{h}_i ; \overleftarrow{h}_i] \quad (47)$$

These vectors are collected into an encoder memory matrix $H = [h_1, h_2, \dots, h_T] \in \mathbb{R}^{2d \times T}$.

2. **Calculating Alignment Scores:** At decoding step t , the decoder must determine which source tokens are relevant to its current hidden state $\mathbf{s}_{t-1}$. It computes an alignment score $e_{t,i}$ —essentially a geometric similarity metric—between $\mathbf{s}_{t-1}$ and every encoder state h_i . A common approach is the Dot-Product attention variant:

$$e_{t,i} = \text{score}(\mathbf{s}_{t-1}, h_i) = \mathbf{s}_{t-1}^T h_i \quad (48)$$

A higher score indicates that the encoder token h_i matches the semantic and grammatical requirements of the decoder’s current state.

3. **Softmax Normalization (Attention Weights):** To transform these raw scores into a valid probability distribution, a softmax function normalizes them into attention weights $\alpha_{t,i}$ that sum to 1:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^T \exp(e_{t,j})} \quad (49)$$

These weights act as a fluid, step-dependent spotlight indicating the relative importance of each input word.

4. **Dynamic Context Vector Generation:** The unique context vector $\mathbf{c}_t$ for step t is computed as a weighted sum of the encoder hidden states, using the normalized attention weights:

$$\mathbf{c}_t = \sum_{i=1}^T \alpha_{t,i} h_i \quad (50)$$

This custom vector $\mathbf{c}_t$ is then concatenated directly with the decoder’s current input to emit the next target word autoregressively.

By maintaining the encoder-decoder paradigm but substituting the single static bottleneck for a dynamically calculated context vector $\mathbf{c}_t$, the model gracefully preserves long-range token relationships across disparate syntactic structures.

8.6 Convolutional Networks for Text Classification

Remark 8.3. While Transformers have become the dominant architecture for text modeling due to their superior scaling and parallelization efficiency, Convolutional Neural Networks (CNNs) remain highly successful at mapping raw text into meaningful latent spaces. A 1D continuous convolution is defined mathematically as:

$$(f \star g)(x) = \int_{-\infty}^{\infty} f(\tau)g(x - \tau)d\tau \quad (51)$$

In digital text architectures, this operator is discretized and applied across sequential dimensions.

Observation 8.2. Instead of processing explicit semantic tokens (words), a character-level CNN treats text as a raw discrete signal. A convolutional filter slides along the temporal axis of the text. The filter parameters act as feature detectors, learning to fire strongly when encountering specific character patterns, prefixes, suffixes, or sub-word structures (n-grams).

8.6.1 Character-Level CNN Architecture (Zhang et al., 2015)

The text sequence is quantified and processed through a fixed, lightweight frame:

- **Input Quantization:** Text is truncated or padded into rigid blocks of $L = 1014$ characters. Each character is converted into a one-hot vector using a fixed alphabet of size $m = 70$ (alphanumeric characters + common symbols). The resulting input is an efficient binary matrix of size 1014×70 .
- **1D Temporal Convolutions:** Filters have a spatial dimension of $K \times m$ (where filter lengths $K \in \{3, 7\}$). Because the filter width matches the alphabet size m , convolutions move strictly in one direction along the character length.
- **Activation & Regularization:** The network applies non-linear Rectified Linear Units ($\text{ReLU}(x) = \max(0, x)$) and utilizes Dropout ($p = 0.5$) between fully connected layers to counter overfitting.
- **Temporal Max-Pooling:** Max-pooling layers compress the temporal dimension by extracting the maximum activation value over local windows, providing translation invariance (detecting a feature regardless of its exact position in the text block).

Remark 8.4 (Key Scaling Properties). Character-level CNNs completely eliminate massive word embedding parameters, making them highly memory-efficient. However, because they lack initial word-level semantics, they require massive training sets (typically $> 10^6$ samples) to learn text structures from scratch. For smaller datasets, traditional sparse representations like bag of n-grams remain highly competitive.

8.6.2 Transition from Statistical N-grams to Deep Networks in Speech Recognition

Historically, Automatic Speech Recognition (ASR) systems relied heavily on statistical language models, such as trigrams, to calculate the probability of a word sequence $P(w_3 \mid w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}$. However, these models suffer from two critical limitations: a severe memory bottleneck restricting context to a few preceding tokens, and a complete lack of semantic generalization (e.g., treating structurally similar words like “cat” and “dog” as entirely independent entities). Modern ASR architectures resolve these flaws by replacing counting-based models with Deep Neural Networks (such as CNNs, RNNs, and Transformers). These deep learning systems leverage continuous vector embeddings to automatically capture semantic similarities and utilize deep temporal or convolutional feature maps to capture long-range contextual information across the acoustic and textual signal.

9 Transformers

Transformers solve the gradient vanishing/exploding problems of RNNs by allowing all tokens in the input to "talk" to each other simultaneously via self-attention. While the hidden dimensions are fixed, the vocabulary size is extremely large nowadays, often on the order of 128,000 tokens.

9.1 Transformer Architecture

The original Transformer (Vaswani et al., 2017) uses an **encoder-decoder** structure. The encoder maps an input sequence $(x_1, \dots, x_n)$ to a sequence of contextual representations; the decoder generates an output sequence $(y_1, \dots, y_m)$ one token at a time.

Key advantages over RNNs:

- **No recurrence:** all tokens processed in parallel – much faster training
- **Level-jumping:** any token attends directly to any other token, regardless of distance
- **No sequential gradient vanishing:** gradients flow through direct connections, not recurrent steps

Encoder. The encoder is a stack of $N = 6$ identical layers. Each layer has two sublayers:

1. **Multi-head self-attention** sublayer
2. **Position-wise feed-forward network** (FFN) sublayer

Each sublayer uses a residual connection followed by layer normalization: $\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$.

Decoder. The decoder is also a stack of $N = 6$ identical layers. Each decoder layer has three sublayers:

1. **Masked multi-head self-attention** (masking prevents attending to future positions during training)
2. **Multi-head encoder-decoder attention** (queries from the decoder; keys and values from the encoder output)
3. **Position-wise feed-forward network**

9.2 Very important nuances:

In the original transformer the embedding dimension is 512, for each word (token) we embed it to a 512 dimensional space. Note that the transformer architecture is totally flexible to the number of tokens as input. Because the input is not a $512 \cdot \text{num tokens}$ 1-D vector, but it is rather a matrix. So for each vector, we pass it to the Key, Query, Value matrices. If we have less words, the key-query matrix will be smaller, if we have more words in the input the key-query matrix will be bigger, the same with the value vector. Moreover, this job is totally parallel, we can compute the key's, values's and queries of each word totally in parallel.

9.3 Self-Attention Mechanism

Self-attention (scaled dot-product attention) computes a representation for each token as a weighted sum of all value vectors, where weights reflect pairwise similarity between tokens.

Q, K, V projections. Each input token x_i (embedding dimension d_m) is projected into three vectors via learned weight matrices $W^Q, W^K, W^V \in \mathbb{R}^{d_m \times d}$:

$$Q_i = x_i W^Q, \quad K_i = x_i W^K, \quad V_i = x_i W^V$$

In matrix form for all n tokens ($X \in \mathbb{R}^{n \times d_m}$, yielding $Q, K, V \in \mathbb{R}^{n \times d}$):

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

In the original Transformer, $d_m = 512$ and $d = 64$. The weight matrices W^Q, W^K, W^V are learnable parameters.

Four computation steps:

1. **Score** – dot product of each query with every key: $\text{sim}_{ij} = Q_i^T K_j$
2. **Scale** – divide by $\sqrt{d}$ (larger d inflates dot products, making softmax too peaked and gradients too small):

$$\text{sim}_{ij} = \frac{Q_i^T K_j}{\sqrt{d}}$$

3. **Softmax** – normalize to obtain attention weights:

$$\alpha_{ij} = \frac{\exp(\text{sim}_{ij})}{\sum_{k=1}^n \exp(\text{sim}_{ik})}$$

4. **Weighted sum** – combine value vectors weighted by attention:

$$Y_i = \sum_{j=1}^n \alpha_{ij} V_j$$

Compact matrix form:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V \quad (52)$$

The softmax score determines how much each token contributes at a given position. High-score tokens keep their values; irrelevant tokens are multiplied by near-zero weights and drowned out. The result is passed to the feed-forward network.

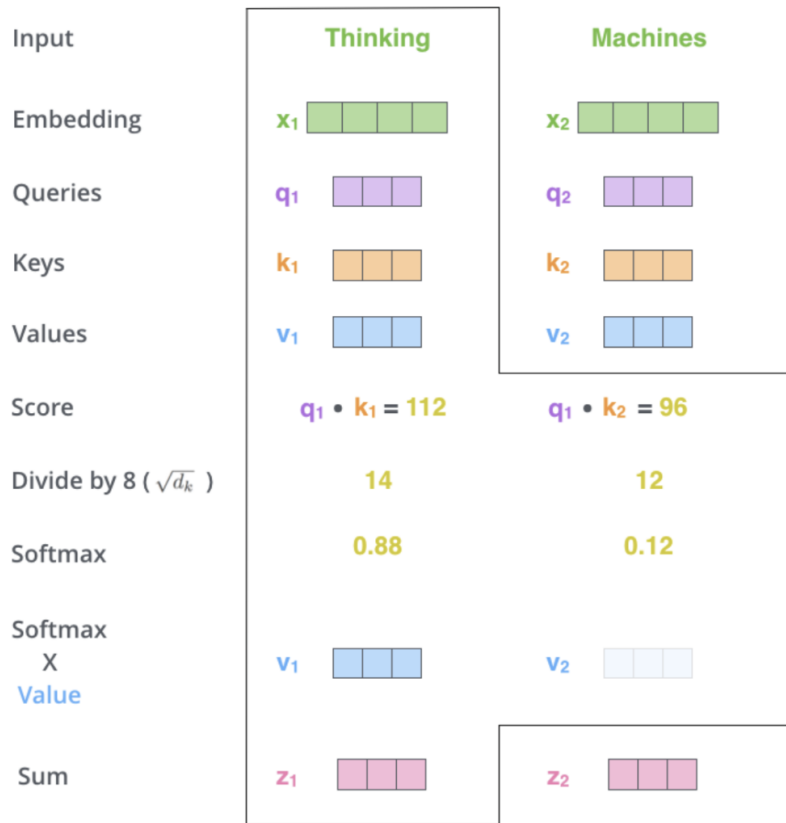


Figure 10: Example of the head attention mechanism, we input a vector we do the positional embeddings and obtain the corresponding queries, keys and values, and then we produce a score, which is normalized and used to weight the values.

9.4 Multi-Head Attention

Remark 9.1. Self-attention is replicated several times, called "multi-head attention." This expands the model's ability to focus on different positions. One attention head might be dominated by the actual word, but other heads might reveal different contextual information. Moreover, multi-head attention layers can cover multiple "representation spaces" at once.

The multi-head attention function is:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (53)$$

$$\text{where } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

The original Transformer uses $h = 8$ attention heads and model dimension $d_{\text{model}} = 512$. To keep the total cost comparable to single-head attention, each head operates in a **reduced dimension** $d_k = d_{\text{model}}/h = 64$: the projection matrices $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ project the full queries, keys, and values down to dimension d_k before computing attention. Each head therefore returns a vector of dimension $d_k = 64$.

After all h heads have run independently, their output vectors are **concatenated**, producing a single vector of dimension $h \cdot d_k = d_{\text{model}}$. The output projection $W^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ then linearly combines this concatenation into the final output vector of dimension d_{model} , which is passed to the feed-forward layer.

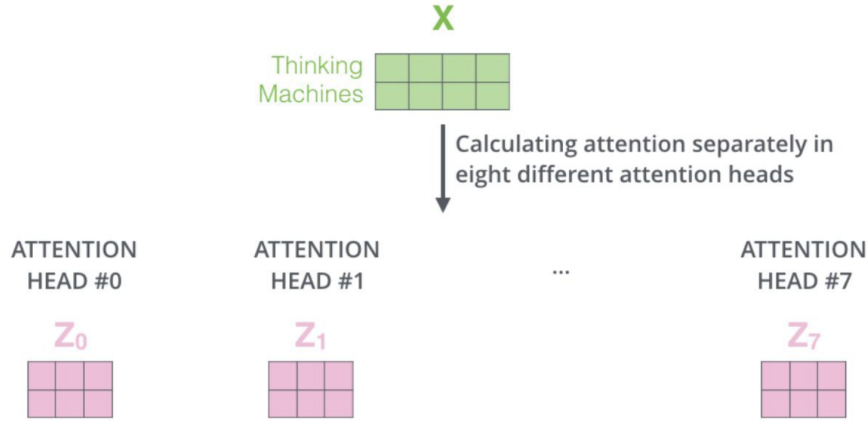


Figure 11: Example of the multihead attention mechanism. Each attention head computes a representation. In this picture, the representation is a 3-dimensional vector and we have two vectors. After this, we concatenate all the representation and we pass them to the feed-forward neural network.

Remark 9.2. *Residual connections are used to pass information without vanishing gradients. By adding these skip connections, we allow the gradients to flow all the way back during backpropagation. These connections also help maintain original position information in the higher layers.*

Remark 9.3. *In the original Transformer, the decoder takes the keys (K) and values (V) from the encoder, while generating the queries (Q) itself. Note that modern GPT-style models are "decoder-only," so they don't require a separate encoder.*

Remark 9.4. *The output is a probability distribution over the space of tokens. For example, if the model knows 100,000 unique words, the logits vector is 100,000 cells long, with each cell corresponding to the score of that specific token.*

Remark 9.5. *In current LLMs, we skip the encoder-decoder cross-attention and use a decoder-only architecture. The prompt is passed directly into the decoder.*

Remark 9.6. *For training, we use cross-entropy loss on a classification task, comparing our predicted token distribution against the ground truth (the actual next token).*

Example 9.1. *Predicted distribution:*

$$\hat{y} = (0.2, 0.2, 0.1, 0.2, 0.2, 0.1) \quad (54)$$

Actual ground truth (one-hot encoded):

$$y = (0, 0, 0, 1, 0, 0) \quad (55)$$

Remark 9.7. • **Greedy Decoding:** *We always choose the token with the highest probability.*

- **Beam Search:** *At each step, we keep the top k sequences (e.g., top 2). We construct a tree of possibilities and prune branches that fall below a certain cumulative probability.*

The whole transformers architecture looks like this:

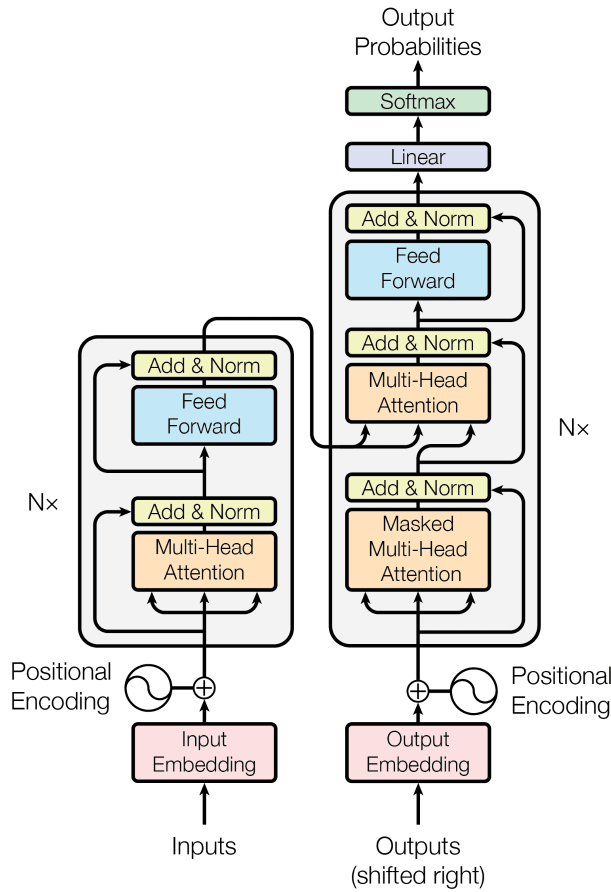


Figure 12: The initial encoder decoder model. The inputs are first processed by the encoder to obtain contextual representations. For this stage we repeat 6 times a block of multi-head attention and feed forward neural networks. Then this representation is given to the decoder, whose job is to output the probabilities of the next token in the sequence.

9.5 Positional Encoding

Self-attention is **permutation-invariant** in the sense that $\text{Attention}(Q, K, V)$ depends only on pairwise dot products between tokens. This means that if we shuffle two positions indexes in the input vector, the output vector will be the same with the two position shuffled. Hence, "Man bites dog" and "dog bites man" is creating the same output with only the position shuffled, which is not really perfect. To solve this, a **positional encoding** must be injected, either by adding a position-dependent vector to the input embeddings, or by modifying the attention computation directly. Four schemes dominate mainstream Transformer-based models, summarised below.

Model	Encoding Type	Implementation	Max-Length Constraint?
Vanilla Transformer	Sinusoidal (Fixed)	Added to input embeddings via fixed math (sine/cosine) functions.	No (can extrapolate)
BERT	Learned Absolute	Added to input embeddings via a learned look-up table.	Yes (hard limit)
T5	Relative Buckets	Added as a scalar bias directly inside the attention score matrix.	No (highly flexible)
ChatGPT (Modern)	RoPE (Rotary)	Multiplies Q/K vectors by a rotation matrix to spin the vector's angle.	No (extrapolates beautifully)

Table 4: Comparison of positional encoding strategies across Transformer-based model families.

Remark 9.8. "RoPE" is not unique to ChatGPT – it is the dominant choice across most modern decoder-only LLMs (GPT-NeoX, LLaMA, Mistral, PaLM), since it combines relative-position generalisation with a simple, parameter-free implementation.

9.5.1 Sinusoidal (Fixed) Encoding – Vanilla Transformer

The original Transformer (Vaswani et al., 2017) uses a fixed, non-learned encoding built from sine and cosine waves of geometrically increasing wavelength:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

where pos is the token’s position in the sequence and i indexes the embedding dimension pairs. The resulting vector $PE_{pos} \in \mathbb{R}^{d_{\text{model}}}$ is **added** directly to the token embedding before the first encoder layer: $z_{pos} = x_{pos} + PE_{pos}$. No parameters are learned – the function is computable for *any* pos , so in principle the model can be evaluated on sequences longer than those seen in training (extrapolation), even though in practice quality degrades for positions far outside the training range.

Example 9.2 (Sinusoidal encoding). *Take a toy model with $d_{\text{model}} = 4$ and compute the encoding for position $pos = 1$:*

$$PE_{(1,0)} = \sin(1/10000^0) = \sin(1) \approx 0.841, \quad PE_{(1,1)} = \cos(1/10000^0) = \cos(1) \approx 0.540$$

$$PE_{(1,2)} = \sin(1/10000^{0.5}) = \sin(0.01) \approx 0.010, \quad PE_{(1,3)} = \cos(1/10000^{0.5}) = \cos(0.01) \approx 0.99995$$

So $PE_1 \approx (0.841, 0.540, 0.010, 0.99995)$, a fixed vector added element-wise to whatever token embedding occupies position 1. Low-index dimension pairs ($i = 0$) oscillate quickly across positions, capturing fine-grained order; high-index pairs (large i) change very slowly, capturing coarse, long-range position.

9.5.2 Learned Absolute Encoding – BERT

BERT (and GPT-1/2/3) instead learns a position embedding matrix $P \in \mathbb{R}^{L_{\text{max}} \times d_{\text{model}}}$ jointly with the rest of the network, L_{max} is the maximum number of positions it can encode, exactly like a word-embedding table, but indexed by *position* rather than by token id. The final input representation is the sum of three learned embeddings: token + segment + position (Figure 13). Segment is specific to BERT.

Example 9.3 (Learned absolute encoding). *BERT-base sets $L_{\text{max}} = 512$. The 6th token of an input sequence (position index 5, zero-indexed) receives the vector P_5 , simply row 5 of the learned 512×768 position-embedding table – a parameter optimised by gradient descent during pretraining, with no fixed mathematical formula. If a 600-token document is fed to BERT-base, positions 512–599 have **no corresponding row** in the table: the model cannot process them at all without truncation or architecture surgery (e.g. interpolating/extending the table and continuing pretraining).*

9.5.3 Relative Position Buckets – T5

T5 (and Transformer-XL before it) drops positional vectors entirely and instead adds a small learned **scalar bias** directly to the raw attention score, based on the relative distance between the query position i and the key position j :

$$\text{sim}_{ij} = \frac{Q_i^\top K_j}{\sqrt{d}} + b_{(j-i)}$$

Since a long sequence has many possible relative distances, T5 **buckets** them logarithmically: each of a handful of nearby offsets ($-1, 0, +1, \dots$) gets its own learned bias, while increasingly distant offsets are grouped into progressively wider shared buckets (e.g. all distances from $+16$ to $+31$ might share one bias). Each attention head has its own set of bucket biases, shared across all layers. No vector is ever added to the embeddings – position only enters through this additive bias inside the softmax, which is why the scheme is so flexible: very distant tokens simply reuse the bias of the largest-magnitude bucket, regardless of sequence length.

Example 9.4 (Relative bucket bias). *Consider a query at position $i = 2$ attending to a key at position $j = 4$. The relative distance is $j - i = +2$, which falls in its own dedicated nearby bucket, so the model adds a single learned scalar b_{+2} (specific to the current attention head) to the dot-product score $Q_2^\top K_4 / \sqrt{d}$ before the softmax. By contrast, a key at $j = 40$ (distance $+38$) would land in a coarse bucket shared with, say, distances $+32$ to $+63$, all of which add the same learned bias $b_{[32,63]}$ – precision is highest for nearby tokens and coarsens gracefully for distant ones.*

9.5.4 Rotary Positional Encoding (RoPE) – Modern Decoder LLMs

RoPE (Su et al., 2021) encodes position by **rotating** the query and key vectors rather than adding anything to them. Split each d -dimensional query/key vector into $d/2$ two-dimensional pairs (x_{2k}, x_{2k+1}) . Each pair is rotated

by an angle proportional to its absolute position pos and to a frequency $\omega_k = 10000^{-2k/d}$ (the same frequency schedule used by the sinusoidal scheme):

$$\begin{pmatrix} x'_{2k} \\ x'_{2k+1} \end{pmatrix} = \begin{pmatrix} \cos(pos \cdot \omega_k) & -\sin(pos \cdot \omega_k) \\ \sin(pos \cdot \omega_k) & \cos(pos \cdot \omega_k) \end{pmatrix} \begin{pmatrix} x_{2k} \\ x_{2k+1} \end{pmatrix}$$

applied to Q_i and K_j before the dot product. A rotation matrix preserves vector length but changes its angle; crucially, the dot product of two rotated vectors depends only on the *difference* between their rotation angles. So $\text{RoPE}(Q_i) \cdot \text{RoPE}(K_j)$ depends only on the relative position $i - j$, even though each vector was rotated using its own *absolute* position. This combines the generalisation benefits of relative encodings with the computational simplicity of a per-token operation – no extra learned parameters, no separate bias term, no lookup table.

Example 9.5 (RoPE rotation). Take a single 2D sub-pair $(x_0, x_1) = (1, 0)$ at position $pos = 2$ with frequency $\omega_0 = 1$, so the rotation angle is $\theta = 2 \cdot 1 = 2$ radians:

$$x'_0 = \cos(2) \cdot 1 - \sin(2) \cdot 0 \approx -0.416, \quad x'_1 = \sin(2) \cdot 1 + \cos(2) \cdot 0 \approx 0.909$$

The pair $(1, 0)$ rotates to $(-0.416, 0.909)$ – same length, different angle. A key vector’s pair at a different position rotates by a different angle; only the angular difference between a query’s and key’s rotation survives in their dot product, which is exactly the relative-position signal the model needs. Because rotation is defined for any real-valued angle, RoPE imposes no hard maximum sequence length, and techniques such as “RoPE scaling”/NTK-aware interpolation are routinely used to extend pretrained LLMs (e.g. LLaMA) to context windows several times longer than they were originally trained on.

9.6 Subword Encoding Tokenization and Embedding

Fixed word-level vocabularies struggle with morphologically rich languages (e.g. Slovene) and rare or unseen words. **Subword tokenization** addresses this by learning a vocabulary of *subword pieces* directly from the corpus statistics — tokens are often parts of words, such as common morphemes like *-er* or *-est* (e.g. *unlikeliest* = *un-* + *likely* + *-est*). This approach is relevant for all modern neural NLP pipelines and is crucial for morphologically rich languages.

Definition 9.1 (Subword tokenization). A subword tokenization system has two components:

- **Token learner:** takes a raw training corpus and induces a vocabulary (a set of tokens).
- **Token segmenter:** takes a raw test sentence and tokenizes it according to that vocabulary.

The two main algorithms are **Byte Pair Encoding** (BPE; Sennrich et al., 2016) and **WordPiece** (Schuster & Nakajima, 2012).

Definition 9.2 (Byte Pair Encoding (BPE)). **Token Learner.** Before running BPE, each word in the corpus is split into characters and an end-of-word marker $_$ is appended (e.g. *low* $\rightarrow$ *l o w _*). The initial vocabulary V is the set of all unique characters. The algorithm then repeats k times:

1. Find the most frequent adjacent pair (t_L, t_R) in the corpus.
2. Create a new token $t_{new} = t_L + t_R$ and add it to V .
3. Replace every occurrence of (t_L, t_R) in the corpus with t_{new} .

Token Segmenter. To tokenize a test word, split it into characters, then apply the learned merge rules greedily in the order they were learned (test-set frequencies play no role).

Example 9.6 (BPE token learner). Consider a small corpus: *low* $\times 5$, *lowest* $\times 2$, *newer* $\times 6$, *wider* $\times 3$, *new* $\times 2$. After adding end-of-word markers the initial vocabulary is $\{_, d, e, i, l, n, o, r, s, t, w\}$. The first merges proceed as follows:

Merge	Vocabulary after merge
<i>e r</i> $\rightarrow$ <i>er</i>	$\dots, er$
<i>er _</i> $\rightarrow$ <i>er_</i>	$\dots, er, er_$
<i>n e</i> $\rightarrow$ <i>ne</i>	$\dots, ne$
<i>ne w</i> $\rightarrow$ <i>new</i>	$\dots, new$
<i>l o</i> $\rightarrow$ <i>lo</i>	$\dots, lo$
<i>lo w</i> $\rightarrow$ <i>low</i>	$\dots, low$
<i>new er_</i> $\rightarrow$ <i>newer_</i>	$\dots, newer_$
<i>low _</i> $\rightarrow$ <i>low_</i>	$\dots, low_$

At the end, the test word *newer* is tokenized as a single token *newer_*, while the unseen word *lower* splits into *low* + *er_* because *lower_* was never merged.

Remark 9.9. BPE is not limited to NLP: in music generation, the same algorithm is applied with a vocabulary of MIDI pitches, pauses, and velocities.

WordPiece (Schuster & Nakajima, 2012). WordPiece is similar to BPE but chooses merges differently: instead of picking the most *frequent* pair, it picks the pair that *maximises the likelihood* of the training data. In practice this is approximated by the mutual-information-like score:

$$\text{score}(u_i, u_j) = \frac{\text{freq}(u_i u_j)}{\text{freq}(u_i) \times \text{freq}(u_j)} \quad (56)$$

A high score means the pair co-occurs far more than expected by chance. Unlike BPE, which runs for a fixed number k of merges, WordPiece optimises the likelihood of the language model’s internal representations, making it slightly more principled in how it handles rare vs. common pairs. WordPiece is used in BERT; BPE (or its byte-level variant) is used in GPT models.

Practical consequence: WordPiece tends to produce more linguistically meaningful subwords because it avoids merging tokens that are frequent simply because their parts are common. BPE is greedier about raw counts.

9.7 Token Embedding Layer in Autoregressive Decoders (GPT)

Autoregressive models like GPT do not rely on external pre-trained embedding networks (such as BERT or Word2Vec). Instead, they utilize an internal, end-to-end learnable token embedding matrix to map discrete tokens directly into the continuous hidden space.

Let $|V|$ denote the vocabulary size of the model’s tokenizer, and let $d_{\text{model}} = 512$ represent the hidden dimension. The model initializes a weight token embedding matrix:

$$E \in \mathbb{R}^{|V| \times d_{\text{model}}} \quad (57)$$

When an input sequence is processed, each token is mapped to its corresponding vocabulary index $y_t \in \{0, 1, \dots, |V| - 1\}$. The continuous vector representation is extracted via a standard row-lookup operation, equivalent to multiplying a one-hot vector $\mathbf{v}_t$ by the embedding matrix:

$$\mathbf{x}_t = \mathbf{v}_t E = E[y_t, :] \quad (58)$$

The row vector $\mathbf{x}_t \in \mathbb{R}^{512}$ serves as the base semantic representation. It is then added to the positional encoding matrix before being passed upward through successive Transformer Decoder blocks. The weights within E are optimized globally via backpropagation, allowing the model to organically learn semantic relationships from raw sequence data.

9.7.1 Computational Complexity Analysis

The computational complexity of a standard Transformer block is dominated by two components: the Multi-Head Self-Attention sub-layer and the Position-Wise Feed-Forward Network (FFN). Let N represent the sequence length (number of tokens) and let D represent the hidden model dimension (d_{model}).

1. **Self-Attention Layer:** Computing the query, key, and value linear projections takes $\mathcal{O}(N \cdot D^2)$ steps. Calculating the attention score matrix via the inner product QK^T requires multiplying matrices of shape $[N \times D]$ and $[D \times N]$, yielding a runtime cost of $\mathcal{O}(N^2 \cdot D)$. Computing the final weighted sum $A \cdot V$ adds another $\mathcal{O}(N^2 \cdot D)$ operations. Thus, the total complexity of the self-attention layer is:

$$\mathcal{O}(N^2 \cdot D + N \cdot D^2) \quad (59)$$

2. **Position-Wise Feed-Forward Network:** The FFN applies two linear projections to each token independently, expanding the inner layer to a dimension of $4D$. This operation scales linearly with sequence length:

$$\mathcal{O}(N \cdot D^2) \quad (60)$$

Table 5: Complexity per Transformer Sub-Layer Module

Sub-Layer Type	Computational Complexity	Primary Operational Scaling
Self-Attention Layer	$\mathcal{O}(N^2 \cdot D + N \cdot D^2)$	Quadratic with sequence length N
Feed-Forward (FFN)	$\mathcal{O}(N \cdot D^2)$	Linear with sequence length N

The Quadratic Sequence Bottleneck: When the sequence length N is small relative to the feature dimension D ($N \ll D$), the computational profile is dominated by the linear matrix projections, scaling as $\mathcal{O}(N \cdot D^2)$. However, as the sequence length expands ($N \gg D$), the quadratic term $\mathcal{O}(N^2 \cdot D)$ becomes the primary computational and memory bottleneck.

10 BERT and T5 models

10.1 Pretrained Language Models

Modern NLP is built around large pretrained neural language models: models trained on massive text corpora to capture statistical structure and relationships in language. The key workflow is:

1. **Pretraining:** Train a large model on a huge corpus (e.g., Wikipedia) using a self-supervised task – no human labels needed.
2. **Fine-tuning:** Take that pretrained model and adapt it to a specific downstream task (e.g., sentiment analysis, named entity recognition) using a much smaller labeled dataset.

This “pretrain once, fine-tune many times” paradigm was revolutionary because it allows practitioners to benefit from expensive large-scale training without doing it themselves. Many such models are publicly available on platforms like HuggingFace and Clarin.si.

10.2 BERT

Definition 10.1 (BERT). ***BERT** (Bidirectional Encoder Representation from Transformers, Devlin et al., 2019) is a pretrained language model based on the **encoder** part of the Transformer architecture. Unlike earlier language models that processed text from left to right (or right to left), BERT reads the entire sentence at once – in both directions simultaneously.*

Core idea: Use large unlabeled text corpora and a self-supervised task (word masking) to pretrain a model that learns general language representations. Then fine-tune it on a specific task using a small labeled dataset.

10.2.1 Motivation: Why Bidirectional?

Classical language models process text in one direction only. But human language understanding is inherently bidirectional. Consider:

“The man, who visited, was **aggressive**.”

To understand how the man is described, you need to read all the way to *aggressive*. This forward context is unavailable if you only look left.

Why were language models unidirectional in the first place? Two reasons:

1. **Probability distribution:** A standard language model predicts $P(w_t \mid w_1, \dots, w_{t-1})$ – directionality avoids trivial solutions. BERT sidesteps this entirely by not generating text.
2. **Words “see themselves”:** In a bidirectional model without masking, when predicting a word the model could trivially attend to that very word, making the task meaningless.

BERT’s solution is **Masked Language Modeling (MLM)**: randomly hide some words and train the model to predict them using *all* the surrounding context.

Observation 10.1. *Too little masking means too few training signals per pass, making training expensive. Too much masking leaves too little context, making prediction impossible. BERT uses $k = 15\%$ as a practical balance.*

10.2.2 BERT Architecture

BERT uses the **encoder** portion of the Transformer (no decoder). Inputs flow through multiple stacked self-attention layers, and every token can attend to every other token simultaneously – there is no left-to-right constraint.

Key architectural properties:

- **Self-attention $\Rightarrow$ no locality bias:** Unlike convolutions or RNNs, there is no preference for nearby words.
- **Long-distance context has “equal opportunity”:** A word at position 1 and a word at position 200 are treated symmetrically.
- **Efficient on modern hardware:** The self-attention computation is a single large matrix multiplication per layer, well-suited to GPU/TPU parallelism.

10.2.3 Input Representation: Subword Encoding

BERT does not operate on whole words. Instead it uses **WordPiece tokenization**, a variant of Byte-Pair Encoding (BPE): common words are kept whole, while rare or unknown words are split into sub-word pieces.

Example 10.1. *The word hypatia is not in the vocabulary, so it becomes: `h ##yp ##ati ##a`. Common words like at, fairfax, 1910s appear as single tokens. The `##` prefix signals a continuation piece within a word.*

For any given input token i , the combined input representation E_i is the sum of three embeddings:

$$E_i = \text{TokenEmbedding}(T_i) + \text{SegmentEmbedding}(S_i) + \text{PositionEmbedding}(P_i)$$

- T_i : The WordPiece token (e.g., [CLS], “playing”, [SEP]).
- S_i : The segment ID (0 for Sentence A, 1 for Sentence B) – tells the model which sentence a token belongs to.
- P_i : The position of the token in the sequence $(0, 1, \dots, n)$.

The full input for a sentence pair is formatted as:

$$\text{Input} = [\text{CLS}], w_{1,A}, \dots, w_{n,A}, [\text{SEP}], w_{1,B}, \dots, w_{m,B}, [\text{SEP}]$$

The [CLS] token at the start accumulates a summary representation of the whole input and is used for classification tasks. The [SEP] token delimits sentence boundaries.

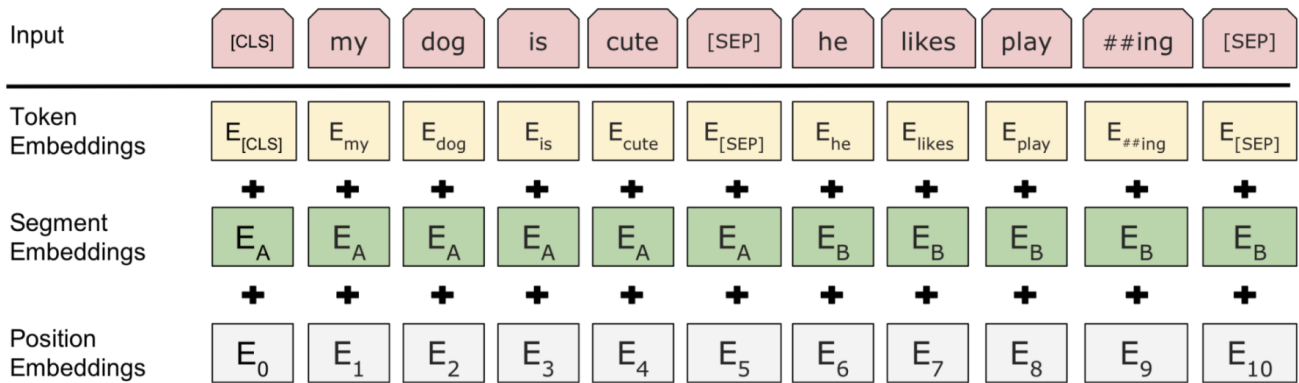


Figure 13: BERT input representation: each token embedding is the sum of token, segment, and position embeddings.

10.2.4 Masked Language Modeling (MLM)

BERT’s primary pretraining task is to predict randomly masked words using context from both sides.

Of the 15% of tokens selected for masking, each is handled as follows:

- 80% are replaced with the special [MASK] token.
- 10% are replaced with a random word from the vocabulary.
- 10% are left unchanged.

Example 10.2. *Suppose “delicious” is the selected word in the sentence “Lunch was delicious”. The three cases produce:*

- 80%: *Lunch was [MASK] → model must predict “delicious”*
- 10%: *Lunch was gasp → random replacement (model must correct it)*
- 10%: *Lunch was delicious → unchanged (model must verify it)*

This mixture prevents the model from only reacting to [MASK] tokens, forcing it to develop a richer, context-aware representation of all tokens.

This avoids a train/fine-tune mismatch: [MASK] never appears during fine-tuning, so always masking would create a distribution shift.

Prediction head. After the masked input passes through all encoder layers, each token position i yields a contextual hidden state $\mathbf{h}_i \in \mathbb{R}^d$ (where d is the hidden size, e.g. 768 for BERT-Base). For each masked position, a linear *prediction head* projects $\mathbf{h}_i$ to a score vector over the entire vocabulary:

$$\mathbf{z}_i = W_{\text{MLM}} \mathbf{h}_i + \mathbf{b}, \quad W_{\text{MLM}} \in \mathbb{R}^{|V| \times d}, \quad \mathbf{z}_i \in \mathbb{R}^{|V|}$$

A softmax then converts these raw scores into a probability distribution over all $|V|$ tokens:

$$P(w \mid i, \tilde{w}) = \frac{\exp(z_{i,w})}{\sum_{w' \in V} \exp(z_{i,w'})}$$

This is a $|V|$ -way classification problem – for BERT-Base, $|V| = 30,522$. The output is one probability per vocabulary token; the model is trained to maximise the probability assigned to the *original* (pre-masking) token at each masked position.

The MLM loss is then computed only over the masked positions $\mathcal{M}$:

$$\mathcal{L}_{\text{MLM}} = -\frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \log P(w_i \mid \tilde{w})$$

where $\tilde{w}$ is the masked input sequence and w_i is the original token at position i .

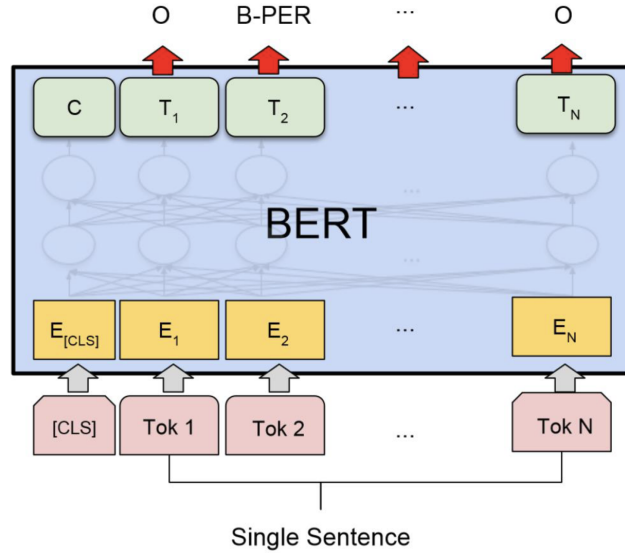


Figure 14: BERT training using masked language modeling (MLM) and word prediction.

10.2.5 Next Sentence Prediction (NSP)

BERT also learns relationships between pairs of sentences and predicts whether or not they are related.

Example 10.3. Given sentences A and B , the model predicts whether B is the actual next sentence in the text (*IsNext*) or a randomly sampled unrelated sentence (*NotNext*). 50% of pairs are genuine; 50% are random.

- Sentence $A = \text{“The man went to the store.”} / B = \text{“He bought a gallon of milk.”} \rightarrow \textbf{IsNext}$
- Sentence $A = \text{“The man went to the store.”} / B = \text{“Penguins are flightless.”} \rightarrow \textbf{NotNext}$

How the architecture changes. The input is formatted as $[\text{CLS}] A [\text{SEP}] B [\text{SEP}]$ and passed through the same encoder. However, instead of reading the hidden state $\mathbf{h}_i$ at each masked position, we only look at the hidden state of the $[\text{CLS}]$ token, $\mathbf{C} \in \mathbb{R}^d$, which accumulates a global summary of the whole input pair. The prediction head now projects to just two classes instead of $|V|$:

$$\mathbf{z}_{\text{NSP}} = W_{\text{NSP}} \mathbf{C} + \mathbf{b}_{\text{NSP}}, \quad W_{\text{NSP}} \in \mathbb{R}^{2 \times d}$$

$$\begin{pmatrix} P(\text{IsNext}) \\ P(\text{NotNext}) \end{pmatrix} = \text{softmax}(\mathbf{z}_{\text{NSP}}) \in \mathbb{R}^2$$

So the output is a vector of two probabilities that sum to 1 – a binary classifier, not a 30k-way vocabulary classifier.

The [CLS] token’s final hidden state $\mathbf{C} \in \mathbb{R}^d$ is thus the only position used for this task. The NSP loss is a standard binary cross-entropy:

$$\mathcal{L}_{\text{NSP}} = -\log P(y \mid \mathbf{C})$$

where $y \in \{\text{IsNext}, \text{NotNext}\}$ is the true label.

The total pretraining loss combines both objectives:

$$\mathcal{L} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{NSP}}$$

Remark 10.1. *Later BERT variants such as **RoBERTa** removed NSP, increased the training set size, and increased training time. This yielded better downstream performance, suggesting NSP was not essential.*

10.2.6 Model Sizes and Training

BERT comes in two configurations:

- **BERT-Base:** 12 encoder layers, 12 attention heads, hidden size 768, $\approx 110\text{M}$ parameters.
- **BERT-Large:** 24 encoder layers, 16 attention heads, hidden size 1024, $\approx 340\text{M}$ parameters.

Training data: English Wikipedia (2.5B words) + BookCorpus (800M words). The original model was trained on a 4×4 or 8×8 TPU slice for 4 days – a significant computational investment at the time of release (2019).

Remark 10.2. *Building your own BERT-like model requires roughly 6B tokens of training text, $\sim 100\text{M}$ parameters, and a cluster of GPUs – feasible but non-trivial. We can use BERT for many downstream tasks: named entity recognition, question answering, sentiment analysis, and more.*

10.3 Fine-tuning BERT for Downstream Tasks

After pre-training on massive unlabeled text corpora using Masked Language Modeling (MLM) and Next Sentence Prediction (NSP), the BERT architecture is fine-tuned end-to-end for specific downstream tasks by appending a minimal, task-specific linear output layer. The foundational architectural value of the Transformer Encoder is its ability to retain its parameter backbone entirely intact across highly disparate problem spaces. Rather than altering the deep contextualization layers, downstream adaptability is achieved simply by altering how we format the initial input matrix $X_{\text{input}} \in \mathbb{R}^{T \times d_{\text{model}}}$ and how we selectively sample or pool from the final output grid $X_{\text{out}} \in \mathbb{R}^{T \times d_{\text{model}}}$.

- **Sentence Classification** (e.g., Sentiment Analysis, Grammatical Correctness)

- **Input Formatting:** The single text sequence is prefixed with the classification marker: $X_{\text{input}} = [\text{[CLS]}, \text{sentence}, \text{[SEP]}]$ of sequence length T .
- **Data Flow & Slicing Strategy:** The sequence matrix is processed through all bidirectional attention layers, yielding the terminal contextualized tensor $X_{\text{out}} \in \mathbb{R}^{T \times d_{\text{model}}}$. Because self-attention allows all tokens to interact without causal constraints, the first row corresponding to the global pooling token position aggregates a holistic structural summary. The model discards rows 1 through $T - 1$ and slices out solely row 0:

$$C = X_{\text{out}}[0, :] \in \mathbb{R}^{d_{\text{model}}} \quad (61)$$

- **Classification Head:** For a classification task containing K unique categories, a linear layer maps the summary vector directly into the target label space:

$$\hat{y} = \text{Softmax}(W_{\text{class}} \cdot C + b_{\text{class}}) \quad \text{where } W_{\text{class}} \in \mathbb{R}^{K \times d_{\text{model}}} \quad (62)$$

- **Natural Language Inference / Textual Entailment**

- **Input Formatting:** To evaluate cross-sentence logic, the premise and hypothesis are packaged as a single sequence delimited by separation boundary markers: $X_{\text{input}} = [\text{[CLS]}, \text{Premise}, \text{[SEP]}, \text{Hypothesis}, \text{[SEP]}]$ of total length T .
- **Data Flow & Segment Conditioning:** Segment Embedding A is added to the premise token representations, and Segment Embedding B is added to the hypothesis token representations. This uniform shift in vector space enables attention heads to track sentence boundaries. Throughout the encoder layers, the hypothesis tokens deeply cross-examine the premise tokens.
- **Slicing & Classification Head:** The global pooling vector is extracted from the terminal encoder matrix: $C = X_{\text{out}}[0, :] \in \mathbb{R}^{d_{\text{model}}}$. Because NLI dictates a 3-way distribution (Entailment, Contradiction, Neutral), a specialized classification head evaluates the pooled interaction:

$$\hat{y}_{\text{NLI}} = \text{Softmax}(W_{\text{NLI}} \cdot C + b_{\text{NLI}}) \quad \text{where } W_{\text{NLI}} \in \mathbb{R}^{3 \times d_{\text{model}}} \quad (63)$$

- **Sentence Similarity (Order-Inversion Ensembling)**

- **Geometric Rationalization:** Semantic similarity must be mathematically symmetric: $\text{sim}(S_1, S_2) = \text{sim}(S_2, S_1)$. However, because absolute positional encodings and segment identifiers alter token coordinates depending on sequence order, passing a pair through a single configuration introduces positional bias.
- **Symmetric Processing Execution:** To eliminate order-dependent variance, the sentence pair is encoded across two opposite permutations within the same training block:

$$C_A = \text{BERT}([\text{CLS}], S_1, [\text{SEP}], S_2, [\text{SEP}])[0, :] \in \mathbb{R}^{d_{\text{model}}} \quad (64)$$

$$C_B = \text{BERT}([\text{CLS}], S_2, [\text{SEP}], S_1, [\text{SEP}])[0, :] \in \mathbb{R}^{d_{\text{model}}} \quad (65)$$

The vectors are summed element-wise to create a joint metric $C_{\text{symmetric}} = C_A + C_B$. Because vector addition is commutative, the pooled representation becomes completely invariant to input sequence placement.

- **Classification Head:** A linear regression or classification layer scales the order-invariant representation:

$$\hat{y}_{\text{sim}} = W_{\text{sim}} \cdot (C_A + C_B) + b_{\text{sim}} \quad \text{where } W_{\text{sim}} \in \mathbb{R}^{\text{num_classes} \times d_{\text{model}}} \quad (66)$$

- **Multiple Choice Framework**

- **Input Formatting:** Given a context question Q and K separate potential candidate answers $\{A_1, A_2, \dots, A_K\}$, the system constructs K independent sequence pairs:

$$X_{\text{input}}^{(k)} = [\text{CLS}], Q, [\text{SEP}], A_k, [\text{SEP}] \quad \forall k \in \{1, \dots, K\} \quad (67)$$

- **Data Flow & Shared Scaling:** All K sequences are passed separately through the encoder parameters. For each candidate matrix, row 0 is sliced out to extract independent compatibility representations: $C_k = X_{\text{out}}^{(k)}[0, :] \in \mathbb{R}^{d_{\text{model}}}$.
- **Categorical Selection Head:** A single, shared linear projection vector $\mathbf{w}_{\text{mc}} \in \mathbb{R}^{1 \times d_{\text{model}}}$ maps each candidate vector down to a scalar logit $s_k = \mathbf{w}_{\text{mc}} \cdot C_k + b_{\text{mc}}$. Rather than evaluating choices independently, a softmax operation is executed across the choice dimension to resolve the selection comparatively:

$$P(\text{Choice } k \mid Q) = \frac{\exp(s_k)}{\sum_{j=1}^K \exp(s_j)} \quad (68)$$

- **Sequence Labeling (NER, POS Tagging, Semantic Role Labeling – SRL)**

- **Input Formatting:** Text sequences are input normally: $X_{\text{input}} = [\text{CLS}], \text{tok}_1, \text{tok}_2, \dots, \text{tok}_N, [\text{SEP}]$.
- **Data Flow & Matrix Retention:** Unlike pooling setups that isolate a single row, token labeling tasks retain the full sequence matrix output: $X_{\text{out}} \in \mathbb{R}^{T \times d_{\text{model}}}$. Every row vector $h_t = X_{\text{out}}[t, :]$ represents a 512-dimensional continuous encoding of that specific word position blended with its full bidirectional context. High-dimensional vector spaces are critical here; they provide orthogonal basis vectors that allow individual labels to remain independent, bypassing the ordinal and proximity biases caused by forcing discrete concepts into low-dimensional scalar markers.
- **Parallel Token Projections:** A linear projection layer $W_{\text{label}} \in \mathbb{R}^{\text{num_labels} \times d_{\text{model}}}$ operates on every row index simultaneously:

$$\hat{y}_t = \text{Softmax}(W_{\text{label}} \cdot h_t + b_{\text{label}}) \quad \forall t \in \{1, \dots, T\} \quad (69)$$

- **Predicate Conditioning in SRL:** To map roles relative to a specific verb, a binary predicate indicator embedding layer maps the target verb position to a focus modifier vector, shifting the attention layers to calculate contextual dependencies relative to that specific action node.

- **Question Answering (e.g., Extractive Span Selection on SQuAD)**

- **Input Formatting:** The prompt pairs the query against a reference passage: $X_{\text{input}} = [\text{CLS}], \text{Question}, [\text{SEP}], \text{Passage}$.
- **Boundary Pointer Design:** The architecture utilizes the full sequence matrix $X_{\text{out}} \in \mathbb{R}^{T \times d_{\text{model}}}$ and introduces two separate, learnable template pointer vectors: $\mathbf{w}_{\text{start}} \in \mathbb{R}^{d_{\text{model}}}$ and $\mathbf{w}_{\text{end}} \in \mathbb{R}^{d_{\text{model}}}$.

- **Span Probability Selection:** Every token vector $h_t = X_{\text{out}}[t, :]$ is evaluated against the pointers using dot products. A softmax calculation is then run across the complete sequence length T to yield two distinct probability distributions:

$$P_{\text{start}}(t) = \frac{\exp(\mathbf{w}_{\text{start}} \cdot h_t)}{\sum_{j=1}^T \exp(\mathbf{w}_{\text{start}} \cdot h_j)} \quad (70)$$

$$P_{\text{end}}(t) = \frac{\exp(\mathbf{w}_{\text{end}} \cdot h_t)}{\sum_{j=1}^T \exp(\mathbf{w}_{\text{end}} \cdot h_j)} \quad (71)$$

The total loss is optimized as the joint sum of cross-entropy metrics across both boundaries: $\mathcal{L}_{\text{QA}} = \mathcal{L}_{\text{start}} + \mathcal{L}_{\text{end}}$. At inference, the exact string span is isolated by computing $\arg \max_{s \leq e} (P_{\text{start}}(s) \times P_{\text{end}}(e))$.

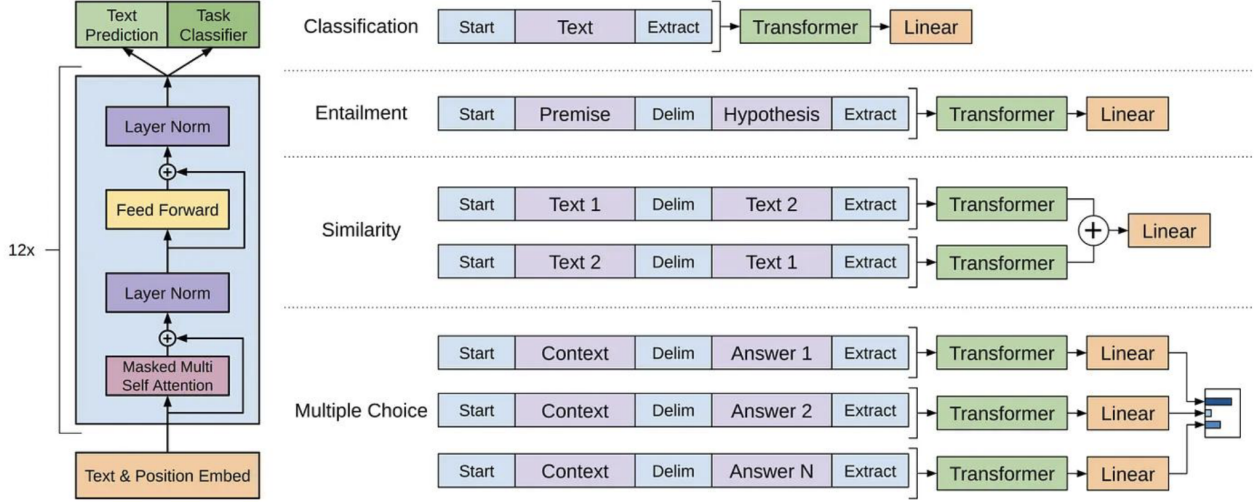


Figure 15: Fine-tuning BERT for different task types: classification, entailment, similarity, and multiple choice.

10.4 BERT as Contextualized Embeddings

Instead of fine-tuning all of BERT’s parameters, one can use BERT as a **feature extractor**: extract fixed-size contextualized vectors from BERT’s hidden layers and feed them into a separate downstream model. This achieves slightly lower accuracy than full fine-tuning but can be much faster and more lightweight.

Each of BERT’s layers produces a different representation of each token. Which layer is most useful?

Token representations evaluated on the CoNLL-2003 NER task (Dev F1):

- Concatenation of last 4 hidden layers: **96.1** (best)
- Sum of last 4 hidden layers: 95.9
- Second-to-last hidden layer: 95.6
- Sum of all 12 layers: 95.5
- Last hidden layer only: 94.9
- First layer (input embedding): 91.0

Combining information from the last several layers works best, as lower layers tend to capture syntactic structure while higher layers capture more task-relevant semantics.

10.5 Evaluating Language Understanding: GLUE and SuperGLUE

Definition 10.2 (GLUE). *GLUE* (General Language Understanding Evaluation) is a benchmark collection of nine NLP tasks used to evaluate how well a model “understands” English. Before GLUE, models were evaluated on single tasks in isolation. GLUE forces a model to perform well across diverse challenges – natural language inference, sentiment, grammar, sentence similarity – proving it has genuinely learned language rather than overfitting one task.

Example 10.4 (GLUE tasks). We present some GLUE tasks here:

What is the best contextualized embedding for “Help” in that context?
For named-entity recognition task CoNLL-2003 NER

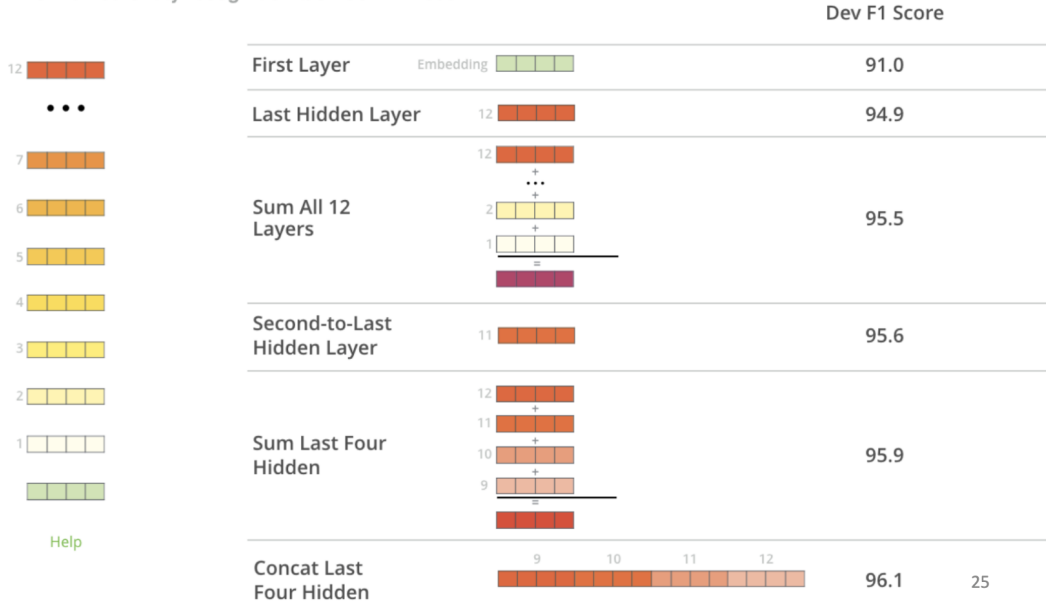


Figure 16: Layer-wise contextualized embeddings: each of BERT’s 12 encoder layers outputs a different representation per token. The output of any layer (or combination of layers) can be used as a feature vector.

- **MultiNLI** (Natural Language Inference): given a premise and hypothesis, label their relationship.
 - Premise: “Hills and mountains are especially sanctified in Jainism.”
 - Hypothesis: “Jainism hates nature.”
 - Label: **Contradiction**
- **CoLA** (Corpus of Linguistic Acceptability): judge whether a sentence is grammatically acceptable.
 - “The wagon rumbled down the road.” → **Acceptable**
 - “The car honked down the road.” → **Unacceptable**

SuperGLUE is a harder follow-up benchmark with more challenging tasks:

- **BoolQ**: boolean questions requiring a yes/no answer from a passage.
- **CB** (Commitment Bank): determine entailment degree in multi-sentence dialogue.
- **COPA**: given a premise, choose the more plausible cause or effect from two alternatives.
- **MultiRC**: multi-sentence reading comprehension with multiple correct answers.
- **ReCoRD**: reading comprehension with commonsense reasoning (fill-in-the-blank from a passage).
- **RTE**: recognizing textual entailment (binary: entails or not).
- **WiC**: determine if a word is used in the same sense across two different sentences.
- **WSC** (Winograd Schema Challenge): resolve pronoun coreference using commonsense reasoning.

Example 10.5 (SuperGLUE examples). *Some SuperGLUE examples:*

- **BoolQ**: passage about Barq’s root beer + question “is barq’s root beer a Pepsi product?” → **No**
- **COPA**: “My body cast a shadow over the grass.” What is the CAUSE? Alt 1: “The sun was rising.” / Alt 2: “The grass was cut.” → **Alternative 1**
- **WiC**: “Room and board.” vs. “He nailed boards across the windows.” – same sense? → **False**
- **WSC**: “Mark told Pete many lies about himself, which he included in his book.” – does “he” refer to Pete? → **False**

10.6 Macro-Evaluation via the GLUE and SuperGLUE Benchmarks

To measure the generalizable natural language understanding (NLU) capabilities of an encoder architecture like BERT, systems are evaluated on standardized multi-task suites, primarily the General Language Understanding Evaluation (GLUE) and its advanced successor, SuperGLUE.

1. **Benchmark Architecture and Scope:** Rather than representing a singular mathematical metric, these benchmarks are collections of diverse NLU tasks spanning single-sentence syntax acceptability, semantic similarity, coreference resolution, and complex logical inference.
2. **Evaluation Methodology:** A pre-trained encoder backbone cannot generate task predictions autonomously. Evaluating a model requires independently training a specialized, minimal task head (e.g., W_{class} , W_{NLI} , or W_{sim}) on top of the pre-trained weights for each constituent dataset.
3. **Scoring and Metric Range:** Each distinct sub-task computes performance via task-appropriate base metrics (such as classification Accuracy, F1-score, Matthews Correlation Coefficient, or Pearson Correlation). Consequently, individual scores are bounded strictly on a percentage scale matching a continuous metric range:

$$\text{Score}_{\text{task}} \in [0, 100] \quad (72)$$

The ultimate benchmark summary performance—the global GLUE or SuperGLUE score—is calculated as the unweighted macro-average of all component task achievements:

$$\text{Score}_{\text{Benchmark}} = \frac{1}{M} \sum_{m=1}^M \text{Score}_{\text{task}_m} \quad (73)$$

This evaluation framework forces models to prioritize robust, transferrable latent word representations during pre-training, as overfitting to a single structural head will result in degraded performance across the remaining tasks of the macro-average suite.

10.7 Pretrained Models for Different Languages

The pretrain-then-fine-tune paradigm has been applied to many languages beyond English.

Example 10.6 (Slovene and Croatian models). *For Slovene and Croatian languages there exists:*

- **Slovene:** *fastText*, *ELMo*, *SloBERTa*, *CroSloEngual BERT*, *SlEng BERT*, *SloT5*, *Slo GPT*.
- **Croatian:** *fastText*, *ELMo*, *BERTic*, *CroSloEngual BERT*.

These models are available on Clarin.si and HuggingFace. Hundreds of papers at major ML/NLP conferences have investigated BERT-like models for diverse languages.

SloBERTa is currently the best Slovene BERT-like LLM:

- Training set: 3.41 billion words from corpora Gigafida, KAS, partially Janes, siParl, and slWaC.
- Training duration: 4 weeks on an Nvidia DGX A100 server using 4 GPUs.
- Many thousands of downloads from HuggingFace.

Example 10.7 (What LLMs learn: masked token prediction). *For the sentence: “We would like to travel to [MASK], ki je najlepši otok v Mediteranu.” (“...which is the most beautiful island in the Mediterranean.”), different models give strikingly different predictions:*

- **SloBERTa:** *Slovenija, I, Koper, Slovenia – the model knows Slovene but not Mediterranean geography.*
- **CSE-BERT** (Croatian/Slovene/English): *Hvar, Rab, Cres, Malta, Brač – Adriatic islands, sensible!*
- **XLM-R:** *Mallorca, Tenerife, otok, Ibiza, Zadar – reasonable Mediterranean islands.*
- **mBERT:** *Ibiza, Gibraltar, Tenerife, Mediterranean, Madeira.*
- **BERT (English-only):** *Belgrade, Italy, Serbia, Prague, Sarajevo – wrong language context entirely.*

This illustrates how a trilingual model trained on closely related languages (CSE-BERT) can outperform a monolingual one on a mixed-language sentence, even if the monolingual model is generally stronger.

10.7.1 Vocabulary and Tokenization Quality

Tokenization quality matters significantly for less-resourced languages. The vocabulary is built statistically from training data (typically BPE or a variant), so a language-specific model will segment its target language much more naturally.

Example 10.8 (Tokenization of a Slovene sentence). *Sentence: “Letenje je bilo predmet precej starodavnih zgodb.”*

- **SloBERTa**: ‘_Le’, ‘_tenje’, ‘_je’, ‘_bilo’, ‘_predmet’, ‘_precej’, ‘_staroda’, ‘_vnihi’, ‘_zgodb’, ‘.’ – 10 meaningful tokens.
- **mBERT**: ‘_Let’, ‘_##en’, ‘_##je’, ‘_je’, ‘_bilo’, ‘_pred’, ‘_##met’, ‘_pre’, ‘_##cej’, ‘_star’, ‘_##oda’, ‘_##vnihi’, ‘_z’, ‘_##go’, ‘_##d’, ‘_##b’, ‘.’ – 17 fragmented tokens.

SloBERTa produces fewer, more linguistically meaningful pieces because its vocabulary was built from Slovene text, not 100 languages simultaneously.

10.8 Cross-Lingual Transfer

10.8.1 Classical Cross-Lingual Transfer

The original approach to applying NLP models across languages was explicit **vector space alignment**: find a transformation matrix W such that the source language word space $WS \approx E$ (the target language embedding space). This required parallel corpora and manual alignment steps. Today, multilingual LLMs handle cross-lingual transfer directly without such alignment.

10.8.2 Multilingual LLMs

Multilingual models are pretrained on text from many languages simultaneously, learning a shared representation space:

- **mBERT**: multilingual BERT supporting 104 languages, trained on Wikipedia in each language.
- **XLNet**: trained on 2.5 TB of web text from 100 languages; stronger than mBERT on most tasks.
- These models enable **zero-shot cross-lingual transfer**: fine-tune on a high-resource language (e.g., English), then evaluate on a low-resource one (e.g., Slovene) – with no task-specific data in that language.
- Practical solution to the lack of labeled data for less-resourced languages.

Remark 10.3 (Curse of multilinguality). *Adding more languages to a model with fixed capacity means each language gets less representational “space”. Performance on individual languages typically drops as the number of languages increases. For Slovene tasks, the monolingual SloBERTa consistently outperforms both mBERT and XLNet.*

Model	NER	POS	DP	SA	WA
mBERT	0.885	0.984	0.681	0.576	0.061
XLNet	0.912	0.988	0.793	0.604	0.146
SloBERTa	0.933	0.991	0.844	0.623	0.405

NER = Named Entity Recognition, POS = Part-of-Speech tagging, DP = Dependency Parsing, SA = Sentiment Analysis, WA = Word Analogy.

10.8.3 Trilingual Models

A practical middle ground: train BERT on only 2–3 closely related languages. This gives:

- More training data per language (capacity not diluted across 100 languages).
- A more specific vocabulary – better tokenization for those languages.
- Strong cross-lingual transfer *within* that language group.

Examples: **CroSloEngual BERT** (Croatian/Slovene/English), **FinEst BERT** (Finnish/Estonian), **LitLat BERT** (Lithuanian/Latvian), **SlavBERT** (Russian/Polish/Czech/Bulgarian).

Performance on Slovene tasks including the trilingual CSE-BERT:

Model	NER	POS	DP	SA	WA
mBERT	0.885	0.984	0.681	0.576	0.061
XLM-R	0.912	0.988	0.793	0.604	0.146
CSE-BERT	0.928	0.990	0.854	0.610	0.195
SloBERTa	0.933	0.991	0.844	0.623	0.405

Remark 10.4. *Cross-lingual transfer quality is task-dependent. Sentiment analysis transfers well between Slovene and Croatian (closely related languages); other tasks may show larger gaps. In general, the more similar the languages, the more effective the transfer.*

10.9 Parameter-Efficient Fine-Tuning

Full fine-tuning updates *all* parameters of a large model for each new task – expensive in both compute and storage (one full copy of the model per task). Parameter-efficient fine-tuning (PEFT) methods keep the pretrained weights frozen and train only a small number of new parameters (e.g. adapters, prefix-tuning, LoRA, QLoRA). These methods are covered in detail in Section 19 (Chapter 19), since they apply equally to encoder models like BERT and to decoder-only LLMs.

10.10 Attention Efficiency and Long Sequences

Standard self-attention has time and space complexity $O(n^2)$ in the sequence length n : every token attends to every other token. This becomes prohibitive for very long inputs such as full documents, character-level text, images (as sequences of pixels), or protein sequences. **Longformer** (Beltagy et al., 2020) replaces full attention with efficient local patterns that scale as $O(n)$:

- **Sliding window attention:** Each token attends only to $w/2$ tokens on each side. Captures local context. Complexity $O(w \times n)$.
- **Dilated window attention:** Like sliding window but with gaps between attended positions, increasing the effective receptive field without increasing computation. Complexity $O(w \times n)$.
- **Global attention:** A small number of special tokens (e.g., [CLS]) attend to and are attended by *all* other tokens. These aggregate global information. Complexity $O(k \times n)$ for k global tokens.

10.11 Modern BERT

The original BERT (2018) has since been significantly improved. **ModernBERT** (Warner et al., 2024) represents the current state of the art in encoder-only models:

- **Longer context:** Sequence length up to 8,192 tokens (vs. 512 in original BERT).
- **Larger sizes:** Base (149M parameters) and Large (395M parameters).
- **Richer training data:** Web documents, code, and scientific articles – 2 trillion tokens total (vs. Wikipedia + BookCorpus in original BERT).
- **Higher masking rate:** 30% of tokens masked during training (vs. 15%), providing more learning signal per pass.
- **Rotary Positional Encoding (RoPE):** Replaces absolute positional embeddings. RoPE encodes relative positions and generalizes better to sequences longer than those seen during training (see Section 9.5 for the full mechanism).
- **GEGLU activation function:** A continuously differentiable gating mechanism (Gaussian Error Gated Linear Unit) that provides better gradient flow than the original ReLU.
- **Alternating attention:** Every 3 layers, the model performs full global attention (every token attends to all others). All other layers use local sliding window attention (each token attends to its 128 nearest neighbors). This dramatically reduces compute while preserving quality.
- **Sequence packing:** Instead of padding sequences to the same length (wasting compute on padding tokens), multiple short sequences are concatenated into one long sequence. Attention masks ensure they are processed independently. Improves hardware utilization significantly.
- Removes unnecessary bias terms; adds an extra normalization layer after embeddings for more stable training.

10.12 T5 – Text-To-Text Transfer Transformer

10.12.1 Motivation

BERT is powerful for language *understanding*, but it has a structural limitation: it cannot generate text. For every new task, you must design a task-specific output head:

- Sentiment analysis → linear classifier on [CLS]
- NER → per-token classifier
- Question answering → span start/end predictor
- Summarization → *impossible with BERT alone*

This proliferation of heads makes multi-task learning awkward: there is no single loss function or output format shared across tasks, and generative tasks like translation or summarization simply cannot be expressed as BERT classification problems.

T5’s key insight: every NLP task can be framed as “read some text, write some text.” A classifier just outputs the word “positive” or “negative”. A translation model outputs the translated sentence. A summarizer outputs a shorter passage. If all tasks share the same text-in / text-out format, you only need *one* model, *one* loss function (cross-entropy on generated tokens), and *one* fine-tuning strategy – regardless of the task.

Definition 10.3 (T5). *T5 (Text-To-Text Transfer Transformer, Raffel et al., 2020) reformulates all NLP tasks as a **text-to-text** problem: both the input and the output are always plain text strings. The same model, loss function, and hyperparameters apply to translation, summarization, classification, and question answering – the only difference between tasks is a short task-specific prefix prepended to the input.*

Example 10.9 (T5 tasks as text-to-text). *These are the tasks that a T5 model can perform, all framed as text-to-text:*

- Translation: *"translate English to German: That is good." → "Das ist gut."*
- Grammatical acceptability: *"cola sentence: The course is jumping well." → "not acceptable"*
- Semantic similarity (score 1–5): *"stsb sentence1: The rhino grazed. sentence2: A rhino is grazing." → "3.8"*
- Summarization: *"summarize: state authorities dispatched emergency crews..." → "six people hospitalized after a storm."*

10.12.2 Architecture: T5 vs. BERT

T5 uses the **full encoder-decoder** Transformer architecture. This is a fundamental difference from BERT, which is encoder-only. The comparison below makes the distinction concrete:

	BERT	T5
Architecture	Encoder only	Encoder + Decoder
Attention (encoder)	Bidirectional (all tokens attend to all)	Bidirectional (same as BERT)
Attention (decoder)	–	Causal (left-to-right) + cross-attention
Output	Hidden states $\mathbf{H} \in \mathbb{R}^{n \times d}$	Generated token sequence
Output head	Task-specific (classifier, span, etc.)	None – always text
Can generate text?	No	Yes
Training objective	Masked Language Modeling	Span corruption

The two components of T5 work as follows:

- **Encoder:** reads the full input string with bidirectional self-attention (identical in spirit to BERT’s encoder). Produces a sequence of contextual hidden states.
- **Decoder:** generates the output token by token. At each step it uses (1) **causal self-attention** – it can only attend to previously generated tokens, not future ones – and (2) **cross-attention** to the encoder’s hidden states, so it can “look at” the full input at every generation step.

10.13 The Span Corruption Paradigm in Sequence-to-Sequence Denoising

Unlike the word-level Masked Language Modeling (MLM) utilized by encoder-only networks, the Text-to-Text Transfer Transformer (T5) relies on an encoder-decoder architecture optimized via *span corruption*. This objective leverages unique sentinel tokens to format self-supervised pre-training identically to downstream auto-regressive generation.

1. **Input Manipulation (Encoder Space):** Given an original sequence X , contiguous blocks of words are sampled (typically following a Poisson distribution with a mean length of 3 tokens) until approximately 15% of the total string is selected. Each unique extracted span is collapsed and mapped to a distinct sentinel variable:

$$X_{\text{corrupted}} = [\text{The}, \langle X \rangle, \text{fox, jumps, over}, \langle Y \rangle, \text{dog.}] \quad (74)$$

The bidirectional encoder ingests $X_{\text{corrupted}}$ to form static global contextual representations.

2. **Target Autoregressive Infilling (Decoder Space):** Rather than calculating terminal loss vectors over the full sequence length T , the causal decoder is tasked strictly with generating the omitted text segments. The true target labels are structured sequentially by concatenating the sentinels with their respective hidden ground truths:

$$Y_{\text{target}} = [\langle X \rangle, \text{quick, brown}, \langle Y \rangle, \text{the, lazy}, \langle Z \rangle] \quad (75)$$

Architectural Efficiency Benefits

Let T_{enc} represent the length of the input sequence and T_{mask} represent the aggregate token count of the corrupted segments (where $T_{\text{mask}} \approx 0.15 \times T_{\text{enc}}$). Under standard MLM, an encoder calculates a vocabulary cross-entropy loss vector over the full horizon T_{enc} . Conversely, the T5 decoder limits its sequential execution steps purely to the sub-sequence length T_{mask} . This bounds the autoregressive computational complexity to $\mathcal{O}(T_{\text{mask}}^2)$, minimizing memory allocation and reducing training FLOP expenditures across massive text datasets.

Remark 10.5. *BERT and T5 are complementary, not competing. BERT excels at tasks requiring deep understanding of a fixed input (classification, NER, extractive QA). T5 excels at tasks requiring generation or flexible output formats (translation, summarization, abstractive QA). In practice, encoder-only models (BERT-family) are preferred for understanding tasks; encoder-decoder models (T5-family) for generation tasks.*

SloT5 is a Slovene T5 model trained on 3.41 billion words of Slovene text (Ulčar & Robnik-Šikonja, 2023), demonstrating the text-to-text approach for a less-resourced language.

10.14 Byte-Level Modeling vs. Subword Tokenization

Traditional encoder architectures rely on subword algorithms—such as WordPiece (BERT) or SentencePiece (T5/mT5)—to map raw character strings into a bounded vocabulary space $\mathcal{V}$ where $|\mathcal{V}| \approx 30,000$ to $250,000$. While computationally efficient, this introduces vulnerabilities to spelling noise, out-of-vocabulary ([UNK]) mutations, and structural cross-lingual tokenization biases.

10.14.1 Character-Based and Multilingual Variants: ByT5 and mT5

While standard T5 uses subword tokenization (SentencePiece) with a large static vocabulary, alternative variants manipulate the input space to prioritize robustness or cross-lingual breadth:

- **ByT5 (Tokenizer-Free Byte Modeling):** Operates directly on raw **UTF-8 byte sequences** ($|\mathcal{V}| = 256 + \text{sentinels}$), completely eliminating artificial subword dictionaries (BPE/WordPiece).
 - *Advantages:* Naturally digests nonstandard spellings, typos, OCR noise, and any language out of the box with zero out-of-vocabulary ([UNK]) tokens.
 - *The Trade-off:* Byte sequences are significantly longer than subword sequences ($T_{\text{byte}} \approx 2.5 \times$ to $4 \times T_{\text{subword}}$). Because morphological and semantic relations must be learned dynamically from scratch using self-attention, ByT5 alters the standard configuration by pairing a **heavier encoder** with a highly streamlined, compact decoder.
- **mT5 (Multilingual Tokenization Scaling):** The massively multilingual adaptation of T5 pre-trained on a web corpus covering 101 languages. Mirroring mBERT’s design methodology, it drastically expands its SentencePiece subword vocabulary to span global scripts, structurally trading per-language contextual depth for raw cross-lingual capability.

11 GPT – Generative Pre-trained Transformers

GPT-style models are **decoder-only** transformers trained as language models: they predict the next token given all previous tokens. Unlike BERT, which uses bidirectional attention and reads the entire sentence at once, GPT uses **causal (masked) self-attention** — each token can only attend to tokens to its left. This makes GPT naturally suited to text generation, but it is a shortcoming for text *understanding*: a unidirectional model cannot use right-context when processing a word.

Remark 11.1. *Bidirectionality matters for understanding. In the sentence “What are those? he said while looking at my crocs”, the meaning of those depends on the word crocs, which appears later. A unidirectional GPT-style model cannot use that right-context, whereas BERT can.*

11.1 GPT Architecture and Training

No encoder — just a decoder stack. A crucial point: GPT contains **no encoder whatsoever**. The original Transformer (Vaswani et al., 2017) had an encoder that read the full input and a decoder that generated output token by token; the decoder could “look at” the encoder’s output via cross-attention. GPT throws the encoder away entirely. There is only a **stack of decoder blocks**, and the input is fed directly into that stack. Because there is no encoder, there is also no cross-attention — each decoder block has only two sublayers: causal self-attention and a feed-forward network. The causal mask ensures each token can only attend to positions to its left.

What does GPT actually output? At each position t in the sequence, the model outputs a **probability distribution over the entire vocabulary**. The forward pass works as follows:

1. **Token + positional embedding:** Each input token is looked up in an embedding matrix to get a dense vector; a positional encoding is added so the model knows the order of tokens.
2. **Stacked decoder blocks:** The embeddings pass through N identical blocks (12 in GPT-1, 48 in GPT-2 large, 96 in GPT-3). Each block applies (a) *causal (masked) self-attention* — no token can see future tokens — followed by (b) a position-wise feed-forward network, with layer normalisation and residual connections around each sublayer.
3. **Linear projection:** The hidden state at position t (a vector in $\mathbb{R}^{d_m}$) is projected to a vector of size $|V|$ (vocabulary size, typically 50,000+ for GPT-2/3) by multiplying with a weight matrix $W_e^\top$ (the same matrix used for token embeddings, just transposed — this is called *weight tying*).
4. **Softmax:** A softmax turns those scores into a valid probability distribution. The model thus outputs $P(w_{t+1} \mid w_1, \dots, w_t)$ — the probability of every possible next token.

At **inference**, we sample (or greedily pick) the highest-probability token, append it to the context, and repeat — one token at a time. At **training**, all positions are processed in parallel thanks to the causal mask; the loss is the average cross-entropy between the predicted distribution and the true next token across every position.

Definition 11.1 (Autoregressive Language Modeling). *A decoder-only transformer models the probability of a sequence autoregressively:*

$$P(w_1, w_2, \dots, w_n) = \prod_{t=1}^n P(w_t \mid w_1, \dots, w_{t-1})$$

At training time, all positions are computed in parallel: the loss is the average cross-entropy $\frac{1}{T} \sum_{t=1}^T L_{CE}$ over all token positions. At inference time, tokens are generated one at a time, each conditioned on all previously generated tokens.

Autoregressive generation works by first feeding a *prefix* (context or prompt) into the model, then sampling new tokens from the output distribution one at a time. Each generated token becomes part of the growing context for the next step — this is called *priming the generator with context*. Beyond text completion, the same approach applies to summarization, question answering, and other generative tasks.

	GPT-2	GPT-3
Layers	48	96
Heads	25	96
Model dim d_m	1600	12,288
Head dim d	64	128
Context size	1,024 tokens	2,048 tokens
Parameters	~1.5B	~175B
Training data	WebText (40 GB)	300B tokens

GPT-3 introduces several changes over GPT-2: layer normalization is applied to the *input* of each sub-block, some sparse attention layers are added, and batch sizes are much larger (3.2 million tokens per batch).

11.2 GPT Evolution and Scaling

The GPT family represents a progressive shift in the training paradigm:

- **GPT-1** (2018, 110M parameters): classic *pre-train then fine-tune* — pretrain on a large corpus, then fine-tune separately for each downstream task with labeled data.
- **GPT-2** (2019, 1.5B parameters): explores *zero-shot transfer* — no fine-tuning; tasks are described in natural language in the prompt alone.
- **GPT-3** (2020, 175B parameters): introduces *in-context learning* — a task description plus a few examples are included in the prompt; no gradient updates ever occur.

ChatGPT (OpenAI, November 2022) is based on GPT-3.5 with additional training for dialogue using RLHF. It had a massive public impact and raised widespread discussion about AI's effects on writing professions, education, and scientific research. **GPT-4** (2023) is even larger, supports longer context windows, and accepts image input in addition to text.

Remark 11.2. *The number of parameters in language models has grown roughly exponentially: from ELMo (94M, Jan 2018) and BERT-Large (340M, 2018) to GPT-3 (175B, 2020) and Switch-C (1.6T, 2021). Despite this growth, training loss continues to decrease with more data and compute. The Llama-2 training curves show models (7B, 13B, 34B, 70B parameters) still improving after 2 trillion training tokens with no sign of saturation — LLMs are genuinely data hungry.*

11.3 In-Context Learning

A key emergent property of large GPT models is **in-context learning**: the model performs new tasks given only a description (and optionally examples) in the prompt, without any gradient updates.

Definition 11.2 (Zero-shot, One-shot, and Few-shot Learning). *Here, we present the differences between zero-shot, one-shot and few-shot learning:*

- **Zero-shot**: only the task description is given; no examples.
- **One-shot**: one input-output example accompanies the task description.
- **Few-shot**: a small number of examples (typically 3–10) are provided in context. No gradient updates occur; examples appear only as part of the input.

Example 11.1 (Translation with In-Context Learning). *We provide some examples of in-context learning:*

Zero-shot:

Translate English to French:
cheese =>

One-shot:

Translate English to French:
sea otter => loutre de mer
cheese =>

Few-shot:

Translate English to French:
sea otter => loutre de mer
peppermint => menthe poivrée
plush giraffe => girafe peluche
cheese =>

Remark 11.3. *The text quality of GPT-3 can be strikingly realistic. When GPT-3 generated a news article, only 12% of human respondents correctly identified it as not written by a human. Larger models benefit disproportionately from in-context learning — GPT-3 approaches state-of-the-art on many benchmarks with few-shot prompting alone.*

11.4 Reinforcement Learning from Human Feedback (RLHF)

The core challenge of aligning LLMs with human preferences is that human feedback is *not present during training*. The solution: train a separate **reward model** on human feedback, which can then generate a reward signal to guide LLM training. RLHF consists of three stages:

1. **Supervised Fine-Tuning (SFT)**: We suppose we have a curated dataset of human-labeled responses, then we fine-tune the base LLM as always using standard cross entropy. This shifts the model towards producing human labelled responses. Then next stages, consist in repeating the training with the dataset but using reinforcement learning. There are two stages, here we present the classical one: training a reward model and using PPO. Another path is to use DPO directly:
2. **Reward Model Training**: collect human preference data by presenting pairs of model outputs and asking which is better. Crucially, humans *rank* responses rather than assigning scores directly — ranking is easier to collect and produces better-calibrated signals (around 50k instances are typical). A reward model r_ϕ is trained to predict human preference via the **Bradley–Terry pairwise loss**:

$$\mathcal{L}(\phi) = -\mathbb{E}_{(x, y_w, y_l)} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

Each term explained:

- ϕ — the parameters of the reward model (trained here; frozen during the subsequent RL step).
- (x, y_w, y_l) — a *preference triplet* drawn from the human-labelled dataset: x is the prompt, y_w (*winner*) is the response humans preferred, y_l (*loser*) is the one they rejected.
- $r_\phi(x, y)$ — the reward model’s scalar score for the pair (prompt, response). Intuitively, higher means “better according to the reward model”.
- $\mathbb{E}_{(x, y_w, y_l)}$ — expectation over all preference triplets in the training set; in practice, a mini-batch average.

The reward model is typically initialised from the SFT model with an added linear head, and outputs a *scalar reward* representing response quality. The loss function is the log-likelihood with σ being the sigmoid-function. Note that this loss function pushes the correct reward to infinity and the incorrect one to -infinity.

3. **RL Fine-Tuning with PPO**: fine-tune the SFT model to maximize the reward model score using Proximal Policy Optimization (PPO). Most parameters are *frozen*; only a small subset are updated. A KL-divergence penalty prevents *reward hacking* (degenerate high-reward outputs):

$$\mathcal{L}_{\text{RLHF}} = \mathbb{E}_x \left[r_\phi(x, y) - \beta \log \frac{\pi_\theta(y | x)}{\pi_{\text{SFT}}(y | x)} \right]$$

Remark 11.4 (The Formula Above Is the Objective, Not the PPO Update). *The expression $\mathcal{L}_{\text{RLHF}}$ describes what is being maximized, not how PPO mechanically updates the weights. Standard PPO optimizes a clipped surrogate to ensure stable updates:*

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t [\min(\rho_t A_t, \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon) \cdot A_t)]$$

where $\rho_t = \pi_\theta(y_t | x, y_{<t}) / \pi_{\text{old}}(y_t | x, y_{<t})$ is the token-level probability ratio between the current and previous policy, and A_t is the **advantage** — how much better the chosen action is compared to the expected baseline. In RLHF, the full PPO setup still requires:

- A **value function** $V_\psi(x, y_{<t})$, trained in parallel, estimating the expected cumulative reward from each token position.
- **Token-level advantages** A_t , typically computed via Generalized Advantage Estimation (GAE) using the end-of-sequence reward from r_ϕ and the per-step value estimates from V_ψ .
- The **clipped ratio** from standard PPO applied per token to prevent excessively large policy updates.

The formula $\mathcal{L}_{\text{RLHF}}$ collapses this into a sequence-level expression for readability: the first term is the reward signal and the second is the KL penalty, together defining the scalar that the advantage function is ultimately derived from.

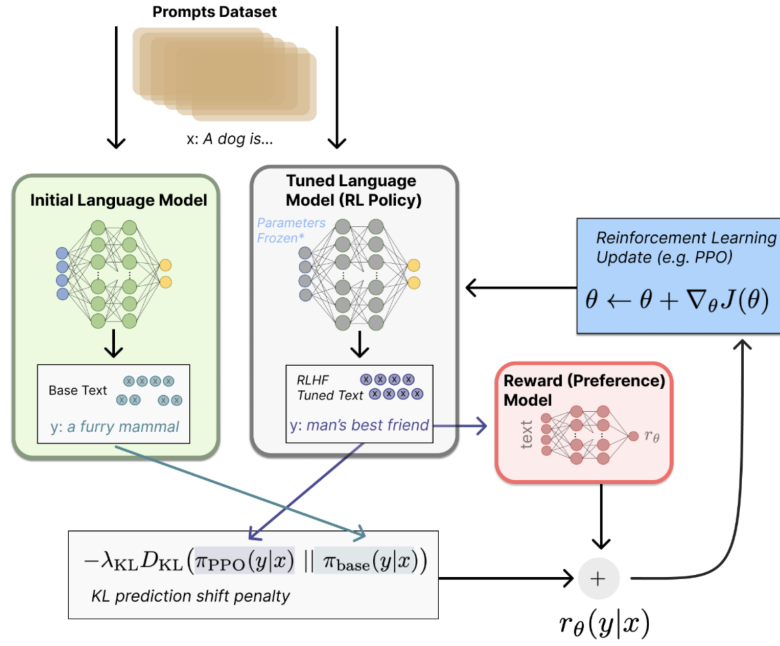


Figure 17: The three stages of RLHF: SFT pretraining, reward model training on human-ranked pairs, and RL fine-tuning with PPO.

11.5 Direct Preference Optimization (DPO)

Definition 11.3 (DPO). *Direct Preference Optimization eliminates the need for a separate reward model and RL training loop. It directly optimizes the policy on preference data using supervised learning:*

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

where π_{ref} is a fixed reference policy (the SFT model) and β controls how far the new policy may deviate from the reference.

Note: We are essentially maximizing the ratio of a good answer, and minimizing the ratio of a bad answer. DPO has two practical advantages over RLHF+PPO:

- **No explicit reward model:** the preference signal is absorbed directly into the policy.
- **No online sampling:** it learns from a fixed preference dataset, avoiding the computationally expensive RL sampling loop.

Remark 11.5. *DPO is simpler, more stable, and achieves comparable alignment to RLHF+PPO. The preference data it requires is also easier to collect via crowdsourced platforms like LLM Arena, where users naturally compare model responses.*

11.6 LLM Evaluation: LLM Arena

Chatbot Arena (LLM Arena) is a crowdsourced evaluation platform: a prompt is sent to two anonymous models simultaneously and the user votes for the better response. Elo ratings derived from these votes produce a human-preference ranking of models, making it one of the primary benchmarks for instruction-following LLMs.

Example 11.2 (Slovenian LLM Arena — PoVeJMo). *The Faculty of Computer and Information Science, University of Ljubljana, runs a Slovenian-language LLM arena at arena.cjvt.si/sl under the name **PoVeJMo** (“Slovenian Large Language Model”). It compares models on Slovenian-language prompts and collects user feedback. Early findings show that the Slovenian model **GaMS** performs just slightly below the 1,000× larger Gemini 2.5 Pro while outperforming other comparably sized models.*

11.7 Prompt Engineering

Prompt engineering (PE) is the practice of designing input prompts to elicit better responses from LLMs. It exploits the attention mechanism and the relatively long context windows of modern LLMs.

PE rests on a key asymmetry: humans typically produce short prompts, but LLMs can exploit very long contexts effectively. A *small* change in wording can produce a *large* change in output quality. PE is somewhere between engineering and guesswork — systematic patterns exist, but extensive experimentation is usually required. Well-established approaches such as RAG and chain-of-thought prompting emerged from this practice.

11.7.1 Prompt Elements

A well-structured prompt typically includes:

- **Instruction:** the specific task to perform
- **Context:** external information or additional context to steer the model toward better responses
- **Input Data:** the actual question or input you want a response for
- **Output Indicator:** the desired format or type of output

Example 11.3 (Sentiment Classification Prompt). *For sentiment classification we do:*

Prompt:

Classify the text into neutral, negative, or positive.

Text: I think the food was okay.

Sentiment:

*This works, but adding explicit **context** (e.g., domain, output format, or persona) steers the model toward more useful responses. Not all four elements are always needed — include what improves your task.*

11.7.2 General Guidelines for Prompt Construction

Designing prompts is an iterative process that requires experimentation:

- Start with a simple prompt and progressively add elements and context.
- Specificity, simplicity, and conciseness often give better results than vague or verbose prompts.
- For large tasks with many subtasks, break them down and build up incrementally.
- **Say what to do, not what not to do:** LLMs handle positive instructions better than negations.

Example 11.4 (Specificity: Entity Extraction with Desired Format). *Here we provide some entity extraction with desired format:*

Prompt:

Extract the name of places in the following text.

Desired format: Place: <comma_separated_list_of_places>

Input: “...Henrique Veiga-Fernandes, a neuroimmunologist at the Champalimaud Centre for the Unknown in Lisbon...”

Output: Place: Champalimaud Centre for the Unknown, Lisbon

Specifying the output format makes responses immediately usable without post-processing.

Example 11.5 (Say What To Do). *Some examples:*

Bad prompt (uses negation — LLMs are not good at negation):

The following is an agent that recommends movies. DO NOT ASK FOR INTERESTS. DO NOT ASK FOR PERSONAL INFORMATION.

Output: “What kind of movie would you like?” — the model still asks.

Better prompt (describes desired behavior positively):

The agent recommends a movie from the top trending list. It should refrain from asking for preferences and respond ‘Sorry, couldn’t find a movie to recommend today’ if unsure.

Output: “I don’t have your interests on file. Here’s a list of top trending movies right now: [...]”

11.7.3 Few-Shot Prompting

Providing examples in the prompt helps the model understand the task format and expected output style.

Example 11.6 (Few-Shot with Novel Words). *Here there is an example of Few-shot with novel words:*

Prompt: A “whatpu” is a small, furry animal native to Tanzania. An example sentence that uses the word whatpu is: We were traveling in Africa and we saw these very cute whatpus. To do a “farduddle” means to jump up and down really fast. An example sentence that uses the word farduddle is:

Output: When we won the game, we all started to farduddle in celebration.

Remark 11.6 (Few-Shot Tips). *Surprisingly, the labels in few-shot examples need not be correct. Using wrong labels (e.g., labelling a positive text as Negative) still helps, because what matters most is: (1) the label space shown in examples and (2) the distribution of input text. Random labels drawn from the true distribution outperform uniform random labels, and any labels at all outperform no labels. If zero-shot and few-shot prompting are still insufficient, consider fine-tuning or more advanced techniques like chain-of-thought.*

Example 11.7 (Random Labels Still Work). *For example:*

Prompt (labels are intentionally wrong):

This is awesome! // Negative

This is bad! // Positive

Wow that movie was rad! // Positive

What a horrible show! //

Output: *Negative (correct answer, despite all wrong training labels)*

11.7.4 Chain-of-Thought (CoT) Prompting

Standard few-shot prompting fails on multi-step reasoning tasks. **Chain-of-Thought** prompting adds explicit reasoning steps to each example, encouraging the model to reason before answering.

Example 11.8 (Standard vs. CoT Prompting). *Here is an example of standard vs CoT prompting:*

Standard prompt (fails):

Q: Roger has 5 tennis balls. He buys 2 cans of 3 tennis balls each. How many does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. Used 20, bought 6 more. How many now?

A: The answer is 27. (incorrect)

CoT prompt (succeeds):

Q: Roger has 5 tennis balls. He buys 2 cans of 3 tennis balls each. How many does he have now?

A: Roger started with 5 balls. 2 cans of 3 = 6 balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. Used 20, bought 6 more. How many now?

A: Originally 23. Used 20: $23 - 20 = 3$. Bought 6: $3 + 6 = 9$. The answer is 9. (correct)

Definition 11.4 (Chain-of-Thought Prompting). *CoT prompting includes explicit reasoning steps before the final answer in each few-shot example. In this case, the user guides the model through different steps. This encourages the model to produce its own reasoning chain before answering, and works even with a single expanded example.*

Remark 11.7. Zero-Shot CoT: *appending “Let’s think step by step” to a prompt often elicits a reasoning chain without any examples. This generalizes across arithmetic, symbolic, and commonsense reasoning tasks, and often corrects errors that zero-shot and few-shot prompting would make.*

11.7.5 Advanced Prompting Techniques

Auto-CoT. Manual construction of CoT demonstrations is labor-intensive. Auto-CoT automates this using Zero-Shot CoT (“Let’s think step by step”) to generate reasoning chains automatically. Two stages: (1) **question clustering** — partition the question dataset into clusters; (2) **demonstration sampling** — select a representative question per cluster and auto-generate its reasoning chain, using simple selection heuristics (e.g., questions ≤ 60 tokens, rationales ≤ 5 reasoning steps). Diverse demonstrations reduce the impact of individual chain mistakes.

Self-Consistency. Instead of greedy decoding, sample multiple diverse reasoning paths and select the answer with the most votes (majority voting). Improves CoT performance on arithmetic and commonsense reasoning.

Example 11.9 (Self-Consistency). *Self-Consistency examples:*

Prompt: *When I was 6 my sister was half my age. Now I'm 70, how old is my sister?*

Zero-shot output: *35 (incorrect — simple halving without tracking the age gap)*

With self-consistency, multiple reasoning paths are sampled:

Output 1: *When I was 6, sister was 3. Age gap is 3. Now I'm 70, so she is $70 - 3 = 67$. Answer: 67.*

Output 2: *Narrator was 6, sister was half = 3. Gap = 3. Now 70, sister is $70 - 3 = 67$. Answer: 67.*

Output 3: *She was $6/2 = 3$. Now she is $70/2 = 35$. Answer: 35.*

Majority vote selects 67 (correct).

Tree of Thoughts (ToT). ToT extends CoT to a **tree** structure: the model generates multiple candidate “thoughts” at each reasoning step, evaluates each (sure / maybe / impossible), and uses BFS or DFS with backtracking to explore the most promising paths. The model self-evaluates progress through intermediate steps. Effective for complex planning tasks.

Example 11.10 (ToT: Game of 24). *The task: make the number 24 from four given numbers (e.g., 4, 9, 10, 13) using any arithmetic operations. At each step, the model proposes partial equations, evaluates whether the remaining numbers can still reach 24, keeps the best $b = 5$ candidates, and backtracks when a branch is hopeless. ToT outperforms standard prompting and CoT on this task.*

Example 11.11 (Alternative ToT Formulation in a Single Prompt). *A simpler in-prompt version evaluates intermediate thoughts within one call:*

“Imagine three different experts are answering this question. All experts will write down 1 step of their thinking, then share it with the group. Then all experts go on to the next step, etc. If any expert realizes they are wrong at any point, they leave. The question is: [...]”

This produces better results than asking the question without breakdown instructions, without requiring multiple API calls.

Generated Knowledge Prompting. The LLM first generates relevant knowledge pieces about the input, then uses those pieces as additional context for the final answer. Conceptually similar to RAG, but knowledge is generated internally rather than retrieved from an external database.

Example 11.12 (Generated Knowledge: Golf). *Example of using generated knowledge in the game of Golf*

Question: *Part of golf is trying to get a higher point total than others. Yes or No?*

Zero-shot output: *Yes. (incorrect — golf awards the lowest score)*

With generated knowledge (LLM first generates facts about golf rules and scoring), the model correctly answers:

“No, the objective of golf is to complete the course in the fewest number of strokes, not a higher point total.”

Prompt Chaining. Break complex tasks into a sequence of subtasks. The output of one prompt becomes input to the next, creating a chain of prompt operations. This improves transparency, controllability, debuggability, and reliability compared to a single monolithic prompt.

Example 11.13 (Prompt Chaining for Document QA). *For document QA for example:*

Prompt 1 (extract relevant quotes):

“Your task is to help answer a question from a document. Extract quotes relevant to the question, delimited by ####. Output as <quotes></quotes>. Respond ‘No relevant quotes found!’ if none found.”

Output of Prompt 1: *a structured list of quotes (e.g., CoT prompting, RAG, Self-consistency, ...)*

Prompt 2 (compose the final answer):

“Given the relevant quotes and the original document, compose an accurate, friendly, helpful answer to the question.”

Separating the tasks makes each step independently debuggable and improvable.

Technique	Core Idea	Key Mechanism	Best For
Chain-of-Thought (CoT)	Guide the model to reason step by step before giving an answer.	Add explicit intermediate reasoning steps to few-shot examples; the model imitates the pattern.	Multi-step arithmetic, logical inference.
Auto-CoT	Automate CoT demonstration construction to avoid manual effort.	Cluster questions by similarity; auto-generate reasoning chains for one representative per cluster via zero-shot CoT (“Let’s think step by step”).	Large-scale CoT without manual chain authoring.
Self-Consistency	Replace greedy decoding with a voting ensemble of reasoning paths.	Sample multiple diverse chains; take the majority-vote answer across all chains.	Robustness; arithmetic and commonsense tasks.
Tree of Thoughts (ToT)	Explore a tree of candidate reasoning steps rather than a single chain.	At each step generate several “thought” candidates; self-evaluate each (sure / maybe / impossible); traverse via BFS or DFS with backtracking.	Complex planning and puzzle tasks (e.g. Game of 24).
Generated Knowledge Prompting	Let the model generate relevant facts first, then use them to answer.	Prompt the LLM to produce auxiliary knowledge about the question; feed that knowledge back as additional context for the final answer.	Commonsense and factual QA where key facts may be overlooked.
Prompt Chaining	Decompose a complex task into a pipeline of simpler sub-prompts.	Output of each prompt feeds the next; optional “gate” checks between steps verify quality before proceeding.	Multi-step workflows, document QA, code generation pipelines.

Table 6: Summary of the six main prompt engineering techniques.

11.7.6 Summary of Prompt Engineering Techniques

11.8 Retrieval Augmented Generation (RAG)

Definition 11.5 (RAG). *Retrieval Augmented Generation augments an LLM with an external knowledge retrieval step. For complex or knowledge-intensive tasks, relevant documents are retrieved from a vector index and concatenated with the original prompt before generation. RAG improves factual consistency, reduces hallucinations, and adapts to dynamic knowledge without retraining the model.*

The RAG pipeline:

1. **Query:** the user submits a question
2. **Semantic search:** the query is embedded and matched against a vector index of document chunks
3. **Context retrieval:** the top- k most relevant chunks are retrieved
4. **Prompt construction:** retrieved context + original query are combined into a prompt
5. **Generation:** the LLM generates an answer conditioned on the enriched prompt
6. **Answer delivery:** the formulated answer is returned to the user

Example 11.14 (RAG vs. No RAG). **Question:** *How do you evaluate OpenAI CEO Sam Altman’s sudden dismissal and rehiring by the board?*

Without RAG: *“I am unable to provide comments on future events. Currently, I do not have information regarding the dismissal and rehiring of OpenAI’s CEO.” (model’s knowledge cutoff)*

With RAG (relevant news chunks retrieved and included in context): *“This suggests significant internal disagreements within OpenAI regarding the company’s future direction. These events reflect power struggles and corporate governance issues within OpenAI.” (factually grounded, up-to-date)*

Remark 11.8. *RAG differs from generated knowledge prompting: in RAG, knowledge is retrieved from an external database (e.g., Wikipedia, enterprise documents, recent news); in generated knowledge prompting, the LLM generates the knowledge from its own parametric memory. RAG is better suited to tasks requiring up-to-date or proprietary information.*

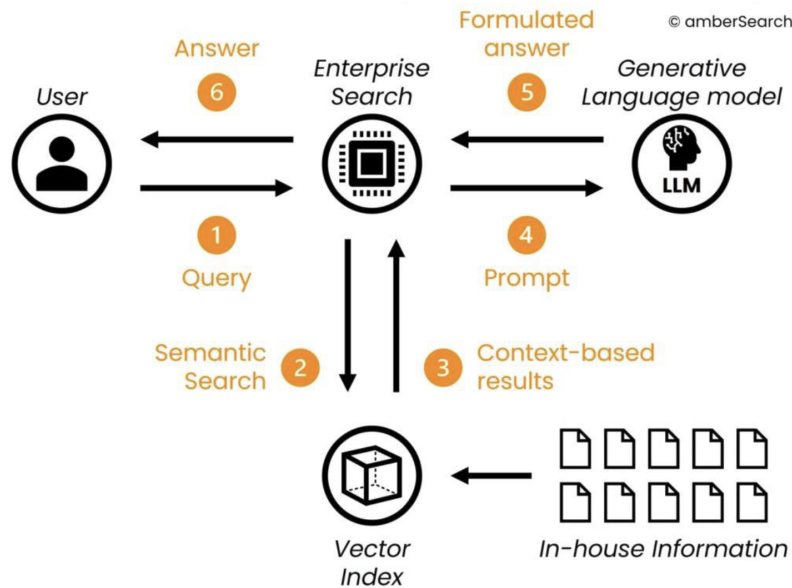


Figure 18: RAG architecture: the user query is semantically matched against a vector index; top- k relevant chunks are retrieved and combined with the query into the LLM prompt before generation.

12 LLM-Based Agents

An **agent** is a system that acts on behalf of a user or another program. In computer software, a **software agent** is “a computer program that acts for a user or another program in a relationship of agency”. In AI, an **intelligent agent** is “an entity that perceives its environment, takes actions autonomously to achieve goals, and may improve its performance through machine learning or by acquiring knowledge”—comprising an AI component, sensors, actuators, memory, and other modules.

12.1 The Agent Loop

The canonical agent execution model is the **sense-plan-act-learn** loop:

1. **Initialise** state and goals.
2. **Repeat** until termination:
 - **Sense:** Gather observations from the environment.
 - **Plan:** Evaluate goals and state, plan/select the next action.
 - **Act:** Execute the chosen action on the environment.
 - **Learn:** Update the internal state, memory, or model based on outcomes.

In sequence: call tool, invoke LLM, test for termination; repeat.

12.2 From ML to LLM-Based Agents

The evolution of AI systems can be characterised across three stages:

1. **Traditional ML:** Individual siloed models, requiring task-specific training and large amounts of human-supervised data.
2. **Foundation models / LLMs:** Massive multi-modal models adapted with minimal training through pre-trained unsupervised learning. It is only one forward pass, given a question they produce the answer.
3. **Reasoning models:** The agent consists of a multi-step logic with self-consistency checks. This means that it is slower but in exchange of a more accurate inference.

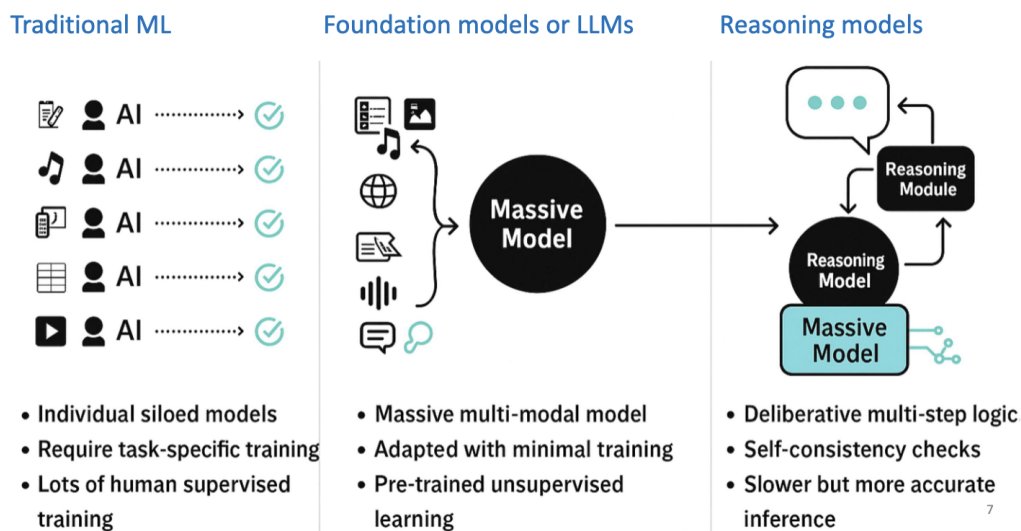


Figure 19: Different approaches to building AI systems: from traditional ML with siloed models, to foundation models (LLMs) with in-context learning, to reasoning models with deliberative multi-step logic.

12.3 LLM-Based Agents

Definition 12.1 (LLM-Based Agent). An **LLM-based agent** is an autonomous system that can reason about tasks and act to achieve goals by leveraging external tools and resources. Modern AI agents are typically powered by large language models (LLMs) connected to external tools or APIs.

LLM-based agents differ from conventional models in important regards:

- They are **interactive and adaptive**.
- Rather than returning fixed outputs, they can take **multi-step actions**, integrate context, and support iterative human–AI collaboration.
- Users can interact with them through human language, substantially reducing usage barriers.

Agents begin their work with either a command from, or interactive discussion with, the human user. Once the task is clear, agents plan and operate independently, potentially returning to the human for further information or judgement. During execution, agents gain “ground truth” from the environment at each step (such as tool call results or code execution) to assess progress. Agents can pause for human feedback at checkpoints or when encountering blockers.

Use case: agents for science A concrete example is **CACTUS**, a chemistry agent that interprets a user’s question, selects the best tool from a set of qualitative and quantitative chemistry tools, executes it, and checks whether the result answers the question—looping until a satisfactory informed output is produced.

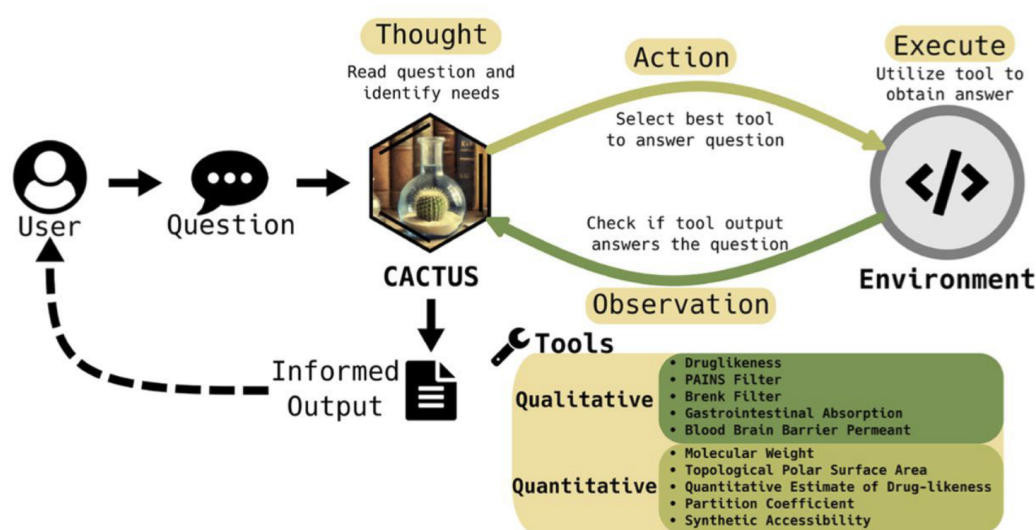


Figure 20: Cactus architecture for chemistry. An example of agent for science.

12.4 How Many Agents?

In **principle**, a single reasoning model (like a single human) can apply a variety of reasoning strategies, hold specialised knowledge on different topics, and improve expertise over time.

In **practice**, it is common to create **multiple agents** (like a team of people) for:

- **Modularity:** Specialisation, reuse, and maintainability.
- **Different roles:** e.g., idea generator, idea critic, program generator.
- **Parallelism:** Run multiple copies of an agent to explore different ideas simultaneously.
- **Expanded capacity:** e.g., larger effective LLM context.

12.5 Agent Roles

Common agent role types include:

- **Computing agent:** Utilises computational models as tools.
- **Decision agent:** Makes decisions in response to given conditions.
- **Database agent:** Retrieves relevant information from databases.
- **Reasoning agent:** Capable of direct reasoning and reasoning with feedback.

- **Expert agent:** Provides professional consultation based on reliable sources, domain expertise, feedback from human experts, and results of specific tools.
- **Hypothesis agent:** Capable of reflective learning and reasoning to generate hypotheses.
- **Planner agent:** Devises plans for future actions.
- **In silico/vitro agent:** Uses tools in silico or in vitro environments.

12.6 Self-Improvement and Feedback Loops

A key trend is applying the foundation model onto itself:

- **Self-consistency:** Generate multiple Chain-of-Thought responses and vote on the final answer.
- **Reflexion & Reflection:** Ask the model to critique and revise its own output.
- **STaR (Self-Taught Reasoner):** Use the model’s own outputs to fine-tune itself.

These methods moved models from **single-shot output** to **deliberative computation**, aligning with Kahneman’s “System 2”-like behaviour.

Reasoning models (e.g., GPT o5) State-of-the-art reasoning models implement several key mechanisms:

- **Parallel reasoning at test time:** Run multiple internal reasoning paths in parallel, then aggregate or vote.
- **Adaptive reasoning effort:** Dynamically scale internal compute based on prompt complexity.
- **Routing layer:** Route between lighter vs. heavier reasoning modes depending on query difficulty; may call tools, execute code, or search as part of the reasoning pipeline.
- **Longer context/memory handling:** Maintain internal memory beyond prompt tokens for multi-step problems.
- **Higher accuracy via verification:** Include internal cross-checking steps after generating a candidate answer.
- **User-controlled verbosity/effort trade-offs:** Parameters to dial reasoning effort up or down.

12.7 Agent Architectures

Definition 12.2 (Workflows vs. Agents). *We define workflows and agents as:*

- **Workflows:** *Systems where LLMs and tools are orchestrated through predefined code paths. The LLM is embedded in or directs control flow through those paths.*
- **Agents:** *Systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks based on environmental feedback.*

12.7.1 The Augmented LLM

The fundamental building block of all workflows is the **augmented LLM**: an LLM connected to three additional modules:

- **Retrieval:** Query/results interface for accessing external knowledge.
- **Tools:** Call/response interface for invoking external functions or APIs.
- **Memory:** Read/write interface for persistent state.

12.7.2 Workflow Patterns

Prompt Chaining Decomposes a task into a sequence of steps, where each LLM call processes the output of the previous one. Programmatic checks (“gates”) can be inserted at intermediate steps to ensure the process stays on track.

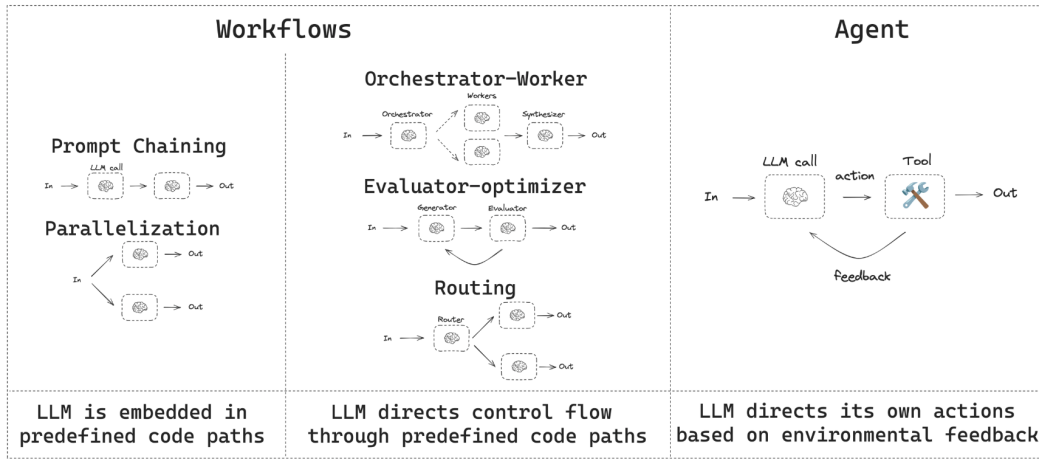


Figure 21: Different approaches to building agent architectures.

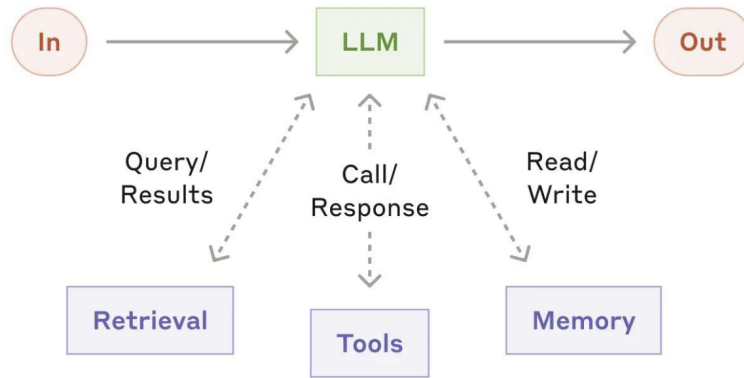


Figure 22: The LLM is able to query results, call tools and read and write from memory.

Routing Classifies an input and directs it to a specialised follow-up task. This allows separation of concerns and more specialised prompts per input type. Optimising a single prompt for one kind of input can hurt performance on others.

Parallelization Runs multiple LLM calls concurrently on the same input (or on independent sub-problems) and aggregates their outputs. Useful when sub-tasks are independent and latency matters, or when redundancy and voting across outputs is desired for robustness.

Evaluator-Optimizer One LLM call generates a response while a second LLM call evaluates it and provides feedback in a loop. The generator revises its output until the evaluator accepts it.

Orchestrator-Worker A central orchestrator LLM decomposes the task, dispatches subtasks to specialised worker LLMs, and synthesises their outputs. Enables fine-grained specialisation and parallelism.

12.7.3 ReAct Loop

ReAct (Reasoning + Acting) frames an agent's behaviour as an interleaving of three steps repeated until the task is complete:

1. **Thought:** The model writes out its reasoning about the current state and what to do next.
2. **Action:** The model picks a tool and calls it.
3. **Observation:** The model reads the tool's result and incorporates it into the next thought.

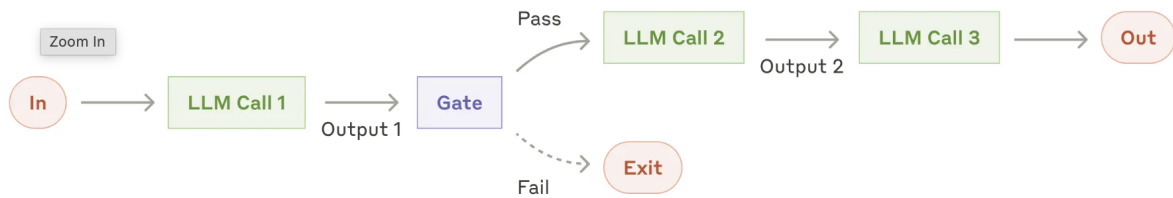


Figure 23: The most common workflow pattern, where the prompt and reasoning is represented by a chain with different outputs and reasoning steps.

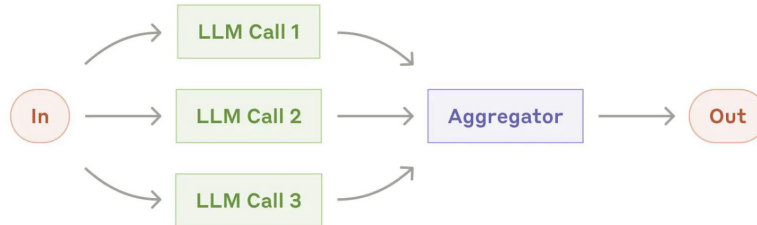


Figure 24: Example of aggregator.

This loop is a practical instantiation of the sense-plan-act-learn cycle applied to LLMs. In frameworks such as **LangChain**, the function `create_agent()` builds a ReAct loop using **LangGraph**: a graph-based agent runtime where **nodes** represent processing steps (e.g. a *model* node that calls the LLM, a *tools* node that executes the selected tool) and **edges** represent the transitions between those steps. The agent traverses this graph, alternating between model calls and tool executions, until a termination condition is met.

12.8 Multi-Agent Systems

Why multiple agents? While a single reasoning model can in principle handle diverse tasks, in practice it is common and beneficial to compose **multiple specialised agents**. The key motivations are:

- **Specialization:** Narrow, role-tuned agents outperform generalists on their particular slice of the problem.
- **Parallelism:** Independent agents work concurrently on sub-tasks, increasing throughput.
- **Heterogeneous tools & environments:** Different agents own different toolchains (HPC, LIMS, robots, databases).
- **Robustness & fault isolation:** One agent can fail, restart, or be replaced without taking down the whole system.
- **Self-checking & consensus:** Cross-agent critique, redundancy, and voting reduce error and bias.
- **Long-running workflow control:** Persistent agents watch queues, schedule jobs, and resume after interruptions.
- **Governance, security, provenance:** Data stays within the minimal-privilege agent; actions are auditable by role.
- **Scalability & cost control:** Orchestrators spawn cheap/ephemeral workers; premium models are used only where needed.

Challenges of multi-agent systems. Composing multiple agents introduces new engineering difficulties:

- **Creating agents:** Defining roles, prompts, tool access, and termination conditions for each agent.
- **Monitoring agents:** Observing internal state, progress, and output quality across concurrent agents.
- **Inter-agent communication:** Deciding what information passes between agents, in what format, and when.
- **Agent lifecycle management:** Spawning, pausing, restarting, and shutting down agents reliably.

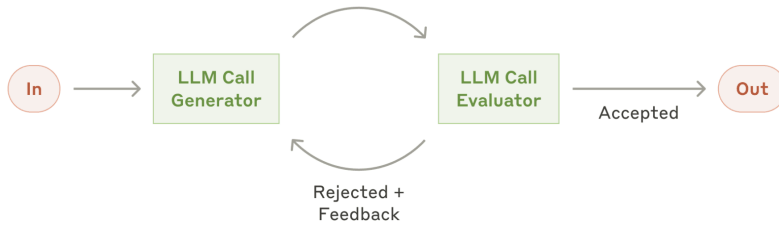


Figure 25: Example of evaluation-optimizer.

Pattern	Structure	How It Works	When to Use
Prompt Chaining	Sequential	Each LLM output feeds the next step; optional gate checks verify quality before proceeding.	Multi-step tasks with clear, ordered sub-steps (e.g. extract → summarize → translate).
Routing	Branching	A classifier LLM directs each input to the most appropriate specialised handler or prompt.	Diverse input types that benefit from tailored prompts (e.g. billing vs. technical vs. general queries).
Parallelization	Parallel	Multiple independent LLM calls run concurrently; outputs are aggregated (merge or majority vote).	Independent sub-tasks where latency matters, or when redundancy and voting improve robustness.
Evaluator-Optimizer	Feedback loop	A generator LLM produces a response; an evaluator LLM critiques it; the generator revises until the evaluator accepts.	Iterative refinement tasks such as writing, code generation, or translation.
Orchestrator-Worker	Hierarchical	A central orchestrator LLM decomposes the task, dispatches subtasks to specialised workers, and synthesises their outputs.	Complex tasks with heterogeneous sub-tasks requiring specialised agents or tools.

Table 7: Summary of the five workflow patterns for LLM-based systems.

Perils of multiple agents. Beyond engineering challenges, multi-agent systems carry specific risks:

Risk	Description	Mitigation
Coordination chaos	Agents loop, contradict, or amplify each other’s errors	Limit turn count; add an arbiter/watchdog agent
Loss of accountability	Hard to trace which agent caused errors or bias	Log every message & decision; require provenance
Emergent behavior	Unpredictable dynamics — agents invent new “protocols”	Constrain roles and allowable message types
Security & safety	Tool calls, code execution, or API use may cause harm	Sandbox actions, validate outputs, require human approval
Human factors	Over-trust, confusion, or reduced oversight	Keep humans in the loop; clear UI for agent reasoning
Resource/cost blow-up	Many agents → exponential token and time costs	Prune redundant roles; use shared memory or caching

How agent systems help. Well-designed agent infrastructure addresses these risks through:

- **Scheduling & Fairness:** Coordinated access to LLM cores, memory, and tools (e.g. the AIOS kernel).

Agent Type	Control Flow	Key Mechanism	Typical Use
Augmented LLM	Single LLM call	Base LLM extended with retrieval, tool calling, and read/write memory interfaces.	Basic tool-using assistant; foundation for all more complex patterns.
ReAct Agent	Thought $\rightarrow$ Action $\rightarrow$ Observation loop	Interleaves reasoning traces with tool calls; observation feeds back into the next thought until a final answer is produced.	Open-ended tasks requiring iterative tool use: web search, code execution, database queries.
Evaluator-Optimizer Agent	Generator / evaluator feedback loop	One LLM generates, a second evaluates and provides critique; loop continues until quality threshold is met.	High-quality generation tasks: writing, code, translation, structured data extraction.
Orchestrator-Worker Agent	Hierarchical multi-LLM	Orchestrator decomposes goals and delegates sub-tasks; workers execute specialized sub-tasks in parallel or sequence.	Large, complex tasks needing specialization: research pipelines, multi-tool workflows.
AutoGen Agent	Conversation-driven	Conversable agents (LLM, tool, human, or hybrid) exchange natural-language messages; supports joint and hierarchical chat.	Rapid prototyping of multi-agent patterns; human-in-the-loop scenarios.
LangGraph Agent	Graph / state machine	Nodes = processing steps; edges = transitions; explicit shared state object enables persistence and deterministic replay.	Production-grade flows needing reliability, debuggability, and human-in-the-loop checkpoints.
AIOS Agent	OS-managed runtime	Kernel treats LLMs and tools as OS resources; handles scheduling, memory, storage, and access control across concurrent agents.	Multi-tenant, long-running, resource-intensive deployments requiring governance and reproducibility.

Table 8: Summary of agent types covered in this section.

- **Isolation & Governance:** Sandboxed agents with explicit privileges and traceable system calls.
- **Reproducibility:** Logged context snapshots and deterministic replay improve scientific auditability.
- **Safety & Oversight:** Policy enforcement and user-intervention checkpoints before critical actions.
- **Scalability:** Architecture that scales from a handful of agents to thousands sharing compute without chaos.

12.8.1 Agent Frameworks: AutoGen, LangGraph, and AIOS

AutoGen is a multi-agent framework from Microsoft that introduces **conversable agents**: agents that can be based on LLMs, tools, humans, or combinations thereof, and that communicate via natural-language conversation. It supports *joint chat* (all agents in one shared conversation) and *hierarchical chat* (agents orchestrating sub-agents). AutoGen is well suited for fast prototyping of multi-agent patterns.

AIOS (LLM Agent Operating System) proposes a **kernel-based architecture** that treats LLMs and their associated tools as *first-class operating-system resources*. It handles scheduling, memory/context management, storage, and access control across concurrent agents — analogous to how an OS manages processes and hardware. AIOS is best suited for multi-tenant, long-running, resource-intensive deployments.

The three frameworks can be compared as follows:

Axis	AutoGen	LangGraph	AIOS
Abstraction	Conversation-programmed agents	Graph / state machine	OS-style runtime (kernel + SDK)
Primary concern	Inter-agent dialog + tools	Deterministic orchestration, persistence, human-in-loop	Scheduling, memory, storage, access control
State	Conversation history + tool results	Explicit shared state object	Kernel-managed contexts/memory
When to use	Fast prototyping	Production flows needing reliability & debuggability	Multi-tenant, long-running, resource-intensive deployments

12.9 Tool Calling and the Model Context Protocol (MCP)

Pure text-based reasoning has fundamental limitations: LLMs lack real-time data, cannot perform precise computation, and have no access to external systems. **Tool calling** (also called *function calling* or *tool-augmented reasoning*) addresses this by letting a model invoke external tools — APIs, databases, simulators, code executors — as part of its reasoning process, much like a scientist using an instrument or a database.

Example 12.1 (Tool calling for precise computation). *Here is an example across different models:*

Question: *What is the pH of a 0.01 M solution of hydrochloric acid?*

Llama-3-8B (no tool): *pH 1.0 (wrong)*

Llama-3-70B (no tool): *pH 2.0 (wrong)*

Correct: $\text{pH} = -\log_{10}(0.01) = 2$

With tool call: *calculator.log10(x=0.01) → result fed back to model → correct answer.*

The challenge is that LLMs express requests in natural language (e.g., “compute the negative log of 0.01”), while tools are executables, Python functions, workflows, or remote APIs that expect structured inputs, may run in diverse environments (local, HPC, cloud), may require specialised permissions, and may change over time. Bridging this gap requires a common protocol.

Definition 12.3 (Model Context Protocol (MCP)). ***MCP** is a standardised, interoperable protocol designed to let models exchange and extend their **context** — everything the model can condition on while reasoning: external knowledge, task parameters, persistent state, and the results of prior computations.*

MCP specifies:

- **Tool description:** *How to expose tools to models (schemas, parameters).*
- **Tool invocation:** *How to issue requests and receive responses.*
- **Metadata carriage:** *How to pass context identifiers, auth tokens, and timestamps.*
- **Context sharing:** *How to share or update context objects across sessions and agents.*

*MCP connects **AI applications** (chat interfaces such as Claude Desktop, IDEs such as Claude Code, and other AI apps) with **data sources and tools** (databases, development tools such as Git, and productivity tools such as Slack or Google Maps) via bidirectional data flow through a single standardised channel.*

13 Neural Information Retrieval

This lecture extends the RAG framework introduced in Section 11 and focuses on the neural retrieval and ranking mechanisms that underpin it.

13.1 Retrieval Augmented Generation (RAG) in Depth

Building on the definition in Section 11, RAG grounds LLM reasoning in *external* knowledge to improve factuality, recency, and provenance. The full pipeline follows:

Embed $\rightarrow$ Retrieve $\rightarrow$ (Re-rank) $\rightarrow$ Generate $\rightarrow$ Cite

Concretely: (1) the user submits a prompt and query; (2) a search module retrieves relevant information from knowledge sources; (3) the relevant information is returned as enhanced context; (4) the prompt, query, and enhanced context are combined; (5) the LLM endpoint produces the final generated response.

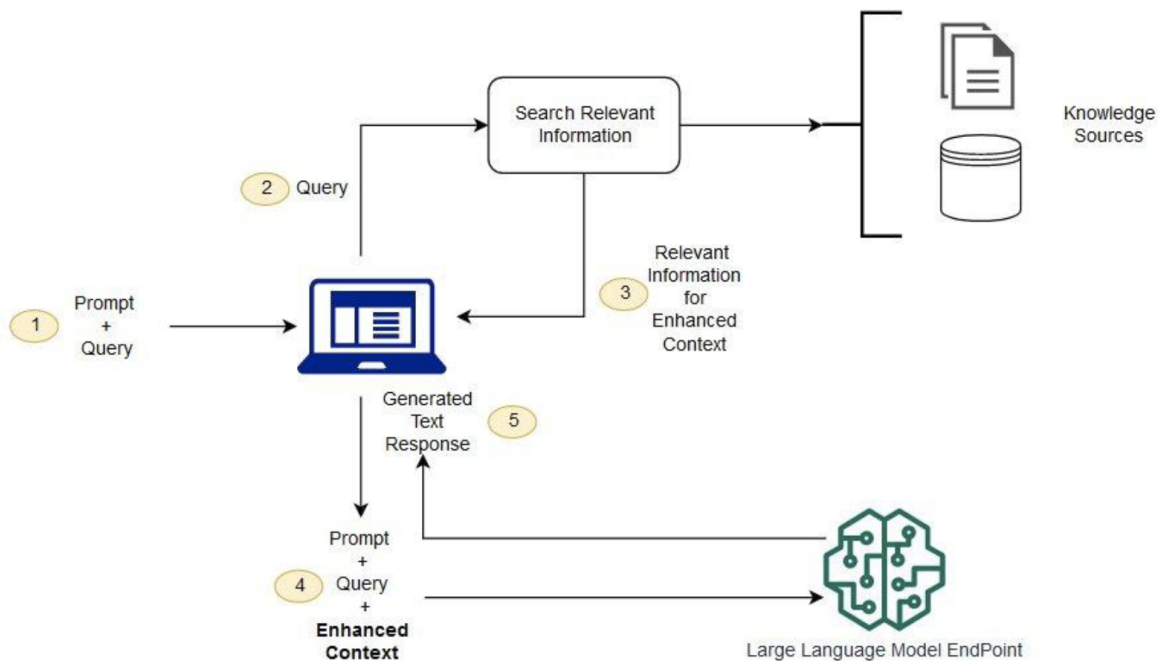


Figure 26: Input of the BERT model

13.2 Key Design Levers

These are the engineering choices that determine how fast, cheap, and accurate a RAG pipeline is.

Chunking Strategy A **chunk** is a contiguous unit of text extracted from a larger document that is indexed and retrieved independently. Chunking is necessary because LLMs have a fixed **context window** — they cannot ingest an entire knowledge base in one prompt. Instead, only the most relevant pieces are retrieved and passed to the model. Smaller, focused chunks produce sharper embeddings and let the retriever surface exactly the right passage rather than a diluted whole document.

The three most common strategies are:

- **Fixed-size:** split every N tokens regardless of content boundaries — fast and simple, but may cut sentences mid-way.
- **Sentence- / paragraph-based:** split at natural linguistic boundaries (sentence ends, paragraph breaks, headers) — semantically coherent units, but variable in length.
- **Overlapping (sliding window):** successive chunks share M overlapping tokens so context at a boundary is not lost on either side — trades index size for completeness.

Core size trade-off: chunks that are **too small** lose their meaning and miss answers that span multiple pieces; chunks that are **too large** dilute the embedding signal and waste context window space.

Example: A 500-page phone manual cannot be fed to an LLM at once. Cutting it into 200-word chunks — keeping all “Camera” instructions together — lets the retriever surface exactly the right paragraph when a user asks “how do I use portrait mode?”.

Embedding Choice The AI model used to convert text chunks into fixed-length numerical vectors. A stronger embedding model maps semantically equivalent phrases to nearby vectors, so the retriever understands that “*my screen is broken*” and “*cracked display*” describe the same problem. Normally we use BERT, SBERT, SloBERTa for Slovene.

Index Type (HNSW / IVF-PQ) How vectors are organised in the database for fast similarity search.

- **HNSW** (Hierarchical Navigable Small World): builds a multi-layer graph — like a road network of motorways and local streets — that jumps quickly to the right neighbourhood. Very fast and accurate, but memory-intensive.
- **IVF-PQ** (Inverted File + Product Quantisation): groups vectors into clusters and compresses each vector. Memory-efficient at the cost of a small accuracy drop.

Filters Hard constraints on document metadata applied before or during the vector search to narrow the candidate pool. For example, a query about “restarting an iPhone 15” adds a filter `brand=Apple, model=15` so the retriever never touches Samsung or legacy manuals.

Hybrid Retrieval Combining dense vector search (semantic similarity) with classical keyword search (exact term matching). Vector search struggles with error codes or acronyms that carry no “meaning” in embedding space; keyword search finds them instantly. Combining both approaches yields the best of both worlds.

Re-ranking (Cross-Encoders) A two-stage approach: the fast vector index returns the top- k candidates, and a slower but more accurate cross-encoder model re-scores each (query, chunk) pair to re-order them before passing the best results to the LLM. See the Re-Ranking subsection below for details.

Answer Grounding Instructing the LLM — via the system prompt — to restrict its answer strictly to the retrieved chunks and to reply “I do not know” when the answer is not present, preventing hallucination of unsupported facts.

13.3 Evaluation Metrics

Retrieval Quality: Recall@ k and nDCG **Recall@ k** checks whether the correct document appears anywhere in the top- k retrieved results. If the ground-truth answer is in one document and the system retrieves $k = 5$ documents that include it, Recall@5 is 100%; if the document is missing, Recall@5 is 0%.

nDCG (Normalised Discounted Cumulative Gain) also rewards rank position: a correct document at rank #1 scores higher than the same document buried at rank #10. It measures both the presence and prominence of relevant results.

Answer Groundedness (Faithfulness) Checks whether the LLM’s answer is fully supported by the retrieved chunks and not hallucinated from parametric memory. Example: if the retrieved text says “water-resistant up to 3m” but the model tells the user “safe for deep-sea diving”, groundedness drops to zero.

Citation Quality Evaluates whether the source references provided by the LLM genuinely point to the exact sentence supporting each claim. A citation evaluator looks up the cited chunk or page and verifies the stated fact is actually written there.

13.4 RAG Improvements

- **Naive RAG:** Indexing, retrieval, prompt + frozen LLM. The simplest pipeline.
- **Advanced RAG:** Adds *pre-retrieval* steps (query routing, query rewriting, query expansion) and *post-retrieval* steps (rerank, summary, fusion) around a retrieval core.
- **Modular RAG:** Decomposes the pipeline into interchangeable modules—Search, Routing, Retrieve, Rewrite, Rerank, Read, Demonstrate, Fusion, Memory, Predict—composable in various patterns (Naive RAG, Advanced RAG, DSP by Khattab et al. 2022, ITER-RETGEN by Shao et al. 2023).

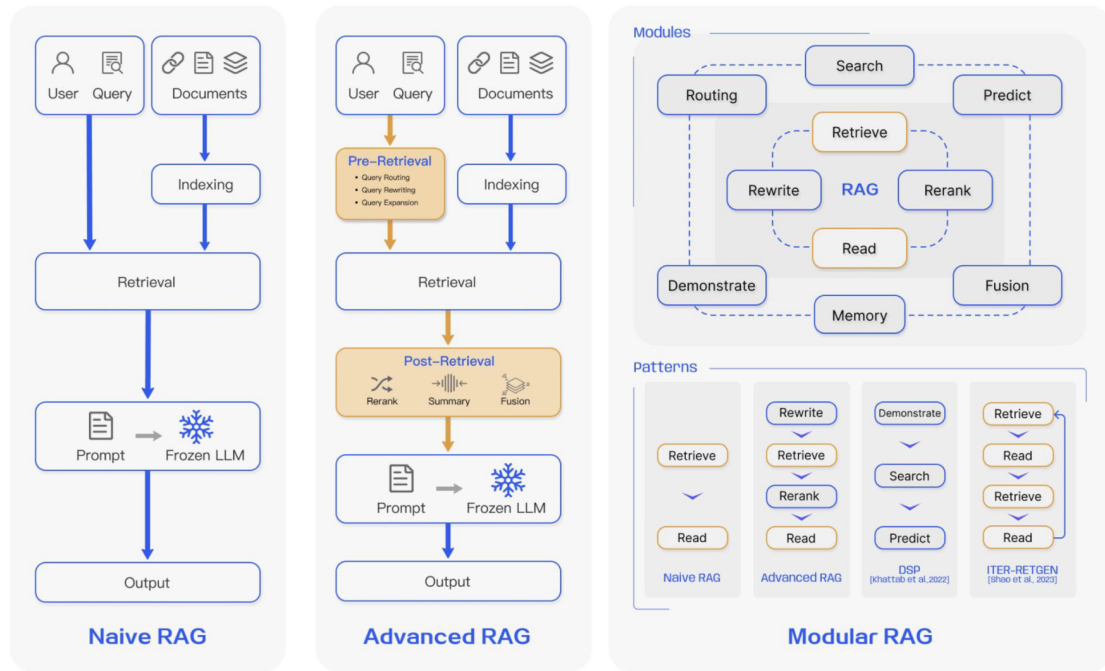


Figure 27: Sketch of the RAG improvements: Navie RAG, Advanced RAG and Modular Rag, and the different steps that compose each one.

13.5 RAG Variants

Three main extension patterns exist for more complex retrieval scenarios:

- **Iterative RAG:** Repeatedly cycles Retrieve–Generate–Judge up to N times or until a quality threshold is met, providing progressively more context to the LLM.
- **Recursive RAG:** Breaks down complex problems step by step. A Query Transformation/Decomposition module splits sub-problems and feeds them back into Retrieve, bounded by a maximum tree depth.
- **Adaptive RAG:** Flexible and active control of retrieval and generation. A Judge component decides whether to retrieve on demand; after retrieval the system either generates a response or decomposes the query further.

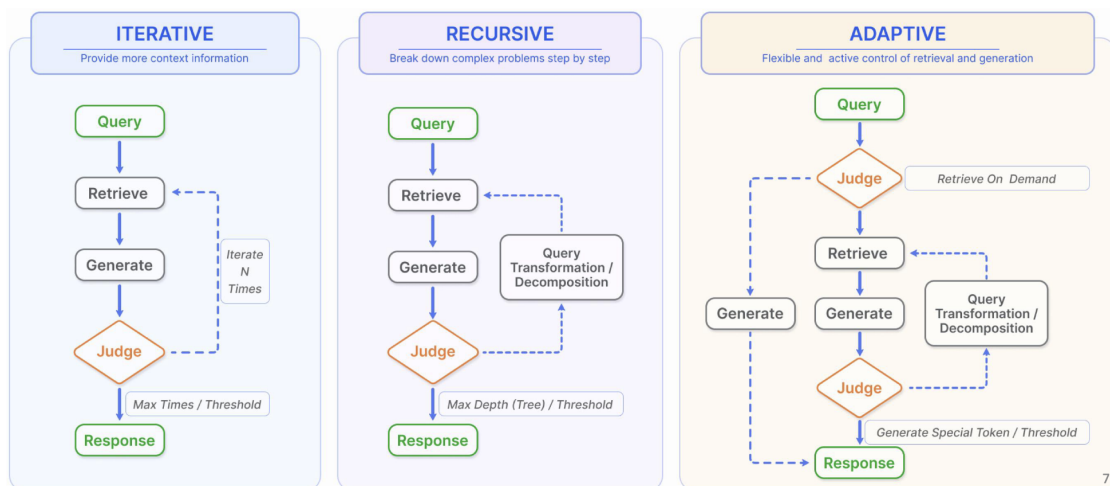


Figure 28: The three different main version of RAG: Iterative, Recursive and Adaptive.

13.6 Re-Ranking

After a retriever returns the top- k candidate documents for a query, a **reranker** sorts them by true relevance. The two-stage pipeline trades off cost vs. accuracy:

- **Stage 1 (cheap, approximate):** Vector DB returns top- k candidates (e.g., 25) via Approximate Nearest Neighbour (ANN) search.
- **Stage 2 (expensive, accurate):** Reranker selects the final top- n (e.g., 3) from those candidates.

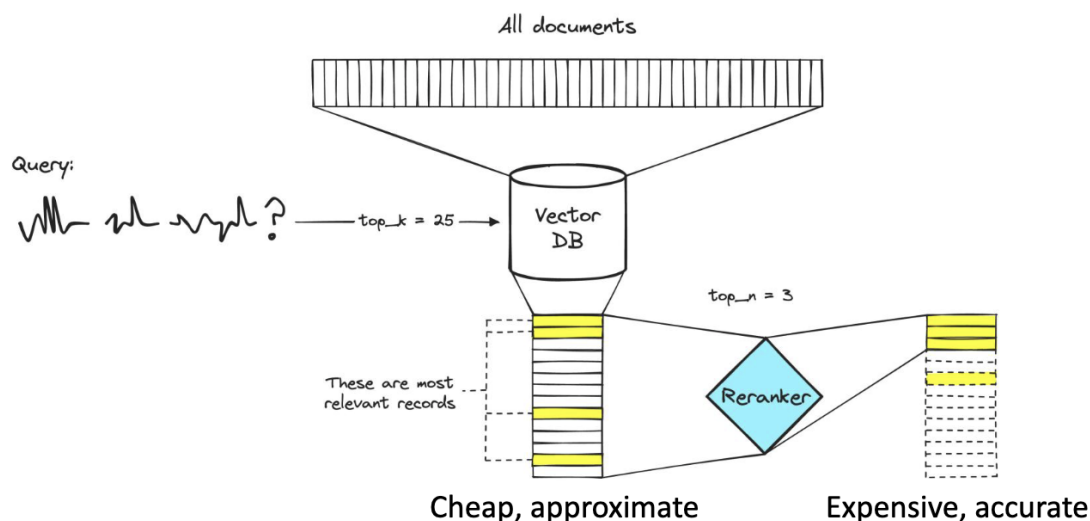


Figure 29: Pipeline showing document ranking for a given query.

13.7 Toolkits for RAG Evaluation: RAGAS

RAGAS (Retrieval Augmented Generation Assessment Suite), developed by Hugging Face & Intel Labs, provides automated scoring of RAG outputs across four dimensions:

- *Faithfulness* – are answers grounded in the retrieved context?
- *Answer relevance* – how relevant is the answer to the query?
- *Context recall* – did the retriever fetch the needed information?
- *Context precision* – was the retrieved information actually used?

RAGAS uses a smaller LLM or NLI (natural-language inference) model for evaluation, produces interpretable scores per query, and integrates into LangChain, LlamaIndex, or Hugging Face pipelines.

13.8 Beyond Textual Data

RAG can be extended to diverse data modalities. The following table summarises the requirements for each type:

Data type	Examples	What the LLM must learn/do
Structured	Databases, CSVs, SQL engines	Generate and interpret queries, aggregate results
Temporal / time-series	Sensors, logs, climate records	Learn temporal reasoning; perform trend analysis
Spatial / geospatial	Maps, satellite imagery	Call GIS APIs; reason with coordinates and regions
Scientific / numeric	Simulation codes, lab data, HPC runs	Request parameters, launch computations, interpret results
Multimodal	Images, spectra, molecules, graphs	Use embeddings or model adapters per modality
Private / dynamic	Company or lab databases, instruments	Handle authentication, provenance, and freshness

Table 9: Data types beyond plain text and the corresponding LLM requirements in RAG settings.

Multimodal RAG: SMURF SMURF (Scientific Multimodal Retrieval in Federations) handles queries such as “Show me examples of catalysts that perform well at 200°C for CO₂ conversion, and include microscopy images of the catalyst surface.” It routes queries through a query processor and resource selector to separate ANN-indexed vector databases (text, image, SMILES modalities), then merges results via an embedding aligner.

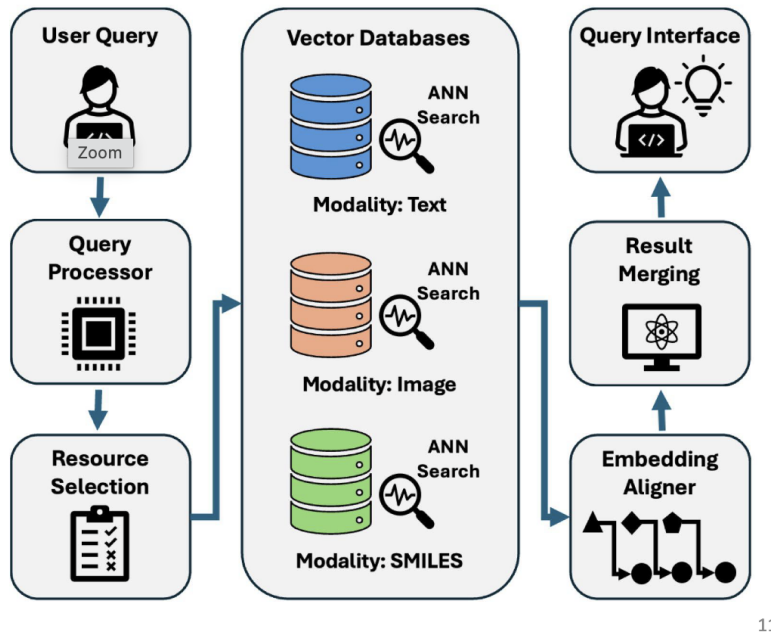


Figure 30: Multimodal RAG architecture for SMURF

13.9 RAG Pitfalls

- **Context window overflow / truncation:** When too many retrieved tokens exceed the LLM’s context limit, older or less-recent chunks are silently dropped, causing loss of critical information or incoherent answers.
- **Irrelevant retrieval hurting generation:** Poorly matched passages mislead the model’s reasoning, pulling the answer toward unrelated topics or introducing factual noise.
- **Citation hallucination:** The model fabricates references or associates statements with the wrong source, creating false confidence and undermining provenance.
- **Embedding drift when corpora evolve:** Updating or re-embedding documents with a new model or altered data distribution changes their vector geometry, breaking previous similarity relationships and degrading retrieval accuracy.

13.10 Neural Text Ranking

Obtaining relevant context for a query is a core component of both traditional IR and modern RAG. Well-known approaches include:

- **BM25 (Best Match 25):** A bag-of-words probabilistic model, works similarly to TF-IDF weighting.
- **DPR (Dense Passage Retrieval):** Dual-encoder architecture with dense embeddings.
- **Dot product on sentence encoders:** e.g., LaBSE or Sentence-BERT.
- **CovBERT:** A BERT-based coverage model.

13.10.1 BM25

Okapi BM25 (Best Match 25) is a bag-of-words ranking function. Given a query Q with terms $q_1, \dots, q_n$, the BM25 score of a document D is:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad (76)$$

where $f(q_i, D)$ is the number of times q_i occurs in D , avgl is the average document length in the collection, and $k_1 \in [1.2, 2.0]$, $b = 0.75$ are parameters.

The IDF component is defined as:

$$\text{IDF}(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right) \quad (77)$$

where N is the total number of documents and $n(q_i)$ is the number of documents containing q_i .

13.10.2 Neural Ranking

Neural rankers use dense representations instead of sparse bag-of-words counts. IR ranking scores query-document pairs, sorts them in descending order, and returns the top K results.

Query-Document Interaction Approach Given one query and a document, we create a matrix and then compute the similarity using a CNN approach:

1. Tokenise query and documents, this is normally a token per word or 2 words and embed tokens to vectors.
2. Build a query-document interaction matrix; compute cosine similarity for each pair of query-document tokens.
3. Compress the matrix to a scalar score using a neural layer (convolution or linear layers).

This approach is considerably better than non-neural methods but computationally expensive at query time. For retrieving, we need to repeat this procedure for all the document, and retrieve the most relevant ones.

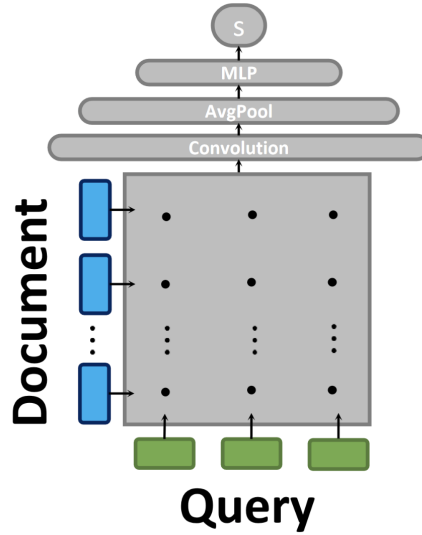


Figure 31: Model of the query-document interaction approach

All-to-All Interaction with BERT (Cross-Encoder)

1. Feed BERT the concatenated string: [CLS] Query [SEP] Document [SEP].
2. Run through all BERT layers (full mutual attention between query and document tokens).
3. Extract the final [CLS] output embedding.
4. Reduce to a single relevance score through a linear layer.

This is essentially a standard BERT classifier for passage ranking. The model must be fine-tuned with positive and negative (query, document) pairs. It achieves much better quality than the interaction-matrix approach but at a dramatic increase in computational cost.

Faster IR: Precomputing A key optimisation is to **pre-compute and cache document representations**. BERT rankers are slow because they encode each document separately for every new query. Since documents are known in advance, their representations can be computed once offline and cached in the index, decoupling document encoding from query-time computation. This can be done in the Query-Document Interaction Approach but not in the All-to-all representation.

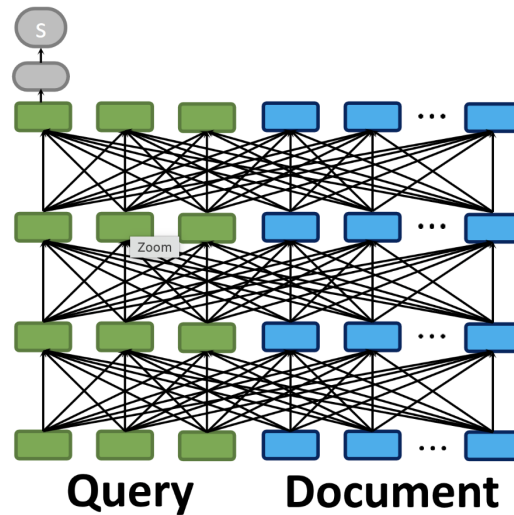


Figure 32: Model of the all-to-all interaction approach.

14 POS Tagging, Dependency Parsing, and Named Entity Recognition

14.1 Lemmatization

Lemmatization groups inflected forms of a word into a single base form (the lemma). Difficulty scales with morphological complexity: English is relatively simple (*walk*, *walked*, *walking* → **walk**; irregular forms like *went* → **go** require a lexicon), while Slovene nouns inflect across six cases and pronouns also inflect heavily (e.g. *jaz*, *mene*, *meni*, *mano* all lemmatise to **jaz**). Approaches include rule-based systems, lexicons, and ML models. Lemmatization is especially important for morphologically rich languages (Slavic, Arabic, Turkish).

14.2 Part-of-Speech Tagging

Definition 14.1 (Part-of-Speech Tagging). *POS tagging* is the task of assigning a grammatical category (noun, verb, adjective, etc.) to each token in a text. Formally, it maps a sequence of words $x_1, \dots, x_n$ to a corresponding sequence of POS tags $y_1, \dots, y_n$.

Example 14.1 (POS Tagging). For the sentence “the girl kissed the boy on the cheek”:

<i>Word</i>	<i>Lemma</i>	<i>Tag</i>
<i>the</i>	<i>the</i>	+DET
<i>girl</i>	<i>girl</i>	+NOUN
<i>kissed</i>	<i>kiss</i>	+VPAST
<i>the</i>	<i>the</i>	+DET
<i>boy</i>	<i>boy</i>	+NOUN
<i>on</i>	<i>on</i>	+PREP
<i>the</i>	<i>the</i>	+DET
<i>cheek</i>	<i>cheek</i>	+NOUN

Words can carry multiple POS categories depending on context: *book* is a VERB in “*book that flight*” and a NOUN in “*hand me that book*”.

POS tagging supports: information retrieval, NER, word sense disambiguation, coreference resolution, speech synthesis (the tag determines stress and pronunciation, e.g. *DIScount* noun vs. *disCOUNT* verb), machine translation, and sentiment analysis (adjectives are the primary opinion holders).

Morphosyntactic tagging extends POS with morphological features (gender, number, case, tense, mood), important for morphologically rich languages. Tags are determined by word morphology (suffixes/prefixes) and distributional context.

14.3 Word Classes and Tag Sets

POS categories divide into:

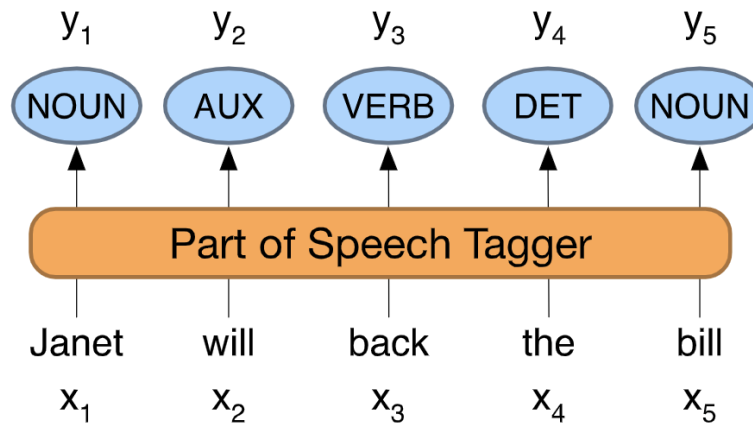


Figure 33: Input of the BERT model for POS tagging.

- **Closed class (function words):** Fixed membership; rarely extended. Examples: prepositions (*of, in, by*), auxiliaries (*may, can, will*), pronouns (*I, you, she*).
- **Open class (content words):** New members created continuously. Four classes in English: **nouns**, **verbs**, **adjectives**, **adverbs**.

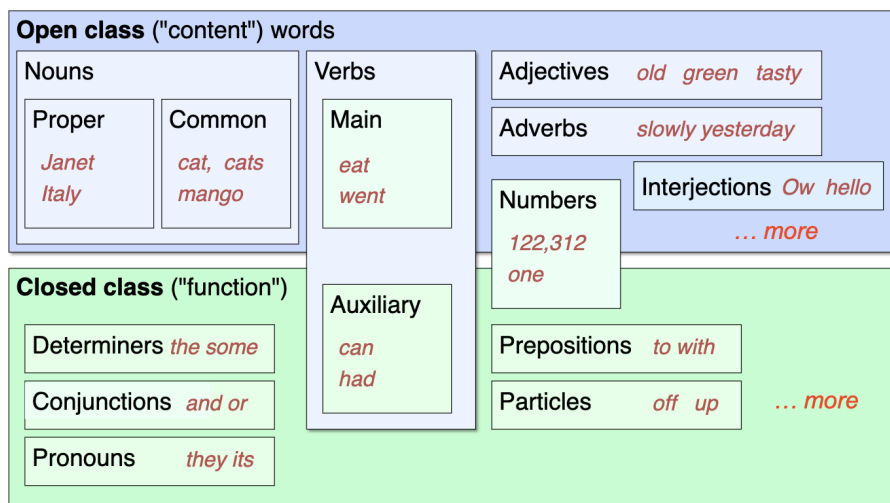


Figure 34: Open and closed class words.

Tag set size varies: the **Penn Treebank** (standard for English) uses 45 tags; the **Universal Dependencies (UD)** framework defines 17 universal POS tags applicable across 100+ languages:

Class	Tag	Description	Example
Open	ADJ	Adjective: noun modifiers	<i>red, young, awesome</i>
	ADV	Adverb: modifiers of time, place, manner	<i>very, slowly, home</i>
	NOUN	Persons, places, things	<i>algorithm, cat, beauty</i>
	VERB	Actions and processes	<i>draw, provide, go</i>
	PROPN	Proper noun	<i>Regina, IBM, Colorado</i>
	INTJ	Interjection	<i>oh, um, yes, hello</i>
Closed	ADP	Adposition (pre/postposition)	<i>in, on, by, under</i>
	AUX	Auxiliary verb	<i>can, may, should, are</i>
	CCONJ	Coordinating conjunction	<i>and, or, but</i>
	DET	Determiner	<i>a, an, the, this</i>
	NUM	Numeral	<i>one, two, first, second</i>
	PART	Particle	<i>up, down, on, off</i>
	PRON	Pronoun	<i>she, who, I, others</i>
	SCONJ	Subordinating conjunction	<i>that, which</i>
Other	PUNCT	Punctuation	<i>; , ()</i>
	SYM	Symbol	<i>\$, %</i>
	X	Other	<i>asdf, qwfg</i>

POS tagging is a **sequence labelling** task: given words $x_1, \dots, x_n$, produce labels $y_1, \dots, y_n$. Modern systems use bidirectional LSTMs or fine-tuned BERT-based models; BERT represents the state of the art across most languages.

14.4 Difficulty, Baselines, and Approaches

The main challenge is **lexical ambiguity**: **$\approx 15\%$ of word types** are ambiguous, but since the most frequent words (*is, that, back*) are the most ambiguous, roughly **60% of running tokens** are ambiguous. For example, *back* is an adjective (*“the back door”*), a noun (*“break his back”*), or a verb (*“back the bill”*).

The simplest baseline assigns each token its most frequent training-corpus tag: **92% word-level** but only **22% sentence-level** accuracy. State-of-the-art systems (fine-tuned BERT) reach **$\approx 97\%$ word-level**, near human agreement of $\approx 98\%$, and **$\approx 65\%$ sentence-level** — meaning one in three sentences still has a tagging error, which matters for downstream tasks like parsing.

Three evidence sources guide tagging: (1) **prior probabilities** (*the* is almost always DET); (2) **neighbouring words** (a determiner is almost always followed by an adjective or noun); (3) **morphology and capitalisation** (suffixes like *-ing, -ness*; capitalisation signals proper nouns).

Main algorithms: **HMMs** (emission + transition probabilities), **CRFs** (discriminative; arbitrary features; avoid label-bias of MEMMs), **bidirectional LSTMs/Transformers**, and **fine-tuned BERT** (current standard; $>98\%$ word-level on Penn Treebank).

14.5 Slovene Morphosyntactic Tagging

For morphologically rich languages, POS is extended with full morphological information. The Slovene standard is **MULTEXT-East 4.0**, defining **1902 distinct MSD (Morphosyntactic Description)** tags encoding word class, gender, number, case, tense, mood, etc. Example: *Ncmsn* = Noun, common, masculine, singular, nominative. Key corpora: **ssj500k** ($\sim 600k$ words, gold POS + lemma + dependency) and **SUK 2023** ($\sim 1M$ words, the largest current Slovene annotated resource).

14.6 Constituency and Dependency Parsing

POS tags alone do not resolve **structural ambiguity**. *“Scientists count whales from space”* is ambiguous: does *from space* modify *count* (scientists in space) or *whales* (whales in space)? Parsing assigns an explicit tree structure that resolves such ambiguities.

Constituency (phrase-structure) parsing organises words into nested constituents: each word is assigned a POS tag, words are grouped into phrases via production rules ($NP \rightarrow Det\ Adj\ N$, $PP \rightarrow P\ NP$), and phrases are combined recursively, yielding a full parse tree with labelled non-terminal nodes ($NP, VP, PP, S, \dots$).

Dependency syntax represents structure as typed binary arcs from a **head** to a **dependent** (e.g. *nsubj*, *obj*, *nmod*). A virtual ROOT node is added so every word has exactly one head; the graph must form a tree. Parses are **projective** (no crossing arcs) or **non-projective** (crossing arcs, common in free-word-order languages).

like Slovene). Advantages over constituency: natural handling of free word order, nodes are words (not abstract phrase labels), clean mapping to predicate-argument structure, easier multilingual development.

14.7 Treebanks and Universal Dependencies

A **treebank** is a corpus annotated with syntactic trees, POS tags, and an annotation scheme — the standard resource for training and evaluating parsers and taggers. Key resources: the **Penn Treebank** (Marcus et al., 1993; ~1M words of Wall Street Journal text, 45-tag English constituency standard) and **Universal Dependencies** (UD; <https://universaldependencies.org>; harmonised dependency treebanks for 100+ languages).

UD is built on the premise that all languages share fundamental structural similarities. It defines 17 universal POS tags (see table above), 40 universal grammatical relations (de Marneffe et al., 2014), and standardised morphological features, enabling consistent cross-lingual comparison. Language-specific subtypes may be added (e.g. `nsubj:pass` for passive subjects). The Slovene UD treebank (SST) is available at https://universaldependencies.org/treebanks/sl_sst/.

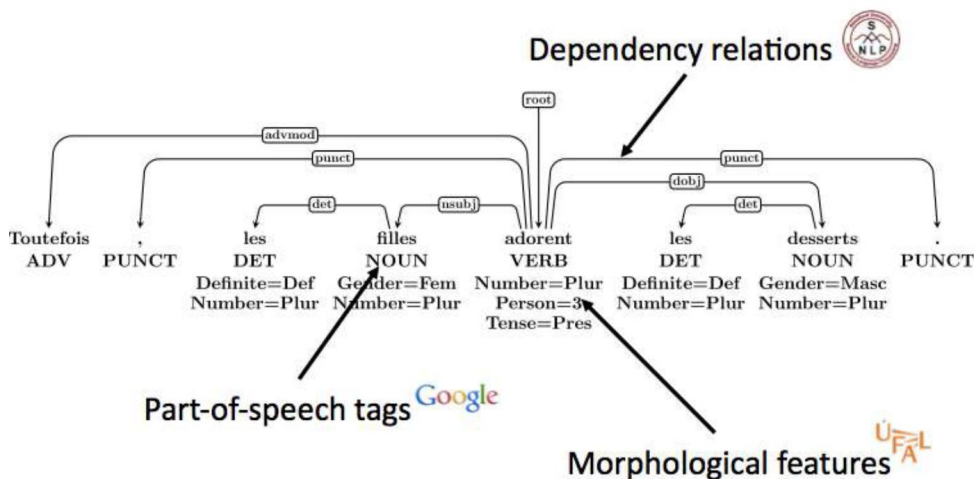


Figure 35: A UD-annotated tree for the French sentence “*Toutefois, les filles adorent les desserts*” (“However, the girls love desserts”), jointly showing the three UD annotation layers: dependency relations (e.g. `nsubj` from *adorent* to *filles*, `obj` to *desserts*, with *adorent* as *root*), universal POS tags (DET, NOUN, VERB, PUNCT, ADV), and morphological features (Gender, Number, Person, Tense).

14.8 Modern NLP Pipelines

Modern NLP integrates all bottom-layer tasks in a **single neural pipeline**: tokenisation, sentence segmentation, lemmatisation, POS and morphological tagging, dependency parsing, and NER. **Stanford Stanza** (Qi et al., 2018) is the leading fully neural, language-agnostic pipeline for 66+ languages; **CLASSLA-Stanza** is its Slovene adaptation, reaching >99% tokenisation accuracy, 98.7% UPOS, and 92.1% labelled attachment score (LAS).

14.9 Named Entity Recognition

Definition 14.2 (Named Entity Recognition (NER)). A **named entity** is anything referred to with a proper name (person, location, organisation). **NER** finds text spans that constitute proper names and classifies each into a type: **PER**, **LOC**, **ORG**, **GPE**, **OTHER** (often extended to dates, prices, drugs, genes, etc.).

NER is the first structured step in **Information Extraction**: source → NER (spans → typed entities) → relation extraction (e.g. CEO(Tim Cook, Apple)) → knowledge-base population. It also supports question answering and entity linking to Wikipedia/Wikidata.

The main challenge is **ambiguity**: the same surface form can map to different types. *Washington* can be PER (born into slavery), ORG (sports team), LOC (city), or GPE (state) depending on context.

14.9.1 BIO Tagging: NER as Sequence Labelling

NER spans cover multiple tokens; the standard per-token encoding is **BIO tagging**:

- **B-X**: begins a named entity span of type X

- **I-X**: inside (continuation of) a span of type X
- **O**: outside any named entity

With n entity types: IO produces $n + 1$ labels; BIO produces $2n + 1$; BIOES (also marking End and Singleton) produces $4n + 1$. The IO scheme cannot distinguish two adjacent same-type entities.

Example 14.2 (BIO Tagging). *Consider the sentence: “Apple CEO Tim Cook met with the European Commission in Brussels.”*

<i>Token</i>	<i>BIO Tag</i>	<i>Entity</i>
<i>Apple</i>	<i>B-ORG</i>	<i>Apple (ORG)</i>
<i>CEO</i>	<i>O</i>	—
<i>Tim</i>	<i>B-PER</i>	<i>Tim Cook (PER)</i>
<i>Cook</i>	<i>I-PER</i>	
<i>met</i>	<i>O</i>	—
<i>with</i>	<i>O</i>	—
<i>the</i>	<i>O</i>	—
<i>European</i>	<i>B-ORG</i>	<i>European Commission (ORG)</i>
<i>Commission</i>	<i>I-ORG</i>	
<i>in</i>	<i>O</i>	—
<i>Brussels</i>	<i>B-LOC</i>	<i>Brussels (LOC)</i>
<i>.</i>	<i>O</i>	—

B-PER marks the start of a multi-token entity and *I-PER* its continuation. Two consecutive *PER* entities would be unambiguous with *BIO* but would merge into a single span under *IO*.

For reference, the full comparison across encodings:

Word	IO	BIO	BIOES
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
Chicago	I-LOC	B-LOC	S-LOC

14.9.2 Algorithms and Evaluation

Algorithms: supervised ML on BIO-tagged text, bidirectional LSTMs/GRUs, and fine-tuned BERT (current standard). Evaluation uses precision, recall, and F_1 against a gold standard:

$$P = \frac{|O \cap G|}{|O|}, \quad R = \frac{|O \cap G|}{|G|}, \quad F_1 = \frac{2PR}{P + R}$$

where O = system output entity set and G = gold standard.

15 Word Senses and Disambiguation

15.1 Word Senses and Lexical Relations

Definition 15.1 (Core Terminology). *Key terms for describing word senses:*

- **Lemma**: the citation/dictionary form of a word (e.g., *run*).
- **Wordform**: an inflected surface form of the lemma (e.g., *runs, ran, running*).
- **Sense**: a discrete representation of one aspect of the meaning of a lemma; one lemma can have multiple senses (e.g., *run*¹ “to move fast on foot” — “She **ran** to the bus” — vs *run*² “to manage or operate” — “He **runs** a restaurant”).

Definition 15.2 (Types of Ambiguity). *Three main types of lexical ambiguity arise in NLP:*

- **Homonymy**: two unrelated words share the same form. Two subtypes:
 - **Homographs**: same spelling. English: *bank*₁ (financial institution) vs *bank*₂ (sloping land); *bat*₁ (club for hitting a ball) vs *bat*₂ (nocturnal flying mammal).
 - **Homophones**: same pronunciation, different spelling. English: *write/right*; *piece/peace*.

Causes issues in IR (ambiguous query results), MT (incorrect word choice), and TTS (mispronunciation, e.g. bass as fish vs instrument).

- **Polysemy**: one word with multiple related senses (unlike homonymy, where meanings are unrelated). Example: *bank*₁ (the building) vs *bank*₂ (the financial institution)—both tied to the same concept. The key distinction: polysemous senses are semantically connected.
- **Metonymy** (also called **systematic polysemy**): a systematic relationship between senses that recurs across multiple words of the same semantic type. Example: *school, university, and hospital* all alternate between an institution and a building sense (“The hospital was evacuated” vs “She works for the hospital”). Also: using one entity to refer to a related entity (“The White House announced...”).

Remark 15.1 (Zeugma Test). *The zeugma test distinguishes homonymy from polysemy: combining both senses of a word in a single sentence sounds odd if the senses are truly unrelated. Example: “?Does Lufthansa serve breakfast and San Jose?” is awkward because serve (food) and serve (operate a route) are distinct senses – confirmed homonymy. If the combined sentence sounds natural, the senses are likely related (polysemy).*

Definition 15.3 (Lexical Relations). *We provide the most important lexical relations*

- **Synonymy**: same meaning in some/all contexts. Perfect synonymy is rare; near-synonyms differ in register or connotation (*brave* vs. *courageous*). Synonymy holds between senses, not words: *big/large* are synonyms in most contexts, but not in “a **big** sister” (*older*).
- **Antonymy**: opposites along one semantic feature (*dark/light, fast/slow*). Types: (1) **scale antonyms** – opposite ends of a scale (*long/short*); (2) **reversives** – opposite directions or actions (*rise/fall*).
- **Hyponymy/Hypernymy** (IS-A): *A* is a **hyponym** of *B* if being an *A* entails being a *B* (*car* $\subset$ *vehicle*). *B* is the **hypernym**. Transitive: *robin* $\rightarrow$ *bird* $\rightarrow$ *animal*.
- **Meronymy/Holonymy** (PART-OF): *A* is a **meronym** of *B* if *A* is part of *B* (*wheel* $\subset$ *car*); *B* is the **holonym**.

15.2 Lexical Resources

Definition 15.4 (WordNet). *WordNet (Princeton) is a hierarchically organised lexical database and online thesaurus, available for many languages (via the Global WordNet Association). Words are grouped into **synsets** (synonym sets) each with a gloss (definition) and example sentences. A synset represents one sense shared by multiple words: e.g. the gloss “a person who is gullible and easy to take advantage of” covers words like *chump*¹, *fool*², *sucker*¹, and others. In lexicography, **splitters** create many fine-grained senses while **lumpers** create fewer coarse senses; WordNet tends to split (e.g. *bass* has 8 noun senses).*

Remark 15.2 (WordNet Noun Relations). *Full set of noun relations in WordNet:*

Relation	Also Called	Definition	Example
Hypernym	Superordinate	Concept to superordinate	breakfast ¹ → meal ¹
Hyponym	Subordinate	Concept to subtype	meal ¹ → lunch ¹
Instance Hypernym	Instance	Instance to its concept	Austen ¹ → author ¹
Instance Hyponym	Has-Instance	Concept to instance	composer ¹ → Bach ¹
Member Meronym	Has-Member	Group to its members	faculty ² → professor ¹
Member Holonym	Member-Of	Member to its group	copilot ¹ → crew ¹
Part Meronym	Has-Part	Whole to its parts	table ² → leg ³
Part Holonym	Part-Of	Part to its whole	course ⁷ → meal ¹
Substance Meronym	—	Substance to subpart	water ¹ → oxygen ¹
Substance Holonym	—	Subpart to substance	gin ¹ → martini ¹
Antonym	—	Semantic opposition	leader ¹ ⇔ follower ¹
Deriv. Related Form	—	Same morphological root	destruction ¹ ⇔ destroy ¹

Remark 15.3 (WordNet Verb Relations). Verbs have their own set of relations (no meronymy; instead troponymy and entailment):

Relation	Definition	Example
Hypernym	From events to superordinate events	fly ⁹ → travel ⁵
Troponym	Subordinate event (often by manner)	walk ¹ → stroll ¹
Entails	From verbs to verbs they entail	snore ¹ → sleep ¹
Antonym	Semantic opposition	increase ¹ ⇔ decrease ¹
Deriv. Related Form	Same morphological root	destroy ¹ ⇔ destruction ¹

Definition 15.5 (Supersenses). **Supersenses** are the top-level hypernym categories in the WordNet hierarchy, providing coarse-grained semantic labels useful as features in NLP tasks. There are 26 noun supersenses (e.g. PERSON, LOCATION, ARTIFACT, FOOD, EVENT) and 15 verb supersenses (e.g. STATIVE, MOTION, COGNITION, COMMUNICATION). Example supersense annotation: “I googled_{comm} restaurants_{group} in the area_{location} and reviews_{comm} were_{stative} great so I made a carry-out_{poss} order_{comm}”

Definition 15.6 (Word Similarity vs Relatedness). **Similarity** (near-synonymy) requires words to be of the same semantic type: car/bicycle are similar; car/gasoline are related but not similar. Similarity is properly a **relation between senses, not words**: the word “bank” is not similar to “slope”, but bank² (river bank) is similar to slope⁵, while bank¹ (financial) is similar to fund³. In practice we often compute wordsim as the max over all sense pairs.

Two classes of similarity algorithms:

- **Thesaurus-based:** are words nearby in the hypernym hierarchy? Do words have similar glosses?
- **Distributional:** do words have similar distributional contexts? (vector semantics / word embeddings)

Path-based similarity (thesaurus-based): $\text{simpath}(c_1, c_2) = 1/\text{pathlen}(c_1, c_2)$, where pathlen counts edges in the shortest hypernym-graph path (1 to itself). Word similarity is then the max over all sense pairs:

$$\text{wordsim}(w_1, w_2) = \max_{c_1 \in \text{senses}(w_1), c_2 \in \text{senses}(w_2)} \text{simpath}(c_1, c_2)$$

Example: $\text{simpath}(\text{nickel}, \text{coin}) = 0.5$ (one hop); $\text{simpath}(\text{nickel}, \text{money}) = 0.17$ (five hops). **Limitation:** treats all edges as equal, but high-level nodes (entity, object) cover far more semantic ground than low-level ones. All WordNet-based refinements (Resnik, Lin, Jiang-Conrath) are now **considered obsolete**, superseded by distributional embeddings.

Evaluation: intrinsic – correlation with human judgements (e.g. Wordsim353: pairs rated 0–10); extrinsic – improvement on downstream tasks (QA, plagiarism detection).

15.3 WSD Task

Definition 15.7 (Word Sense Disambiguation (WSD)). Given word w in context c and a sense inventory $\{s_1, \dots, s_k\}$, determine which sense s^* applies. Applications: machine translation (choose the correct translation per sense, e.g. English-to-Spanish), QA, speech synthesis (homographs like bass, bow have different pronunciations per sense), and LLM benchmarking.

Four families of WSD methods: (1) Supervised ML, (2) Thesaurus/Dictionary methods, (3) Semi-supervised learning, (4) WiC-based approaches.

15.3.1 Supervised WSD

Remark 15.4 (Training Data). *Two corpus types: **lexical sample corpora** densely annotate a few target words (e.g. line, serve, interest); **all-words corpora** annotate every content word in running text (e.g. SemCor, SENSEVAL-3, ElxisWSD in 10 languages). The main challenge is the cost of manual sense annotation.*

Remark 15.5 (WSD with BERT). *Compute contextual embedding of the target word from BERT, then either:*

1. *Add a softmax classification layer and fine-tune on sense-labelled data; or*
2. *Use 1-nearest-neighbour in BERT embedding space with sense prototype vectors (averaged SemCor embeddings per sense). Fallback: Most Frequent Sense, or average over WordNet taxonomy neighbours.*

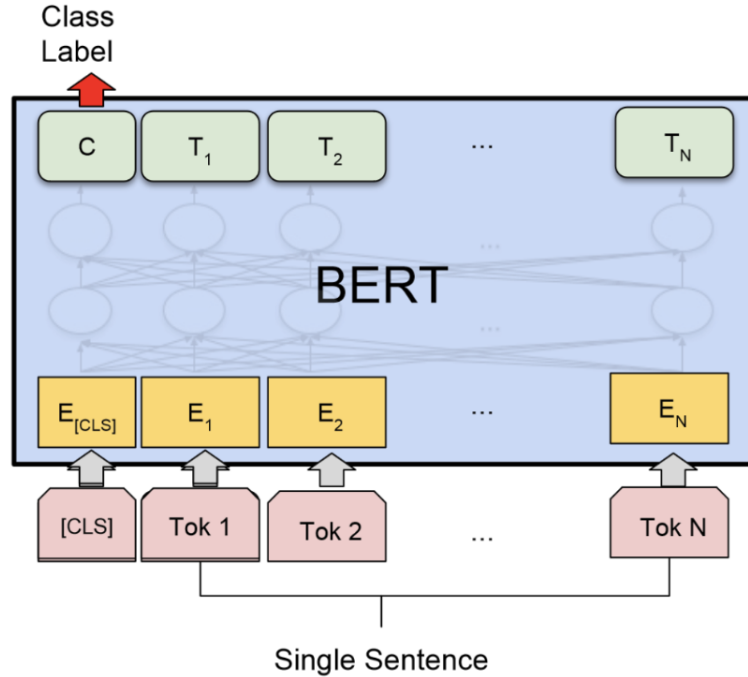


Figure 36: Word sense disambiguation with BERT: fine-tuning vs. nearest neighbour.

15.3.2 Baselines

Definition 15.8 (Most Frequent Sense (MFS)). *Always assign the sense with the highest frequency in SemCor. WordNet senses are listed in decreasing frequency order, so “most frequent sense” = “take the first sense”. Sense frequencies come from SemCor. This is a surprisingly strong baseline. Example for plant (note: the ordering is counterintuitive):*

Freq	Synset	Gloss
338	plant ¹ , works, industrial plant	buildings for carrying on industrial labor
207	plant ² , flora, plant life	a living organism lacking power of locomotion
2	plant ³	something planted secretly for discovery by another
0	plant ⁴	an actor planted in the audience

MFS always predicts plant¹ (industrial), even though plant² (living organism) is more salient to most speakers. Frequencies reflect the Brown Corpus, which skews toward news/non-fiction.

Definition 15.9 (Simplified Lesk Algorithm). *For each candidate sense s_i : count the overlap between content words in s_i ’s WordNet gloss/examples and the context words of w . Return the sense with maximum overlap.*

Example: *Disambiguating bank in “The bank can guarantee deposits will cover tuition costs”. Context words: {guarantee, deposits, cover, tuition, costs}. Gloss for bank¹ (financial): “accepts deposits and channels money into lending” – overlap: {deposits} = 1. Gloss for bank² (river): “sloping land beside a body of water” – overlap: {} = 0. Lesk selects bank¹. ✓*

Definition 15.10 (Corpus Lesk with IDF Weighting). *Augment the gloss with SemCor sentences labelled with each sense (the signature). Weight overlapping words by IDF to down-weight generic function words:*

$$\text{idf}_w = \log \frac{N}{\text{df}_w}, \quad \text{score}(\text{sense}_i, \text{context}_j) = \sum_{w \in \text{overlap}(\text{sig}_i, \text{ctx}_j)} \text{idf}_w$$

Remark 15.6 (Evaluation). *Intrinsic: sense accuracy = % of tokens that match the human-annotated sense on held-out data. Human inter-annotator agreement is 75–80%, setting the performance ceiling. Extrinsic (task-based) evaluation is considered the gold standard.*

15.3.3 Semi-Supervised WSD – Bootstrapping

Definition 15.11 (Yarowsky Bootstrapping (1995)). *Motivated by the scarcity of sense-labelled data. Start from a small hand-labelled seed set and iteratively label unlabelled instances using two heuristics:*

1. **One sense per collocation:** a word reoccurring with the same neighbour almost always has the same sense (*play* → music sense of bass; *fish* → animal sense).
2. **One sense per discourse:** sense is highly consistent within a document, especially for topic-specific nouns.

Each iteration adds high-confidence instances to training, expanding the labelled set. The seed set can be bootstrapped by hand-labelling a few examples or by applying the same two heuristics directly to known collocations/document topics.

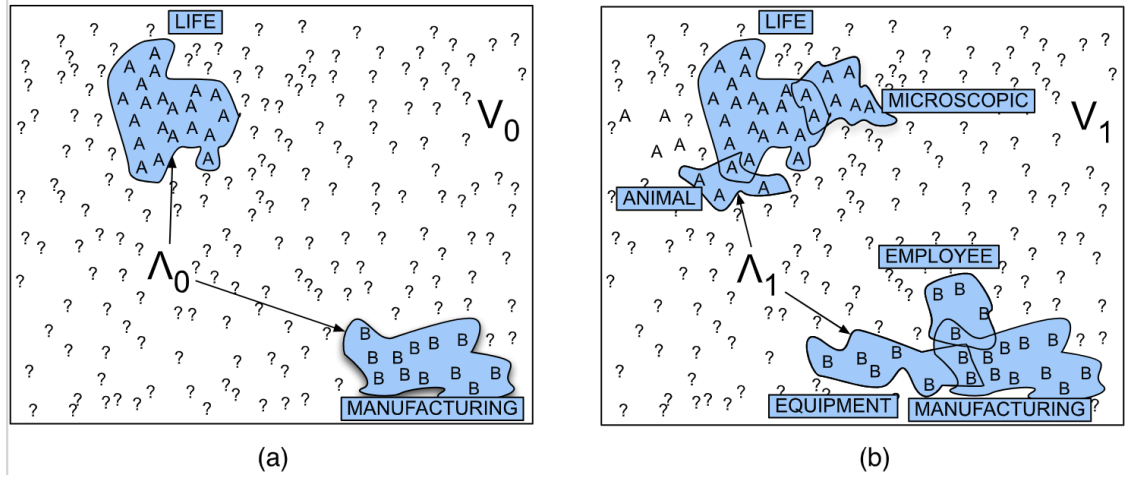


Figure 37: Semi-supervised WSD and bootstrapping.

15.3.4 Word Sense Induction and WiC

Definition 15.12 (Word Sense Induction (WSI)). *WSI is fully unsupervised – it induces senses without a pre-defined inventory:*

1. For each token w_i of word w , compute a context vector (e.g., BERT embedding)
2. Cluster context vectors; each cluster defines one sense
3. Represent each sense by the cluster centroid s_j

Weakness: induced clusters have no human-readable names (solvable post-hoc with LLMs). Evaluation against a hand-labelled gold standard; significant cluster overlap is a practical challenge.

Definition 15.13 (WiC for WSD/WSI (Škvorc & Robnik-Šikonja 2025)). *The **Word-in-Context (WiC)** dataset poses WSD as binary classification: given two sentences containing the same word, do they share the same sense? WiC data can be sourced from dictionaries or generated synthetically with LLMs. Note: WordNet senses are sometimes too fine-grained for machine recognition.*

WiC examples (T = same sense, F = different sense):

F “There’s a lot of trash on the **bed** of the river” / “I keep a glass of water next to my **bed** when I sleep”

F “**Justify** the margins” / “The end **justifies** the means”

T “**Air** pollution” / “Open a window and let in some **air**”

T “The expanded **window** will give us time to catch the thieves” / “You have a two-hour **window** of clear weather”

WSD/WSI pipeline using WiC: (1) build a WiC classifier (same/different sense in two contexts; data from dictionaries or synthetic); (2) for a new sentence, apply the classifier against all example sentences from the sense inventory; (3) select the sense with the highest match score; (4) if no sense matches, this may indicate a new sense; (5) for WSI, check if the non-matching sentences match each other to discover new sense clusters.

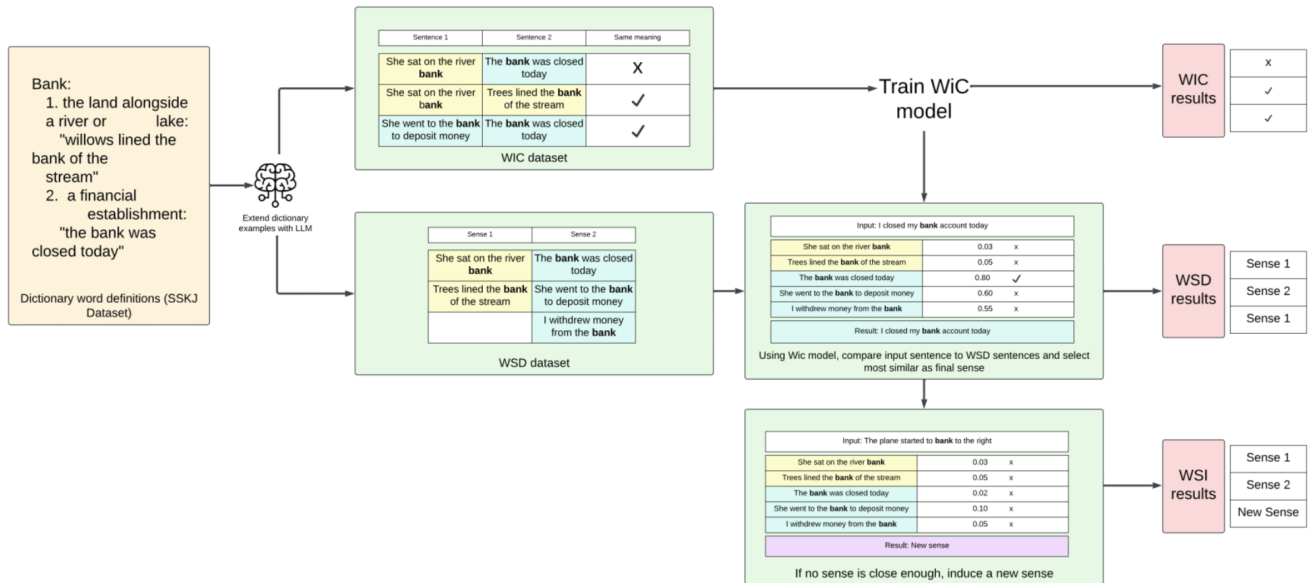


Figure 38: Example of WiC for WSD and WSI.

Remark 15.7 (Summary). Four main WSD/WSI approaches: (1) **Supervised ML** via BERT contextual embeddings (fine-tuning or 1-NN with sense prototype vectors; fallback: MFS or WordNet taxonomy averaging); (2) **Thesaurus/Dictionary** methods (Lesk, Corpus Lesk with IDF weighting) – outdated but used as baselines; (3) **Semi-supervised bootstrapping** (Yarowsky 1995; one-sense-per-collocation and one-sense-per-discourse heuristics); (4) **WiC-based** approach (train a same/different-sense binary classifier, apply against sense inventory). Key insight: a word's context contains sufficient information to disambiguate. Simple word-count baselines (MFS, Lesk) are competitive; LLMs and contextual embeddings greatly improved performance. Applications: machine translation, QA, speech synthesis, **LLM benchmarking**.

16 Affective Computing

16.1 Emotion Theory

Remark 16.1 (Affective Meaning and Motivation). **Connotation:** *an idea or feeling which a word invokes beyond its literal meaning (e.g., “discipline” carries connotations of punishment and repression). Words carry both sense and connotation – the affective dimension that affective lexicons aim to capture.*

Why compute affective meaning?

- **Detecting:** *sentiment towards politicians, products, countries, ideas; frustration of call-centre callers; stress in drivers or pilots; depression and other medical conditions; confusion in students talking to e-tutors; emotions in novels (e.g., studying groups that are feared over time).*
- **Generating:** *emotions/moods for literacy tutors in children’s storybooks; emotions for computer games; personalities for dialogue systems to match the user.*

Emotional arcs in fiction: 85% of 1,327 Project Gutenberg fiction stories follow one of six narrative arcs: Rags to Riches (*rise*), Tragedy (*fall*), Man in a Hole (*fall-to-rise*), Cinderella (*rise-fall-rise*), Icarus (*rise-to-fall*), Oedipus (*fall-rise-fall*). Arcs are traced by the happiness/sadness of words across the running text.

Definition 16.1 (Scherer’s Typology of Affective States). *Scherer identifies five affective categories:*

- **Emotions:** *relatively brief episode of synchronized response of all or most organismic subsystems in response to the evaluation of an event as being of major significance. Examples: angry, sad, joyful, fearful, ashamed, proud, desperate.*
- **Moods:** *diffuse affect state; change in subjective feeling, of low intensity but relatively long duration, often without apparent cause. Examples: cheerful, gloomy, irritable, listless, depressed, buoyant.*
- **Interpersonal stance:** *affective stance taken toward another person in a specific interaction, coloring the interpersonal exchange. Examples: distant, cold, warm, supportive, contemptuous.*
- **Attitudes:** *relatively enduring, affectively colored beliefs and preferences toward objects or persons. Examples: liking, loving, hating, valuing, desiring. Sentiment analysis targets this category.*
- **Personality traits:** *emotionally laden, stable personality dispositions and behavior tendencies typical for a person. Examples: nervous, anxious, reckless, morose, hostile, envious, jealous.*

Definition 16.2 (Categorical Emotion Theories). **Ekman (1969):** 6 basic emotions with universal facial expressions: surprise, happiness, anger, fear, disgust, sadness.

Plutchik (1980): 8 basic emotions arranged in 4 opposing pairs:

joy – sadness anger – fear trust – disgust anticipation – surprise

Complex emotions arise from combinations (e.g., love = joy + trust). The **Plutchik wheel** also represents intensities: ecstasy > joy > serenity; rage > anger > annoyance, etc.

Remark 16.2 (Distinctive vs. Dimensional Theories). *Two competing paradigms for modelling emotion:*

<i>Distinctive (atomic)</i>	<i>Dimensional</i>
<i>Emotions are discrete units</i>	<i>Emotions are points in a continuous space</i>
<i>Finite list of 6 or 8 basic emotions</i>	<i>Limited labels, unlimited possible emotions</i>
<i>Basic emotions are innate and universal</i>	<i>Emotions are culturally learned</i>

NRC EmoLex uses the distinctive (Plutchik) paradigm; Warriner et al. use the dimensional (VAD) paradigm.

Definition 16.3 (Dimensional Theories). *Emotions are modelled as points in a continuous space:*

- **Valence** (*hedonic tone*): *pleasantness of the stimulus; positive vs negative*
- **Arousal:** *intensity of emotion provoked by the stimulus; excited vs calm*
- **Dominance/Control:** *degree of control exerted by the stimulus; in control (active) vs controlled (passive)*

Key quadrants in valence/arousal space: high arousal + low pleasure = anger; high arousal + high pleasure = excitement; low arousal + low pleasure = sadness; low arousal + high pleasure = relaxation.

The **Hourglass of Emotion** (Cambria et al.) uses four dimensions each ranging from −3 to +3:

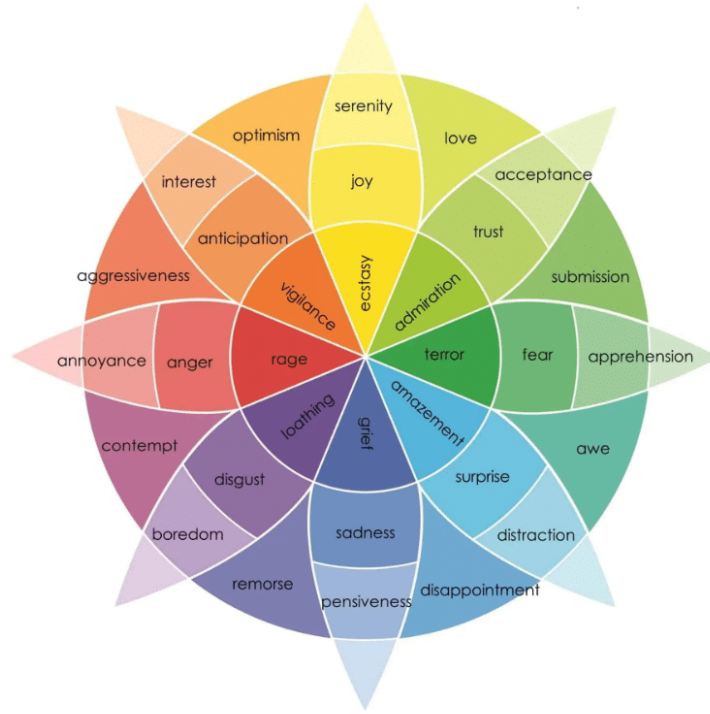


Figure 39: 8 dimensions of emotions.

<i>Level</i>	<i>Pleasantness</i>	<i>Attention</i>	<i>Sensitivity</i>	<i>Aptitude</i>
+3	<i>ecstasy</i>	<i>vigilance</i>	<i>rage</i>	<i>admiration</i>
+2	<i>joy</i>	<i>anticipation</i>	<i>anger</i>	<i>trust</i>
+1	<i>serenity</i>	<i>interest</i>	<i>annoyance</i>	<i>acceptance</i>
0	—	—	—	—
−1	<i>pensiveness</i>	<i>distraction</i>	<i>apprehension</i>	<i>boredom</i>
−2	<i>sadness</i>	<i>surprise</i>	<i>fear</i>	<i>disgust</i>
−3	<i>grief</i>	<i>amazement</i>	<i>terror</i>	<i>loathing</i>

16.2 Sentiment and Emotion Lexicons

Remark 16.3 (Key English Lexicons). *We have the following english lexicons:*

- **General Inquirer** (Stone et al. 1966): Harvard psychosocial dictionary. Categories include Positive/Negative, Strong/Weak, Active/Passive, Pleasure/Pain, Virtue/Vice. Free.
- **LIWC** (Pennebaker et al. 2007): 2,300 words, >70 classes. Covers affective processes (positive/negative emotion), cognitive processes (tentative, inhibition), pronouns, negation, quantifiers. Commercial.
- **MPQA** (Wilson et al. 2005): 6,885 words (2,718 pos., 4,912 neg.), annotated for intensity (strong/weak). GNU GPL.
- **Bing Liu Opinion Lexicon** (Hu & Liu 2004): 6,786 words (2,006 pos., 4,783 neg.). Widely used polarity resource.
- **SentiWordNet** (Baccianella et al. 2010): All WordNet synsets scored for positivity, negativity, and objectivity. E.g., *estimable*(J,1) “deserving respect” → Pos .75, Neg 0, Obj .25.
- **NRC EmoLex** (Mohammad & Turney 2011): 10,170 words labeled for 8 Plutchik emotions + 2 polarity labels via crowdsourcing.
- **Warriner et al. 2013**: 13,915 words rated on 9-point VAD (Valence–Arousal–Dominance) scales via AMT.

Remark 16.4 (Lexicon Disagreements). *Polarity lexicons frequently disagree. Potts (2011) reports disagreement rates on words shared between lexicons (format: disagreements/overlap):*

	<i>Opinion Lex</i>	<i>Gen. Inquirer</i>	<i>SentiWordNet</i>	<i>LIWC</i>
<i>MPQA</i>	33/5402 (0.6%)	49/2867 (2%)	1127/4214 (27%)	12/363 (3%)
<i>Opinion Lex</i>		32/2411 (1%)	1004/3994 (25%)	9/403 (2%)
<i>Gen. Inquirer</i>			520/2306 (23%)	1/204 (0.5%)
<i>SentiWordNet</i>				174/694 (25%)

SentiWordNet shows the highest disagreement because it is automatically constructed; hand-built lexicons (*MPQA*, *Opinion Lex*) agree most with each other.

Definition 16.4 (Lexicon Construction Methods). We present now some lexicon construction methods:

1. **By hand / crowdsourcing:** highest quality; expensive (NRC EmoLex via AMT; Warriner et al. VAD also via AMT).
2. **Semi-supervised from seed words:** propagate polarity through WordNet edges (synonyms share polarity; antonyms flip it).
3. **Fully supervised via natural signal:** use review star ratings as distant supervision (IMDB, Goodreads, Amazon, TripAdvisor); compute **scaled likelihood** $P(w|c)/P(w)$ per rating class. **Potts diagrams** reveal four word types: positive scalars (good, great, excellent – monotone increase with rating), negative scalars (disappointing, bad, terrible – monotone decrease), emphatics (totally, absolutely, utterly – appear in extreme reviews; U-shaped), attenuators (somewhat, fairly, pretty – peak at mid ratings; arch-shaped).

Important: pure lexicon-based detection is now largely **obsolete**; only used as a baseline or for extremely low-computation settings.

- **Unsupervised method:** sum weights of positive words in the document; sum weights of negative words; choose the higher. For emotion: repeat for each emotion category.
- **Supervised method:** use counts of lexicon categories as classifier features (e.g., LIWC “cognition” count = 7, NRC “anticipation” count = 2). Baseline alternative: use all word/bigram counts from training data (only works when train and test domains are very similar).

16.3 Personality Analysis

Definition 16.5 (Big Five Personality Traits). The five dimensions are:

- **Extraversion** vs Introversion: sociable, assertive, playful vs aloof, reserved, shy
- **Emotional Stability** vs Neuroticism: calm, unemotional vs insecure, anxious
- **Agreeableness** vs Disagreeableness: friendly, cooperative vs antagonistic, faultfinding
- **Conscientiousness** vs Unconscientiousness: self-disciplined, organized vs inefficient, careless
- **Openness to experience:** intellectual, insightful vs shallow, unimaginative

Remark 16.5 (Personality from Text). Key corpora: Pennebaker & King stream-of-consciousness essays (Big Five self-reports); Mehl EAR recorded speech (96 participants); Schwartz et al. Facebook corpus (75,000 volunteers, 309M words, self-reported Big Five). Word cloud findings: Extraversion $\leftrightarrow$ Introversion (party/girls/chillin vs anime/internet/computer); Neuroticism $\leftrightarrow$ Emotional Stability (depression/lonely/sick_of vs basketball/soccer/blessings).

Remark 16.6 (Interpersonal Stance – Speed Dating (Ranganath & McFarland)). 1000 four-minute dates; labels: friendly/awkward/flirtatious/assertive (1–10). Logistic regression with LIWC features, hedges, prosody (pitch/energy), discourse acts (interruptions, questions). Results: Friendliness correlates with fewer negative emotion words, fewer hedges, higher pitch; Awkwardness correlates with more negation, hedges, and questions.

16.4 Sentiment Analysis

Remark 16.7 (Why Sentiment Analysis?). Movies: is this review positive or negative? Products: what do people think about the new iPhone? Public sentiment: how is consumer confidence? Is despair increasing? Politics: what do people think about a candidate or issue? Prediction: predict election outcomes or market trends from sentiment.

Definition 16.6 (Sentiment Analysis). Also called opinion extraction/mining, subjectivity analysis (= sentiment mining). Targets **attitudes** (Scherer): “enduring, affectively colored beliefs, dispositions towards objects or persons”. A sentiment instance has four components:

- **Holder** (source): who expresses the attitude

- **Target (aspect):** what the attitude is about
- **Type:** from a set of types (like, love, hate, value, desire) or more commonly simple weighted **polarity** (positive/negative/neutral) with strength
- **Text:** the span expressing the attitude (sentence or full document)

Complexity levels: (1) Simplest – binary positive/negative; (2) More complex – rank from 1 to 5; (3) Advanced – detect target, source, or complex attitude types.

Remark 16.8 (Applications). Movie review polarity – Pang, Lee & Vaithyanathan (EMNLP 2002); Polarity Data 2.0 (IMDB). Product sentiment at feature level (Google/Bing). Public opinion: Twitter sentiment correlates with Gallup polls ($r = 0.804$; O’Connor et al. 2010). Stock market prediction (Bollen et al. 2011). Political opinion. Distant supervision via emoticons on Twitter (Go, Bhayani & Huang 2009).

Remark 16.9 (Challenges in Sentiment Classification). Here are the main challenges in sentiment classification:

- **Tokenization:** HTML/XML markup, Twitter hashtags and usernames, capitalization (ALL-CAPS carries emphasis), emoticons, phone numbers, dates
- **Subtlety and irony:** “If you are reading this because it is your darling fragrance, please wear it at home exclusively, and tape the windows shut.” – negative, but contains no negative lexical items. Also: Dorothy Parker on Katherine Hepburn: “She runs the gamut of emotions from A to B.” – sarcastic despite no negative words.
- **Thwarted expectations:** many positive words followed by a negative conclusion flip the overall polarity (e.g., “This film should be brilliant... However, it can’t hold up.”); ordering effects matter

Remark 16.10 (Models). Standard ML pipelines (logistic regression, SVM on n -grams): 60–70% accuracy. Neural approaches add a few percent. **BERT for sentiment:** feed [CLS]+tokens into BERT, attach a classification head on the [CLS] representation, fine-tune end-to-end.

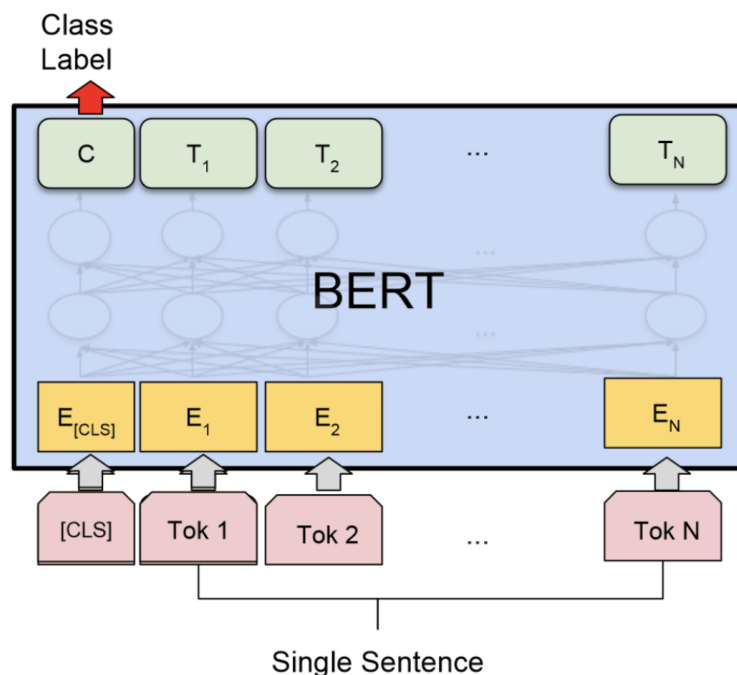


Figure 40: Input of the BERT model

Remark 16.11 (Cross-lingual Sentiment Transfer (Robnik-Šikonja et al. 2021)). Twitter sentiment dataset in 13 languages (Bosnian, Croatian, English, German, Slovene, etc.). Three representations compared:

1. **LASER:** 93-language joint embedding; MLP (8 hidden, 3 output neurons), ReLU, Adam
 2. **mBERT:** multilingual BERT (104 languages), fine-tuned
 3. **CroSloEngual BERT (CSE BERT):** fine-tuned on Croatian/Slovene/English; best for Slavic languages
- Expanding the training set with other languages does not help when the target language dataset is already large.

16.5 Related Tasks

Remark 16.12 (Aspect-Based Sentiment Detection). *Also called targeted sentiment. Assign polarity per aspect (fixed attributes like product properties, or auto-detected). Example: “We had a great experience at the restaurant, food was delicious, but the service was kinda bad” $\Rightarrow$ food: positive; service: negative.*

Remark 16.13 (Depression Detection from Text). *Reddit dataset (Kayalvizhi & Thenmozhi 2022): 13,000 posts from subreddits r/stress, r/loneliness, r/Anxiety, r/depression, 3 severity levels. Twitter dataset (Hu 2021): 66,000 labelled tweets, 4 severity levels. Baselines: majority class, TF-IDF, Doc2Vec. Best models: fine-tuned BERT variants (BERT, RoBERTa, mentalBERT, BERTweet) and Bayesian ensembles; hyperparameter tuning; relatively small differences between models. Transfer from Reddit to Twitter works; reverse does not (Tavchioski, Robnik-Šikonja & Pollak, LTC 2023).*

Remark 16.14 (Hate Speech and Offensive Language Detection). *Related to sentiment analysis but requires modelling intention, subtlety, and social context beyond lexical polarity. Best approach: supervised fine-tuning (SFT) of BERT models or LLMs on task-specific datasets. Raw LLMs with prompting lag behind fine-tuned models. Recent Slovene results (SlEng-BERT): $F_1 = 0.666$ for sentiment, $F_1 = 0.850$ for offensive language prediction.*

17 Machine Translation

17.1 Motivation and Classical Paradigms

Machine translation aims to automatically convert text from a source language into a fluent and faithful target language text. Key tensions:

- **Faithfulness** (fidelity): preserves the meaning of the source
- **Fluency**: sounds natural and grammatical in the target language

Practical motivation: digital divide (most web content is in few languages), Computer-Assisted Translation (CAT tools), lexical divergence across languages (e.g. English *leg/foot/paw* overlap non-trivially with French *jambe/pied/patte/étape* — Hutchins & Somers 1992).

Remark 17.1 (CAT, Localisation, and the Faithfulness–Fluency Tradeoff). ***CAT (Computer-Aided Translation)**: MT produces a draft translation that is fixed in a post-editing phase by a human translator. CAT is commonly used in **localisation**: the task of adapting content or a product to a particular language community.*

***Perfect translation is impossible.** Translating Hebrew *adonai roi* (“the lord is my shepherd”) for a culture with no sheep or shepherds forces a choice:*

- *Fluent but not faithful: The Lord will look after me.*
- *Faithful but not fluent: The Lord is for me like somebody who looks after animals with cotton-like hair.*

Real translations always trade off faithfulness and fluency.

Definition 17.1 (Three Translation Paradigms). *We present three translation paradigms:*

1. **Direct**: word-for-word substitution with local morphological adjustment; specific to one language pair
2. **Transfer**: analyse source to an abstract representation; apply transfer rules; generate target; requires rules per language pair
3. **Interlingual**: map source to a universal interlingua, then generate any target; most general but hardest to build

Remark 17.2 (MT as Decoding). *Norbert Wiener (1947, in a letter): “When I look at an article in Russian, I say, ‘This is really written in English, but it has been coded in some strange symbols. I will now proceed to decode.’ ” This cryptographic framing motivates the noisy channel model: the foreign sentence is treated as a “noisy encoding” of the English original, and the decoder recovers the most probable English.*

17.2 Statistical Machine translation (MT)

Definition 17.2 (Noisy Channel Model).

$$\hat{E} = \arg \max_E P(F | E) P(E)$$

where $P(E)$ is the **language model** (target-side fluency) and $P(F | E)$ is the **translation model** (source–target fidelity). A **decoder** searches the hypothesis space.

Three components of SMT:

1. **Language Model**: computes $P(E)$ – fluent target strings get high probability.
2. **Translation Model**: computes $P(F | E)$ – pairs that look like translations get high probability.
3. **Decoding algorithm**: given LM, TM, and F , find $\hat{E}$.

Division of labour: $P(F | E)$ models which source words map to which target words (no ordering); $P(E)$ handles ordering – putting words in the correct sequence. Key advantage: $P(E)$ requires only monolingual data.

Classical SMT sub-families: **word-based**, **phrase-based**, **tree-based**, **factored**.

Parallel corpora: **EuroParl** ($\approx 50M$ words/EU language; Koehn 2005), **DGT** (24 EU languages; largest parallel corpus; available on CLARIN), **OPUS** (subtitles and web data). Sentence alignment uses multilingual embeddings + dynamic programming.

Remark 17.3 (Sentence Alignment and Corpus Cleanup). *Sentence alignment finds minimal sets of sentences that are mutual translations. Alignment types: **single-sentence** (E_i, F_j), **many-to-one** (2-1) ($E_i/E_{i+1}, F_j$), and **null alignments** (a sentence with no counterpart in the other language). After alignment, **corpus cleanup** removes: too long or too short sentence pairs, too-similar pairs (copies rather than translations), and pairs with low multilingual-embedding cosine score.*

17.3 Neural MT

Definition 17.3 (Seq2Seq NMT). *Proposed by Sutskever et al. (2014) and Cho et al. (2014). An encoder reads the source sequence and produces a hidden representation; an autoregressive decoder generates the target. NMT is a **conditional language model**:*

$$P(y_1, \dots, y_m \mid x_1, \dots, x_n) = \prod_{t=1}^m P(y_t \mid y_{<t}, \mathbf{h})$$

Trained by minimising negative log-likelihood end-to-end via backpropagation.

Definition 17.4 (Decoding). *The main strategies include:*

- **Greedy decoding:** $\hat{y}_t = \arg \max_y P(y \mid y_{<t}, x)$ at each step; complexity $O(V \cdot T)$; fast but suboptimal — cannot undo earlier decisions since each choice is locally optimal but not necessarily globally optimal.
- **Exhaustive search:** track all V^t partial translations at each step; complexity $O(V^T)$ — theoretically optimal but completely intractable in practice (e.g. $V = 50,000$, $T = 20$ gives $50,000^{20}$ hypotheses).
- **Beam search** ($k = 5-10$): maintain exactly k most probable partial hypotheses at each step. At each step, expand all k hypotheses by V tokens ($k \cdot V$ candidates), then prune back to the top k by cumulative score — keeping the frontier size constant throughout. Complexity $O(k \cdot V \cdot T)$, linear in k .

$$\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P(y_i \mid y_{<i}, x)$$

All terms are negative (log of a probability); higher (less negative) is better. Apply length normalisation $\frac{1}{t} \sum_{i=1}^t \log P(y_i \mid y_{<i}, x)$ only when selecting the final hypothesis (not during expansion, to avoid prematurely favouring short sequences); backtrack through stored decisions to recover the full sequence.

- **Stopping criterion:** in greedy decoding, stop at **<END>**. In beam search, different hypotheses complete at different steps; set each completed hypothesis aside and continue until (a) a pre-defined time step T is reached, or (b) at least n completed hypotheses have been found.
- **Effect of beam size k :** small k ($= 1$ recovers greedy) produces ungrammatical output; moderate k improves quality at cost $O(k \cdot V \cdot T)$. Counterintuitively, too large k hurts: for NMT it decreases BLEU (Tu et al.; Koehn et al.) — primarily because it produces overly short translations even with length normalisation, and can degenerate to empty translations (Stahlberg & Byrne 2019). In open-ended tasks (e.g. chit-chat dialogue), large k converges to generic, high-frequency responses, since the globally safest path through the distribution tends to be bland and off-topic. The deeper issue is that more search is not always better when the model's probability distribution is imperfectly calibrated with respect to human judgement.

Remark 17.4 (Transformer Architecture for NMT). *The **encoder** applies multi-head self-attention + feed-forward + layer normalisation to produce $\mathbf{H}_{\text{enc}}$. The **decoder** applies (1) causal (masked) self-attention, (2) **cross-attention** using $\mathbf{H}_{\text{enc}}$ as keys and values, and (3) feed-forward + layer normalisation.*

Remark 17.5 (NMT vs SMT). **Advantages of NMT** over SMT: better performance, more fluent output, better use of context, better use of phrase similarities, no feature engineering, same method for all language pairs, optimised end-to-end as a single network (no subcomponents to individually optimise). **Disadvantages:** less interpretable (hard to debug); difficult to control (cannot easily specify rules or guidelines for translation; safety concerns with biased outputs).

Remark 17.6 (Open Problems and Biases). *Out-of-vocabulary words, domain mismatch, very long documents, low-resource language pairs, common sense, idiomatic expressions. **Gender bias:** Malay gender-neutral “dia” becomes gendered English (nurse = she, programmer = he). **Uninterpretable failures:** repeated nonsense tokens in Google Translate once produced hallucinated religious prophecies.*

17.4 MT Evaluation

17.4.1 Human Evaluation

Definition 17.5 (Human Evaluation Dimensions). *Two key dimensions are:*

- **Fluency:** *is the output intelligible and natural in the target language? (monolingual judges)*
- **Adequacy / Fidelity:** *does it express the same meaning as the source? (bilingual judges score; monolinguals measure informativeness as % of reference questions answerable from the translation alone)*

17.4.2 Automatic Evaluation Metrics

Definition 17.6 (WER and PER). **WER** (Word Error Rate) = Levenshtein distance (insert/delete/substitute) to the reference, normalised by reference length. Good for fluency; not so good for fidelity; inflexible — e.g. “he saw a man and a woman” vs. reference “he saw a woman and a man” gives high WER despite being semantically equivalent.

PER (Position-Independent WER) = unigram matching ignoring order. Not good for fluency; too flexible for fidelity — e.g. “he saw a man” and “a man saw he” receive the same PER score against reference “he saw a man”.

Definition 17.7 (BLEU (Papineni et al. 2002)). *N*-gram precision (word-level) against one or more reference translations:

1. For each *n*-gram in the MT output, count matches against any reference; clip to the maximum count in any single reference
2. Compute p_n (clipped precision) for $n = 1, 2, 3, 4$
3. Apply brevity penalty $BP = \min(1, e^{1-r/c})$ where c is output length, r reference length
4. $BLEU = BP \times \exp\left(\frac{1}{4} \sum_{n=1}^4 \log p_n\right)$

Example: hypothesis “the cat sat on the mat” vs. reference “the cat sat on the rug”: $p_1 = 5/6$, $p_2 = 4/5$, $p_3 = 3/4$, $p_4 = 2/3$, $BP = 1$ (same length) $\Rightarrow BLEU \approx 0.74$. Now consider the semantically identical hypothesis “a feline rested on the rug”: only unigrams on, the, rug match $\Rightarrow BLEU \approx 0.18$, despite perfect meaning preservation. This is the core failure mode: BLEU is purely lexical and blind to synonyms and paraphrases.

Note: BLEU is precision-based — recall is ignored; the brevity penalty partially compensates for short outputs. Other automatic metrics: **NIST**, **TER** (Translation Edit Rate), **METEOR**. BLEU works well for comparing similar systems; less reliable across different system types. No good sentence-level variant exists; no measure of absolute translation quality.

Definition 17.8 (BERTScore). Compute contextual BERT embeddings for each token in both system output ($\tilde{x}$) and reference (x); measure pairwise cosine similarities:

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_i \in x} \max_{\tilde{x}_j \in \tilde{x}} \cos(x_i, \tilde{x}_j), \quad P_{\text{BERT}} = \frac{1}{|\tilde{x}|} \sum_{\tilde{x}_j \in \tilde{x}} \max_{x_i \in x} \cos(x_i, \tilde{x}_j)$$

F_{BERT} is the harmonic mean. IDF weighting down-weights common tokens.

Example: same pair “a feline rested on the rug” vs. “the cat sat on the rug”. Each hypothesis token is greedily matched to its closest reference token in embedding space: feline $\leftrightarrow$ cat (cos $\approx$ 0.91), rested $\leftrightarrow$ sat (cos $\approx$ 0.87), exact matches for on, the, rug (cos = 1.00) $\Rightarrow F_{\text{BERT}} \approx 0.93$, compared to BLEU ≈ 0.18 for the same pair. The gap is explained entirely by synonyms living close together in embedding space.

18 Summarization and Question Answering

18.1 Automatic Summarization

Definition 18.1 (Summary Types). *The goal of automatic text summarization is to produce a succinct summary preserving the most important information from one or more source documents about the same topic. Neural summarization uses the same seq2seq technology as MT.*

- **Output style:** *abstract* (concise overview placed at the start of a document), *outline* (structural plan organising main ideas hierarchically), *headline* (brief attention-grabbing title, as used in journalism)
- **Source:** *SDS* (single-document) vs *MDS* (multi-document)
- **Generation method:** **Extractive** (copy-paste phrases/sentences from source; problem: redundancy), **Compressive** (shorten extracted sentences via dependency-tree subtrees; now considered outdated), **Abstractive** (generate new cohesive text not necessarily present in the source)
- **Goal:** **Generic** vs **Query-based** (also targeted or aspect-based): summarise a document with respect to a user's information need; a kind of complex QA where the answer is produced by summarising relevant passages
- **ML approach:** supervised or unsupervised

Remark 18.1 (Summarization Applications). *Outlines, abstracts, or headlines of any document or article; summaries of email threads and web commentaries; extracting action items from meetings; simplifying text by compressing sentences; summarization for a specific purpose, customer, or point of view; generating informative headlines to replace click-bait ones.*

Remark 18.2 (Datasets). *English SDS datasets (varying source length and style): **Gigaword** (first 1–2 sentences → headline; aka sentence compression), **LCSTS** (Chinese microblogging: paragraph → sentence summary), **NYT / CNN/DailyMail** (news article → multi-sentence summary), **Wikihow** (how-to article → summary sentences), **XSum** (Narayan et al., 2018) and **Newsroom** (Grusky et al., 2018) (article → 1-sentence summary), **BookSum** (Kryściński et al., 2021: novels, plays, stories; human-written summaries at paragraph-, chapter-, and book-level). Slovene: **STA** (127,563 news articles, 1,000–3,000 chars each, first paragraph as summary), **Wikipedia** (2,100 Slovene articles of sufficient length, first paragraph as summary), **KAS-abstracts** (Slovene theses with Slovene and English summaries; used for cross-lingual evaluation).*

*Sentence simplification is a related but different task: rewrite source text in a simpler (sometimes shorter) way. Key datasets: **Simple Wikipedia** (standard Wikipedia sentence → simple version) and **Newsela** (news article → four increasingly simplified versions).*

Remark 18.3 (Neural Summarization Approaches). *We present some neural summarization approaches:*

- **Paulus et al. (2017):** seq2seq + RL to directly optimise ROUGE-L (standard ML training cannot, as ROUGE-L is non-differentiable). Key finding: RL achieves higher ROUGE scores but lower human judgment scores than ML training.
- **Wang et al. (2019):** BERT encoder + extractive head + abstractive decoder + RL; extractor and abstractor are trained end-to-end with a reinforcement reward.
- **Pegasus** (Zhang et al. 2020): transformer encoder-decoder pre-trained with gap sentence generation – whole sentences are masked and predicted as a single output sequence; sentences are chosen by importance (not randomly), making the objective directly analogous to summarization. Combines masked language modelling (token-level) with gap sentence generation (sentence-level).
- **Cross-lingual** (Žagar & Robnik-Šikonja 2022): English pre-trained model with MUSE cross-lingual embedding mapping → fine-tune on STA → generate 64 hypotheses and re-score with ROUGE, multilingual BERT, and transformer LM.
- **Unsupervised** (TextRank and variants): represent each sentence as a node; connect nodes by pairwise sentence similarity scores to build a document graph; rank nodes via graph centrality / PageRank; extract the most central sentences. LaBSE sentence embeddings enable multilingual user-comment summarization (Žagar & Robnik-Šikonja 2021, EACL Hackashop).

Definition 18.2 (ROUGE (Lin 2004)). **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation) compares an automatically produced summary against one or more human reference summaries.

ROUGE-N: n -gram recall between system and reference:

$$\text{ROUGE-N} = \frac{\sum_{\text{gram}_n \in \text{ref}} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{\text{gram}_n \in \text{ref}} \text{Count}(\text{gram}_n)}$$

ROUGE-L: longest common subsequence (LCS) recall/precision/ F_1 . Most commonly reported scores: ROUGE-1 (unigram), ROUGE-2 (bigram), ROUGE-L. ROUGE scores are reported separately for each n -gram level; an F_1 version is commonly reported alongside recall.

ROUGE vs. BLEU: ROUGE has no brevity penalty; ROUGE is recall-based while BLEU is precision-based. Recall is more appropriate for summarization (given a max-length constraint); precision is more appropriate for MT (then a brevity penalty compensates for under-translation). BLEU combines $n = 1 \dots 4$ into a single score; ROUGE reports each level separately.

Example 18.1 (ROUGE-2 with Multiple References). For the question “What is water spinach?”, a system produces: “Water spinach is a leaf vegetable commonly eaten in tropical areas of Asia.” Three human references are available:

Human 1	Water spinach is a green leafy vegetable grown in the tropics.
Human 2	Water spinach is a semi-aquatic tropical plant grown as a vegetable.
Human 3	Water spinach is a commonly eaten leaf vegetable of Asia.

Since there are multiple references, the ROUGE-N formula above sums matched and total counts across all references rather than averaging separate scores. Concretely: take every bigram of the system summary (e.g. water spinach, spinach is, is a, a leaf, ..., 12 bigrams in total) and, for each reference separately, count how many of that reference’s bigrams also occur among the system’s bigrams:

- Human 1 (10 bigrams): matches are water spinach, spinach is, is a $\rightarrow$ 3 matches.
- Human 2 (10 bigrams): the shared opening gives the same 3 matches (water spinach, spinach is, is a); nothing else overlaps.
- Human 3 (9 bigrams): matches are water spinach, spinach is, is a, commonly eaten, leaf vegetable, of asia $\rightarrow$ 6 matches.

Summing matches gives the numerator $3 + 3 + 6 = 12$; summing each reference’s total bigram count gives the denominator. Human 1 and Human 3 contribute 10 and 9 respectively; Human 2’s sentence also tokenises to 10 bigrams, so the literal denominator is $10 + 9 + 9 = 28$ only if Human 2 is (as in the original slide) counted as 9 – a one-bigram rounding slip, since 11 words give 10 bigrams. Using the slide’s reported tally:

$$\text{ROUGE-2} = \frac{3 + 3 + 6}{10 + 9 + 9} = \frac{12}{28} \approx 0.43,$$

while the arithmetically exact count ($10 + 10 + 9 = 29$ total bigrams) gives $12/29 \approx 0.41$ – both express the same idea: recall is pooled over all references at once, not averaged per-reference.

Remark 18.4 (QA-Based Summarization Evaluation). Standard n -gram overlap metrics (ROUGE, BLEU) do not capture factual consistency – a summary can score well while hallucinating facts. Two QA-based metrics address this:

QAGS (Question Answering and Generation for Summarization; Wang et al. 2020): Core intuition – if questions about a summary and its source receive similar answers, the summary is factually consistent with the source. Pipeline: (1) a QG module generates questions using named entities and noun phrases extracted from the summary as answer candidates; (2) low-quality and duplicate questions are filtered; (3) a QA model answers each question twice – once conditioned on the summary, once on the source document; (4) answer similarity is scored via exact match, token-level F_1 , edit distance, or BERTScore. Requires high-quality QG and QA models. Achieves significantly higher correlation with human factual consistency judgments than ROUGE.

QuestEval (Scialom et al. 2021): Extends QAGS with both a precision and a recall component, plus importance weighting. **Precision side**: questions Q_y are generated from the evaluated summary; the QA model answers them conditioned on the source (score eq. 1). **Recall side**: questions Q_x are generated from the source; a weighting module W down-weights unimportant questions; the QA model answers them conditioned on the summary (weighted score eq. 2). Both sides are combined to give the final QuestEval score.

Remark 18.5 (RQUGE: Reference-Free Evaluation of Question Generation). *QAGS* and *QuestEval* evaluate a summary by checking QA-answer agreement; **RQUGE** (Reference-free QUestion Generation Evaluation) instead evaluates the quality of generated questions themselves, without needing a human-written reference question – conceptually similar to ROUGE, but scoring overlap of important answer spans rather than n-grams. It has two learned components:

1. A **QA module**: given a candidate question q_c and the source context D , it predicts an answer span a_c (the position in D where the answer lies).
2. A **span scorer**: compares the predicted span a_c against the gold span a_r – the answer (named entity or text segment) that was originally selected before q_c was generated – conditioned on the context, and outputs a learned score.

This lets a QG system’s output be ranked for how well it targets answerable, salient content, without requiring a human reference question for every input; it acts as a learned proxy for human judgment of which parts of a text are worth asking about. **Weakness**: strongly dependent on the quality of the QA component – a weak QA module makes the whole score unreliable.

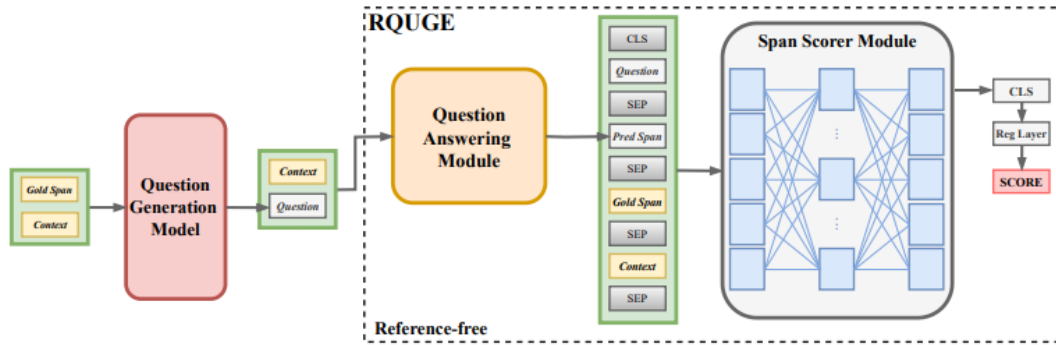


Figure 41: RQUGE architecture: a Question Generation model produces a question from a gold span and context; the (reference-free) QA module predicts an answer span from the question and context; the Span Scorer Module compares predicted and gold spans to output a score.

Remark 18.6 (Query-Focused Summarization and QFS-Composer). **Query-focused summarization (QFS)** produces a summary relevant to a specific query or named entity (e.g. an organisation), and can be viewed as a special type of QA. **QFS-Composer** (Duranović & Robnik-Šikonja 2026) is a pipeline for QFS in less-resourced languages:

1. An LLM analyses the input to determine the type of focus and decomposes the query into sub-questions.
2. If the focus is a named entity, a QG module generates additional questions based on that entity and the source text.
3. A QA engine answers all questions against the source.
4. The original query is expanded with the QA answers and fed to a summarization LLM to produce the final focused summary.

This re-uses the QG + QA building blocks shared with QAGS/QuestEval but for generation rather than evaluation.

Remark 18.7 (Summarization Challenges). **Core difficulties**: (1) meaning representation and construction; (2) abstractive summarization of long texts (books, scientific papers). Large generative LMs now excel at summarization and can be further improved for specific domains or summary types (e.g. query-focused summarization) by fine-tuning on a suitable dataset.

18.2 Question Answering

Remark 18.8 (QA: Definition and History). **Question answering (QA)** systems are designed to fill human information needs that arise when interacting with a virtual assistant, search engine, or database. The goal is to automatically answer questions posed by humans in natural language ($Q \rightarrow A$). QA is one of the oldest NLP tasks (punched-card systems in 1961; Simmons, Klein & McConlogue 1964). A QA system can be characterised along four axes: **information source** (text passage, web, knowledge base, table, image, ...); **question type** (factoid vs non-factoid; simple vs compositional); **answer type** (short span, paragraph, list, yes/no, ...); **knowledge store**

(parametric/closed-book, or non-parametric/open-book/RAG). Landmark practical system: IBM Watson defeated two Jeopardy! champions on February 16, 2011.

Definition 18.3 (QA Task Types). *The main task types are:*

- **Factoid**: Who/What/Where/How-many questions; short answer spans
- **Complex/narrative**: multi-sentence or multi-document reasoning
- **Closed-book**: answer with no access to external documents; relies on parametric knowledge
- **Open-book / Reading comprehension**: given passage P and question Q , produce answer A ; $(P, Q) \rightarrow A$
- **Open-domain**: no passage given; retrieve from a large collection ($Q \rightarrow A$ over $\mathcal{D}$)

18.2.1 IR-Based and Retrieval-Augmented QA

Remark 18.9 (IR-Based QA Pipeline). *Classic pipeline (modern RAG equivalent):*

1. **Question Processing**: reformulate as keyword query; detect expected answer type (NER category: person, location, number, ...)
2. **Document Retrieval**: retrieve candidate documents from a large collection
3. **Passage Retrieval**: rank and select relevant passages
4. **Answer Processing**: extract or generate the answer span

Definition 18.4 (Retriever-Reader Framework / RAG). *Input: large document collection $\mathcal{D} = \{D_1, \dots, D_N\}$ and question Q .*

$$\begin{aligned} \text{Retriever: } f(\mathcal{D}, Q) &\rightarrow P_1, \dots, P_K \\ \text{Reader: } g(Q, \{P_1, \dots, P_K\}) &\rightarrow A \end{aligned}$$

The retriever is any neural retriever (e.g., dense passage retrieval); the reader is a reading-comprehension model (BERT-based or zero-shot LLM). A reranker often re-scores retrieved passages before the reader. DrQA (Chen et al. 2017) pioneered this on Wikipedia. **Retrieval-Augmented Generation (RAG)** extends this with a generative reader.

Remark 18.10 (Advantages of RAG over Closed-Book LMs). *We present the advantages of RAG over closed-book LMs:*

- **Dynamic**: retrieval corpus is updated without retraining
- **Interpretable**: system can cite retrieved passages for human verification
- Handles private documents not available during pre-training

Remark 18.11 (Problem: LLMs Cannot Use Many Documents). *If the retriever is restricted to a single passage, that passage must be the right one. The obvious fix – retrieve many passages and let the reader sort it out – runs into the **long-context problem**: LLMs do not pay attention to their entire context well, so in practice they cannot effectively use a large number of retrieved documents.*

Figure 42 (Liu et al. 2023) makes this concrete: as the number of retrieved documents grows from 5 to 50, retriever recall (e.g. Contriever) keeps climbing steadily toward $\sim 90\%$, but downstream QA accuracy of several LLMs (Claude-1.3, Claude-1.3-100k, GPT-3.5-turbo-0613/16k, MPT-30B-Instruct, LongChat-13b-16k) saturates almost immediately, flattening out around 55–60% after only 10–20 documents – even for models explicitly built for long contexts (the “-100k”/“-16k” variants). Adding more correctly-retrieved evidence does not translate into proportionally better answers, because the model fails to make full use of everything in its context window.

This is why retriever precision (getting the few best passages, not just many plausible ones) matters more than retriever recall alone, and is a key open limitation of RAG-based QA systems alongside imperfect retrievers and citation hallucination.

18.2.2 BERT for Reading Comprehension

Definition 18.5 (Span Extraction with BERT). *Input: $[CLS] q_1 \dots q_n [SEP] p_1 \dots p_m$. Two linear layers on top of BERT predict $P_{\text{start}}(i)$ and $P_{\text{end}}(i)$ for each passage token i . Answer span: $\arg \max_{i \leq j} (P_{\text{start}}(i) + P_{\text{end}}(j))$.*

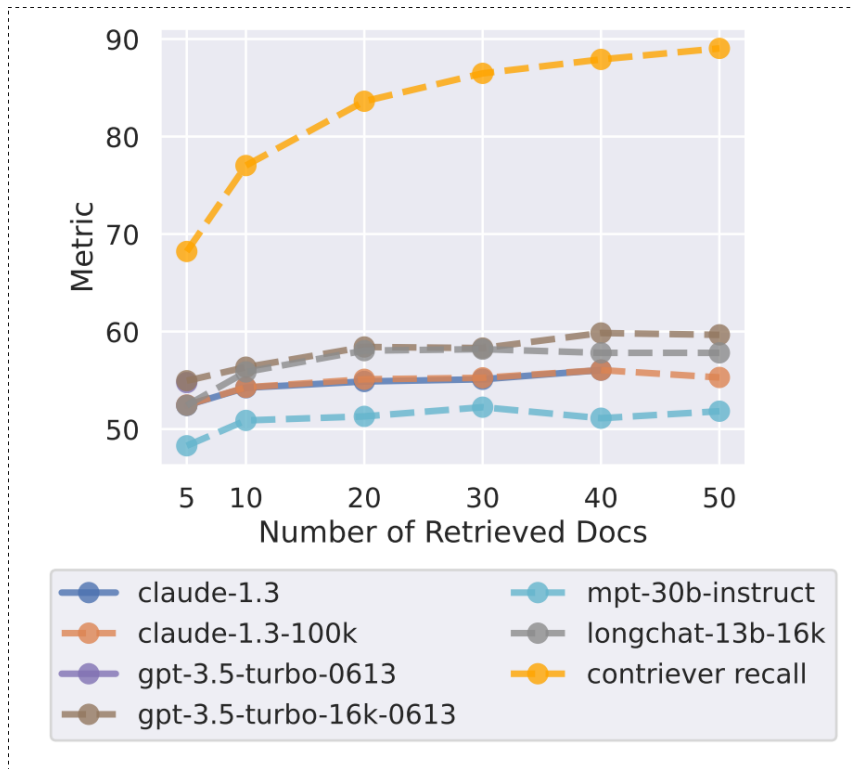


Figure 42: Retriever recall rises steadily with more retrieved documents, while RAG question-answering accuracy across several LLMs saturates after 10–20 documents (Liu et al. 2023).

18.2.3 Closed-Book QA and LLM-Based QA

Definition 18.6 (T5 Model for Closed-Book QA). *T5* (see Section 10 for the full architecture) is pre-trained with **span corruption**: contiguous input spans are replaced by sentinel tokens (marked $\langle M \rangle$), and the decoder learns to generate the missing spans. For closed-book QA, T5 is then **fine-tuned directly on QA datasets given only the question** – no passage or other context is supplied at fine-tuning or inference time. Since the model cannot look anything up, any correct answer must come from facts stored in its parameters during pre-training; this makes T5 a clean testbed for asking “how much knowledge can you pack into the parameters of a language model?” (Roberts et al. 2020).

Remark 18.12 (Parametric Knowledge in LLMs). This closed-book setting generalises beyond T5: any sufficiently large pre-trained LM can answer factoid questions without retrieving any document, since knowledge is stored in its weights. **Closed-book QA** then became a standard capability benchmark (MMLU, GPQA, ARC-C, Humanity’s Last Exam). Acquiring knowledge robustly requires roughly **1000 exposures** per fact during pre-training to achieve 2 bits/parameter storage capacity (Allen-Zhu & Li 2023); rare facts (~ 100 exposures) are stored at only 1 bit/parameter.

Remark 18.13 (Challenges with Parametric Knowledge). Some challenges with parametric knowledge include:

- **Robustness**: entailed knowledge is hard to learn without chain-of-thought (e.g., inferring “Biden’s birth year is even” from “Biden was born in 1946”)
- **Reversal curse**: knowing $A \rightarrow B$ does not guarantee $B \rightarrow A$ (e.g., model knows Tom Cruise’s mother but fails the reverse query)
- **Hallucinations**: LLMs aggressively confabulate facts they don’t know; real-world consequences (Air Canada chatbot lawsuit, defamation claims)
- **Abstention**: ideal systems should estimate confidence and back off when confidence is below threshold (via sampling or self-scoring), communicating uncertainty rather than hallucinating

Remark 18.14 (Updating Parametric Knowledge). Continued pre-training on domain-specific data can add knowledge, but requires $50+B$ tokens. Synthetic continued pre-training (LLM-transformed data varying content and style) can significantly boost closed-book QA performance.

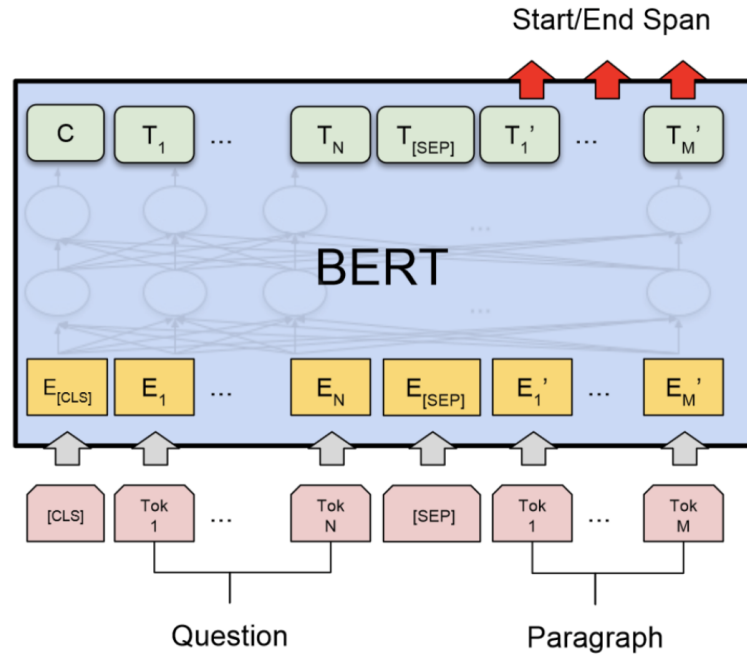


Figure 43: Input of the BERT model

18.2.4 QA Datasets

Dataset	Type	Notes
SQuAD 2.0	Span extraction	Wikipedia; 50K unanswerable questions
BoolQ	Yes/No	Wikipedia paragraphs (Clark et al. 2019)
COPA	Causal reasoning	2-choice causal inference (Roemmele 2011)
MultiRC	Multi-sentence RC	True/false answer list (Khashabi 2018)
ReCoRD	Cloze / commonsense	CNN/DailyMail (Zhang 2018)
RiddleSense	Riddle commonsense	Multiple choice (Lin 2021)
TriviaQA	Open-domain RC	Trivia Q&A + independent web evidence
NaturalQuestions	Open-domain	Google search queries; Wikipedia spans
HotpotQA	Multi-hop	Requires 2 Wikipedia pages
MMLU	Closed-book	Multi-subject multiple-choice; capability test
GPQA	Expert-level	PhD-level science multiple-choice; o1-preview $\approx 70\%$, o1-mini $\approx 60\%$
Humanity's Last Exam (HLE)	Expert-level	Cross-domain expert questions; GPT-4o 3.1%, o3-mini(high) 14.0%

19 Parameter-Efficient Fine-Tuning and Language Agents

19.1 Parameter-Efficient Fine-Tuning (PEFT)

Full fine-tuning updates *all* parameters of a large model for each new task – expensive in both compute and storage (one full copy of the model per task). Parameter-efficient fine-tuning (PEFT) methods keep the pretrained weights frozen and train only a small number of new parameters.

Definition 19.1 (PEFT). *Parameter-Efficient Fine-Tuning adapts a pre-trained model to a downstream task by training only a small subset of parameters, matching full fine-tuning performance while drastically reducing computational and environmental cost.*

19.1.1 Adapter Fine-Tuning

Small **adapter modules** are inserted inside each transformer layer. Each adapter:

1. **Down-projects** the hidden representation to a much lower dimension.
2. Applies a nonlinearity.
3. **Up-projects** back to the original dimension.

Formally, with a residual connection around the bottleneck:

$$f_{\phi}(\mathbf{x}) = W^U(\sigma(W^D\mathbf{x})) + \mathbf{x}$$

where $W^D \in \mathbb{R}^{h \times r}$ projects down and $W^U \in \mathbb{R}^{r \times h}$ projects back up. Only the adapter parameters ($\approx 2\%$ of total parameters) are trained; the pretrained transformer weights remain frozen. Different tasks get different adapters on the same shared frozen backbone.

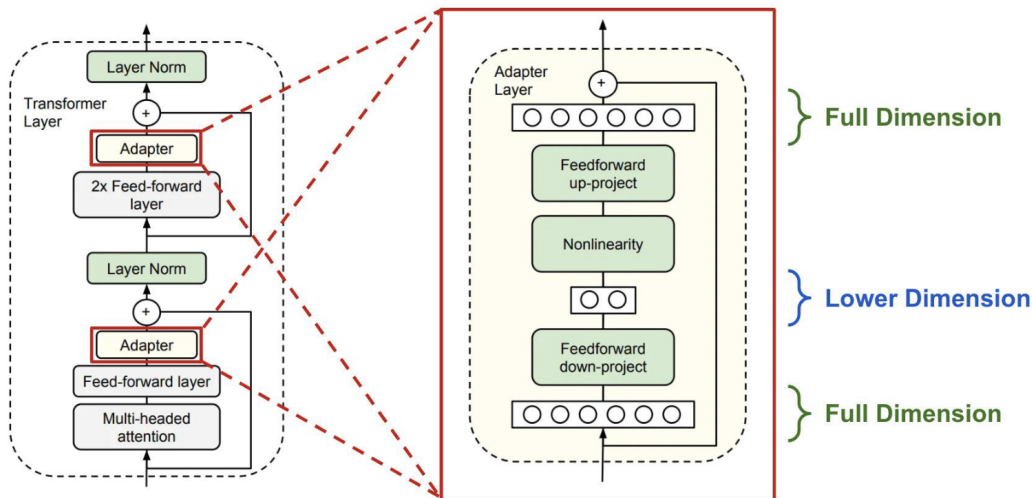


Figure 44: Adapter fine-tuning: small adapter modules (down-project, nonlinearity, up-project) are inserted into each transformer layer; only adapter weights are trained.

19.1.2 Prefix Tuning and Prompt Tuning

The core problem with classical fine-tuning is **storage and cost**: you need a separate full copy of the pretrained model for each downstream task – one for translation, one for summarisation, one for NER, and so on. At hundreds of millions of parameters each, this quickly becomes impractical.

Prefix tuning (Li & Liang, 2021) solves this by keeping *one* shared, completely frozen pretrained transformer and learning only a small set of task-specific **prefix vectors**:

- A set of k continuous, learnable vectors $P = [p_1, \dots, p_k]$, $p_j \in \mathbb{R}^d$, are prepended to the key and value matrices of *every* attention layer.
- These are **not real tokens from the vocabulary** – they have no text equivalent and cannot be read as words. They are free-floating parameters optimised directly by gradient descent.

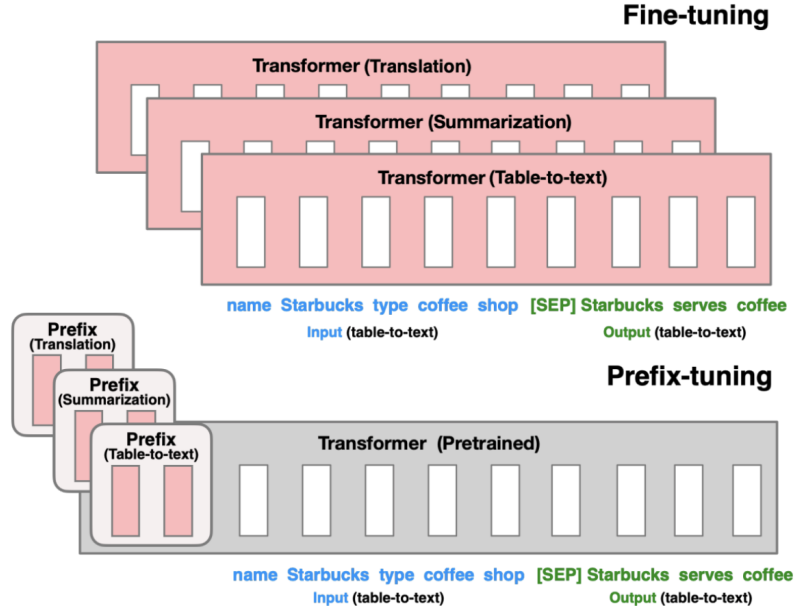


Figure 45: Classical fine-tuning (top) requires one full model per task. Prefix tuning (bottom) keeps one frozen backbone and only trains small task-specific prefix vectors.

- The pretrained transformer weights are **completely frozen**. Only the prefix parameters P are updated during training.

At inference time, switching tasks means swapping in a different set of prefix vectors – the backbone model itself stays the same and is shared across all tasks.

Prompt-Tuning is a simpler variant: soft, trainable prompt tokens are prepended only to the input embedding layer (rather than to every layer’s keys and values). It has fewer parameters than prefix-tuning and its performance scales well with model size, approaching full fine-tuning at very large scale.

Example 19.1 (Prefix tuning vs. classical fine-tuning). *Suppose we need models for three tasks: translation, summarisation, and table-to-text.*

- **Classical fine-tuning:** Three separate full copies of the pretrained transformer, each with all parameters updated independently. Storage cost: $3 \times |W|$.
- **Prefix tuning:** One shared frozen transformer backbone, plus three small sets of prefix vectors P_{transl} , P_{summ} , P_{table} . Storage cost: $|W| + 3 \times k \times d$, where $k \ll n$.

The prefix parameters typically account for less than 0.1% of the model size, so the saving is dramatic.

Remark 19.1. Prefix tuning is **not** the same as prepending a text prompt. Prompt tokens are real vocabulary tokens with fixed embeddings; prefix vectors are unconstrained continuous parameters optimised end-to-end. This gives prefix tuning strictly more expressive power than discrete prompting.

19.1.3 LoRA (Low-Rank Adaptation)

LoRA models the weight update as a low-rank factorization. Instead of learning a full matrix update $\Delta W \in \mathbb{R}^{d \times d}$, it freezes the pretrained weight matrix W_0 and learns two small matrices:

$$\Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k)$$

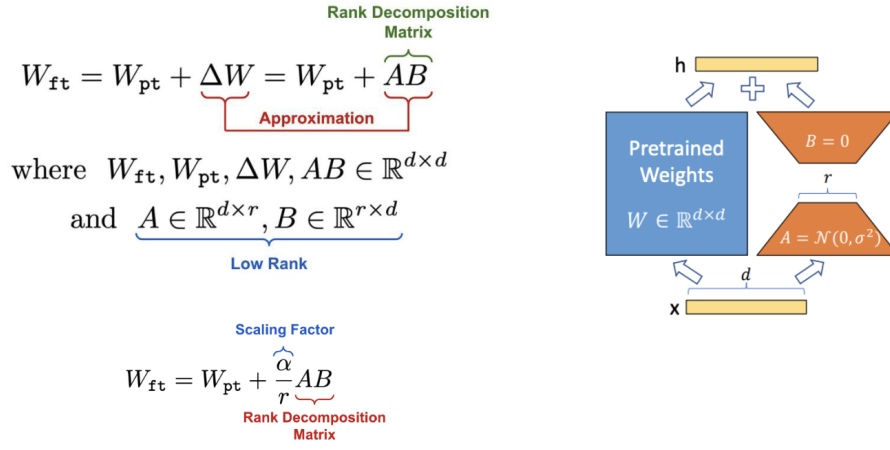
The fine-tuned weight matrix becomes:

$$W = W_0 + \alpha BA$$

where α is a scaling constant. Only A and B are trained; r can be as small as 1% of d , making LoRA extremely parameter-efficient. At inference, BA can be merged into W_0 , adding no latency overhead.

19.1.4 QLoRA: Quantised LoRA

QLoRA (Dettmers et al., 2023) combines LoRA with aggressive quantisation of the frozen base model, so that fine-tuning a model with tens of billions of parameters becomes possible on a *single* consumer or research GPU (e.g., one 24–48GB card, instead of a multi-GPU cluster). The base weights W_0 are never updated during PEFT



43

Figure 46: LoRA: the pretrained weight matrix W_0 is frozen; only the low-rank factors A and B are trained, and their product $\Delta W = BA$ is added at inference.

training, so they only need to be stored precisely enough to be *read* during the forward/backward pass – they do not need full 16- or 32-bit precision. QLoRA exploits this with three ideas used together:

1. **4-bit NormalFloat (NF4) quantisation:** The frozen base weights are stored in a custom 4-bit data type, NF4, designed specifically for weights that follow a zero-centered normal distribution (as pretrained neural network weights typically do). Unlike a generic 4-bit integer quantisation, NF4’s quantisation bins are chosen so that each bin holds an equal proportion of the expected probability mass under a normal distribution, which minimises quantisation error for this specific weight distribution.
2. **Double quantisation:** Quantisation itself requires storing per-block scaling constants (the factors that map low-precision integers back to real values). With small block sizes (needed for good accuracy), these constants add non-trivial memory overhead. Double quantisation quantises *the quantisation constants themselves* (typically from 32-bit to 8-bit), saving roughly 0.5 bits per parameter on average with negligible accuracy loss.
3. **Paged optimisers:** GPU memory usage can spike unpredictably during training (e.g., when a long sequence triggers a large activation memory peak), which can cause out-of-memory crashes. QLoRA uses NVIDIA’s *unified memory* feature to automatically page optimiser states between GPU and CPU memory on demand, similarly to how an operating system pages virtual memory to disk, avoiding crashes from transient memory spikes.

During the forward and backward pass, the 4-bit weights are de-quantised on the fly into a higher-precision compute type (typically bfloat16) only for the duration of the matrix multiplication, then discarded again – the model never needs the full-precision weights materialised all at once. The LoRA adapter matrices A and B injected into each layer are still trained in higher precision (bfloat16), since these are the only parameters receiving gradient updates; only the much larger frozen backbone benefits from 4-bit storage.

Remark 19.2 (Why this matters). *QLoRA reduces the memory footprint of fine-tuning a 65B-parameter model from >780GB (full fine-tuning, mixed precision) to under 48GB, with no measurable drop in downstream task performance compared to full 16-bit fine-tuning. This was the technique that made fine-tuning very large open LLMs accessible outside large industrial labs.*

19.1.5 Types of Fine-Tuning: A Summary

- **Fine-tuning as continued pretraining:** Take a pretrained LLM and continue training all parameters on domain-specific data using the same pretraining objective. Good for domain adaptation (e.g., medical or legal text).
- **Parameter-Efficient Fine-Tuning (PEFT, e.g., LoRA, QLoRA):** Train only a small number of new parameters on task data. Cheap and effective; widely used in practice.
- **MLM Fine-Tuning:** Train only a task-specific classification head (e.g., for NER or sentiment) using supervised data and a task-specific loss. The backbone is frozen or lightly updated.

- **Instruction Tuning (Supervised Fine-Tuning / SFT):** Fine-tune on diverse instruction-following examples so the model generalizes to unseen tasks at inference. This is how chat models are created from base LLMs.

Definition 19.2 (Zero-shot and Few-shot Transfer). *Zero-shot and few-shot transfer consists in:*

- **Zero-shot transfer:** *The model is applied to a downstream task without any gradient updates or task-specific training data, relying solely on its pretrained weights (and possibly an instruction prompt).*
- **Few-shot transfer:** *The model is adapted to a task using a small amount of supervised data (typically k examples), either through minimal fine-tuning or in-context prompting.*

Remark 19.3 (Unifying View (He et al. 2022)). *LoRA, prefix-tuning, adapters, and prompt-tuning all modify the hidden representation h and can be expressed in a unified framework. They are approximately equivalent under certain conditions.*

19.2 Knowledge Distillation (Teacher-Student Framework)

Knowledge Distillation is a compression paradigm designed to transfer the dark knowledge of a high-capacity, computationally expensive *Teacher* network (T) into a highly compact, parameterized *Student* network (S). Rather than training the student strictly on hard categorical indicators, it minimizes the divergence between the continuous logit distributions of both systems.

1. **Probability Softening via Temperature Calibration:** To amplify the structural nuances and dark knowledge contained within the non-target classes of the teacher’s output, a temperature scaling hyperparameter $T > 1$ is injected directly into the softmax function for both architectures:

$$p_i^\tau = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \quad (78)$$

where z_i represents the raw unnormalized logit score for class index i .

2. **Dual Objective Loss Function:** The student network is optimized end-to-end using a convex combination of standard categorical cross-entropy ($\mathcal{L}_{\text{CE}}$) against the hard ground-truth targets y , and Kullback-Leibler (KL) divergence ($\mathcal{L}_{\text{KL}}$) against the softened teacher outputs:

$$\mathcal{L}_{\text{total}} = (1 - \alpha)\mathcal{L}_{\text{CE}}(y, \sigma(z_S)) + \alpha\tau^2\mathcal{L}_{\text{KL}}(p_T^\tau \parallel p_S^\tau) \quad (79)$$

where α is a balancing scaling coefficient, and the τ^2 multiplier preserves the relative scale of the gradients during backpropagation.

This objective forces the student to mimic the complete geometric topology of the teacher’s latent decision boundaries, yielding comparable downstream classification capabilities within a significantly minimized parameter footprint.

19.3 Language Agents

Definition 19.3 (Language Agents). *LLMs extended beyond text generation to interact with external environments: web browsing, software engineering (SWE-bench), robotics, scientific discovery pipelines. The agent alternates between **perceiving** observations and **acting** through a set of tools.*

Definition 19.4 (CoALA Framework (Sumers et al., TMLR 2024)). *Cognitive Architectures for Language Agents organises agent design into three components:*

- **Memory:** *short-term (working memory / context window); long-term (procedural, semantic, episodic stores)*
- **Action space:**
 - Internal: *Reasoning (chain-of-thought), Retrieval (from memory), Learning (parameter update)*
 - External: *Grounding (perception and actuation in the world)*
- **Decision making:** *planning cycle – Observation $\rightarrow$ Proposal $\rightarrow$ Evaluation $\rightarrow$ Selection $\rightarrow$ Execution*

Remark 19.4 (Analogies). *The Von Neumann computer architecture is an analogy: memory stores data, the control unit (LLM) executes instructions, and tools act as the ALU. Cognitive architectures from cognitive science (Soar, ACT-R) provide earlier models of the same modular reasoning idea.*

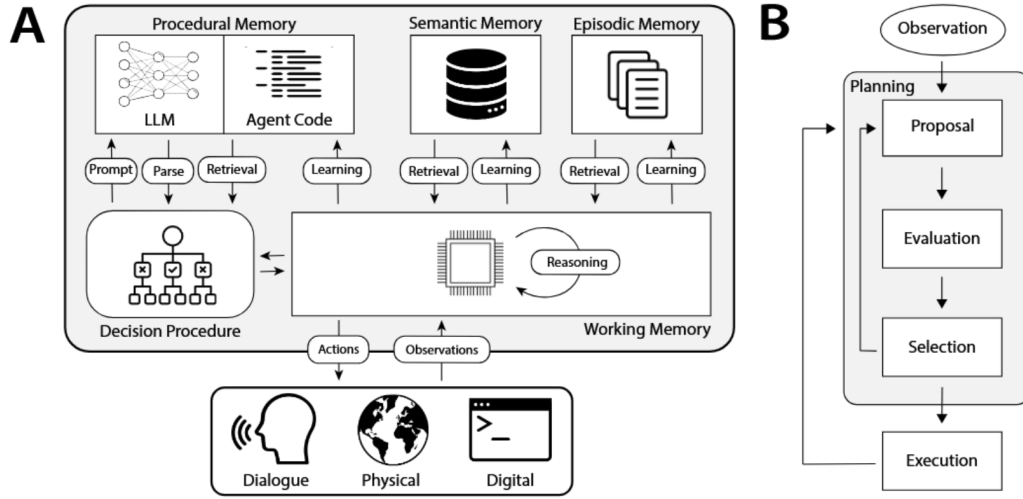


Figure 47: Cognitive Architectures for Language Agents

Remark 19.5 (Progression of LLM-Based Systems).

$LLMs \rightarrow \text{Language Agents} \rightarrow \text{Cognitive Language Agents}$

Each step adds structured capability: Language Agents gain tools and environment interaction; Cognitive Language Agents additionally gain structured memory and explicit reasoning modules. Each step increases autonomy but also alignment and safety challenges.

Summary of Key Concepts by Chapter

This section provides a compact reference of the most important terms, concepts, and methods introduced in each chapter, intended for revision and exam preparation.

Chapter 1 — Introduction

Core definition: NLP = techniques for language processing + attempts at language understanding (syntax, semantics, pragmatics, AGI).

Key terms:

- **Ambiguity types:** lexical, syntactic, semantic, referential.
- **Linguistic levels:** prosody, phonology, morphology, syntax, semantics, pragmatics, world knowledge.
- **Disambiguation approaches:** POS tagging, word sense disambiguation, probabilistic parsing, speech act interpretation, user profiles.
- **History of NLP:** Symbolic era (ELIZA, SHRDLU) → Statistical era (HMM, N-grams, SMT) → Neural era (Word2Vec, LSTM) → Transformer & LLM era (Attention, BERT, GPT, LLaMA).
- **Two approaches:** symbolic (rule-based, GOFAI) vs. statistical (data-driven, pretrained LLMs).
- **Classical pipeline:** syntactic analysis → semantic interpretation → use of world knowledge.
- **Turing Test, ARC-AGI, ELIZA effect, hallucination.**
- **CJVT UL:** centre producing NLP tools and resources for Slovene (GaMS, CLASSLA-Stanza).

Chapter 2 — Text Processing

Core task: transforming raw text into structured linguistic units ready for downstream analysis.

Key terms:

- **Token / Type / Term:** instance vs. unique form vs. normalized dictionary entry.
- **Tokenization:** splitting text into tokens; challenges with abbreviations, clitics, possessives, multi-word expressions, language-specific issues (French elision, German compounds, Chinese/Japanese segmentation).
- **Heaps' Law (Herdan's Law):** $V = kN^\beta$, $\beta \in [0.67, 0.75]$ — vocabulary grows sublinearly with corpus size.
- **Term normalization:** case folding, asymmetric expansion; creating equivalence classes.
- **Lemmatization:** reducing word forms to their dictionary headword (lemma); requires morphological analysis; critical for morphologically rich languages (Slovene, Arabic, Turkish).
- **Morphology:** stems, affixes; inflectional vs. derivational morphology; morphological parsers; agglutination (Turkish).
- **Stemming:** crude heuristic suffix-stripping (Porter's Algorithm); operates without context; rarely used in modern NLP.
- **Sentence segmentation:** detecting sentence boundaries; ambiguity of the period; binary classifiers (Decision Tree) using length, case, abbreviation lists.
- **Tools:** NLTK, spaCy, Gensim, CLASSLA-Stanza.
- **Corpora:** monolingual / multilingual, genres, code-switching, corpus datasheets.

Chapter 3 — Formal Languages

Core idea: mathematically defining and processing the structure of language via automata and grammars.

Key terms:

- **Regular Expressions (RE):** anchors ($\wedge$, $\$$), wildcards ($\cdot$), quantifiers ($*$, $+$, $?$, $\{m,n\}$), greedy vs. non-greedy, character sets ($[...] , [\wedge\dots]$), alternation ($|$), grouping ($(\dots)$), flags (i, m, s).
- **Pattern matching errors:** false positives (Type I) $\leftrightarrow$ increase precision; false negatives (Type II) $\leftrightarrow$ increase recall.
- **Formal language:** a (possibly infinite) set of strings over an alphabet Σ ; Kleene star Σ^* .
- **Chomsky Hierarchy:**
 - Type 3 (Regular) — Finite State Automata, Regular Expressions.
 - Type 2 (Context-Free) — Pushdown Automata, CFGs.
 - Type 1 (Context-Sensitive) — Linear Bounded Automata.
 - Type 0 (Unrestricted) — Turing Machines.
- **Context-Free Grammar (CFG):** 4-tuple (V, Σ, R, S) ; terminals, non-terminals, productions, start symbol; derivation; parse tree; ambiguous grammar.
- **CYK Algorithm:** dynamic programming parsing for CFGs in Chomsky Normal Form (CNF); $O(n^3 \cdot |G|)$ time complexity.
- **ELIZA:** direct application of RE substitution rules ($s/pattern/response/$).

Chapter 4 — Sparse Text Representations

Core idea: represent text as high-dimensional sparse vectors based on word co-occurrence counts.

Key terms:

- **Distributional semantics:** “You shall know a word by the company it keeps” (Firth); first-order (syntagmatic) vs. second-order (paradigmatic) co-occurrence.
- **Bag-of-Words (BoW):** count vector over vocabulary; sparse, dimension $|V|$; ignores word order.
- **Term-document matrix:** $|V| \times |D|$ matrix; term counts per document; basis of IR.
- **TF-IDF:** $\text{tf}(t, d) \times \text{idf}(t)$; down-weights frequent but uninformative words; sparse embedding baseline.
- **Cosine similarity:** $\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}| |\mathbf{v}|}$; angle-based similarity in vector space; not sensitive to vector magnitude.
- **Edit distance (Levenshtein):** minimum insertions, deletions, substitutions to transform one string into another; spell correction, fuzzy matching.
- **Precision and Recall:** $P = \frac{TP}{TP+FP}$, $R = \frac{TP}{TP+FN}$; F1 score = harmonic mean; precision–recall trade-off.

Chapter 5 — N-gram Models

Core idea: estimate the probability of word sequences using local context windows.

Key terms:

- **Language model (LM):** assigns probability $P(W)$ or $P(w_n | w_1, \dots, w_{n-1})$ to sequences.
- **Chain rule of probability:** $P(w_1, \dots, w_n) = \prod_i P(w_i | w_1, \dots, w_{i-1})$.
- **Markov assumption:** $P(w_n | w_1, \dots, w_{n-1}) \approx P(w_n | w_{n-k}, \dots, w_{n-1})$.
- **N-gram models:** unigram (no context), bigram (1 previous word), trigram (2 previous words); estimated via Maximum Likelihood Estimation (MLE): $\hat{P}(w_i | w_{i-1}) = c(w_{i-1}, w_i) / c(w_{i-1})$.
- **Smoothing:** Laplace (add-1), add- k , Kneser-Ney; necessary because MLE assigns zero probability to unseen n-grams.
- **Perplexity (PP):** $\text{PP}(W) = P(w_1, \dots, w_n)^{-1/N}$; lower is better; intrinsic evaluation metric.
- **Log probabilities:** used to avoid numerical underflow when multiplying many small probabilities.

Chapter 6 — Revision of Neural Networks

Core idea: neural networks as universal function approximators trained by gradient-based optimization.

Key terms:

- **Artificial neuron / Perceptron:** weighted sum + activation function; step function for the perceptron.
- **Activation functions:** sigmoid, ReLU ($\max(0, x)$, default in modern DNNs), softplus; nonlinearity is essential — without it, stacked linear layers collapse to one.
- **Feed-Forward Neural Network (FFNN):** input layer, hidden layers, output layer; no cycles; deep $\Rightarrow$ Deep Learning.
- **Gradient Descent:** $\mathbf{x}^{t+1} = \mathbf{x}^t - \eta \nabla f(\mathbf{x}^t)$; batch GD vs. SGD vs. mini-batch.
- **Backpropagation:** chain rule applied to compute gradients layer by layer; empirical risk minimization.
- **Softmax:** converts logit vector to probability distribution; $\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}$.
- **Cross-entropy loss:** $L = -\sum_i y_i \log \hat{y}_i$; standard classification loss.
- **Overfitting and regularization:** L1/L2 weight decay, dropout, early stopping.

Chapter 7 — Static Neural Embeddings

Core idea: train shallow networks to produce dense, fixed word vectors capturing semantic similarity.

Key terms:

- **Static vs. contextual:** one fixed vector per word type; polysemy not handled (cf. BERT).
- **Word2Vec (Mikolov et al., 2013):** input $\rightarrow$ projection $\rightarrow$ output; after training, the projection matrix rows are the embeddings.
 - **Skip-gram:** target word $\rightarrow$ predict context words.
 - **CBOW:** context words $\rightarrow$ predict target word.
 - **Negative Sampling:** binary classification; positive pairs from corpus window, negative pairs from random sampling; $P(+ | t, c) = \sigma(t \cdot c)$.
- **GloVe (Pennington et al., 2014):** global co-occurrence matrix; log-bilinear model; combines local context (Word2Vec) with global statistics.
- **FastText (Facebook):** subword n-grams; handles OOV words and morphologically rich languages.
- **Properties:** analogical reasoning ($king - man + woman \approx queen$); dimension ≈ 100 –300; bias inherited from corpus.
- **Doc2Vec:** extends Word2Vec to document-level embeddings.
- **Cross-lingual embeddings:** align embedding spaces across languages (MUSE); enable zero-shot cross-lingual transfer.

Chapter 8 — Recurrent and Convolutional NNs for Text

Core idea: process variable-length sequences by introducing memory into the network.

Key terms:

- **RNN:** $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$; shared weights across time steps; architectures: one-to-many, many-to-one, many-to-many.
- **BPTT (Backpropagation Through Time):** unrolling the RNN and applying standard backprop; causes vanishing / exploding gradients.
- **LSTM (Hochreiter & Schmidhuber, 1997):** forget gate f_t , input gate i_t , output gate o_t , cell state c_t ; protects long-term dependencies; dominant from 2011 to 2017.
- **GRU (Cho et al., 2014):** simplified LSTM with update and reset gates; fewer parameters, similar performance.

- **Bidirectional RNN (BiRNN)**: processes sequence in both directions; concatenates forward and backward hidden states; better context for understanding tasks.
- **Seq2Seq**: encoder RNN compresses input into a context vector; decoder RNN generates the output sequence; used in MT and summarization.
- **Attention in RNNs (Bahdanau et al., 2015)**: dynamic, weighted context vector over all encoder states; precursor to self-attention; resolved the information bottleneck of Seq2Seq.
- **CNN for text**: 1-D convolutions over token embeddings; max-over-time pooling; good for classification; captures local n-gram features.

Chapter 9 — Transformers

Core idea: replace sequential recurrence with parallel self-attention, enabling direct long-range dependencies.

Key terms:

- **Transformer (Vaswani et al., 2017)**: encoder-decoder; no recurrence; all tokens processed in parallel.
- **Encoder**: N identical layers; each with multi-head self-attention + position-wise FFN sublayers; residual connections + LayerNorm.
- **Decoder**: N layers; masked self-attention (causal) + encoder-decoder cross-attention + FFN.
- **Scaled dot-product attention**: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$; four steps: score, scale, softmax, weighted sum.
- **Q, K, V projections**: learned matrices W^Q, W^K, W^V ; each token projected to query, key, value.
- **Multi-head attention**: h parallel attention heads; each captures different representation subspaces; concatenated and linearly projected.
- **Positional encoding**: injected since self-attention is permutation-invariant. Four schemes: *sinusoidal* (fixed, Vanilla Transformer), *learned absolute* (look-up table, BERT), *relative buckets* (additive attention bias, T5), *RoPE* (rotates Q/K , modern decoder LLMs).
- **Subword tokenization (BPE)**: merge frequent byte pairs; handles OOV; vocabulary $\sim 30\text{k}-128\text{k}$.

Chapter 10 — BERT and T5

Core idea: pretrain deep Transformer encoders on large corpora with self-supervised objectives; fine-tune for downstream tasks.

Key terms:

- **Pretrain-fine-tune paradigm**: pretrain once (self-supervised) $\rightarrow$ fine-tune many times (task-specific small dataset).
- **BERT (Devlin et al., 2019)**: encoder-only; reads full sentence bidirectionally.
 - **MLM (Masked Language Modeling)**: 15% of tokens masked; model predicts them using all surrounding context.
 - **NSP (Next Sentence Prediction)**: binary classification of sentence pairs; helps pair-level tasks.
 - **WordPiece tokenization**: [CLS], [SEP], ## continuation tokens; input = token + segment + position embeddings.
- **Fine-tuning BERT**: add task head to [CLS] or per-token representations; classification, NER, QA, NLI.
- **Contextual embeddings**: each token’s representation depends on full context; handles polysemy.
- **GLUE / SuperGLUE**: benchmark suites for evaluating language understanding.
- **PEFT (LoRA, adapters, prefix-tuning, prompt-tuning, QLoRA)**: covered in Chapter 19, alongside language agents.
- **Cross-lingual transfer**: mBERT, XLM-R; zero-shot cross-lingual transfer via shared multilingual representations.
- **T5 (Raffel et al., 2020)**: encoder-decoder; *text-to-text* framing — every task is a string-to-string problem; pretrained with masked span prediction on Colossal Clean Crawled Corpus (C4).

Chapter 11 — GPT: Generative Pre-trained Transformers

Core idea: decoder-only autoregressive Transformer scaled to billions of parameters, enabling emergent in-context learning and alignment.

Key terms:

- **Decoder-only:** causal (masked) self-attention; each token attends only to its left; no encoder, no cross-attention.
- **Autoregressive LM:** $P(w_1, \dots, w_n) = \prod_t P(w_t \mid w_1, \dots, w_{t-1})$; at inference: sample one token at a time.
- **GPT evolution:** GPT-1 (110M, pretrain+finetune) → GPT-2 (1.5B, zero-shot) → GPT-3 (175B, in-context learning) → ChatGPT / GPT-4.
- **In-context learning:** zero-shot, one-shot, few-shot; no gradient updates; examples in the prompt only.
- **Scaling laws:** performance improves predictably with more parameters, data, and compute.
- **RLHF (Reinforcement Learning from Human Feedback):** SFT → reward model (Bradley-Terry loss on human-ranked pairs) → PPO fine-tuning with KL penalty; aligns model with human preferences.
- **DPO (Direct Preference Optimization):** eliminates separate reward model and RL loop; directly optimizes on preference pairs; simpler and more stable than RLHF.
- **Prompt Engineering:** instruction, context, input data, output indicator; chain-of-thought (CoT), zero-shot CoT (“Let’s think step by step”), RAG, self-consistency.
- **LLM Arena (Chatbot Arena / PoVeJMo):** crowdsourced Elo-based evaluation of LLMs.

Chapter 12 — LLM-Based Agents

Core idea: LLMs augmented with tools, memory, and multi-step planning to act autonomously in environments.

Key terms:

- **Agent:** system that perceives, plans, and acts to achieve goals; sense-plan-act-learn loop.
- **LLM-based agent:** LLM as the reasoning core, connected to external tools/APIs; interactive and adaptive; multi-step actions.
- **Agent roles:** computing, decision, database, user-interaction, monitoring agents.
- **Multi-agent systems:** multiple specialised agents collaborating; modularity, parallelism, expanded capacity.
- **Self-improvement and feedback loops:** reflection, critique, self-consistency checking; outer loop optimization.
- **Agent architectures:** ReAct (Reason + Act), Reflexion, Plan-and-Execute, memory-augmented.
- **Tool calling and MCP (Model Context Protocol):** standardised protocol for LLMs to call external tools and services.
- **CoALA framework:** Memory (short-term/long-term) + Action space (internal: reasoning/retrieval/learning; external: grounding) + Decision-making cycle.

Chapter 13 — Neural Information Retrieval

Core idea: neural models for retrieval and ranking, powering Retrieval-Augmented Generation (RAG) pipelines.

Key terms:

- **RAG pipeline:** Embed → Retrieve → (Re-rank) → Generate → Cite; grounds LLM reasoning in external, up-to-date knowledge.
- **Chunking strategy:** how documents are split for indexing; critical design lever.
- **Embedding choice:** dense bi-encoders (e.g., sentence-transformers); sparse BM25; hybrid retrieval.
- **Index types:** HNSW, IVF-PQ; Approximate Nearest Neighbour (ANN) search for efficient retrieval.

- **RAG variants:** Naive RAG → Advanced RAG (pre/post-retrieval steps) → Modular RAG (interchangeable modules); Iterative RAG, Recursive RAG, Adaptive RAG.
- **Re-ranking:** Stage 1 (cheap, bi-encoder/ANN, top- k) → Stage 2 (expensive, cross-encoder, top- n).
- **RAGAS:** automated RAG evaluation; faithfulness, answer relevance, context recall, context precision.
- **Neural text ranking:** BM25 (sparse baseline), dense retrieval (bi-encoder), cross-encoder re-rankers; BEIR benchmark.

Chapter 14 — POS Tagging, Dependency Parsing, and NER

Core idea: assigning structural and semantic labels to tokens within sentences.

Key terms:

- **Lemmatization:** maps inflected forms to base lemma; essential for morphologically rich languages (Slovene, Arabic, Turkish).
- **POS tagging:** assigns grammatical category (NOUN, VERB, ADJ, ...) to each token; supports IR, NER, WSD, MT, sentiment analysis.
- **Morphosyntactic tagging:** POS + morphological features (gender, number, case, tense, mood).
- **Word classes:** open class (nouns, verbs, adjectives, adverbs) vs. closed class (prepositions, auxiliaries, pronouns, determiners).
- **Tag sets:** Penn Treebank (45 tags); Universal Dependencies (UD, 17 tags across 100+ languages).
- **Constituency parsing:** hierarchical phrase-structure tree (NP, VP, PP); CYK algorithm.
- **Dependency parsing:** head-dependent arcs; Universal Dependencies treebank; arc-standard/arc-eager transition systems.
- **Named Entity Recognition (NER):** identifies and classifies named entities (PERSON, ORG, LOC, DATE, ...); IOB/BIOES tagging scheme; CRF, BiLSTM-CRF, BERT-based models.

Chapter 15 — Word Senses and Disambiguation

Core idea: representing and disambiguating the multiple meanings of words in context.

Key terms:

- **Lemma / Wordform / Sense:** citation form / inflected form / discrete meaning; one lemma → multiple senses.
- **Types of ambiguity:** homonymy (unrelated meanings: homographs / homophones), polysemy (related meanings), metonymy (systematic polysemy, e.g. institution vs. building).
- **Zeugma test:** distinguishes homonymy from polysemy.
- **Lexical relations:** synonymy, antonymy (scale antonyms, reversives), hyponymy/hypernymy (IS-A), meronymy/holonymy (PART-OF).
- **WordNet:** Princeton lexical database; synsets with glosses and examples; splitters vs. lumpers.
- **WSD (Word Sense Disambiguation) approaches:** knowledge-based (Lesk algorithm: overlap of context with gloss), supervised (Naive Bayes, SVMs, BERT), semi-supervised (bootstrapping).
- **Evaluation:** SemEval shared tasks; coarse-grained vs. fine-grained WSD.

Chapter 16 — Affective Computing

Core idea: automatically detecting and representing emotions, sentiment, and personality from text.

Key terms:

- **Connotation:** affective meaning beyond literal sense; target of affective lexicons.
- **Scherer’s typology:** emotions, moods, interpersonal stances, attitudes (target of sentiment analysis), personality traits.
- **Categorical models:** Ekman’s 6 basic emotions (surprise, happiness, anger, fear, disgust, sadness); Plutchik’s wheel (8 emotions in 4 opposing pairs; complex emotions by combination).
- **Dimensional models:** VAD space — Valence (pleasant/unpleasant), Arousal (excited/calm), Dominance (in-control/controlled); Hourglass of Emotion (Cambria).
- **Affective lexicons:** NRC EmoLex (Plutchik paradigm), Warriner et al. (VAD), LIWC, SentiWordNet.
- **Sentiment analysis:** opinion mining; aspect-based SA; subjectivity detection; stance detection.
- **Personality analysis:** Big Five (OCEAN: Openness, Conscientiousness, Extraversion, Agreeableness, Neuroticism); Personality Recognizer (Mairesse et al.).
- **Related tasks:** irony/sarcasm detection, hate-speech detection, emotion arc detection in fiction.

Chapter 17 — Machine Translation

Core idea: automatically converting text between languages; tension between faithfulness and fluency.

Key terms:

- **Faithfulness vs. fluency:** two fundamental competing objectives; perfect translation is impossible.
- **MT paradigms:** direct (word-for-word), transfer (intermediate abstract representation), interlingual (universal interlingua).
- **Noisy channel model (SMT):** $\hat{E} = \arg \max_E P(F | E) P(E)$; language model ($P(E)$, fluency) + translation model ($P(F | E)$, fidelity) + decoder.
- **SMT sub-families:** word-based, phrase-based, tree-based, factored; parallel corpora (EuroParl, DGT, OPUS).
- **Sentence alignment:** multilingual embeddings + dynamic programming; alignment types (1-1, 2-1, null).
- **Neural MT (NMT):** Seq2Seq encoder-decoder (Sutskever 2014, Cho 2014); attention mechanism; Transformer-based MT.
- **Decoding strategies:** greedy, beam search, sampling; length penalty.
- **BLEU score:** precision-based n-gram overlap + brevity penalty; standard MT evaluation metric.
- **CAT (Computer-Aided Translation):** MT draft + human post-editing; localisation.

Chapter 18 — Summarization and Question Answering

Core idea: generating concise information or direct answers from text sources.

Key terms:

- **Summary types:** extractive (copy sentences), compressive (shorten), abstractive (generate new text); SDS vs. MDS; generic vs. query-focused.
- **Neural summarization:** seq2seq; Pegasus (gap sentence generation pre-training); BERT+RL; unsupervised (TextRank, graph centrality).
- **ROUGE (Lin 2004):** recall-oriented n-gram overlap metric; ROUGE-1, ROUGE-2, ROUGE-L (LCS); complement to BLEU (which is precision-oriented).
- **Factual consistency evaluation:** QAGS (QA-based), QuestEval (precision + recall sides); standard ROUGE does not capture hallucination.

- **Query-focused summarization (QFS):** answer + summarize for a specific query; QFS-Composer pipeline.
- **Question Answering (QA):** oldest NLP task; axes: information source, question type, answer type, knowledge store.
- **QA types:** factoid QA, open-domain QA (IBM Watson / Jeopardy!), reading comprehension (SQuAD, CoQA), knowledge-base QA, visual QA, conversational QA.
- **Retriever-reader architecture:** bi-encoder retriever + reading comprehension reader (e.g., DPR + BERT); RAG as a modern variant.

Chapter 19 — Parameter-Efficient Fine-Tuning and Language Agents

Core idea: efficient adaptation of LLMs and their extension to cognitive agent architectures.

Key terms:

- **PEFT (Parameter-Efficient Fine-Tuning):** adapt a pretrained model by training only a small fraction of parameters; matches full fine-tuning performance at a fraction of the cost.
 - **Adapters:** small bottleneck modules inserted after each Transformer sublayer; $\sim 100\times$ fewer trainable parameters.
 - **Prefix-tuning / Prompt-tuning:** prepend trainable soft tokens to key/value matrices (every layer) or input embeddings only (prompt-tuning).
 - **LoRA:** freeze W_0 ; learn low-rank update $W_0 + \alpha BA$ with $r \ll \min(d, k)$; no inference overhead.
 - **QLoRA:** LoRA on top of a 4-bit NormalFloat (NF4) quantised frozen base, plus double quantisation of the quantisation constants and paged optimisers to absorb memory spikes; fine-tunes 65B-parameter models on a single GPU with no measurable quality loss vs. full 16-bit fine-tuning.
- **Knowledge Distillation:** teacher model's soft output distribution teaches a smaller student model; e.g. DistilBERT (40% fewer parameters, 97% of BERT performance).
- **Language Agents:** LLMs extended with tool use, web browsing, code execution, robotics.
- **CoALA (Cognitive Architectures for Language Agents):** Memory (short-term / long-term: procedural, semantic, episodic) + Action space (internal: reasoning, retrieval, learning; external: grounding) + Decision cycle (Observe $\rightarrow$ Propose $\rightarrow$ Evaluate $\rightarrow$ Select $\rightarrow$ Execute).
- **Progression:** LLMs $\rightarrow$ Language Agents $\rightarrow$ Cognitive Language Agents; each step adds structured capability, autonomy, and alignment challenges.